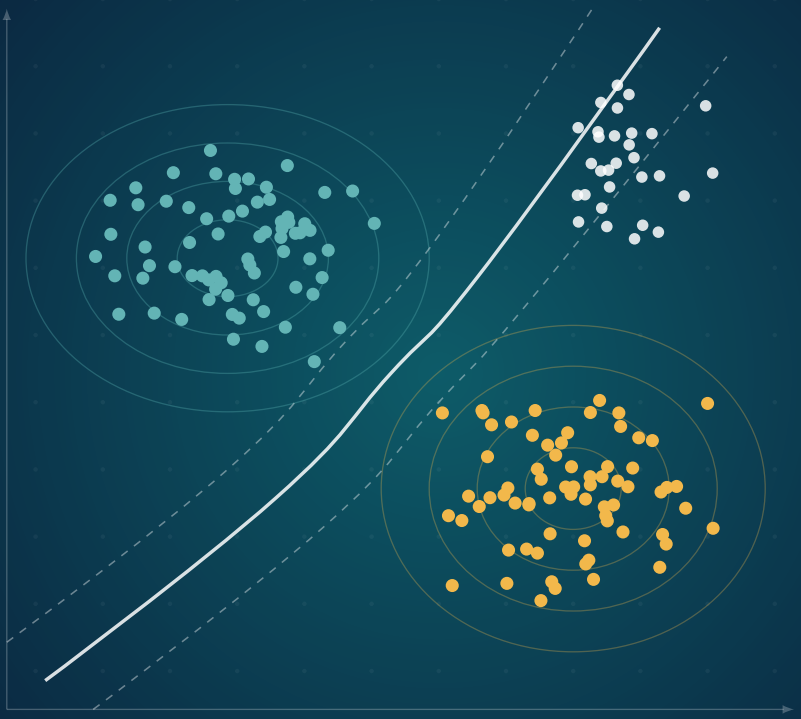
MACHINE LEARNING AND DEEP LEARNING

Foundations, Methods and Practice



CHAPTER

01 Machine Learning

48 sections · 59 figures · 50 tables · 23 algorithms · 79 numbered equations · 28 code listings

LUXSHI KARUNAKARAN

B.Sc. (Hons) Computer Science and Technology · M.Sc. Data Science and AI (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka

Foundations, Methods and Practice

CHAPTER 1

Machine Learning

48 sections · 59 figures · 50 tables · 23 algorithms · 79 numbered equations · 28 code listings

Luxshi Karunakaran

B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)

Department of Social Statistics
University of Sri Jayewardenepura, Sri Lanka

Machine Learning and Deep Learning: Foundations, Methods and Practice
Chapter 1 – Machine Learning

Author. Luxshi Karunakaran, B.Sc. (Hons) in Computer Science and Technology; M.Sc. in Data Science and Artificial Intelligence (Reading). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

Copyright. © 2026 Luxshi Karunakaran. All rights reserved.

Suggested citation. Karunakaran, L. (2026). Machine learning. In *Machine Learning and Deep Learning: Foundations, Methods and Practice* (Chapter 1). Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka.

Edition. Version 1.0, October 2026. Distributed as a book-chapter preprint via ResearchGate.

Conventions. Sections, figures, tables, equations, algorithms, code listings and highlight panels are numbered in the form *chapter.sequence*; see *How to Read This Chapter* for details. Code listings are written in Python 3.

Typesetting. Typeset with XeLaTeX in TeX Gyre Pagella, Inter and DejaVu Sans Mono.

Contents

How to Read This Chapter xv

Chapter Overview 1

1.1 Introduction to Machine Learning 3

1.1.1 What Is Machine Learning? 3

1.1.2 Machine Learning, Artificial Intelligence and Deep Learning 5

1.1.3 Why Machine Learning Now? 6

1.2 History and Evolution of Machine Learning 7

1.2.1 Early Foundations (1940s–1960s) 8

1.2.2 Winters and Revivals (1969–1990s) 8

1.2.3 The Statistical Learning Era (1990s–2000s) 8

1.2.4 The Deep Learning Era (2006–present) 9

1.3 Fundamentals and Key Concepts of Machine Learning 11

1.3.1 Data, Features and Labels 11

1.3.2 Models, Parameters and Hyperparameters 11

1.3.3 Loss, Risk and Empirical Risk Minimisation 11

1.3.4 Inductive Bias and the No Free Lunch Theorem 12

1.4 Types of Machine Learning 13

1.4.1 Supervised Learning 14

1.4.2 Unsupervised Learning 15

1.4.3 Semi-Supervised Learning 15

1.4.4 Self-Supervised Learning 15

1.4.5 Reinforcement Learning 16

1.5 Machine Learning Workflow and Lifecycle 17

1.5.1 Stages of the Lifecycle 17

1.5.2 Iteration and Reproducibility 19

1.6 Data Collection, Integration, and Preparation 20

1.6.1 Sources of Data 20

1.6.2 Sampling, Labelling and Bias 21

1.6.3 Data Integration 21

1.6.4 Organising Data for Learning 22

1.7 Data Cleaning and Data Preprocessing 23

1.7.1 Handling Missing Values 23

1.7.2 Outliers and Noise 24

1.7.3 Encoding Categorical Variables 24

1.7.4 Other Preprocessing Steps 25

1.8 Exploratory Data Analysis (EDA) 26
1.8.1 Univariate Analysis 26
1.8.2 Bivariate and Multivariate Analysis 27
1.8.3 Checking Data Quality and Assumptions 28
1.9 Feature Engineering and Feature Selection 29
1.9.1 Feature Construction 29
1.9.2 Why Select Features? 29
1.9.3 Filter, Wrapper and Embedded Methods 30
1.10 Feature Scaling and Normalization 32
1.10.1 Why Scaling Matters 33
1.10.2 Scaling Methods 33
1.11 Training, Validation, and Testing Datasets 35
1.11.1 Roles of the Three Sets 35
1.11.2 Splitting Strategies 35
1.11.3 How Large Should the Test Set Be? 36
1.12 Bias, Variance, and the Bias–Variance Trade-off 37
1.12.1 Intuition 37
1.12.2 Mathematical Decomposition 38
1.12.3 Managing the Trade-off 39
1.13 Overfitting and Underfitting 40
1.13.1 Diagnosing the Problem 40
1.13.2 Causes of Overfitting 42
1.14 Model Generalization and Regularization 42
1.14.1 Why Generalisation Is Possible 43
1.14.2 L2 (Ridge) and L1 (Lasso) Regularisation 43
1.14.3 Other Regularisation Techniques 44
1.15 Supervised Learning Algorithms 46
1.15.1 Families of Supervised Learners 46
1.15.2 Comparing the Algorithms 46
1.16 Linear Regression 48
1.16.1 The Model 48
1.16.2 Least Squares and the Normal Equations 48
1.16.3 Gradient Descent 49
1.16.4 Assumptions, Diagnostics and Evaluation 50
1.17 Logistic Regression 51
1.17.1 From Linear Scores to Probabilities 51
1.17.2 Maximum Likelihood and Cross-Entropy 52

1.17.3 Multiclass Logistic Regression . . . 53

1.18 Decision Trees 54

1.18.1 Structure of a Tree . . . 54
1.18.2 Impurity Measures: Entropy and Gini . . . 55
1.18.3 Information Gain and the Splitting Criterion . . . 56
1.18.4 Worked Example: Choosing the Root Split . . . 57
1.18.5 Overfitting, Pruning and Practical Considerations . . . 58

1.19 Random Forests 59

1.19.1 Two Sources of Randomness . . . 59
1.19.2 Out-of-Bag Error and Feature Importance . . . 60
1.19.3 Hyperparameters and Practical Use . . . 62

1.20 Support Vector Machines (SVM) 62

1.20.1 The Maximum-Margin Hyperplane . . . 63
1.20.2 Soft Margin and Hinge Loss . . . 63
1.20.3 The Dual Problem and the Kernel Trick . . . 64

1.21 k-Nearest Neighbors (k-NN) 67

1.21.1 The Algorithm and Distance Metrics . . . 67
1.21.2 Choosing k . . . 68
1.21.3 Computational Cost and the Curse of Dimensionality . . . 69

1.22 Naive Bayes 70

1.22.1 Bayes' Theorem and the Naive Assumption . . . 70
1.22.2 Variants of Naive Bayes . . . 70
1.22.3 Laplace Smoothing . . . 71
1.22.4 Strengths, Weaknesses and Use . . . 72

1.23 Ensemble Learning 73

1.23.1 Why Ensembles Work . . . 73
1.23.2 Combining Predictions and Families of Ensembles . . . 74

1.24 Bagging, Boosting, and Stacking 75

1.24.1 Bagging . . . 75
1.24.2 Boosting and AdaBoost . . . 76
1.24.3 Gradient Boosting . . . 78
1.24.4 XGBoost, LightGBM and CatBoost . . . 79
1.24.5 Stacking . . . 80

1.25 Unsupervised Learning Algorithms (overview) 83

1.25.1 Why Unsupervised Learning Matters . . . 83
1.25.2 Evaluating Without Labels . . . 84

1.26 Clustering Techniques 85

1.26.1 Measuring Similarity . . . 85
1.26.2 Families of Clustering Algorithms . . . 86
1.26.3 Choosing a Clustering Method . . . 86

1.27 k-Means Clustering 87
1.27.1 Objective Function . . . 87
1.27.2 Lloyd's Algorithm . . . 88
1.27.3 Initialisation with k-Means++ . . . 89
1.27.4 Choosing k: the Elbow Method and the Silhouette Coefficient . . . 90
1.27.5 Implementation, Variants and Limitations . . . 90

1.28 Hierarchical Clustering 92
1.28.1 Agglomerative and Divisive Strategies . . . 92
1.28.2 Linkage Criteria . . . 93
1.28.3 The Dendrogram . . . 94
1.28.4 Implementation and Practical Considerations . . . 94

1.29 DBSCAN and Density-Based Clustering 95
1.29.1 Core, Border and Noise Points . . . 95
1.29.2 The DBSCAN Algorithm . . . 96
1.29.3 Choosing ε and minPts . . . 97
1.29.4 Strengths, Limitations and Extensions . . . 98

1.30 Dimensionality Reduction 99
1.30.1 The Curse of Dimensionality . . . 99
1.30.2 Feature Selection versus Feature Extraction . . . 100
1.30.3 Linear and Non-Linear Methods . . . 100

1.31 Principal Component Analysis (PCA) 102
1.31.1 Intuition . . . 103
1.31.2 Derivation via Eigen-Decomposition of the Covariance Matrix . . . 103
1.31.3 Explained Variance and Choosing the Number of Components . . . 105
1.31.4 Worked Example . . . 105
1.31.5 Practical Considerations and Limitations . . . 107

1.32 Association Rule Learning 107
1.32.1 Itemsets, Rules and Interestingness Measures . . . 108
1.32.2 Worked Market-Basket Example . . . 108
1.32.3 The Apriori Algorithm . . . 109
1.32.4 FP-Growth and Practical Use . . . 111

1.33 Anomaly and Outlier Detection 112
1.33.1 Types of Anomalies and Problem Settings . . . 112
1.33.2 Statistical Methods: z-Score and IQR . . . 112
1.33.3 Isolation Forest . . . 113

1.33.4 Local Outlier Factor and One-Class SVM . . . 114

1.34 Semi-Supervised and Self-Supervised Learning Methods 117

1.34.1 Assumptions Behind Semi-Supervised Learning . . . 117
1.34.2 Self-Training and Pseudo-Labelling . . . 117
1.34.3 Co-Training . . . 118
1.34.4 Graph-Based Label Propagation . . . 118
1.34.5 Self-Supervised Learning: Pretext Tasks and Contrastive Methods . . . 120

1.35 Reinforcement Learning Fundamentals 121

1.35.1 Markov Decision Processes . . . 122
1.35.2 Policies and Value Functions . . . 122
1.35.3 The Bellman Equations . . . 123
1.35.4 Exploration versus Exploitation . . . 124

1.36 Q-Learning and Deep Q-Networks 124

1.36.1 The Q-Learning Update . . . 125
1.36.2 From Tables to Function Approximation . . . 127
1.36.3 Deep Q-Networks . . . 128
1.36.4 Improvements to DQN . . . 129

1.37 Model Evaluation and Performance Metrics 130

1.37.1 Choosing a Metric . . . 131
1.37.2 Baselines, Uncertainty and Significance . . . 131

1.38 Classification Metrics: Accuracy, Precision, Recall, F1-Score, and ROC-AUC 132

1.38.1 Accuracy, Precision, Recall and F1 . . . 133
1.38.2 Thresholds, ROC Curves and AUC . . . 134
1.38.3 Computing Classification Metrics in Practice . . . 134

1.39 Regression Metrics: MAE, MSE, RMSE, and R^2 136

1.39.1 Interpreting the Metrics . . . 137
1.39.2 Sensitivity to Outliers and Other Metrics . . . 138

1.40 Cross-Validation and Hyperparameter Tuning 139

1.40.1 k-Fold and Its Variants . . . 140
1.40.2 Hyperparameter Search Strategies . . . 141
1.40.3 Nested Cross-Validation and Practical Implementation . . . 142

1.41 Optimization Algorithms and Gradient Descent 143

1.41.1 Convexity . . . 144
1.41.2 Gradient Descent and Its Variants . . . 144
1.41.3 The Learning Rate . . . 145
1.41.4 Second-Order Methods: Newton's Method . . . 146

1.42 Explainable and Interpretable Machine Learning 147
1.42.1 A Taxonomy of Explanation Methods 148
1.42.2 Feature Importance and Partial Dependence 148
1.42.3 SHAP: Shapley Additive Explanations 149
1.42.4 LIME: Local Interpretable Model-Agnostic Explanations 151
1.43 Machine Learning Pipelines and Automation 152
1.43.1 Pipelines in Code 152
1.43.2 From Pipelines to MLOps 154
1.44 AutoML and Automated Feature Engineering 155
1.44.1 The CASH Problem 156
1.44.2 Automated Feature Engineering 156
1.45 Machine Learning Tools and Frameworks 157
1.45.1 The Scientific Python Stack 158
1.45.2 Classical Machine Learning Libraries 158
1.45.3 Deep Learning Frameworks and Platforms 159
1.46 Practical Machine Learning Using Python, Scikit-learn, Google Colab, and KNIME 160
1.46.1 Working in Google Colab 160
1.46.2 An End-to-End scikit-learn Project 161
1.46.3 Visual Workflows with KNIME 162
1.47 Machine Learning Applications Across Different Industries 163
1.47.1 Healthcare and Life Sciences 163
1.47.2 Finance, Commerce and Industry 164
1.48 Challenges, Limitations, and Future Trends in Machine Learning 165
1.48.1 Technical Challenges 165
1.48.2 Ethical, Social and Legal Challenges 166
1.48.3 Future Trends 166
Chapter Summary 168
Key Terms 169
Review Questions 171
Exercises 172
Further Reading 174

List of Figures

1.1 Traditional programming maps rules and data to answers . . . 5

1.2 Deep learning is a subset of machine learning, which is itself a subset of artificial intelligence . . . 6

1.3 Timeline of selected milestones in machine learning, from the McCulloch–Pitts neuron to large language models . . . 9

1.4 Taxonomy of machine learning paradigms with representative tasks and algorithms beneath each . . . 14

1.5 The agent–environment interaction loop in reinforcement learning: the agent acts, and the environment returns a reward and the next state . . . 16

1.6 The machine learning lifecycle . . . 18

1.7 Three ways to encode the nominal feature 'colour': ordinal integers, one-hot binary columns, and target (mean) encoding using a binary target . . . 25

1.8 Typical EDA plots for a synthetic housing dataset: (a) a right-skewed price distribution, (b) price against floor area with a clear outlier, and (c) a correlation heatmap of four features . . . 27

1.9 Feature engineering can turn a non-linear problem into a linear one: (a) two concentric classes cannot be separated by a line in (x_1, x_2) . . . 30

1.10 Effect of scaling on a two-feature dataset containing one outlier: min–max scaling squeezes the bulk of the data into a narrow band, standardisation is less affected, and robust scaling keeps the bulk well spread . . . 34

1.11 A typical hold-out partition: the model is fitted on the training set, tuned on the validation set and assessed once on the test set . . . 35

1.12 Bias and variance as dart throws: each point is the prediction of a model trained on a different sample . . . 38

1.13 Bias^2, variance and total expected error as a function of polynomial degree, estimated by fitting 400 noisy training sets of 15 points from $y = \sin(2\pi x) + \varepsilon$ with $\sigma = 0.4$ . . . 39

1.14 Underfitting, a good fit and overfitting: polynomials of degree 1, 4 and 15 fitted to the same 20 noisy samples of a sine curve (dashed) . . . 40

1.15 Learning curves for (a) a high-bias model (linear regression on a non-linear target) and (b) a high-variance model (unpruned decision tree): the gap between training and validation error reveals overfitting . . . 41

1.16 Geometry of regularisation: loss contours (ellipses) centred on the unregularised solution $\hat{w}$ meet the L2 constraint disc at a non-sparse point but the L1 diamond at a corner where $w_1 = 0$ . . . 44

1.17 Decision boundaries learned by six classifiers on the same two-moons dataset, illustrating the inductive bias of each family . . . 47

1.18 Left: the sigmoid function maps a linear score z to a probability . . . 52
1.19 A depth-2 classification tree (left) and the axis-aligned partition of the feature space it induces (right) . . . 55
1.20 Impurity measures for a binary node as a function of the proportion p of the positive class . . . 56
1.21 Test and out-of-bag error of a random forest as the number of trees grows, compared with a single fully grown decision tree . . . 61
1.22 A linear soft-margin SVM: the solid line is the decision boundary $w^T x + b = 0$, dashed lines are the margin boundaries $w^T x + b = \pm 1$, and circled points are support vectors 63
1.23 Effect of the RBF-kernel parameters on an SVM's decision boundary: small γ and C give smooth, underfitting boundaries, while large values give complex boundaries that overfit . . . 66
1.24 k-NN decision boundaries for k = 1, 15 and 75 on the same 200-point dataset: small k overfits noise . . . 68
1.25 Error of a majority vote of B independent classifiers, each with error rate ε . . . 74
1.26 Boosting builds learners sequentially: each weak learner is trained on reweighted data emphasising the previous learners' mistakes (marker size $\propto$weight), and the final model is a weighted vote . . . 77
1.27 Gradient boosting for regression with depth-2 trees and learning rate 0.1: the ensemble prediction (red) after 1, 10 and 100 stages . . . 79
1.28 Stacking: diverse level-0 base models produce out-of-fold predictions, which become the input features of a level-1 meta-learner that outputs the final prediction . . . 81
1.29 The same data viewed through supervised (left, labels known) and unsupervised (right, labels unknown) lenses . . . 84
1.30 Four clustering algorithms on blob-shaped (top) and moon-shaped (bottom) data . . . 87
1.31 Lloyd's algorithm on three groups: centroids (stars) start bunched in one corner and migrate to the group centres over successive iterations, while the inertia J falls monotonically until convergence . . . 89
1.32 Choosing k for data drawn from four groups: the elbow in the inertia curve (left) and the peak of the mean silhouette coefficient (right) both indicate k = 4 . . . 91
1.33 Ward-linkage dendrogram of 20 points from three groups (left) and the resulting clusters (right) . . . 94
1.34 DBSCAN point types for $\varepsilon = 0.5$ and minPts = 4: core points (filled) have at least four neighbours within ε, border points (open) lie within ε of a core point, and noise points (crosses) are isolated . . . 96
1.35 Distance concentration: for 500 uniformly random points, the relative contrast $(d_{max} - d_{min})/d_{min}$ between a query's farthest and nearest neighbour shrinks rapidly as dimension grows . . . 100

1.36 Two-dimensional embeddings of the 64-dimensional handwritten digits data: PCA (left) preserves global variance but overlaps classes, whereas t-SNE (right) separates the ten digits into distinct clusters . . . 102
1.37 Principal components of correlated two-dimensional data . . . 103
1.38 Scree plot (bars, individual explained variance) and cumulative explained variance (line) for the standardised 30-feature Wisconsin breast-cancer data . . . 106
1.39 Itemset lattice for four frequent items (B = bread, M = milk, D = diapers, E = beer) in the basket example . . . 110
1.40 Isolation Forest, LOF and One-Class SVM on two normal clusters of different density plus scattered anomalies . . . 115
1.41 Label propagation on the two-moons data with only one labelled point per class (stars) . . . 119
1.42 The agent–environment interaction loop: at each step the agent observes the state, selects an action according to its policy, and the environment returns a reward and the next state . . . 122
1.43 Average reward on a ten-armed Gaussian bandit (averaged over 300 runs) . . . 125
1.44 DQN architecture and training components: a convolutional Q-network maps stacked frames to one Q-value per action . . . 129
1.45 ROC curves (left) and precision–recall curves (right) for logistic regression and a random forest on imbalanced data (5% positives) . . . 135
1.46 Precision, recall and F1 of a logistic-regression model as functions of the decision threshold . . . 135
1.47 Effect of a single growing error on MAE and RMSE for 100 predictions with otherwise small errors: RMSE is far more sensitive to the outlier than MAE . . . 138
1.48 Five-fold cross-validation: in each of five iterations a different fold (orange) is held out for validation and the remaining four (blue) are used for training . . . 140
1.49 Grid search versus random search with nine trials when only one of two hyperparameters matters (Bergstra and Bengio, 2012) . . . 141
1.50 Optimisation paths on the convex least-squares loss of a two-parameter linear model: batch gradient descent moves smoothly . . . 145
1.51 Effect of the learning rate on gradient descent for $J(\theta) = \theta^2$: a small rate converges slowly, a well-chosen rate converges quickly, and a rate above the stability limit ($\eta > 1$ here) diverges . . . 146
1.52 Partial dependence (thick line) and individual conditional expectation curves (thin lines, 50 houses) for income and house age in a gradient-boosted model of synthetic house prices . . . 150

1.53 Structure of a scikit-learn pipeline for tabular data: numerical and categorical columns are processed in parallel branches of a ColumnTransformer, concatenated and passed to the estimator . . . 153
1.54 An automated machine learning lifecycle (MLOps): production monitoring feeds back into data collection and triggers retraining, forming a continuous loop . . . 154
1.55 A generic AutoML workflow: the system searches jointly over preprocessing, feature engineering, algorithms and hyperparameters, evaluates candidates by cross-validation and returns the best model or an ensemble . . . 156
1.56 Layers of a typical Python machine learning stack, from hardware and numerical foundations up to modelling libraries, workflow tools and applications . . . 160
1.57 Steps of the end-to-end project implemented in Python and, equivalently, as KNIME nodes . . . 161
1.58 How often each machine learning problem type appears among the thirteen representative applications in the industry table . . . 165
1.59 Shifts in the dominant machine learning paradigm, from hand-engineered features and classical models to representation learning and pre-trained foundation models adapted to many tasks (approximate periods) . . . 167

List of Tables

1.1 Roadmap of Chapter 1: the 48 sections grouped into ten themes . . . 4
1.2 Representative application domains of machine learning . . . 7
1.3 Selected milestones in the history of machine learning . . . 10
1.4 Core terminology used throughout the chapter . . . 13
1.5 Comparison of the five machine learning paradigms . . . 17
1.6 Stages of the machine learning lifecycle, their outputs and common pitfalls . . . 19
1.7 Common data sources and their characteristics . . . 21
1.8 Missing-data mechanisms and appropriate treatments . . . 23
1.9 Comparison of feature-selection families . . . 31
1.10 Feature scaling methods and when to use them . . . 34
1.11 Data-splitting strategies . . . 36
1.12 Actions that reduce bias or variance . . . 40
1.13 Diagnosing and remedying underfitting and overfitting . . . 41
1.14 Regularisation techniques and where they are used . . . 45
1.15 Comparison of classical supervised learning algorithms . . . 47
1.16 Assumptions of ordinary least squares and how to check them . . . 50
1.17 The Play Tennis dataset (14 examples, 4 categorical features) . . . 58
1.18 Key random-forest hyperparameters (scikit-learn names) . . . 62
1.19 Common SVM kernel functions . . . 65
1.20 Strengths and weaknesses of k-NN . . . 69
1.21 Naive Bayes variants . . . 71
1.22 The three main families of ensemble methods . . . 75
1.23 Comparison of the major gradient-boosting libraries . . . 80
1.24 Main families of unsupervised learning and the questions they answer . . . 83
1.25 Comparison of the main clustering families . . . 86
1.26 Strengths and limitations of k-means . . . 92
1.27 Properties of common linkage criteria . . . 93
1.28 Strengths and limitations of DBSCAN . . . 98
1.29 Comparison of common dimensionality-reduction methods . . . 102
1.30 Eight market-basket transactions used in the worked examples . . . 109
1.31 Comparison of frequent-itemset mining algorithms . . . 111
1.32 Comparison of anomaly-detection methods . . . 116
1.33 Semi-supervised and self-supervised methods at a glance . . . 121

1.34 A taxonomy of reinforcement learning approaches . . . 125
1.35 Key extensions of the Deep Q-Network . . . 130
1.36 Typical evaluation metrics by task type . . . 131
1.37 Layout of the binary confusion matrix . . . 132
1.38 Summary of classification metrics . . . 136
1.39 Comparison of regression metrics . . . 139
1.40 Cross-validation schemes and when to use them . . . 141
1.41 Comparison of hyperparameter search strategies . . . 143
1.42 Comparison of optimisation methods for machine learning . . . 147
1.43 Common explanation methods and their properties . . . 149
1.44 Tools supporting ML pipelines and MLOps . . . 155
1.45 Selected AutoML systems . . . 158
1.46 Major machine learning libraries and frameworks . . . 159
1.47 Equivalent steps in scikit-learn and KNIME . . . 163
1.48 Representative machine learning applications by industry . . . 164
1.49 Major challenges in machine learning and common mitigations . . . 167
1.50 Key terms introduced in Chapter 1 . . . 169

List of Algorithms

1.1 Greedy Forward Feature Selection . . . 31
1.2 Batch Gradient Descent for Linear Regression . . . 49
1.3 Logistic Regression Trained by Gradient Descent . . . 52
1.4 Recursive Decision-Tree Induction (ID3/CART style) . . . 56
1.5 Random Forest . . . 60
1.6 Linear SVM by Stochastic Sub-gradient Descent (Pegasos) . . . 64
1.7 k-Nearest Neighbours Prediction . . . 67
1.8 AdaBoost (Discrete, Binary) . . . 76
1.9 Gradient Boosting with Regression Trees . . . 78
1.10 Lloyd's Algorithm for k-Means . . . 88
1.11 k-Means++ Seeding . . . 90
1.12 Agglomerative Hierarchical Clustering . . . 92
1.13 DBSCAN . . . 97
1.14 Principal Component Analysis . . . 104
1.15 Apriori Frequent-Itemset Mining . . . 109
1.16 Isolation Forest . . . 114
1.17 Self-Training with Confidence Threshold . . . 117
1.18 Value Iteration . . . 123
1.19 Tabular Q-Learning with ε-Greedy Exploration . . . 126
1.20 Deep Q-Learning with Experience Replay and Target Network . . . 128
1.21 Bayesian Optimisation for Hyperparameter Tuning . . . 142
1.22 Mini-Batch Stochastic Gradient Descent . . . 145
1.23 LIME for Tabular Data . . . 151

List of Code Listings

1.1 A reproducible end-to-end scikit-learn pipeline with a held-out test set. 19
1.2 Imputation, outlier capping and encoding with pandas and scikit-learn. 25
1.3 A compact EDA routine with pandas and Matplotlib. 28
1.4 Filter, wrapper and embedded feature selection in scikit-learn. 32
1.5 Stratified, group and temporal splits. 37
1.6 Comparing ridge, lasso and elastic net with cross-validated regularisation strength. 45
1.7 Linear regression via the normal equations (NumPy) and scikit-learn. 51
1.8 Regularised multinomial logistic regression and odds ratios. 54
1.9 Training, pruning and printing a decision tree. 59
1.10 Tuning an RBF-kernel SVM with a scaled pipeline and grid search. 66
1.11 A multinomial naive Bayes text classifier. 72
1.12 Hard and soft voting ensembles of heterogeneous classifiers. 74
1.13 AdaBoost, gradient boosting with early stopping, and stacking in scikit-learn. 81
1.14 Selecting k with the silhouette score and fitting k-means in scikit-learn. 91
1.15 Agglomerative clustering, dendrogram and flat cut with SciPy. 95
1.16 Choosing ε with a k-distance plot and running DBSCAN. 98
1.17 PCA with scikit-learn: standardise, retain 95% of variance, and reconstruct. 106
1.18 Frequent itemsets and association rules with mlxtend. 111
1.19 Unsupervised anomaly detection with Isolation Forest and LOF. 115
1.20 Self-training and label spreading in scikit-learn with 5% labelled data. 119
1.21 Tabular Q-learning on FrozenLake with Gymnasium. 127
1.22 Confusion matrix, classification report, ROC-AUC and average precision. 135
1.23 Regression metrics with scikit-learn. 138
1.24 Grid search, randomised search and nested cross-validation with scikit-learn. 142
1.25 Permutation importance, partial dependence and SHAP for a random forest. 151
1.26 A leakage-safe preprocessing and modelling pipeline for mixed-type data. 153
1.27 AutoML on tabular data with AutoGluon and FLAML. 157
1.28 An end-to-end classification project with scikit-learn (runs unchanged in Google Colab). 161

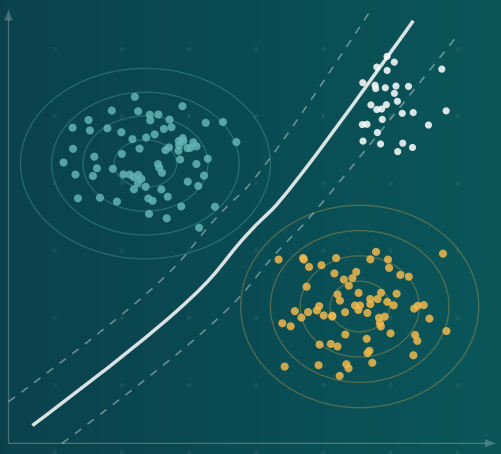
How to Read This Chapter

Every element in this chapter carries a label and a number so that it can be identified, cited and cross-referenced precisely. Numbers restart in each chapter and take the form *chapter.sequence*. The conventions are summarised in the table below.

Element	Label format	Purpose
Section / Subsection	1.4 / 1.4.2	Numbered heading; subsections are nested inside sections.
Figure	Figure 1.1	Diagram or chart; caption appears below the figure.
Table	Table 1.1	Structured comparison or summary; caption appears above the table.
Equation	(1.1)	Display equation; number at the right margin; symbols explained beneath.
Algorithm	Algorithm 1.1	Pseudocode with inputs, outputs and numbered steps.
Listing	Listing 1.1	Runnable Python code in a shaded panel.
Definition	Definition 1.1	Formal definition of a key term (blue panel).
Worked Example	Worked Example 1.1	Step-by-step numerical illustration (green panel).
Case Study	Case Study 1.1	Real-world application (orange panel).
Key Point / Note / Practice Tip	—	Unnumbered highlight panels: takeaways, caveats and practical advice.

CHAPTER 1

Machine Learning



48 sections · 59 figures · 50 tables · 23 algorithms · 79 numbered equations · 28 code listings

Chapter Overview

Machine learning is the study of computer programs that improve their performance on a task by learning from data rather than by following rules written explicitly by a programmer. Over the last three decades it has moved from a specialised corner of artificial intelligence research to a core engineering discipline: it ranks web search results, filters spam, detects fraudulent card transactions, recommends films and products, reads medical images, forecasts demand and translates between languages. Behind each of these applications sit a small number of recurring ideas — representing examples as feature vectors, choosing a family of candidate models, measuring error with a loss function, fitting parameters by optimisation and, above all, checking that what has been learned generalises to data the model has never seen.

This chapter builds those ideas from the ground up. It begins with what machine learning is, where it came from and how its main paradigms — supervised, unsupervised, semi-supervised, self-supervised and reinforcement learning — differ. It then follows the life of a real project: collecting and integrating data, cleaning and preprocessing it, exploring it visually and statistically, engineering and selecting features, scaling them, and partitioning the data into training, validation and test sets. With the data pipeline in place, the chapter turns to the central theoretical tension of the field, the bias–variance trade-off, and to the practical tools used to manage it: diagnosing overfitting and underfitting, and controlling model complexity through regularisation.

The second half of the chapter is a tour of the algorithms that every practitioner should

know. The classical supervised learners — linear and logistic regression, decision trees, random forests, support vector machines, k-nearest neighbours and naive Bayes — are each developed from intuition to mathematics to pseudocode to working Python. Ensemble methods (bagging, boosting and stacking) follow, including the gradient-boosting libraries that dominate competitions on tabular data. The chapter then covers unsupervised learning (clustering, dimensionality reduction, density estimation and anomaly detection), semi-supervised, self-supervised and reinforcement learning, model evaluation metrics, cross-validation, hyperparameter tuning and optimisation by gradient descent, and finishes with interpretability, automated machine learning, tools, deployment and an outlook on the field.

The emphasis throughout is on understanding *why* a method works and *when* it fails, not merely on calling a library function. Every major algorithm is accompanied by its objective function, a worked numerical example that can be checked with a calculator, and a short scikit-learn listing. Chapter 2 builds on this material to develop deep learning, and Chapter 3 treats advanced and integrated topics; the foundations laid here — loss functions, generalisation, regularisation and honest evaluation — recur in every later chapter.

Learning Objectives

After studying this chapter, you should be able to:

- ✓ Define machine learning formally and distinguish it from traditional rule-based programming, statistics and the broader field of artificial intelligence.
- ✓ Outline the historical development of machine learning and identify the milestones that shaped today's methods.
- ✓ Classify a problem as supervised, unsupervised, semi-supervised, self-supervised or reinforcement learning, and choose an appropriate formulation.
- ✓ Describe each stage of the machine learning lifecycle, from problem framing and data collection to deployment and monitoring.
- ✓ Clean, preprocess, explore, encode, scale and split a raw dataset without introducing data leakage.
- ✓ Engineer informative features and apply filter, wrapper and embedded feature-selection methods.
- ✓ Explain the bias–variance decomposition, diagnose overfitting and underfitting from learning curves, and apply L1, L2 and elastic-net regularisation.
- ✓ Derive the objective functions of linear regression, logistic regression and support vector machines, and explain how each is optimised.
- ✓ Build decision trees using entropy, Gini impurity and information gain, and compute a split by hand.
- ✓ Apply k-nearest neighbours and naive Bayes classifiers and reason about their assumptions.

- ✓ Explain how bagging, random forests, AdaBoost, gradient boosting and stacking reduce error, and compare XGBoost, LightGBM and CatBoost.
- ✓ Evaluate, tune, interpret and deploy machine learning models responsibly using modern tools.

Prerequisites

The chapter assumes familiarity with first-year university mathematics: linear algebra (vectors, matrices, dot products, matrix inversion), differential calculus (partial derivatives, gradients and the chain rule) and elementary probability and statistics (random variables, expectation, variance, conditional probability, Bayes' theorem and the normal distribution). Code listings use Python 3 with NumPy, pandas, Matplotlib and scikit-learn; readers should be comfortable writing functions, loops and list comprehensions and importing packages. No prior exposure to machine learning is required.

Chapter Roadmap

The 48 sections of this chapter fall into ten themes, summarised in Table 1.1. Readers new to the field should read the themes in order; readers with some background may treat each theme as a self-contained unit and use the table to navigate.

1.1 Introduction to Machine Learning

Consider the problem of deciding whether an incoming e-mail is spam. A traditional programmer would write rules: if the message contains the phrase "you have won", or comes from a domain on a blocklist, or has more than three exclamation marks in the subject, then flag it. Such rules are brittle. Spammers change their wording, legitimate mail occasionally trips a rule, and the rule base grows into thousands of hand-tuned conditions that nobody fully understands. Machine learning takes a different route. Instead of writing the rules, we collect many examples of e-mails that users have already labelled as spam or not spam, describe each message by a set of measurable properties, and let an algorithm discover the decision rule that best separates the two groups. When spammers change tactics, we retrain on fresh examples rather than rewrite code.

1.1.1 What Is Machine Learning?

The term *machine learning* was popularised by Arthur Samuel, who in 1959 described his checkers-playing program as giving computers the ability to learn without being

Table 1.1 Roadmap of Chapter 1: the 48 sections grouped into ten themes.

Theme	Sections	What you will learn
Foundations	1.1–1.5	Definition and history of machine learning, key concepts, the five learning paradigms, and the end-to-end workflow and lifecycle
Data	1.6–1.11	Collecting, integrating, cleaning and exploring data; feature engineering, selection and scaling; training/-validation/test splits
Model building & generalisation	1.12–1.14	Bias–variance trade-off, overfitting and underfitting, generalisation and regularisation
Supervised algorithms	1.15–1.22	Linear and logistic regression, decision trees, random forests, SVMs, k-NN and naive Bayes
Ensembles	1.23–1.24	Ensemble principles; bagging, AdaBoost, gradient boosting, XGBoost, LightGBM, CatBoost, stacking
Unsupervised	1.25–1.33	Clustering (k-means, hierarchical, density-based), dimensionality reduction and PCA, association rules and anomaly detection
Semi/self-supervised & RL	1.34–1.36	Learning from few labels, pretext and contrastive tasks, Markov decision processes, Q-learning and deep Q-networks
Evaluation & tuning	1.37–1.41	Classification and regression metrics, cross-validation, hyperparameter search, gradient descent and its variants
Interpretability & automation	1.42–1.44	Explainable AI (feature importance, PDP, SHAP, LIME), ML pipelines, AutoML and automated feature engineering
Practice & outlook	1.45–1.48	Tools and frameworks, hands-on practice with scikit-learn, Colab and KNIME, industry applications, challenges and future trends

explicitly programmed. A more operational definition, due to Tom Mitchell (1997), is given in the box below and is the one used throughout this book. Its value lies in forcing us to name three things for any learning problem: the task, the experience from which the system learns, and the measure by which we judge improvement.

D Definition 1.1 | Machine Learning (Mitchell, 1997)

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E.

- **Spam filtering:** T = classify e-mails; E = a corpus of labelled e-mails; P = proportion classified correctly.
- **House-price estimation:** T = predict sale price; E = historical sales with features; P = mean absolute error in currency units.
- **Game playing:** T = play Go; E = games played against itself; P = win rate against a refer-

ence opponent.

Figure 1.1 contrasts the two programming paradigms. In traditional programming, a human supplies the rules and the data and the computer produces answers. In machine learning, the human supplies data together with the desired answers, and the computer produces the rules — a *model* — which can then be applied to new data in the ordinary way. The model is a mathematical function with adjustable parameters; *learning* is the process of adjusting those parameters so that the function agrees with the examples.

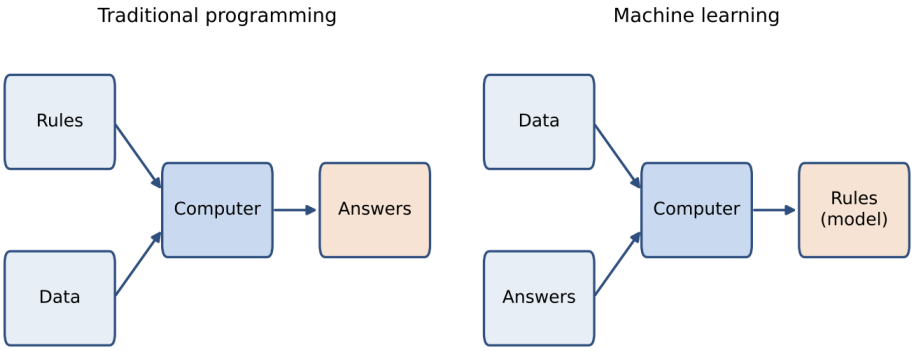


Figure 1.1 Traditional programming maps rules and data to answers; machine learning maps data and answers to rules (a model), which are then applied to new inputs.

1.1.2 Machine Learning, Artificial Intelligence and Deep Learning

Machine learning is frequently confused with the broader and narrower terms around it. *Artificial intelligence* (AI) is the umbrella field concerned with building systems that perform tasks we associate with intelligence: reasoning, planning, perception, language and learning. Much early AI was symbolic, relying on hand-crafted knowledge bases and logical inference; machine learning is the subfield that acquires its competence from data. *Deep learning* is in turn a subfield of machine learning that uses neural networks with many layers to learn hierarchical representations directly from raw inputs such as pixels, audio samples or characters. The nesting is shown in Figure 1.2. Machine learning also overlaps heavily with statistics: both fit models to data, but statistics has traditionally emphasised inference about parameters and uncertainty, whereas machine learning has emphasised predictive accuracy on new data and computational scalability. The distinction has blurred considerably, and modern practitioners draw freely on both traditions.

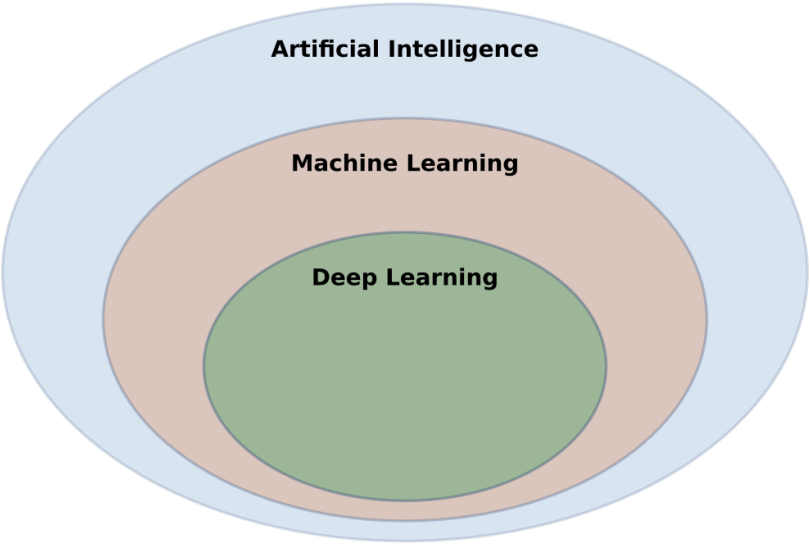


Figure 1.2 Deep learning is a subset of machine learning, which is itself a subset of artificial intelligence; data science overlaps all three.

1.1.3 Why Machine Learning Now?

The core algorithms in this chapter are decades old, yet machine learning only became ubiquitous in the 2010s. Three forces converged. First, *data*: the digitisation of commerce, communication, science and sensing created datasets orders of magnitude larger than before. Second, *computation*: cheap commodity clusters, cloud computing and, crucially, graphics processing units (GPUs) made it feasible to fit models with millions or billions of parameters. Third, *software and algorithms*: open-source libraries such as scikit-learn, XGBoost, TensorFlow and PyTorch lowered the barrier to entry, while algorithmic advances in optimisation, regularisation and network design made large models trainable. Machine learning is the right tool when the rules are too complex to write by hand, when they change over time, when the task must be personalised to individual users, or when we wish to discover structure in data that no human has yet articulated. It is the wrong tool when a simple deterministic rule suffices, when there is too little representative data, or when errors are unacceptable and cannot be bounded.

Table 1.2 lists representative domains together with the learning paradigm most commonly used; the paradigms themselves are defined in the section on Types of Machine Learning. Notice that the same paradigm — supervised classification, say — appears in finance, healthcare and manufacturing. This generality is what makes ma-

Table 1.2 Representative application domains of machine learning.

Domain	Typical task	Learning paradigm
Finance	Credit scoring, fraud detection, algorithmic trading	Supervised, anomaly detection
Healthcare	Disease diagnosis from images or records, readmission risk	Supervised
Retail and media	Product and content recommendation, customer segmentation	Supervised, unsupervised
Manufacturing	Predictive maintenance, visual quality inspection	Supervised, anomaly detection
Language	Translation, sentiment analysis, question answering	Self-supervised then supervised
Transport	Demand forecasting, route optimisation, driver assistance	Supervised, reinforcement
Science	Protein structure prediction, materials discovery, astronomy surveys	Supervised, self-supervised

chine learning a discipline rather than a collection of domain tricks: the algorithms and the methodology for evaluating them transfer across fields.

Key Point | Key Points

- Machine learning replaces hand-written rules with a model whose parameters are fitted to data.
- Every learning problem should be stated in terms of a task T, experience E and performance measure P.
- Deep learning $\subset$ machine learning $\subset$ artificial intelligence.
- Machine learning is appropriate when rules are complex, changing or personalised and representative data are available.

1.2 History and Evolution of Machine Learning

The history of machine learning is not a straight line of progress but a series of waves, each driven by a new idea, inflated by optimism, and followed by a period of disappointment when the idea met its limits. Understanding these waves helps explain why today's toolkit contains such a diverse mix of methods, and why claims about imminent breakthroughs deserve healthy scepticism. The main milestones are plotted in Figure 1.3 and listed in Table 1.3.

1.2.1 Early Foundations (1940s–1960s)

In 1943 Warren McCulloch and Walter Pitts proposed a mathematical model of a neuron as a threshold logic unit, showing that networks of such units could compute logical functions. In 1950 Alan Turing published *Computing Machinery and Intelligence*, which introduced the imitation game and explicitly discussed "learning machines". Arthur Samuel's checkers program at IBM, developed through the 1950s, improved by playing against itself and used an early form of what we would now call temporal-difference learning; Samuel's 1959 paper is generally credited with popularising the phrase "machine learning". In 1958 Frank Rosenblatt introduced the *perceptron*, a single-layer neural network with a learning rule guaranteed to converge when the classes are linearly separable. The perceptron attracted great publicity, and with it inflated expectations.

1.2.2 Winters and Revivals (1969–1990s)

In 1969 Marvin Minsky and Seymour Papert's book *Perceptrons* rigorously analysed the limits of single-layer networks, showing, for example, that they cannot represent the exclusive-or (XOR) function. Funding for neural-network research declined, contributing to the first "AI winter". Meanwhile other lines of work progressed: Cover and Hart (1967) analysed the nearest-neighbour rule; Breiman, Friedman, Olshen and Stone published *Classification and Regression Trees* (CART) in 1984; and Ross Quinlan introduced the ID3 decision-tree algorithm (1986) and later C4.5 (1993). The popularisation of *backpropagation* by Rumelhart, Hinton and Williams in 1986 showed how to train multilayer networks and revived neural-network research. Chris Watkins introduced Q-learning in his 1989 thesis, and Gerald Tesauro's TD-Gammon (1992) reached near-expert backgammon play by reinforcement learning.

1.2.3 The Statistical Learning Era (1990s–2000s)

The 1990s saw machine learning grow closer to statistics and optimisation. Vladimir Vapnik's statistical learning theory provided generalisation bounds, and Cortes and Vapnik's *support-vector networks* (1995) became a dominant classification method. Freund and Schapire's AdaBoost (1995–1997) showed that combining many weak learners could yield a strong one, and Leo Breiman's bagging (1996) and random forests (2001) and Jerome Friedman's gradient boosting machine (2001) established ensemble methods as some of the most accurate off-the-shelf tools. Hochreiter and Schmidhuber's long short-term memory (LSTM, 1997) and LeCun and colleagues' convolutional network LeNet-5 (1998) laid foundations that would bear fruit a decade later. Kernel methods, graphical models and Bayesian approaches matured, and textbooks such

as Mitchell (1997), Hastie, Tibshirani and Friedman (2001; 2nd ed. 2009) and Bishop (2006) codified the field.

1.2.4 The Deep Learning Era (2006–present)

In 2006 Geoffrey Hinton and colleagues showed how deep belief networks could be pretrained layer by layer, rekindling interest in deep architectures. The decisive moment came in 2012 when AlexNet (Krizhevsky, Sutskever and Hinton), a GPU-trained convolutional network, won the ImageNet Large Scale Visual Recognition Challenge by a wide margin. A rapid sequence followed: generative adversarial networks (Goodfellow et al., 2014), residual networks (He et al., 2015), AlphaGo's defeat of Lee Sedol at Go (2016), the Transformer architecture (Vaswani et al., 2017), BERT (Devlin et al., 2018) and large language models such as GPT-3 (Brown et al., 2020). On tabular data, meanwhile, the gradient-boosting libraries XGBoost (Chen and Guestrin, 2016), LightGBM (Ke et al., 2017) and CatBoost (Prokhorenkova et al., 2018) became the default choice of practitioners — a reminder that classical methods remain highly relevant. Deep learning is developed fully in Chapter 2.

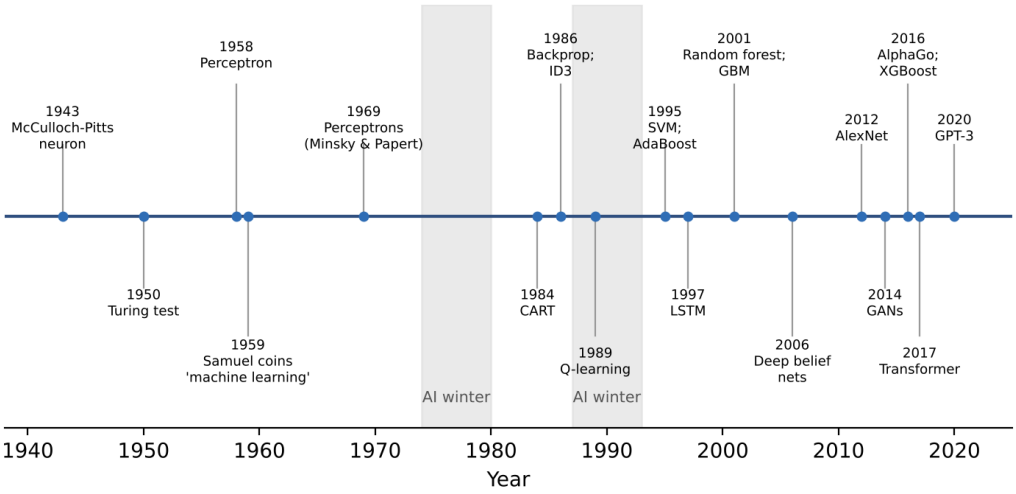


Figure 1.3 Timeline of selected milestones in machine learning, from the McCulloch–Pitts neuron to large language models; shaded bands mark the two periods of reduced funding known as AI winters.

Table 1.3 Selected milestones in the history of machine learning.

Year	Milestone	Contributors	Significance
1943	Threshold-logic neuron model	McCulloch & Pitts	First mathematical model of a neuron
1950	Computing Machinery and Intelligence	Turing	Framed machine intelligence and learning machines
1958	Perceptron	Rosenblatt	First trainable neural classifier
1959	Self-improving checkers program	Samuel	Popularised the term machine learning
1967	Nearest-neighbour rule analysis	Cover & Hart	Error bound for k-NN
1969	Perceptrons (book)	Minsky & Papert	Exposed limits of single-layer networks
1984	CART	Breiman, Friedman, Olshen & Stone	Classification and regression trees
1986	Backpropagation	Rumelhart, Hinton & Williams	Training of multilayer networks
1986	ID3	Quinlan	Entropy-based tree induction
1989	Q-learning	Watkins	Model-free reinforcement learning
1995	Support-vector networks	Cortes & Vapnik	Maximum-margin kernel classifiers
1997	AdaBoost (journal paper)	Freund & Schapire	Boosting weak learners
2001	Random forests; gradient boosting	Breiman; Friedman	Robust ensemble methods
2012	AlexNet	Krizhevsky, Sutskever & Hinton	Deep learning breakthrough on ImageNet
2016	AlphaGo; XGBoost	Silver et al.; Chen & Guestrin	RL milestone; scalable boosting
2017	Transformer	Vaswani et al.	Attention-based foundation of modern language models
2020	GPT-3	Brown et al.	Few-shot learning with large language models

Note | Lessons from History

Many ideas that "failed" were simply ahead of the available data and computation: neural networks were proposed in the 1940s and 1950s but only dominated after 2012, when GPUs and large labelled datasets arrived. Conversely, methods declared obsolete — trees, linear models, nearest neighbours — remain indispensable baselines. A sound practitioner judges methods by evidence on the problem at hand, not by fashion.

1.3 Fundamentals and Key Concepts of Machine Learning

Before studying particular algorithms we need a common vocabulary. This section introduces the objects every learning problem involves — instances, features, labels, models, parameters, loss functions and risk — and states learning as an optimisation problem. These concepts are summarised in Table 1.4 and will be used without further comment in the remainder of the chapter.

1.3.1 Data, Features and Labels

A dataset is a collection of n *instances* (also called examples, samples, observations or records). Each instance is described by d *features* (attributes, predictors, input variables or covariates), collected into a feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)^{\mathrm{T}}$. Stacking the n feature vectors as rows gives the *design matrix* $\mathbf{X} \in \mathbb{R}^{n \times d}$. In supervised learning each instance also carries a *label* or *target* y: a real number for regression, or one of K discrete classes for classification. Features may be numerical (continuous such as temperature, or discrete such as a count) or categorical (nominal such as colour, or ordinal such as an education level). How features are represented — the subject of the sections on feature engineering and scaling — often matters more than which algorithm is chosen.

1.3.2 Models, Parameters and Hyperparameters

A *model* is a function f that maps an input $\mathbf{x}$ to a prediction $\hat{y} = f(\mathbf{x}; \theta)$. The *parameters* θ are the quantities adjusted during training: the weights of a linear model, the split thresholds of a tree, the connection weights of a neural network. The set of all functions the learning algorithm can choose from — say, all linear functions of $\mathbf{x}$ — is called the *hypothesis space* $\mathcal{H}$. *Hyperparameters* are settings chosen before training that control the learning process or the capacity of $\mathcal{H}$: the learning rate, the depth of a tree, the number of neighbours k, the strength of a regularisation penalty. Parameters are learned from training data; hyperparameters are chosen using validation data, as described in the section on training, validation and testing datasets.

1.3.3 Loss, Risk and Empirical Risk Minimisation

To learn, we need to measure how wrong a prediction is. A *loss function* $L(y, \hat{y})$ assigns a non-negative penalty to predicting $\hat{y}$ when the truth is y. Common choices are the squared loss $(y - \hat{y})^2$ for regression and the 0–1 loss, which is 1 for a misclassification and 0 otherwise. We assume that data are drawn independently from an unknown joint distribution $P(\mathbf{x}, y)$. What we really want to minimise is the *expected risk* of Equation (1.1) — the average loss over all possible future data. Because P is unknown, we

instead minimise the *empirical risk* of Equation (1.2), the average loss over the n training examples. This principle, *empirical risk minimisation* (ERM), underlies almost every algorithm in this chapter.

$$R(f) = \mathbb{E}_{(\mathbf{x},\, y)\,\sim\, P}[L(y,\, f(\mathbf{x}))] \tag{1.1}$$

where P is the unknown data-generating distribution, L the loss function and f the model; R(f) is also called the generalisation error.

$$\begin{aligned} \hat{R}_n(f) &= \tfrac{1}{n}\sum_{i=1}^{n} L(y_i,\, f(\mathbf{x}_i)) \\ \hat{f} &= \arg\min_{f \in \mathcal{H}}\; \hat{R}_n(f) + \lambda\, \Omega(f) \end{aligned} \tag{1.2}$$

where (x_i, y_i) are the training examples, $\mathcal{H}$ is the hypothesis space, $\Omega(f)$ is a complexity penalty (regulariser) and $\lambda \geq 0$ controls its strength.

The second line of Equation (1.2) adds a *regulariser* $\Omega(f)$ that penalises complex models; this is *structural* or *regularised* risk minimisation, developed in the section on model generalisation and regularisation. The gap between the empirical risk on training data and the true risk on new data is the *generalisation gap*. A model that drives training error to zero while the true risk stays high has *overfitted*: it has memorised the training set rather than learned the underlying pattern. Controlling the generalisation gap is the central concern of machine learning.

E Worked Example 1.1 | Computing Empirical Risk

A model predicts house prices (in \$1000s) for four houses whose true prices are $y = (200, 250, 310, 180)$ as $\hat{y} = (210, 240, 300, 200)$.

- Errors $y - \hat{y}$: $(-10, 10, 10, -20)$.
- Squared losses: $(100, 100, 100, 400)$; empirical risk under squared loss $= (100+100+100+400)/4 = 175$.
- Absolute losses: $(10, 10, 10, 20)$; empirical risk under absolute loss $= 50/4 = 12.5$.

The single large error on the fourth house contributes 57% of the squared-loss risk but only 40% of the absolute-loss risk — squared loss penalises large errors disproportionately, which is why it is sensitive to outliers.

1.3.4 Inductive Bias and the No Free Lunch Theorem

Learning from a finite sample requires assumptions. Infinitely many functions agree with any finite training set, and they disagree on unseen inputs; the learner must prefer some over others. The set of assumptions a learner uses to generalise beyond the training data is its *inductive bias*. Linear regression assumes the target is approximately

linear in the features; k-nearest neighbours assumes nearby points have similar labels; decision trees prefer short, axis-aligned rules. The *No Free Lunch* theorems of Wolpert and Macready (1997) and Wolpert (1996) formalise the consequence: averaged over all possible problems, no learning algorithm outperforms any other. An algorithm is good only relative to a class of problems whose structure matches its inductive bias, which is why empirical comparison on the problem at hand is indispensable.

Table 1.4 Core terminology used throughout the chapter.

Term	Symbol	Meaning
Instance / example	x_i	One observation described by d feature values
Feature	x_j	A measurable input property of an instance
Label / target	y	The quantity to be predicted (real-valued or categorical)
Design matrix	$X \in \mathbb{R}^{n \times d}$	All feature vectors stacked as rows
Model / hypothesis	$f(x; \theta)$	Function mapping inputs to predictions
Parameters	θ, w, b	Values learned from training data
Hyperparameters	λ, k, η	Settings chosen before training, tuned on validation data
Loss function	$L(y, \hat{y})$	Penalty for a single prediction
Empirical risk	$\hat{R}_n(f)$	Average loss on the training set
Generalisation error	$R(f)$	Expected loss on new data from the same distribution
Inductive bias	—	Assumptions that allow generalisation beyond training data

Key Point | Key Points

- Learning means choosing f from a hypothesis space $\mathcal{H}$ to minimise empirical risk, usually plus a complexity penalty.
- Parameters are fitted on training data; hyperparameters are chosen on validation data.
- What matters is the generalisation error on unseen data, not the training error.
- Every learner has an inductive bias; no algorithm is best for all problems.

1.4 Types of Machine Learning

Learning problems differ chiefly in the kind of feedback the learner receives. When every training input comes with the correct output, the problem is *supervised*. When no outputs are given at all, it is *unsupervised*. Between these extremes lie *semi-supervised* learning, which combines a few labelled examples with many unlabelled ones, and

self-supervised learning, which manufactures labels from the structure of the data itself. *Reinforcement learning* differs again: the learner acts in an environment and receives only a scalar reward signal, often delayed. Figure 1.4 arranges these paradigms and their principal tasks, and Table 1.5 compares them. Each paradigm is introduced briefly here; the supervised algorithms are developed in detail later in this part of the chapter, and the other paradigms in dedicated sections later in the chapter.

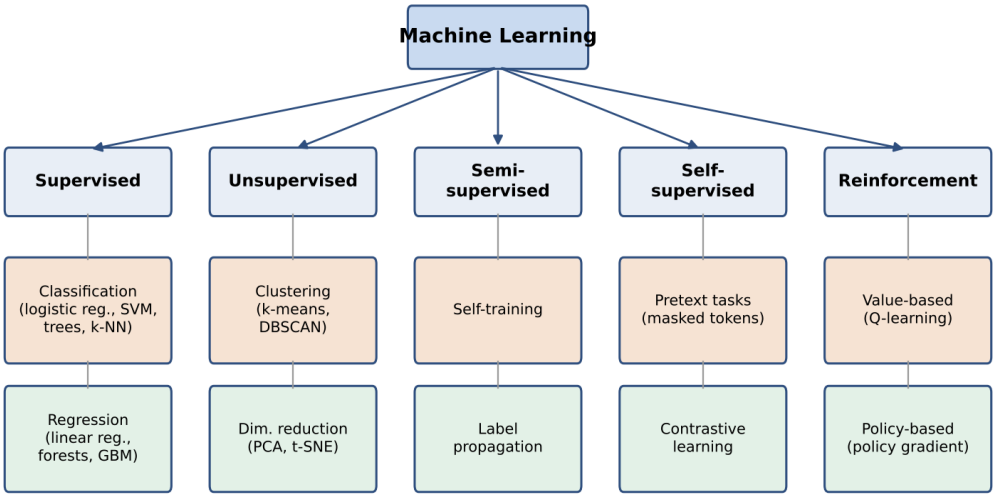


Figure 1.4 Taxonomy of machine learning paradigms with representative tasks and algorithms beneath each.

1.4.1 Supervised Learning

In supervised learning the training set is a collection of input–output pairs $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, and the goal is to learn a function f such that $f(x) \approx y$ for new inputs. The "supervisor" is the set of correct answers. When y is continuous — a price, a temperature, a length of stay — the task is *regression*; when y takes one of a finite set of values — spam or not, one of ten digits, benign or malignant — it is *classification*, which may be binary or multiclass. Variants include multilabel classification (each instance may carry several labels), ordinal regression and structured prediction (outputs are sequences or graphs). The supervised objective is the regularised empirical risk of Equation (1.2). Supervised learning is the most mature and widely deployed paradigm, but labelled data are often expensive: a radiologist's time, a lawyer's review, or months of waiting for a loan to default.

1.4.2 Unsupervised Learning

In unsupervised learning only inputs $\{x_1, \ldots, x_n\}$ are available, and the goal is to discover structure in them. *Clustering* partitions instances into groups whose members are similar — customer segments, document topics, cell types. *Dimensionality reduction* finds a low-dimensional representation that preserves most of the information, useful for visualisation and compression. *Density estimation* models $P(x)$ itself, and *anomaly detection* flags instances that are improbable under that model. *Association-rule mining* discovers co-occurrence patterns such as products bought together. A representative objective is the k-means criterion of Equation (1.3), which seeks K cluster centres minimising within-cluster squared distance. Because there is no ground truth, evaluating unsupervised results is harder and often relies on domain judgement.

$$J = \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \mu_k\|^2 \tag{1.3}$$

where C_k is the set of points assigned to cluster k and μ_k is its centroid.

1.4.3 Semi-Supervised Learning

Semi-supervised learning uses a small labelled set together with a much larger unlabelled set. It rests on assumptions linking the input distribution to the labels: the *smoothness* assumption (nearby points share labels), the *cluster* assumption (decision boundaries pass through low-density regions) and the *manifold* assumption (data lie near a low-dimensional surface). Simple methods include *self-training*, in which a model trained on labelled data assigns *pseudo-labels* to the unlabelled points it is most confident about and is retrained, and *label propagation*, which spreads labels through a similarity graph. Semi-supervised learning is attractive in medical imaging, speech and web classification, where raw data are plentiful but annotation is costly; it can, however, hurt performance when its assumptions are violated.

1.4.4 Self-Supervised Learning

Self-supervised learning creates a supervised task from unlabelled data by hiding part of each input and asking the model to predict it. A language model predicts the next word or a masked word from its context; an image model predicts a missing patch or learns that two augmented views of the same photograph should have similar representations (*contrastive learning*). The labels are free, so training can use billions of examples. The goal is rarely the pretext task itself but the *representations* it produces, which are then *fine-tuned* on a small labelled dataset for the real task. Self-supervised

pretraining underlies modern large language models and vision foundation models; it is technically a form of unsupervised learning but is usually treated separately because of its supervised-style training objective. Chapter 2 covers the deep architectures involved.

1.4.5 Reinforcement Learning

In reinforcement learning (RL) an *agent* interacts with an *environment* over discrete time steps, as shown in Figure 1.5. At each step t the agent observes a state s_t, takes an action a_t, and receives a reward r_{t+1} and a new state s_{t+1}. The agent's behaviour is a *policy* $\pi(a \mid s)$, and its goal is to maximise the expected *return* — the discounted sum of future rewards in Equation (1.4). Unlike supervised learning, the agent is never told the correct action; it must discover good actions by trial and error, balancing *exploration* of untried actions against *exploitation* of actions known to be good. Rewards may be delayed, so the agent must solve a *credit-assignment* problem. RL has produced landmark results in games (TD-Gammon, AlphaGo), robotics and resource allocation, and is used to fine-tune language models from human feedback.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{1.4}$$

where $\gamma \in [0, 1)$ is the discount factor that weights near-term rewards more heavily than distant ones.

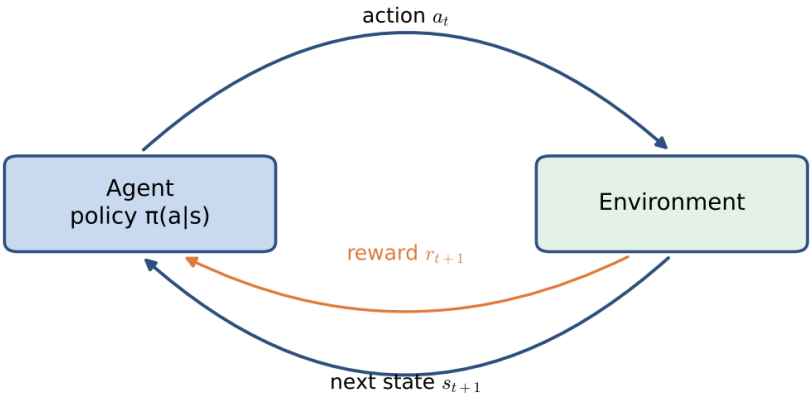


Figure 1.5 The agent–environment interaction loop in reinforcement learning: the agent acts, and the environment returns a reward and the next state.

Table 1.5 Comparison of the five machine learning paradigms.

Aspect	Supervised	Unsupervised	Semi-supervised	Self-supervised	Reinforcement
Training signal	Correct output for every input	None	Labels for a few inputs	Labels derived from the data	Scalar reward, often delayed
Typical goal	Predict y from x	Discover structure	Predict y using unlabelled data too	Learn reusable representations	Learn a policy maximising return
Example tasks	Spam filtering, price prediction	Segmentation, PCA	Medical image labelling	Language-model pretraining	Game playing, robotics
Evaluation	Held-out accuracy, error	Internal indices, expert review	Held-out labelled data	Downstream task performance	Cumulative reward
Main challenge	Cost of labels	No ground truth	Validity of assumptions	Designing pretext tasks; compute	Exploration, credit assignment

T Practice Tip | Choosing a Paradigm

- Do you have a reliable label for each example? Use **supervised** learning.
- Only a few labels but abundant raw data? Try **semi-supervised** learning, or **self-supervised** pretraining followed by supervised fine-tuning.
- No labels and the goal is exploration, segmentation or compression? Use **unsupervised** learning.
- Does the system take sequential actions whose consequences unfold over time? Consider **reinforcement** learning — but first check whether a simpler supervised formulation suffices.

1.5 Machine Learning Workflow and Lifecycle

Algorithms are only one ingredient of a successful machine learning system. Surveys of practitioners consistently report that most of a project's effort goes into framing the problem, obtaining and preparing data, and deploying and maintaining the resulting model; fitting the model itself is often the shortest phase. A disciplined *workflow* makes this effort repeatable and auditable. Several process models exist — the Cross-Industry Standard Process for Data Mining (CRISP-DM, published in 2000) is the best known — but they share the iterative structure shown in Figure 1.6.

1.5.1 Stages of the Lifecycle

Problem framing. A project begins with a business or scientific objective, which must be translated into a learning task with a measurable target. "Reduce customer churn"

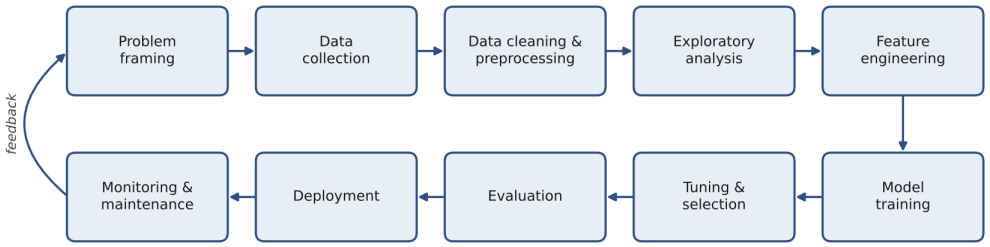


Figure 1.6 The machine learning lifecycle. The process is iterative: monitoring in production feeds back into problem definition and data collection.

might become "predict, for each active subscriber, the probability of cancelling within 30 days". Framing fixes the unit of prediction, the target definition, the time horizon, the evaluation metric and the baseline to beat. It should also establish whether machine learning is needed at all: a simple rule or a database query may solve the problem more cheaply.

Data collection and preparation. Relevant data are gathered, integrated, cleaned and transformed into a form a model can consume; these steps are the subject of the next three sections. **Exploratory data analysis** examines distributions, relationships and anomalies, and frequently sends the team back to collect more data or revise the problem framing. **Feature engineering** constructs informative inputs from raw fields.

Model training and selection. Candidate algorithms are trained on the training set, their hyperparameters tuned on validation data or by cross-validation, and the best configuration chosen. A simple *baseline* — predicting the majority class, the mean, or using a linear model — should always be fitted first, so that the value of more complex models can be judged. **Evaluation** then measures the chosen model once on an untouched test set, using metrics aligned with the original objective, and examines errors by subgroup to detect unfairness or blind spots.

Deployment and monitoring. The model is packaged and integrated into a product or decision process — as a batch scoring job, a real-time web service or an on-device component. In production the world keeps changing: customer behaviour shifts, sensors are recalibrated, upstream data pipelines change format. *Data drift* (a change in the distribution of inputs) and *concept drift* (a change in the relationship between inputs and target) degrade accuracy silently, so live performance and input statistics must be monitored and the model retrained when needed. The engineering discipline that automates this loop, known as MLOps, is discussed in the later section on deployment.

Table 1.6 Stages of the machine learning lifecycle, their outputs and common pitfalls.

Stage	Key output	Common pitfall
Problem framing	Target definition, metric, baseline	Optimising a proxy unrelated to the real goal
Data collection	Raw dataset with documentation	Sampling bias; unrepresentative data
Cleaning & preprocessing	Tidy, validated feature table	Leakage from using test data in transformations
Exploratory analysis	Insights, hypotheses, data-quality report	Skipping it and missing obvious errors
Feature engineering	Model-ready feature matrix	Features unavailable at prediction time
Training & selection	Tuned candidate models	Tuning on the test set
Evaluation	Unbiased performance estimate, error analysis	Single aggregate metric hiding subgroup failures
Deployment	Serving infrastructure	Training–serving skew in preprocessing
Monitoring	Alerts, retraining triggers	Silent degradation from drift

1.5.2 Iteration and Reproducibility

The lifecycle is not a waterfall. Error analysis after evaluation commonly reveals mislabelled examples, missing features or a poorly chosen metric, sending the project back several stages. To make iteration safe, every experiment should be reproducible: code under version control, data snapshots versioned, random seeds fixed, and all preprocessing encapsulated in a single *pipeline* object that is fitted on training data and applied unchanged to validation, test and production data. Listing 1.1 shows such a pipeline in scikit-learn; because the scaler and the classifier are fitted together, there is no opportunity for test-set statistics to leak into training.

Listing 1.1 A reproducible end-to-end scikit-learn pipeline with a held-out test set.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          stratify=y, random_state=42)

# Always fit a trivial baseline first
baseline = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
```

```

# Preprocessing and model are fitted together on training data only
model = Pipeline([("scale", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=1000))]).fit(X_tr, y_tr)

print("baseline accuracy:", accuracy_score(y_te, baseline.predict(X_te)))
print("model accuracy:   ", accuracy_score(y_te, model.predict(X_te)))
```

Case Study 1.1 | Predicting Hospital Readmissions

A hospital wishes to reduce 30-day readmissions. Framing: for each discharged patient, predict the probability of unplanned readmission within 30 days; the metric is recall at a fixed review capacity of 50 patients per week, because nurses can follow up only that many. Data: electronic health records, prior admissions and medication lists, joined on a patient identifier. EDA reveals that a "discharge disposition" field is filled in only after the readmission occurs — a leaking feature that must be removed. A gradient-boosted model beats the logistic-regression baseline; after deployment, monitoring detects drift when a new coding standard is introduced, triggering retraining. Each lifecycle stage contributed to the outcome, and the leak discovered during EDA would have produced a model that looked excellent offline and failed in practice.

1.6 Data Collection, Integration, and Preparation

A model can be no better than the data from which it learns. The phrase "garbage in, garbage out" is a cliché precisely because it is so often true: subtle problems in how data were collected — who was sampled, what was measured, how labels were assigned — propagate into every downstream prediction and cannot be fixed by a cleverer algorithm. This section discusses where data come from, how heterogeneous sources are combined, and how the combined data are organised for learning.

1.6.1 Sources of Data

Data for machine learning arrive from many sources, summarised in Table 1.7. *Internal operational data* — transaction logs, customer records, sensor streams — are usually the most relevant but are collected for operational rather than analytical purposes, so they are often incomplete or inconsistently coded. *Public and open datasets*, such as those in the UCI Machine Learning Repository, ImageNet or government open-data portals, are invaluable for research and benchmarking. *Web data* obtained through APIs or scraping must respect terms of service, copyright and privacy law. *Surveys and experiments* allow the analyst to control what is measured but are expensive. Finally, *syn-*

thetic data generated by simulation or by generative models can supplement scarce or sensitive data, though models trained on it may not transfer perfectly to reality.

Table 1.7 Common data sources and their characteristics.

Source	Examples	Strengths	Caveats
Internal databases	CRM, ERP, transaction logs	Highly relevant; large volume	Collected for operations; inconsistent coding
Sensors and IoT	Machine telemetry, wearables	High frequency; objective	Noise, calibration drift, gaps
Public datasets	UCI repository, ImageNet, open government data	Free; benchmarked	May not match target population
Web and APIs	Social media, product reviews	Rich, current	Legal/ethical constraints; noisy text
Surveys and experiments	Questionnaires, A/B tests	Controlled design	Costly; response bias
Synthetic data	Simulations, generative models	Unlimited; privacy-preserving	Reality gap

1.6.2 Sampling, Labelling and Bias

The fundamental assumption behind supervised learning is that the training data are drawn from the same distribution as the data on which the model will be used. Violations are common. *Selection bias* arises when the sampled population differs from the deployment population — for example, a credit model trained only on applicants who were previously approved never sees how rejected applicants would have behaved. *Measurement bias* arises when features or labels are recorded differently across groups. *Historical bias* arises when the data faithfully record a past that society no longer wishes to reproduce. Labels themselves are frequently produced by human annotators who disagree; measuring inter-annotator agreement, writing clear labelling guidelines and auditing a sample of labels are essential steps. Collecting *metadata* — when, where and how each record was obtained — makes these biases visible later.

1.6.3 Data Integration

Real projects seldom have a single tidy table. Customer attributes live in one system, transactions in another, and support tickets in a third. *Data integration* combines them into a unified view. Typical challenges include *entity resolution* (recognising that "J. Smith, 12 High St" and "John Smith, 12 High Street" are the same customer), *schema matching* (aligning fields with different names, units or encodings), *temporal alignment* (joining events recorded at different granularities) and handling *conflicting values* from different sources. Integration typically follows an extract–transform–load (ETL) or

extract–load–transform (ELT) process into a data warehouse or data lake. A particularly dangerous integration error is the *time-travel join*: attaching to a training record information that was only recorded after the moment of prediction, a form of data leakage.

D Definition 1.2 | Data Leakage

Data leakage occurs when information that would not be available at prediction time is used, directly or indirectly, during training or model selection. Leakage makes offline evaluation over-optimistic and the deployed model disappointing.

- **Target leakage:** a feature is a consequence or proxy of the target (e.g. "treatment administered" when predicting diagnosis).
- **Train–test contamination:** preprocessing statistics, feature selection or hyperparameter tuning use the test data.
- **Temporal leakage:** future information is joined to past records, or a random split mixes future and past in time-series data.

1.6.4 Organising Data for Learning

The output of collection and integration is usually a *tidy* table: one row per unit of prediction, one column per feature, and a target column. Each column should have a documented meaning, type, unit and permitted range — a *data dictionary*. Formal *data validation* checks, run automatically on every new batch, enforce these constraints (non-negative ages, valid category codes, no duplicate keys) and catch upstream changes before they corrupt a model. Datasheets for datasets (Gebru et al., 2021) provide a structured template for documenting motivation, composition, collection process and recommended uses. Finally, data volume should match model complexity: a rough heuristic for classical models is at least ten examples per feature per class, while deep networks typically require far more unless pretrained representations are available.

T Practice Tip | Practical Checklist for Data Collection

- Define the unit of prediction and the exact moment at which a prediction is made.
- Use only information available at that moment; record timestamps for every field.
- Compare the sampled population with the population the model will serve.
- Document provenance, licences and consent; remove or protect personal data.
- Write automated validation checks for schema, ranges and uniqueness.

1.7 Data Cleaning and Data Preprocessing

Raw data are rarely ready for modelling. Values are missing, mistyped, duplicated or implausible; categorical variables are stored as text; numerical variables have wildly different scales and skewed distributions. *Data cleaning* detects and corrects errors and inconsistencies; *data preprocessing* transforms clean data into the numeric form that learning algorithms require. The two overlap, and together they commonly consume the majority of a project's time. Feature scaling is an important preprocessing step treated separately in the section on feature scaling and normalisation.

1.7.1 Handling Missing Values

Missing values arise from non-response, sensor failure, data-entry omissions or fields that do not apply. How they should be treated depends on *why* they are missing. Rubin (1976) distinguished three mechanisms, listed in Table 1.8. Under *missing completely at random* (MCAR), missingness is unrelated to any variable, and deleting incomplete rows loses efficiency but introduces no bias. Under *missing at random* (MAR), missingness depends only on observed variables, so imputation conditioned on those variables is appropriate. Under *missing not at random* (MNAR), missingness depends on the unobserved value itself — high earners declining to report income — and no method can fully correct the bias without modelling the missingness mechanism.

Table 1.8 Missing-data mechanisms and appropriate treatments.

Mechanism	Definition	Example	Typical treatment
MCAR	Missingness independent of all data	Random sensor packet loss	Deletion or simple imputation
MAR	Depends only on observed variables	Older patients skip an online form	Model-based or multiple imputation
MNAR	Depends on the missing value itself	High earners withhold income	Model the mechanism; add missing indicators; sensitivity analysis

Practical strategies include *listwise deletion* (dropping rows with any missing value), which is simple but wasteful; *simple imputation* with the mean, median (robust to outliers) or mode; *k-nearest-neighbour imputation*, which fills a value from similar rows; *iterative* or *multiple imputation* (e.g. MICE), which models each incomplete variable as a function of the others; and adding a binary *missing indicator* feature so that the model can learn whether missingness is itself informative. Tree-based gradient-boosting libraries can also handle missing values natively by learning a default branch direction. Whatever the method, imputation parameters must be estimated on the training set only.

1.7.2 Outliers and Noise

An *outlier* is an observation that deviates markedly from the bulk of the data. Some outliers are errors (a recorded human height of 17 m), which should be corrected or removed; others are genuine and important (a fraudulent transaction), which should be kept. Two simple univariate detection rules are the *z-score* rule of Equation (1.5) and the *interquartile-range* (IQR) rule of Equation (1.6), which underlies the box plot. The IQR rule is more robust because quartiles, unlike the mean and standard deviation, are not themselves distorted by the outliers. Multivariate outliers — points unusual only in combination — require methods such as the Mahalanobis distance, isolation forests or local outlier factor, covered in the section on anomaly detection.

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad \text{flag if } |z_i| > 3 \tag{1.5}$$

where $\bar{x}$ and s are the sample mean and standard deviation of the feature.

$$\begin{aligned} \mathrm{IQR} &= Q_3 - Q_1 \\ \text{flag if } x_i < Q_1 - 1.5\,\mathrm{IQR} &\quad \text{or} \quad x_i > Q_3 + 1.5\,\mathrm{IQR} \end{aligned} \tag{1.6}$$

where Q_1 and Q_3 are the first and third quartiles (25th and 75th percentiles).

E Worked Example 1.2 | IQR Outlier Detection

Daily orders recorded over eleven days: 12, 14, 15, 15, 16, 17, 18, 19, 20, 22, 61.

- Sorted, $n = 11$. Using the median-of-halves method, $Q_1 = \text{median}(12, 14, 15, 15, 16) = 15$ and $Q_3 = \text{median}(18, 19, 20, 22, 61) = 20$.
- $\mathrm{IQR} = 20 - 15 = 5$; lower fence $= 15 - 7.5 = 7.5$; upper fence $= 20 + 7.5 = 27.5$.
- Only 61 lies outside $[7.5, 27.5]$, so it is flagged.
- For comparison, the mean is $229/11 \approx 20.8$ and $s \approx 13.6$, giving $z(61) \approx 2.95$ — below the threshold of 3 so the z-score rule misses it, because the outlier itself inflates s. This illustrates the robustness of the IQR rule.

Options for confirmed outliers include removal, correction, *capping* (winsorising) at a chosen percentile, transformation (e.g. a logarithm that compresses large values) or using models and losses that are robust to them, such as trees or the Huber loss. *Noise* — random error in features or labels — cannot be removed entirely; its effect is managed through regularisation and by collecting more data.

1.7.3 Encoding Categorical Variables

Most algorithms require numerical inputs, so categorical variables must be encoded. Figure 1.7 illustrates the main schemes. *Ordinal encoding* maps categories to integers

and is appropriate when the categories have a natural order (low < medium < high). *One-hot encoding* creates one binary column per category and is the standard choice for nominal variables with few levels; it avoids implying a spurious order but can create very wide matrices for high-cardinality variables such as postcodes. *Target (mean) encoding* replaces each category with the mean target value of training rows in that category; it is compact but leaks the target unless computed out-of-fold with smoothing. *Hashing* and learned *embeddings* are alternatives for very high cardinality.

Ordinal

colour	ordinal
red	0
green	1
blue	2
green	1
red	0

One-hot

colour	is_red	is_green	is_blue
red	1	0	0
green	0	1	0
blue	0	0	1
green	0	1	0
red	1	0	0

Target (mean)

colour	y	target enc.
red	1	0.50
green	1	0.50
blue	0	0.00
green	0	0.50
red	0	0.50

Figure 1.7 Three ways to encode the nominal feature 'colour': ordinal integers, one-hot binary columns, and target (mean) encoding using a binary target.

1.7.4 Other Preprocessing Steps

Further routine steps include removing exact and near-duplicate records; correcting inconsistent formats (dates, units, capitalisation, trailing spaces); converting types; *discretisation* of a continuous variable into bins; *transformations* such as the logarithm or Box–Cox transform to reduce skewness; and, for text and images, tokenisation, resizing and similar domain-specific operations. For imbalanced classification, *resampling* (undersampling the majority class, oversampling the minority, or generating synthetic minority examples with SMOTE) may be applied — but only to the training set, never to validation or test data. Listing 1.2 combines several of these steps into a single scikit-learn `ColumnTransformer`.

Listing 1.2 Imputation, outlier capping and encoding with pandas and scikit-learn.

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

df = pd.read_csv("customers.csv").drop_duplicates()
```

```
df["income"] = np.log1p(df["income"])                # reduce right skew

# Cap 'age' at the IQR fences (fences computed on the training data in practice)
q1, q3 = df["age"].quantile([0.25, 0.75])
df["age"] = df["age"].clip(q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1))

numeric = ["age", "income", "tenure"]
nominal = ["region", "plan"]
ordinal = ["satisfaction"]                           # low < medium < high

preprocess = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
                          StandardScaler()), numeric),
    ("nom", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), nominal),
    ("ord", OrdinalEncoder(categories=[["low", "medium", "high"]]), ordinal),
])
# preprocess.fit(X_train) ... preprocess.transform(X_test)
```

1.8 Exploratory Data Analysis (EDA)

Exploratory data analysis, a term introduced by John Tukey in his 1977 book of the same name, is the practice of examining data — largely visually — to understand its structure, spot anomalies, check assumptions and generate hypotheses before formal modelling. EDA is where analysts discover that a supposedly numeric column contains the string "N/A", that two features are near-duplicates, that a class is extremely rare, or that a feature correlates suspiciously perfectly with the target (a likely leak). Time spent here is repaid many times over.

1.8.1 Univariate Analysis

Univariate analysis examines one variable at a time. For numerical variables we compute summary statistics — count, mean, median, standard deviation, minimum, maximum and quartiles — and inspect the distribution with histograms, kernel density plots and box plots. Key questions are: Is the distribution symmetric or skewed? Unimodal or multimodal? Are there impossible or extreme values? The *skewness* of Equation (1.7) quantifies asymmetry; a strongly right-skewed variable such as income is often log-transformed. For categorical variables we examine frequency tables and bar charts, looking for rare levels, inconsistent spellings and class imbalance in the target.

$$g_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2\right)^{3/2}} \tag{1.7}$$

where $g_1 > 0$ indicates a long right tail, $g_1 < 0$ a long left tail and $g_1 \approx 0$ symmetry.

1.8.2 Bivariate and Multivariate Analysis

Bivariate analysis examines pairs of variables. Scatter plots reveal the form of relationships between two numerical variables — linear, curved, clustered or heteroscedastic. The *Pearson correlation coefficient* of Equation (1.8) measures the strength of linear association; the *Spearman* rank correlation captures monotonic but non-linear relationships and is robust to outliers. Grouped box plots compare a numerical variable across categories, and contingency tables with the chi-squared test assess association between categorical variables. For many variables, a *correlation heatmap* and a *scatter-plot matrix* give an overview, and dimensionality-reduction methods such as PCA or t-SNE (covered in the section on dimensionality reduction) project the data into two dimensions for visual inspection. Figure 1.8 shows typical EDA plots for a synthetic housing dataset.

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\,\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{1.8}$$

where $r \in [-1, 1]$; $r = \pm 1$ indicates a perfect linear relationship and $r = 0$ no linear relationship.

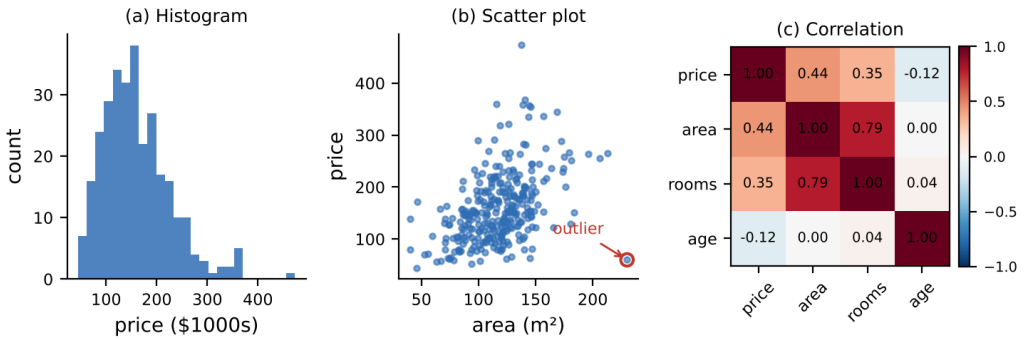


Figure 1.8 Typical EDA plots for a synthetic housing dataset: (a) a right-skewed price distribution, (b) price against floor area with a clear outlier, and (c) a correlation heatmap of four features.

E Worked Example 1.3 | Pearson Correlation by Hand

Five students record hours studied $x = (2, 4, 6, 8, 10)$ and exam scores $y = (55, 60, 70, 75, 90)$.

- Means: $\bar{x} = 6$, $\bar{y} = 70$. Deviations: $x - \bar{x} = (-4, -2, 0, 2, 4)$; $y - \bar{y} = (-15, -10, 0, 5, 20)$.
- $\Sigma(x - \bar{x})(y - \bar{y}) = 60 + 20 + 0 + 10 + 80 = 170$.
- $\Sigma(x - \bar{x})^2 = 16 + 4 + 0 + 4 + 16 = 40$; $\Sigma(y - \bar{y})^2 = 225 + 100 + 0 + 25 + 400 = 750$.
- $r = 170 \,/\, (\sqrt{40} \times \sqrt{750}) = 170 \,/\, (6.325 \times 27.386) = 170 \,/\, 173.2 \approx 0.98$.

There is a very strong positive linear association — though, as always, correlation alone does not establish that studying causes higher scores.

1.8.3 Checking Data Quality and Assumptions

EDA also serves as a quality audit and a check on modelling assumptions. Tabulating the fraction of missing values per column, and visualising missingness patterns across rows, shows whether data are plausibly MCAR. Plotting the target over time reveals trends, seasonality and drift that argue for a time-based split. Comparing feature distributions between training and test sets detects distribution shift. Checking pairwise correlations among features identifies *multicollinearity*, which destabilises linear-model coefficients. Most importantly, any feature that predicts the target almost perfectly should be treated with suspicion until its provenance is understood. Listing 1.3 gathers the most common EDA commands in pandas.

Listing 1.3 A compact EDA routine with pandas and Matplotlib.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("housing.csv")
print(df.shape)
print(df.dtypes)
print(df.describe(include="all").T)              # summary statistics
print(df.isna().mean().sort_values(ascending=False))  # missing fraction
print(df.select_dtypes("number").skew())          # skewness per column
print(df["neighbourhood"].value_counts().head(10))

num = df.select_dtypes("number")
corr = num.corr(method="spearman")
print(corr["price"].sort_values(ascending=False))  # check for leaks

num.hist(bins=30, figsize=(10, 6))
pd.plotting.scatter_matrix(num[["price", "area", "rooms", "age"]], figsize=(8, 8))
df.boxplot(column="price", by="neighbourhood", rot=90)
plt.tight_layout(); plt.show()
```

> **Note | Anscombe's Quartet**
>
> In 1973 the statistician Francis Anscombe constructed four small datasets with almost identical means, variances, correlations ($r \approx 0.816$) and fitted regression lines, yet completely different shapes when plotted: one linear, one curved, one linear with a single outlier, and one with all points but one at the same x value. The lesson is that summary statistics can conceal structure — always plot the data.

1.9 Feature Engineering and Feature Selection

A *feature* is a measurable property used as model input, and the choice of features frequently determines success more than the choice of algorithm. *Feature engineering* is the creative process of constructing new, more informative features from raw data using domain knowledge; *feature selection* is the complementary process of discarding features that are irrelevant or redundant. Both aim at the same end: a representation in which the relationship between inputs and target is as simple as possible for the chosen learner to capture. Deep networks (Chapter 2) learn many representations automatically, but for tabular data hand-crafted features remain decisive.

1.9.1 Feature Construction

Common construction techniques include: *mathematical transformations* (logarithm, square root, reciprocal) that linearise relationships or reduce skew; *interaction features* such as the product of two variables, which let a linear model capture effects that depend on combinations; *polynomial features* (x^2, x^3) for curved relationships; *ratios* that encode domain meaning (debt-to-income, price per square metre, body-mass index = weight / height^2); *aggregations* over related records (a customer's number of purchases in the last 30 days, average basket value); *date and time features* (hour of day, day of week, holiday flag, time since last event); *binning* a continuous variable into intervals; and *text features* such as word counts or TF-IDF weights. Cyclical quantities such as hour of day should be encoded with a sine–cosine pair so that 23:00 and 00:00 are close. Figure 1.9 shows how a single engineered feature can make a non-linear problem linearly separable.

1.9.2 Why Select Features?

Adding features is not free. Irrelevant features add noise that the model may fit; redundant features inflate variance and make coefficients unstable; every feature must

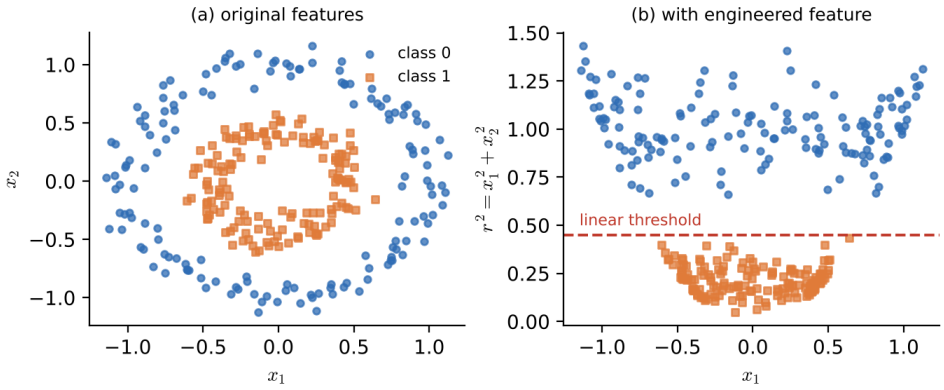


Figure 1.9 Feature engineering can turn a non-linear problem into a linear one: (a) two concentric classes cannot be separated by a line in (x_1, x_2); (b) adding the feature $r^2 = x_1^2 + x_2^2$ makes them separable by a threshold.

be collected, validated and served in production. Moreover, as the number of dimensions grows, data become exponentially sparse — the *curse of dimensionality*: to keep the same density of samples, the number of examples must grow exponentially with d, and distances between points become nearly uniform, which particularly harms distance-based methods such as k-NN. Feature selection mitigates these problems, reduces training time and improves interpretability. Methods fall into three families, compared in Table 1.9.

1.9.3 Filter, Wrapper and Embedded Methods

Filter methods score each feature by a statistic computed independently of any model and keep the top-ranked ones. Examples include variance thresholds (remove near-constant features), correlation with the target, the chi-squared test for categorical features, the ANOVA F-test, and *mutual information*. Mutual information, defined in Equation (1.9), measures how much knowing a feature reduces uncertainty about the target and captures non-linear dependence. Filters are fast and model-agnostic but ignore interactions between features.

$$I(X;Y) = \sum_{x}\sum_{y} p(x,y)\,\log \frac{p(x,y)}{p(x)\,p(y)} \tag{1.9}$$

where $p(x, y)$ is the joint distribution of feature X and target Y and $p(x)$, $p(y)$ are the marginals; $I(X; Y) = 0$ if and only if X and Y are independent.

Wrapper methods search over subsets of features, training a model on each and evaluating it on validation data. *Forward selection*, given in Algorithm 1.1, starts from the empty set and greedily adds the feature that most improves validation performance;

backward elimination starts from all features and removes the least useful; *recursive feature elimination* (RFE) repeatedly fits a model and drops the features with the smallest importance. Wrappers account for interactions and for the specific learner, but are computationally expensive and can overfit the validation set when many subsets are compared.

Algorithm 1.1 Greedy Forward Feature Selection

Input: Feature set $F = \{f_1, \ldots, f_d\}$; learner A; validation score function S; maximum subset size m
Output: Selected feature subset S^*

$S^* \leftarrow \emptyset$; best $\leftarrow -\infty$
while $|S^*| < m$ **do**
  for each feature $f \in F \setminus S^*$ **do**
    train A on features $S^* \cup \{f\}$ using the training set
    score$(f) \leftarrow S(A)$ on the validation set (**or** by cross-validation)
  $f^* \leftarrow \operatorname{argmax}_f$ score(f)
  if score$(f^*) \le$ best **then**
    break ▷ *no further improvement*
  $S^* \leftarrow S^* \cup \{f^*\}$; best $\leftarrow$ score(f^*)
return S^*

Embedded methods perform selection as part of model training. L1 (lasso) regularisation, discussed in the section on regularisation, drives the coefficients of unhelpful features exactly to zero. Decision trees and their ensembles compute *feature importances* from the impurity reduction each feature provides. Embedded methods offer a good balance of cost and accuracy. Note also the distinction between *feature selection*, which keeps a subset of the original features, and *feature extraction*, which creates new features as combinations of the originals (e.g. principal component analysis, covered in the section on dimensionality reduction).

Table 1.9 Comparison of feature-selection families.

Family	Examples	Cost	Captures interactions?	Risk
Filter	Variance threshold, correlation, χ^2, ANOVA F, mutual information	Low	No	Discards features useful only in combination
Wrapper	Forward/backward selection, RFE, genetic search	High	Yes	Overfitting the validation set
Embedded	Lasso, elastic net, tree importances	Moderate	Partly	Importances biased toward high-cardinality features

E Worked Example 1.4 | Mutual Information of a Binary Feature

In 100 e-mails, 40 are spam ($Y = 1$). A binary feature X indicates whether the word "free" appears. Counts: $X = 1$ & spam: 30; $X = 1$ & ham: 10; $X = 0$ & spam: 10; $X = 0$ & ham: 50.

- Marginals: $p(X{=}1) = 0.4$, $p(X{=}0) = 0.6$; $p(Y{=}1) = 0.4$, $p(Y{=}0) = 0.6$.
- Terms $p(x,y)\,\log_2[p(x,y)/(p(x)p(y))]$:
- $(1,1)$: $0.30 \log_2(0.30/0.16) = 0.30 \times 0.907 = 0.272$
- $(1,0)$: $0.10 \log_2(0.10/0.24) = 0.10 \times (-1.263) = -0.126$
- $(0,1)$: $0.10 \log_2(0.10/0.24) = -0.126$
- $(0,0)$: $0.50 \log_2(0.50/0.36) = 0.50 \times 0.474 = 0.237$
- $I(X;Y) \approx 0.272 - 0.126 - 0.126 + 0.237 = 0.257$ bits, out of a maximum $H(Y) \approx 0.971$ bits. The word "free" is moderately informative about spam.

Listing 1.4 Filter, wrapper and embedded feature selection in scikit-learn.

```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, mutual_info_classif, RFE,
↪  SelectFromModel
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
X = StandardScaler().fit_transform(X)   # in practice: fit on training data only

# Filter: top 10 features by mutual information
filt = SelectKBest(mutual_info_classif, k=10).fit(X, y)

# Wrapper: recursive feature elimination with a linear model
rfe = RFE(LogisticRegression(max_iter=1000), n_features_to_select=10).fit(X, y)

# Embedded: L1-regularised logistic regression zeroes out weak features
l1 = LogisticRegression(penalty="l1", solver="liblinear", C=0.1)
emb = SelectFromModel(l1).fit(X, y)

for name, sel in [("filter", filt), ("wrapper", rfe), ("embedded", emb)]:
    print(name, sel.get_support().nonzero()[0])
```

1.10 Feature Scaling and Normalization

Features measured in different units often have very different ranges: annual income may vary from 10,000 to 500,000 while the number of dependants varies from 0 to 6. Many algorithms are sensitive to such disparities. *Feature scaling* transforms numerical features onto comparable scales. (The words *normalisation* and *standardisation* are used inconsistently in the literature; here *standardisation* means z-scoring and *normalisation* means rescaling to a fixed range or unit norm.)

1.10.1 Why Scaling Matters

Scaling matters for three reasons. First, **distance-based methods** — k-NN, k-means, SVMs with RBF kernels — compute Euclidean distances in which a large-range feature dominates; income differences of thousands would swamp differences in the number of dependants. Second, **gradient-based optimisation** converges faster when features have similar scales, because the loss surface is closer to spherical than to an elongated valley in which gradient descent zig-zags. Third, **regularisation** penalties such as $\lambda\|w\|^2$ treat all coefficients alike, which is only fair when features are on comparable scales. By contrast, tree-based models split on one feature at a time using thresholds and are invariant to monotonic transformations, so they do not require scaling.

1.10.2 Scaling Methods

The main methods are given in Equation (1.10) and compared in Table 1.10. *Min–max normalisation* maps each feature linearly to $[0, 1]$; it preserves the shape of the distribution but is sensitive to outliers, since a single extreme value compresses all others. *Standardisation* subtracts the mean and divides by the standard deviation, giving zero mean and unit variance; it is the default for most linear models, SVMs and neural networks. *Robust scaling* uses the median and interquartile range and is therefore resistant to outliers. *Max-abs scaling* divides by the maximum absolute value and preserves sparsity. *Unit-norm normalisation* rescales each *sample* (row) to unit length, which is common for text vectors compared by cosine similarity. Non-linear transformations such as the logarithm, Box–Cox, Yeo–Johnson or quantile transforms change the shape of a distribution, for example making it closer to Gaussian.

$$
\begin{aligned}
x'_{\text{minmax}} &= \frac{x - x_{\min}}{x_{\max} - x_{\min}} \\
x'_{\text{std}} &= \frac{x - \mu}{\sigma} \\
x'_{\text{robust}} &= \frac{x - \operatorname{median}(x)}{Q_3 - Q_1} \\
\mathbf{x}'_{\text{unit}} &= \frac{\mathbf{x}}{\|\mathbf{x}\|_2}
\end{aligned}
\tag{1.10}
$$

where μ and σ are the training-set mean and standard deviation of the feature, $x_{\min}$ and $x_{\max}$ its training-set minimum and maximum, and Q_1, Q_3 its quartiles.

Figure 1.10 illustrates the methods on a dataset with a single outlier in x_1. After min–max scaling, the outlier sits at 1 and the remaining eighty points occupy less than half of the unit interval; standardisation spreads them more but the outlier still inflates σ; robust scaling, which ignores the tails, keeps the bulk of the data on a comfortable scale.

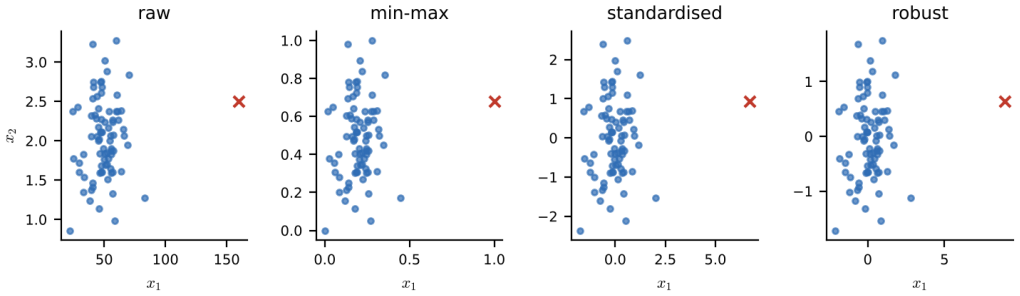


Figure 1.10 Effect of scaling on a two-feature dataset containing one outlier: min–max scaling squeezes the bulk of the data into a narrow band, standardisation is less affected, and robust scaling keeps the bulk well spread.

Table 1.10 Feature scaling methods and when to use them.

Method	Resulting range	Outlier-robust?	Typical use
Min–max	$[0, 1]$	No	Bounded inputs (pixel intensities), neural networks
Standardisation (z-score)	Mean 0, s.d. 1, unbounded	Partly	Linear models, SVM, PCA, k-NN
Robust (median/IQR)	Median 0, unbounded	Yes	Data with outliers
Max-abs	$[-1, 1]$	No	Sparse data (preserves zeros)
Unit norm (per sample)	$\|x\| = 1$	—	Text vectors, cosine similarity
Log / Box–Cox / quantile	Changes shape	Yes	Heavily skewed features

E Worked Example 1.5 | Min–Max and Standard Scaling

Training values of a feature: $x = (20, 30, 40, 50, 60)$.

- Min–max: $x_{min} = 20$, $x_{max} = 60$. For $x = 50$: $(50 - 20)/(60 - 20) = 0.75$.
- Standardisation: $\mu = 40$; $\sigma = \sqrt{[(400 + 100 + 0 + 100 + 400)/5]} = \sqrt{200} \approx 14.14$. For $x = 50$: $(50 - 40)/14.14 \approx 0.71$.
- A **test** value $x = 70$ must be transformed with the **training** statistics: min–max gives $(70 - 20)/40 = 1.25$ (outside $[0, 1]$, which is acceptable) and the z-score is $(70 - 40)/14.14 \approx 2.12$.

N Note | Fit on Training Data Only

Scaling parameters (μ, σ, min, max, quantiles) are themselves estimated from data and must be computed from the training set alone, then applied unchanged to validation, test and production data. Computing them on the full dataset leaks information about the test distribution into training. Wrapping the scaler and the model in a single pipeline, as in Listing 1.1, enforces this automatically, including within each fold of cross-validation.

1.11 Training, Validation, and Testing Datasets

A model's performance on the data it was trained on is an optimistically biased estimate of its performance on new data, because the parameters were chosen precisely to fit those examples. To obtain honest estimates we must evaluate on data the model has not seen. Standard practice divides the available labelled data into three disjoint subsets with distinct roles, as shown in Figure 1.11.

1.11.1 Roles of the Three Sets

The **training set** is used to fit the model's parameters. The **validation set** (also called the development or hold-out set) is used to compare models and tune hyperparameters — the depth of a tree, the regularisation strength, the number of neighbours — and for early stopping. Because many configurations are compared on it, the best validation score is itself optimistically biased. The **test set** is therefore kept completely untouched until the very end and used exactly once to estimate the generalisation performance of the final, chosen model. If the test set is consulted repeatedly and decisions are made on its basis, it silently becomes a second validation set and its estimate is no longer unbiased.



Figure 1.11 A typical hold-out partition: the model is fitted on the training set, tuned on the validation set and assessed once on the test set; after tuning, the model may be refitted on training plus validation data.

1.11.2 Splitting Strategies

Common split ratios are 60/20/20 or 70/15/15 for moderate datasets, and as little as 98/1/1 for datasets with millions of examples, where 1% still contains tens of thousands of instances. The split must respect the structure of the data. A *random* split assumes examples are independent and identically distributed (i.i.d.). A *stratified* split preserves the class proportions in each subset and is essential for imbalanced classification. A *group* split keeps all records from the same entity — patient, customer, device —

in a single subset, so that the model cannot exploit entity-specific idiosyncrasies seen during training. A *temporal* split trains on the past and tests on the future, mimicking deployment for time-series and any data subject to drift. Table 1.11 summarises when each is required. When data are scarce, a single validation split is noisy; *k-fold cross-validation*, discussed in a later section, rotates the validation role across folds to use every example for both training and validation.

Table 1.11 Data-splitting strategies.

Strategy	How it works	When to use
Random hold-out	Shuffle and cut into proportions	Large i.i.d. datasets
Stratified	Preserve class proportions in each subset	Classification, especially imbalanced
Group	All rows of an entity go to one subset	Repeated measurements per patient, user or device
Temporal	Train on earlier period, test on later	Time series; drifting data; forecasting
k-fold cross-validation	Rotate validation over k folds	Small or moderate datasets; model selection
Nested cross-validation	Inner loop tunes, outer loop evaluates	Unbiased estimate when tuning on small data

1.11.3 How Large Should the Test Set Be?

The test set must be large enough for its performance estimate to be precise. For an accuracy estimate $\hat{p}$ on n_{test} independent examples, the standard error is given by Equation (1.11), and an approximate 95% confidence interval is $\hat{p} \pm 1.96 \cdot \text{SE}$. The worked example below shows that a few hundred test examples give an interval of several percentage points — too wide to distinguish models whose accuracies differ by one point.

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}\,(1-\hat{p})}{n_{\text{test}}}} \tag{1.11}$$

where $\hat{p}$ is the observed test accuracy and n_{test} the number of test examples.

E Worked Example 1.6 | Precision of a Test-Set Estimate

A classifier achieves $\hat{p} = 0.90$ accuracy on a test set.

- With $n_{\text{test}} = 200$: $\text{SE} = \sqrt{(0.9 \times 0.1 \,/\, 200)} = \sqrt{0.00045} \approx 0.0212$; 95% CI $\approx 0.90 \pm 0.042 = [0.858, 0.942]$.
- With $n_{\text{test}} = 5000$: $\text{SE} = \sqrt{(0.09/5000)} \approx 0.0042$; 95% CI $\approx 0.90 \pm 0.008 = [0.892, 0.908]$.

Increasing the test set 25-fold shrinks the interval 5-fold ($\sqrt{25} = 5$).

Listing 1.5 Stratified, group and temporal splits.

```python
from sklearn.model_selection import train_test_split, GroupShuffleSplit,
↪  TimeSeriesSplit

# Stratified 70/15/15 split
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.15,
                                                stratify=y, random_state=0)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.15/0.85,
                                                  stratify=y_tmp, random_state=0)

# Group split: no patient appears in both training and test data
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=0)
train_idx, test_idx = next(gss.split(X, y, groups=patient_id))

# Temporal split: each fold trains on the past and validates on the future
for tr_idx, va_idx in TimeSeriesSplit(n_splits=5).split(X):
    print(f"train up to {tr_idx[-1]}, validate {va_idx[0]}-{va_idx[-1]}")
```

Key Point | Key Points

- Train on the training set, tune on the validation set, report on the test set — once.
- Split before any data-dependent preprocessing, and fit preprocessing on training data only.
- Match the split to the data's structure: stratify classes, group entities, respect time.

1.12 Bias, Variance, and the Bias–Variance Trade-off

Why do simple models sometimes outperform complex ones, and vice versa? The *bias–variance decomposition* answers this question precisely for squared-error regression and provides intuition that carries over to classification. It decomposes a model's expected error on a new point into three parts: an irreducible noise term, a *bias* term measuring systematic error, and a *variance* term measuring sensitivity to the particular training sample.

1.12.1 Intuition

Imagine repeating the whole experiment many times: each time we draw a fresh training set of size n from the same distribution, fit the model, and record its prediction at a fixed query point x. The predictions form a distribution. *Bias* is the distance between the average of these predictions and the true value: a model that is too rigid —

a straight line fitted to a curve — is consistently wrong in the same direction, however much data it sees. *Variance* is the spread of the predictions around their own average: a model that is too flexible — a degree-15 polynomial — changes drastically from one training set to the next because it fits the noise. The classic dartboard analogy is shown in Figure 1.12: low bias means the throws centre on the bull's-eye; low variance means they cluster tightly.

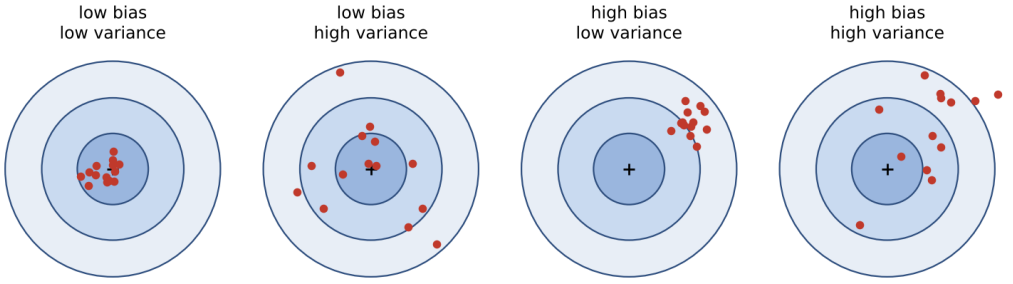


Figure 1.12 Bias and variance as dart throws: each point is the prediction of a model trained on a different sample; the centre is the true value.

1.12.2 Mathematical Decomposition

Assume the data are generated as $y = f(x) + \varepsilon$, where f is the true function and ε is noise with mean zero and variance σ^2. Let $\hat{f}_{\mathcal{D}}(x)$ denote the prediction of a model fitted to training set $\mathcal{D}$, and let the expectation $\mathbb{E}_{\mathcal{D}}$ be taken over random training sets. Then the expected squared error at x decomposes as in Equation (1.12).

$$\mathbb{E}_{\mathcal{D},\varepsilon}\left[\left(y - \hat{f}_{\mathcal{D}}(\mathbf{x})\right)^2\right] = \text{Bias}^2 + \text{Variance} + \sigma^2$$

$$\text{Bias}^2 = \left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x})\right)^2 \tag{1.12}$$

$$\text{Variance} = \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})]\right)^2\right]$$

where f is the true regression function, $\hat{f}_{\mathcal{D}}$ the model trained on dataset $\mathcal{D}$, and σ^2 the variance of the irreducible noise ε.

The derivation adds and subtracts $\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]$ inside the square and uses the facts that ε has mean zero and is independent of $\mathcal{D}$, so all cross-terms vanish in expectation. The noise term σ^2 is a floor that no model can beat. The remaining error is shared between bias^2 and variance, and model complexity moves error from one to the other: increasing complexity typically lowers bias and raises variance. Total error is therefore U-shaped in complexity, as Figure 1.13 shows, and the best model sits at the bottom of the U — the *bias–variance trade-off*.

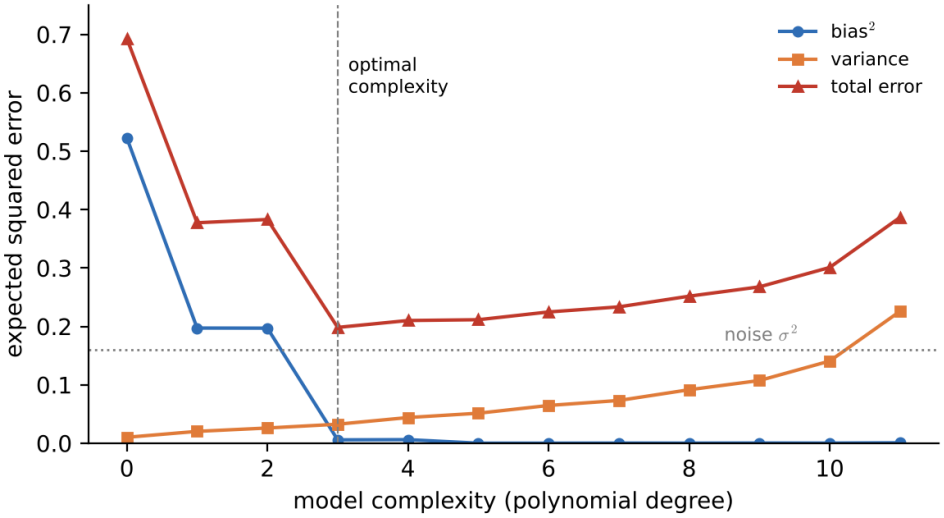


Figure 1.13 Bias^2, variance and total expected error as a function of polynomial degree, estimated by fitting 400 noisy training sets of 15 points from $y = \sin(2\pi x) + \varepsilon$ with $\sigma = 0.4$.

E Worked Example 1.7 | Estimating Bias and Variance

The true value at a query point is $f(x) = 10$ and the noise variance is $\sigma^2 = 1$. Five models trained on five independent samples predict $\hat{f} = (12, 13, 11, 12, 12)$ — a rigid model — while a flexible model predicts $(7, 14, 9, 12, 8)$.

- Rigid: mean $= 12$; bias$^2 = (12 - 10)^2 = 4$; variance $= [0 + 1 + 1 + 0 + 0]/5 = 0.4$; expected error $= 4 + 0.4 + 1 = 5.4$.
- Flexible: mean $= 10$; bias$^2 = 0$; variance $= [9 + 16 + 1 + 4 + 4]/5 = 6.8$; expected error $= 0 + 6.8 + 1 = 7.8$.

Despite zero bias, the flexible model has the larger expected error, because its variance dominates.

1.12.3 Managing the Trade-off

Different levers act on different components, as summarised in Table 1.12. Bias is reduced by using a more expressive model, adding informative features or weakening regularisation. Variance is reduced by collecting more training data, simplifying the model, strengthening regularisation, selecting fewer features, or *averaging* many models — the principle behind bagging and random forests. Boosting, conversely, primarily reduces bias by sequentially combining simple models. In modern heavily overparameterised models such as large neural networks, test error can fall again after the interpolation threshold, a phenomenon called *double descent* (Belkin et al., 2019); the classical U-curve nevertheless remains an excellent guide for the methods in this chapter.

Table 1.12 Actions that reduce bias or variance.

Action	Effect on bias	Effect on variance
Increase model complexity (deeper tree, higher degree)	Decreases	Increases
Add informative features	Decreases	May increase
Collect more training data	Unchanged	Decreases
Stronger regularisation (larger λ)	Increases	Decreases
Bagging / random forests	Roughly unchanged	Decreases
Boosting	Decreases	May increase if overrun
Increase k in k-NN	Increases	Decreases

1.13 Overfitting and Underfitting

The bias–variance trade-off has two failure modes with everyday names. A model *underfits* when it is too simple to capture the underlying pattern: it performs poorly on the training data and equally poorly on new data. A model *overfits* when it is so flexible that it fits the noise and accidental regularities of its particular training sample: it performs very well on the training data but poorly on new data. A *well-fitted* model captures the signal and ignores the noise. Figure 1.14 illustrates the three regimes with polynomial regression.

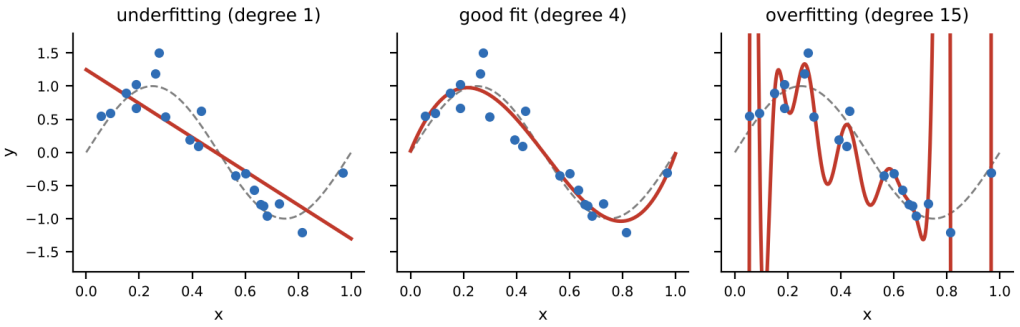


Figure 1.14 Underfitting, a good fit and overfitting: polynomials of degree 1, 4 and 15 fitted to the same 20 noisy samples of a sine curve (dashed).

1.13.1 Diagnosing the Problem

The key diagnostic is to compare training error with validation error. Underfitting shows as *high training error* with validation error close to it — the model cannot even fit the data it has seen. Overfitting shows as *low training error* with *much higher val-*

idation error — a large generalisation gap. Two plots make this visible. A *validation curve* (or complexity curve) plots training and validation error against a complexity hyperparameter such as tree depth or polynomial degree; the validation curve is U-shaped and its minimum marks the best complexity. A *learning curve* plots both errors against training-set size, as in Figure 1.15. For a high-bias model the two curves converge quickly to a high plateau, so more data will not help. For a high-variance model a wide gap persists, which shrinks as data are added, so collecting more data is likely to help. Table 1.13 summarises the diagnosis and remedies.

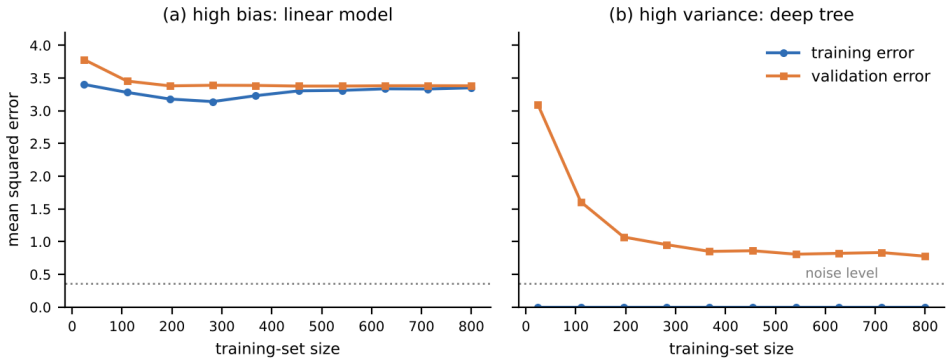


Figure 1.15 Learning curves for (a) a high-bias model (linear regression on a non-linear target) and (b) a high-variance model (unpruned decision tree): the gap between training and validation error reveals overfitting.

Table 1.13 Diagnosing and remedying underfitting and overfitting.

Symptom	Diagnosis	Remedies
High training error; validation error similar	Underfitting (high bias)	More expressive model; add or engineer features; reduce regularisation; train longer
Low training error; much higher validation error	Overfitting (high variance)	More data; simpler model; stronger regularisation; feature selection; early stopping; ensembles
Both errors low and close	Good fit	Confirm on the test set; monitor in production
Validation error better than training error	Possible leakage, small or unrepresentative validation set, or dropout/augmentation effect	Check the split and pipeline

1.13.2 Causes of Overfitting

Overfitting becomes likely when the number of effective parameters is large relative to the number of training examples; when features are numerous and many are irrelevant; when the labels are noisy; when training continues too long for iteratively optimised models; and — a subtle but common cause — when many models or hyperparameter settings are compared on the same validation set, so that the selection procedure itself overfits. A model that has memorised its training data will often reveal itself by extreme parameter values, such as very large polynomial coefficients, which motivates the regularisation methods of the next section.

E Worked Example 1.8 | Reading Training and Validation Errors

Three models are trained to predict delivery times (minutes). Root mean squared errors are:

- Model A (linear): training 14.2, validation 14.9. Gap 0.7, but both high relative to a domain target of 8 → **underfitting**; add features such as distance × traffic interaction.
- Model B (depth-30 tree): training 0.4, validation 13.8. Gap 13.4 → **overfitting**; limit depth or use a random forest.
- Model C (gradient boosting with early stopping): training 7.1, validation 8.0. Small gap and near target → **good fit**.

K Key Point | Key Points

- Underfitting: high training error. Overfitting: low training error, high validation error.
- Learning curves tell you whether more data will help; validation curves tell you how much complexity to use.
- Repeatedly selecting on the same validation set can itself cause overfitting.

1.14 Model Generalization and Regularization

Generalisation is the ability of a model to perform well on previously unseen data drawn from the same distribution as its training data. It is the true objective of learning; low training error is only a means to that end. This section looks briefly at why generalisation is possible at all, and then at *regularisation* — the family of techniques that deliberately constrain a model to improve its generalisation, usually at the price of a slightly higher training error.

1.14.1 Why Generalisation Is Possible

Statistical learning theory bounds the gap between the true risk $\mathrm{R(f)}$ and the empirical risk $\hat{\mathrm{R}}_\mathrm{n}(\mathrm{f})$ in terms of the sample size n and the *capacity* of the hypothesis space $\mathcal{H}$. Capacity can be measured by the *Vapnik–Chervonenkis* (*VC*) *dimension* — the largest number of points that $\mathcal{H}$ can label in all possible ways (linear classifiers in $\mathbb{R}^\mathrm{d}$ have VC dimension $\mathrm{d}+1$). A representative bound, holding with probability at least $1-\delta$ for every $\mathrm{f}\in\mathcal{H}$, is shown in Equation (1.13). Its message is qualitative but important: the generalisation gap grows with capacity h and shrinks with the number of examples roughly as $\sqrt{(\mathrm{h/n})}$. Simple hypotheses fitted to plenty of data generalise; complex hypotheses fitted to little data may not. This is a formal version of *Occam's razor*: among hypotheses that explain the data equally well, prefer the simplest.

$$R(f) \le \hat{R}_n(f) + \sqrt{\frac{h\,(\log(2n/h)+1) + \log(4/\delta)}{n}} \tag{1.13}$$

where h is the VC dimension of $\mathcal{H}$, n the number of training examples and $1-\delta$ the confidence level (Vapnik, 1998).

1.14.2 L2 (Ridge) and L1 (Lasso) Regularisation

The most common regularisers add a penalty on the size of the weights to the training loss, giving the regularised objective in Equation (1.14). *L2 regularisation* (ridge regression, Hoerl and Kennard, 1970; *weight decay* in neural networks) penalises the sum of squared weights. It shrinks all weights smoothly towards zero, spreading influence across correlated features, and has a closed-form solution for linear regression. *L1 regularisation* (the lasso, Tibshirani, 1996) penalises the sum of absolute weights. Because the absolute value has a corner at zero, L1 drives many weights exactly to zero, performing automatic feature selection and yielding sparse, interpretable models. *Elastic net* (Zou and Hastie, 2005) combines both, retaining sparsity while handling groups of correlated features more gracefully than the lasso. The intercept w_0 is conventionally not penalised, and features should be standardised first so that the penalty treats them equally.

$$\begin{aligned} J(\mathbf{w}) &= \tfrac{1}{n}\sum_{i=1}^{n} L(y_i,\, f(\mathbf{x}_i;\mathbf{w})) + \lambda\,\Omega(\mathbf{w}) \\ \Omega_{L2} &= \|\mathbf{w}\|_2^2 = \sum_j w_j^2, \qquad \Omega_{L1} = \|\mathbf{w}\|_1 = \sum_j |w_j| \\ \Omega_{EN} &= \alpha\|\mathbf{w}\|_1 + \tfrac{1-\alpha}{2}\|\mathbf{w}\|_2^2 \end{aligned} \tag{1.14}$$

where $\lambda \geq 0$ is the regularisation strength and $\alpha \in [0, 1]$ the elastic-net mixing parameter.

The geometric reason for L1's sparsity is shown in Figure 1.16. Minimising the loss subject to a constraint on the penalty is equivalent to the penalised form. The L2 constraint region is a disc, and the elliptical contours of the loss typically touch it at a point where both weights are non-zero. The L1 region is a diamond whose corners lie on the axes, and the contours very often touch it first at a corner, where one weight is exactly zero.

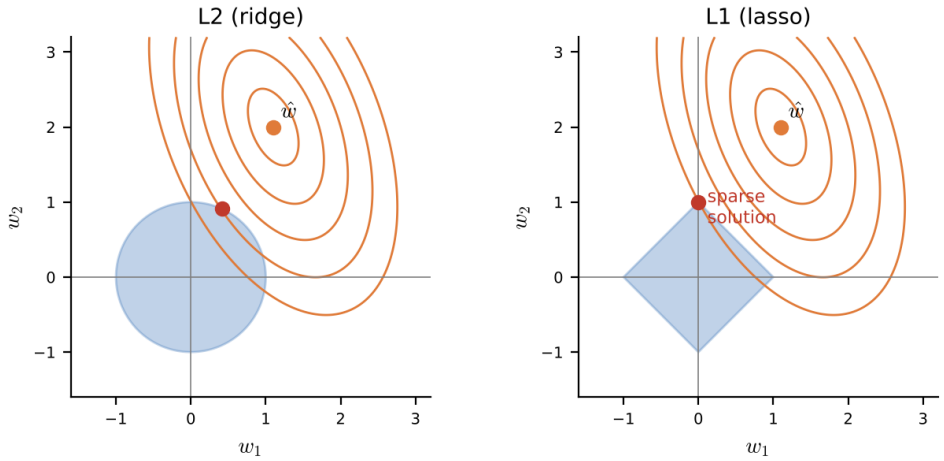


Figure 1.16 Geometry of regularisation: loss contours (ellipses) centred on the unregularised solution $\hat{w}$ meet the L2 constraint disc at a non-sparse point but the L1 diamond at a corner where $w_1 = 0$.

E Worked Example 1.9 | Ridge Shrinkage in One Dimension

For a single standardised feature with no intercept, the ridge solution is $w = \Sigma x_i y_i \,/\, (\Sigma x_i^2 + \lambda)$. Take $x = (1, 2, 3)$ and $y = (2, 4, 7)$.

- $\Sigma x_i y_i = 2 + 8 + 21 = 31$; $\Sigma x_i^2 = 1 + 4 + 9 = 14$.
- $\lambda = 0$ (ordinary least squares): $w = 31/14 \approx 2.214$.
- $\lambda = 1$: $w = 31/15 \approx 2.067$. $\lambda = 10$: $w = 31/24 \approx 1.292$. $\lambda = 100$: $w = 31/114 \approx 0.272$.

As λ grows the weight shrinks smoothly toward zero but never reaches it. The corresponding lasso solution is a *soft-thresholded* estimate, $w = \text{sign}(31)\cdot\max(31 - \lambda/2, 0)/14$ for the loss $\Sigma(y_i - w x_i)^2 + \lambda|w|$, which becomes exactly zero once $\lambda \geq 62$.

1.14.3 Other Regularisation Techniques

Penalising weights is only one way to restrict effective capacity. **Early stopping** halts iterative training when validation error stops improving, and for linear models with gradient descent it is approximately equivalent to L2 regularisation. **Data augmentation**

enlarges the training set with label-preserving transformations (flipped or cropped images, paraphrased sentences). **Dropout**, which randomly disables units during training, and **batch normalisation** are regularisers for neural networks discussed in Chapter 2. **Pruning** and depth limits regularise decision trees, and **ensembling** — averaging many models — reduces variance. **Noise injection** into inputs or weights and **label smoothing** are further options. Table 1.14 compares these methods.

Table 1.14 Regularisation techniques and where they are used.

Technique	Mechanism	Typical models
L2 / weight decay	Penalise squared weights; smooth shrinkage	Linear, logistic, SVM, neural networks
L1 / lasso	Penalise absolute weights; sparsity	Linear and logistic models
Elastic net	Mix of L1 and L2	Linear models with correlated features
Early stopping	Stop when validation error rises	Gradient boosting, neural networks
Pruning / depth limits	Restrict tree size	Decision trees
Dropout	Randomly drop units during training	Neural networks
Data augmentation	Label-preserving transformations	Images, audio, text
Ensembling	Average many models	Any (bagging, random forests)

Listing 1.6 Comparing ridge, lasso and elastic net with cross-validated regularisation strength.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV
import numpy as np

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
alphas = np.logspace(-3, 3, 50)    # scikit-learn calls lambda "alpha"

models = {"ridge": RidgeCV(alphas=alphas),
          "lasso": LassoCV(alphas=alphas, cv=5),
          "elastic net": ElasticNetCV(alphas=alphas, l1_ratio=0.5, cv=5)}
for name, reg in models.items():
    pipe = make_pipeline(StandardScaler(), reg).fit(X_tr, y_tr)
    coef = pipe[-1].coef_
    print(f"{name:12s} alpha={pipe[-1].alpha_:.3f}  R2={pipe.score(X_te, y_te):.3f}  "
          f"zero weights={np.sum(np.abs(coef) < 1e-8)}")
```

1.15 Supervised Learning Algorithms

The remainder of this part of the chapter presents the classical supervised learning algorithms. Although they differ widely in mechanics, each can be understood through the same three questions: *what hypothesis space does it search* (linear functions, axis-aligned partitions, weighted combinations of kernels, local neighbourhoods, probabilistic models)? *what loss does it minimise*? and *how does it control complexity*? This section provides an overview and a basis for comparison; the following sections develop each algorithm in detail.

1.15.1 Families of Supervised Learners

Supervised learners can be grouped along several dimensions. **Parametric** models, such as linear and logistic regression and naive Bayes, summarise the data with a fixed number of parameters that does not grow with n; they are fast and data-efficient but limited by their assumed functional form. **Non-parametric** models, such as k-NN, decision trees and kernel SVMs, let complexity grow with the data; they are more flexible but need more data and may be slower at prediction time. **Discriminative** models learn $P(y \mid x)$ or a decision boundary directly (logistic regression, SVM), whereas **generative** models learn $P(x \mid y)$ and $P(y)$ and apply Bayes' theorem (naive Bayes, linear discriminant analysis). **Linear** models have decision boundaries that are hyperplanes in feature space; **non-linear** models — trees, kernel methods, k-NN, neural networks — can represent curved boundaries. Finally, **ensemble** methods combine many base learners. Figure 1.17 shows how differently these families carve up the same two-dimensional dataset.

1.15.2 Comparing the Algorithms

Table 1.15 compares the algorithms developed in the following sections, together with gradient boosting from the section on bagging, boosting and stacking. No algorithm dominates: linear models are excellent baselines and remain competitive in high-dimensional sparse problems such as text; tree ensembles, particularly gradient boosting, are usually the most accurate on heterogeneous tabular data; SVMs excel on medium-sized datasets with clear margins; k-NN and naive Bayes are simple, fast to build and surprisingly strong in the right settings. Neural networks, covered in Chapter 2, dominate on images, audio and text.

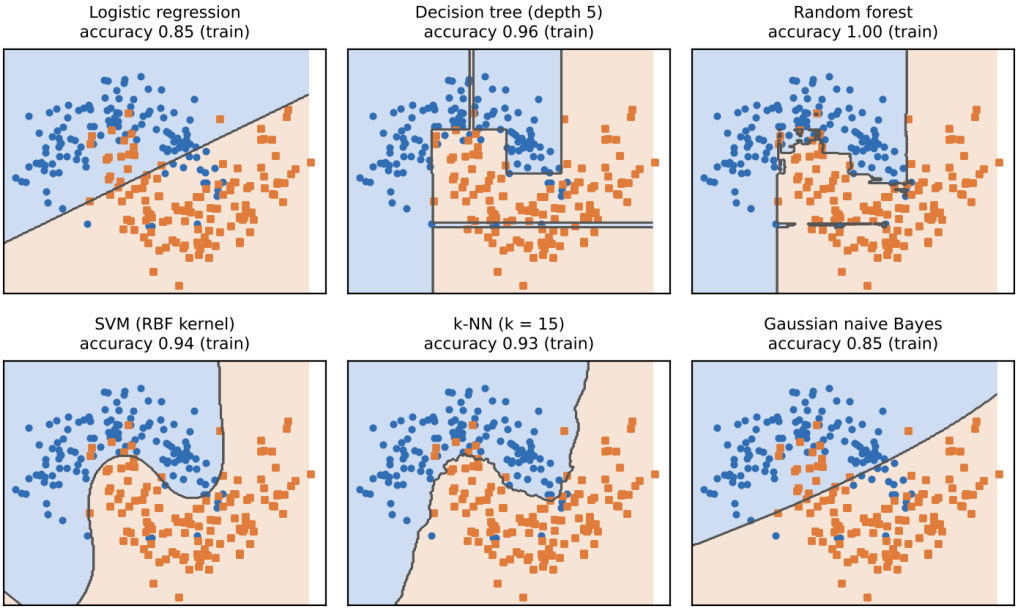


Figure 1.17 Decision boundaries learned by six classifiers on the same two-moons dataset, illustrating the inductive bias of each family.

Table 1.15 Comparison of classical supervised learning algorithms.

Algorithm	Type	Boundary	Needs scaling	Interpretabili	Main strength	Main weakness
Linear regression	Parametric	Linear	For regularised fits	High	Fast, simple, closed form	Only linear relations
Logistic regression	Parametric, discriminative	Linear	Yes	High	Calibrated probabilities	Linear boundary
Decision tree	Non-parametric	Axis-aligned	No	High (if small)	Handles mixed data, rules	High variance
Random forest	Ensemble	Non-linear	No	Medium	Robust, little tuning	Large, slower to predict
SVM	Non-parametric (kernel)	Linear or kernel	Yes	Low–medium	Effective in high dimensions	Scales poorly to large n
k-NN	Non-parametric, lazy	Non-linear	Yes	Medium	No training; adapts locally	Slow prediction; curse of dimensionality
Naive Bayes	Parametric, generative	Linear or quadratic	No	Medium	Very fast; good for text	Independence assumption
Gradient boosting	Ensemble	Non-linear	No	Low–medium	State of the art on tabular data	Many hyperparameters

T Practice Tip | A Practical Order of Attack

- Start with a trivial baseline (majority class or mean), then a regularised linear or logistic model.
- Try a random forest: it needs little tuning and gives a strong reference score.
- Try gradient boosting (XGBoost, LightGBM or CatBoost) with early stopping and moderate tuning.
- Use SVMs or k-NN for small-to-medium datasets with meaningful distances; naive Bayes for text.
- Prefer the simplest model whose validation performance is within noise of the best.

1.16 Linear Regression

Linear regression is the oldest and most widely used supervised learning method, with roots in the least-squares method published by Legendre (1805) and Gauss (1809). It models a continuous target as a weighted sum of the input features. Its simplicity makes it fast, interpretable and a natural baseline, and its mathematics — loss minimisation, closed-form solutions, gradient descent, regularisation — forms the template for much of machine learning.

1.16.1 The Model

Given a feature vector $x = (x_1, \ldots, x_d)^T$, linear regression predicts $\hat{y}$ as in Equation (1.15). It is convenient to prepend a constant feature $x_0 = 1$ so that the intercept (bias) w_0 is absorbed into the weight vector. Each weight w_j is the expected change in $\hat{y}$ for a one-unit increase in x_j, holding the other features fixed. "Linear" refers to linearity in the *parameters*: the features may be non-linear transformations of raw inputs (x^2, $\log x$, $x_1 x_2$), so *polynomial regression* is still linear regression on expanded features.

$$\hat{y} = w_0 + \sum_{j=1}^{d} w_j x_j = \mathbf{w}^\top \mathbf{x}, \qquad \hat{\mathbf{y}} = X\mathbf{w} \tag{1.15}$$

where $w = (w_0, w_1, \ldots, w_d)^T$ is the weight vector, x includes $x_0 = 1$, and $X \in \mathbb{R}^{n \times (d+1)}$ is the design matrix with a leading column of ones.

1.16.2 Least Squares and the Normal Equations

Ordinary least squares (OLS) chooses w to minimise the mean squared error between predictions and targets, Equation (1.16). The loss is a convex quadratic in w, so its

minimum is found by setting the gradient to zero, which yields the *normal equations* and their closed-form solution in Equation (1.17). If $X^{T}X$ is singular or ill-conditioned — because features are collinear or $d > n$ — the inverse does not exist or is numerically unstable; adding a ridge penalty λI fixes this, giving the ridge solution on the last line. In practice, libraries solve the system with a QR or singular value decomposition rather than forming an explicit inverse. Under the assumption of Gaussian noise, $y = w^{T}x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \sigma^{2})$, the OLS estimate is also the *maximum-likelihood* estimate, which connects least squares to probabilistic modelling.

$$J(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \mathbf{w}^{\top}\mathbf{x}_i\right)^2 = \frac{1}{n}\lVert \mathbf{y} - X\mathbf{w}\rVert_2^2 \tag{1.16}$$

$$\begin{aligned} \nabla_{\mathbf{w}} J &= -\tfrac{2}{n} X^{\top}(\mathbf{y} - X\mathbf{w}) = \mathbf{0} \quad \Rightarrow \quad X^{\top}X\,\mathbf{w} = X^{\top}\mathbf{y} \\ \hat{\mathbf{w}}_{OLS} &= (X^{\top}X)^{-1}X^{\top}\mathbf{y}, \qquad \hat{\mathbf{w}}_{ridge} = (X^{\top}X + \lambda I)^{-1}X^{\top}\mathbf{y} \end{aligned} \tag{1.17}$$

1.16.3 Gradient Descent

Solving the normal equations costs $O(nd^2 + d^3)$ time, which becomes prohibitive for very many features or for data that do not fit in memory. *Gradient descent* instead starts from an initial guess and repeatedly moves w a small step in the direction of steepest decrease of the loss, as in Algorithm 1.2. The step size η is the *learning rate*: too small and convergence is slow, too large and the iterates oscillate or diverge. *Stochastic* and *mini-batch* variants estimate the gradient from one example or a small batch at a time and scale to arbitrarily large datasets; they, together with momentum and adaptive methods such as Adam, are discussed in the later section on gradient descent and optimisation.

Algorithm 1.2 Batch Gradient Descent for Linear Regression

Input: Design matrix $X \in \mathbb{R}^{n \times (d+1)}$ (first column ones), targets $y \in \mathbb{R}^{n}$, learning rate η, regularisation $\lambda \geq 0$, tolerance ε, maximum iterations T
Output: Weight vector w

initialise $w \leftarrow 0$
for $t = 1$ **to** T **do**
  $\hat{y} \leftarrow Xw$ ▷ *predictions*
  $g \leftarrow -(2/n)\, X^{T}(y - \hat{y}) + 2\lambda\, \tilde{w}$ ▷ $\tilde{w} = w$ *with* w_0 *set* **to** 0
  $w \leftarrow w - \eta\, g$
  if $\lVert g \rVert < \varepsilon$ **then**
    break ▷ *converged*
return w

E Worked Example 1.10 | Fitting a Line by Least Squares

Advertising spend x (in \$1000s) and sales y (in units): (1, 3), (2, 5), (3, 6), (4, 8).

- Means: $\bar{x} = 2.5$, $\bar{y} = 5.5$.
- Slope $w_1 = \Sigma(x - \bar{x})(y - \bar{y})/\Sigma(x - \bar{x})^2 = [(-1.5)(-2.5) + (-0.5)(-0.5) + (0.5)(0.5) + (1.5)(2.5)] / [2.25 + 0.25 + 0.25 + 2.25] = (3.75 + 0.25 + 0.25 + 3.75)/5 = 8/5 = 1.6$.
- Intercept $w_0 = \bar{y} - w_1\bar{x} = 5.5 - 1.6 \times 2.5 = 1.5$. Fitted line: $\hat{y} = 1.5 + 1.6x$.
- Predictions (3.1, 4.7, 6.3, 7.9); residuals (−0.1, 0.3, −0.3, 0.1); SSE = 0.01 + 0.09 + 0.09 + 0.01 = 0.20.
- SST = $\Sigma(y - \bar{y})^2 = 6.25 + 0.25 + 0.25 + 6.25 = 13$; $R^2 = 1 - 0.20/13 \approx 0.985$.

1.16.4 Assumptions, Diagnostics and Evaluation

For valid inference (confidence intervals and hypothesis tests on coefficients) OLS relies on the assumptions listed in Table 1.16; for pure prediction, violations matter mainly insofar as they reduce accuracy. Residual plots are the primary diagnostic: residuals plotted against fitted values should show no pattern and constant spread. Regression performance is commonly reported with the mean squared error, its square root (RMSE), the mean absolute error and the *coefficient of determination* R^2 of Equation (1.18), the fraction of target variance explained by the model; these and other metrics are treated in the section on evaluation metrics. Listing 1.7 fits linear regression both in closed form and with scikit-learn.

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\mathrm{SSE}}{\mathrm{SST}} \tag{1.18}$$

where SSE is the residual sum of squares and SST the total sum of squares about the mean $\bar{y}$.

Table 1.16 Assumptions of ordinary least squares and how to check them.

Assumption	Meaning	Diagnostic	Remedy if violated
Linearity	$E[y \mid x]$ is linear in the parameters	Residuals vs fitted plot	Transform features; add polynomial terms
Independence	Errors are uncorrelated	Durbin–Watson test; residuals vs time	Time-series models; group-aware methods
Homoscedasticity	Constant error variance	Breusch–Pagan test; funnel shape in residuals	Transform target; weighted least squares
Normality of errors	$\varepsilon \sim$ Normal (for inference)	Q–Q plot of residuals	Transform target; robust or bootstrap inference
No perfect multicollinearity	Features not linearly dependent	Variance inflation factor (VIF)	Drop or combine features; ridge

Listing 1.7 Linear regression via the normal equations (NumPy) and scikit-learn.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

rng = np.random.default_rng(0)
X = rng.uniform(0, 10, size=(200, 2))
y = 3.0 + 1.5 * X[:, 0] - 2.0 * X[:, 1] + rng.normal(0, 1.0, 200)

# Closed form: solve (X^T X) w = X^T y with a column of ones for the intercept
Xb = np.c_[np.ones(len(X)), X]
w = np.linalg.lstsq(Xb, y, rcond=None)[0]     # numerically stable solve
print("normal equations:", w.round(3))

# scikit-learn
model = LinearRegression().fit(X, y)
print("sklearn:", model.intercept_.round(3), model.coef_.round(3))
y_hat = model.predict(X)
print("RMSE:", mean_squared_error(y, y_hat) ** 0.5, " R2:", r2_score(y, y_hat))
```

1.17 Logistic Regression

Despite its name, logistic regression is a *classification* algorithm. It models the probability that an instance belongs to the positive class as a smooth function of a linear score, and it is the standard baseline for binary classification in fields from medicine to marketing. Its outputs are well-calibrated probabilities, its coefficients are interpretable as log-odds ratios, and it generalises naturally to more than two classes.

1.17.1 From Linear Scores to Probabilities

Applying linear regression directly to a $0/1$ target produces predictions outside $[0,1]$ and is sensitive to points far from the boundary. Logistic regression instead passes the linear score $z = \mathbf{w}^{\mathrm{T}}\mathbf{x}$ through the *logistic* (*sigmoid*) *function* σ, defined in Equation (1.19) and plotted in Figure 1.18, which squashes any real number into $(0,1)$. Inverting the relation shows that the model is linear in the *log-odds* (logit): each unit increase in x_j multiplies the odds of the positive class by e^{w_j}. The decision rule "predict 1 if $P(y=1 \mid \mathbf{x}) \geq 0.5$" is equivalent to $\mathbf{w}^{\mathrm{T}}\mathbf{x} \geq 0$, so the decision boundary is the hyperplane $\mathbf{w}^{\mathrm{T}}\mathbf{x} = 0$. The threshold 0.5 can be moved to trade off false positives against false negatives.

$$P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\top}\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^{\top}\mathbf{x}}}$$
$$\log \frac{P(y=1 \mid \mathbf{x})}{1-P(y=1 \mid \mathbf{x})} = \mathbf{w}^{\top}\mathbf{x} \tag{1.19}$$

where $\sigma(z) = 1/(1+e^{-z})$ is the sigmoid function, with derivative $\sigma'(z) = \sigma(z)(1-\sigma(z))$.

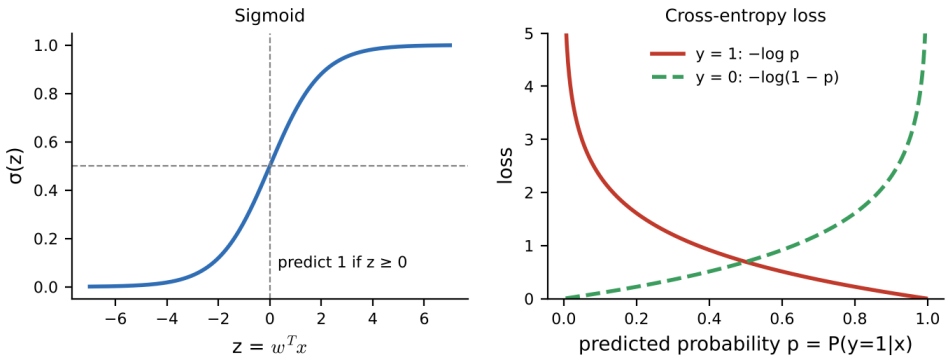


Figure 1.18 Left: the sigmoid function maps a linear score z to a probability; the 0.5 threshold corresponds to $z = 0$. Right: the cross-entropy loss for a positive example grows without bound as the predicted probability approaches zero.

1.17.2 Maximum Likelihood and Cross-Entropy

Writing $p_i = \sigma(w^T x_i)$, the likelihood of the observed labels under the model is $\prod_i p_i^{y_i}(1-p_i)^{1-y_i}$. Maximising it is equivalent to minimising the average negative log-likelihood, known as the *binary cross-entropy* or *log loss*, Equation (1.20). The loss heavily penalises confident wrong predictions, as the right panel of Figure 1.18 shows. Unlike least squares, there is no closed-form solution, but the loss is convex, so gradient descent or second-order methods (Newton's method, known in this context as *iteratively reweighted least squares*, or quasi-Newton L-BFGS) converge to the global minimum. Remarkably, the gradient has exactly the same form as for linear regression — prediction error times input — with $\hat{y}$ replaced by the probability p.

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right] + \frac{\lambda}{2}\|\mathbf{w}\|_2^2$$
$$\nabla_{\mathbf{w}} J = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)\,\mathbf{x}_i + \lambda \mathbf{w} = \frac{1}{n} X^{\top}(\mathbf{p}-\mathbf{y}) + \lambda \mathbf{w} \tag{1.20}$$

where $p_i = \sigma(w^T x_i)$ and $y_i \in \{0, 1\}$.

Regularisation is particularly important for logistic regression: if the classes are perfectly separable, the unregularised likelihood keeps increasing as $\|w\| \to \infty$, and the weights diverge. An L2 penalty (the default in scikit-learn, where the inverse strength $C = 1/\lambda$ is specified) or an L1 penalty for sparsity resolves this. Algorithm 1.3 summarises training by gradient descent.

Algorithm 1.3 Logistic Regression Trained by Gradient Descent

Input: Training data $X \in \mathbb{R}^{n \times (d+1)}$ (first column ones), labels $y \in \{0,1\}^n$, learning rate η, regularisation λ, iterations T

Output: Weight vector w; classifier $\hat{y}(x) = \mathbb{1}[\sigma(w^T x) \geq \tau]$ for threshold τ

initialise $w \leftarrow 0$
for $t = 1$ **to** T **do**
  $p \leftarrow \sigma(Xw)$ ▷ *element-wise sigmoid*
  $g \leftarrow (1/n)\, X^T(p - y) + \lambda\, \tilde{w}$ ▷ *$\tilde{w}$ = w with w_0 set to 0*
  $w \leftarrow w - \eta\, g$
return w

Worked Example 1.11 | **One Gradient Step for Logistic Regression**

Model with one feature: $P(y{=}1 \mid x) = \sigma(w_0 + w_1 x)$, starting from $w_0 = 0$, $w_1 = 0$, with $\eta = 0.5$ and $\lambda = 0$. Data: $(x, y) = (1, 0), (2, 0), (3, 1), (4, 1)$.

- With $w = 0$, every $p_i = \sigma(0) = 0.5$; errors $p_i - y_i = (0.5, 0.5, -0.5, -0.5)$.
- Gradient for w_0: $(1/4)(0.5 + 0.5 - 0.5 - 0.5) = 0$.
- Gradient for w_1: $(1/4)(0.5{\times}1 + 0.5{\times}2 - 0.5{\times}3 - 0.5{\times}4) = (1/4)(-2) = -0.5$.
- Update: $w_0 = 0$, $w_1 = 0 - 0.5 \times (-0.5) = 0.25$.
- New probabilities for $x = 1 \ldots 4$: $\sigma(0.25) = 0.562$, $\sigma(0.5) = 0.622$, $\sigma(0.75) = 0.679$, $\sigma(1.0) = 0.731$. Probabilities now increase with x; further steps will make w_0 negative and shift the boundary between $x = 2$ and $x = 3$. The initial log loss of $-\log 0.5 = 0.693$ has fallen to $(1/4)[-\log(0.438) - \log(0.378) - \log(0.679) - \log(0.731)] \approx (0.826 + 0.973 + 0.387 + 0.313)/4 \approx 0.625$.

1.17.3 Multiclass Logistic Regression

For $K > 2$ classes, two strategies are common. *One-vs-rest* trains K binary classifiers, each separating one class from all others, and predicts the class with the highest score. *Multinomial* or *softmax regression* learns one weight vector per class and converts the K scores into a probability distribution with the *softmax* function of Equation (1.21), trained by minimising the categorical cross-entropy. Softmax regression is exactly the output layer of a neural-network classifier, a connection developed in Chapter 2. Listing 1.8 fits a multinomial model and inspects its odds ratios.

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x})}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x})}, \qquad k = 1, \ldots, K \tag{1.21}$$

where w_k is the weight vector for class k.

Listing 1.8 Regularised multinomial logistic regression and odds ratios.

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

data = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target, stratify=data.target,
                                          random_state=0)
# C is the inverse regularisation strength (C = 1 / lambda); lbfgs fits a softmax model
clf = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter=1000))
clf.fit(X_tr, y_tr)
print("test accuracy:", clf.score(X_te, y_te))
print("class probabilities:", clf.predict_proba(X_te[:2]).round(3))

# Odds ratio per standard deviation of each feature, for class 'virginica'
coef = clf[-1].coef_[2]
for name, c in zip(data.feature_names, coef):
    print(f"{name:20s} odds ratio = {np.exp(c):.2f}")
```

1.18 Decision Trees

A decision tree predicts by asking a sequence of simple questions about the features — "Is income above 40,000?", "Is the outlook sunny?" — and following the answers down a branching structure until it reaches a *leaf*, which holds the prediction. Trees mirror the way people make decisions, can be drawn and read by non-specialists, handle numerical and categorical features without scaling, and capture non-linear relationships and interactions automatically. They are also the building blocks of random forests and gradient boosting, the most successful methods for tabular data. The main algorithms are ID3 and its successor C4.5 (Quinlan, 1986; 1993) and CART (Breiman et al., 1984).

1.18.1 Structure of a Tree

A tree consists of *internal nodes*, each testing a feature against a condition; *branches*, one per outcome of the test; and *leaves*, each assigned a class label (or class distribution) for classification or a constant value for regression. The topmost node is the *root*. Geometrically, each internal node with a threshold test $x_j \le t$ splits the feature space with an axis-aligned hyperplane, so a tree partitions the space into axis-aligned rectangular regions, each with a constant prediction. Figure 1.19 shows a tree learned on two features together with the partition it induces.

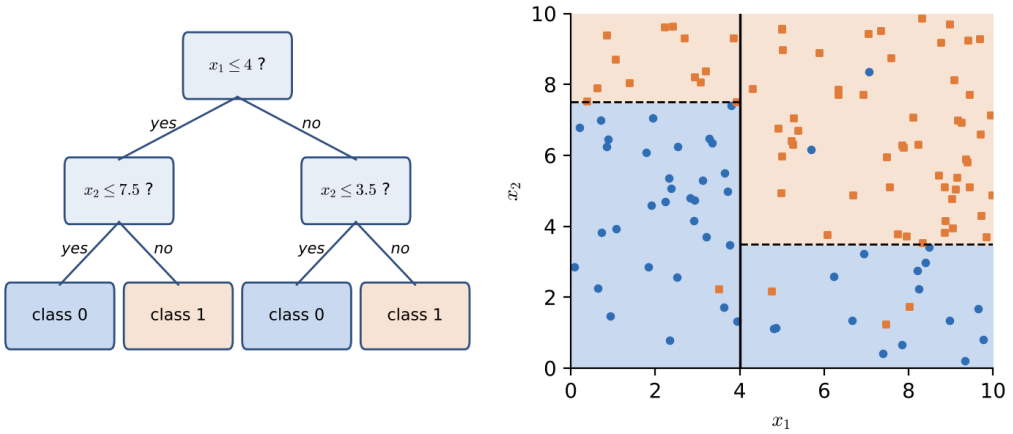


Figure 1.19 A depth-2 classification tree (left) and the axis-aligned partition of the feature space it induces (right); each leaf corresponds to one rectangular region.

1.18.2 Impurity Measures: Entropy and Gini

Trees are grown greedily from the root: at each node the algorithm chooses the split that makes the resulting child nodes as *pure* as possible, meaning dominated by a single class. Purity is measured by an *impurity function* of the class proportions p_k in a node, given in Equation (1.22). *Entropy*, from information theory, measures the average number of bits needed to encode the class of a random member of the node; it is 0 for a pure node and $\log_2 K$ for a uniform distribution over K classes. *Gini impurity*, used by CART, is the probability that two items drawn at random from the node (with replacement) have different classes. *Misclassification error* $1 - \max_k p_k$ is a third option but is less sensitive to changes in the class distribution and is rarely used for growing trees. As Figure 1.20 shows, entropy and Gini have very similar shapes, and in practice they usually produce similar trees; Gini is slightly cheaper to compute because it avoids logarithms.

$$
\begin{aligned}
H(S) &= -\sum_{k=1}^{K} p_k \log_2 p_k \\
\mathrm{Gini}(S) &= 1 - \sum_{k=1}^{K} p_k^2 \\
\mathrm{Err}(S) &= 1 - \max_k p_k
\end{aligned}
\tag{1.22}
$$

where p_k is the proportion of examples in node S that belong to class k (with $0 \log 0 = 0$).

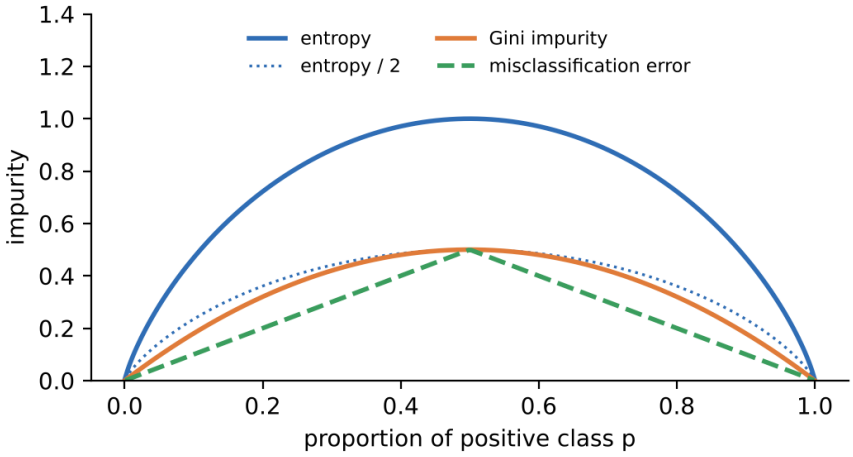


Figure 1.20 Impurity measures for a binary node as a function of the proportion p of the positive class. Entropy is scaled by ½ for comparison; all three are maximal at p = 0.5 and zero for a pure node.

1.18.3 Information Gain and the Splitting Criterion

The quality of a candidate split is the reduction in impurity it achieves, weighting each child by its share of the examples. With entropy as the impurity this reduction is called *information gain*, Equation (1.23). ID3 chooses the attribute with the highest information gain. Information gain is biased towards attributes with many values — an identifier column splits every example into its own pure leaf and has maximal gain but no predictive value. C4.5 corrects this by dividing by the *split information* (the entropy of the partition itself), giving the *gain ratio*. For numerical features, candidate thresholds are the midpoints between consecutive sorted distinct values, and the best threshold is found by a linear scan after sorting. CART produces strictly binary splits and, for regression, uses the reduction in variance (equivalently, in squared error) as its criterion, predicting the mean target value in each leaf.

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

$$\text{GainRatio}(S, A) = \frac{IG(S, A)}{-\sum_{v} \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}} \tag{1.23}$$

where S_v is the subset of S for which attribute A takes value v.

Algorithm 1.4 Recursive Decision-Tree Induction (ID3/CART style)

Input: Training set S of labelled examples; feature set F; impurity function I; stopping parameters (max_depth, min_samples_split, min_impurity_decrease)
Output: A decision tree T

```
function BuildTree(S, depth)
    if all examples in S have the same label, or depth = max_depth, or |S| < min_samples_split then
        return Leaf(majority label of S)                    ▷ mean of y for regression
    for each feature j ∈ F and each candidate threshold t do
        S_L ← {x ∈ S : x_j ≤ t}; S_R ← S \ S_L
        Δ(j, t) ← I(S) − (|S_L|/|S|) I(S_L) − (|S_R|/|S|) I(S_R)
    (j*, t*) ← argmax Δ(j, t)
    if Δ(j*, t*) < min_impurity_decrease then
        return Leaf(majority label of S)
    T_L ← BuildTree(S_L*, depth + 1); T_R ← BuildTree(S_R*, depth + 1)
    return Node(test x_j* ≤ t*, left = T_L, right = T_R)
T ← BuildTree(S, 0)
return T
```

Algorithm 1.4 summarises the greedy recursive procedure. Because each split is chosen to be locally optimal, the resulting tree is not guaranteed to be globally optimal — finding the smallest consistent tree is NP-hard — but greedy induction works well in practice and is fast: roughly $O(d \cdot n \log n)$ per level of the tree.

1.18.4 Worked Example: Choosing the Root Split

We apply information gain to the classic *Play Tennis* dataset used by Quinlan (1986) and Mitchell (1997), shown in Table 1.17. It contains 14 days, of which 9 are labelled Yes (play) and 5 No.

E Worked Example 1.12 | **Information Gain and Gini on Play Tennis**

Root entropy. $H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = 0.410 + 0.530 = \mathbf{0.940}$ bits.

Split on Outlook. Sunny: 2 Yes, 3 No → H = 0.971. Overcast: 4 Yes, 0 No → H = 0. Rain: 3 Yes, 2 No → H = 0.971.

- Weighted child entropy $= (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694$.
- $\mathrm{IG}(\text{Outlook}) = 0.940 - 0.694 = \mathbf{0.247}$.

Split on Humidity. High: 3 Yes, 4 No → H = 0.985. Normal: 6 Yes, 1 No → H = 0.592.

- $\mathrm{IG}(\text{Humidity}) = 0.940 - (7/14)(0.985) - (7/14)(0.592) = 0.940 - 0.789 = \mathbf{0.151}$.

Split on Wind. Weak: 6 Yes, 2 No → H = 0.811. Strong: 3 Yes, 3 No → H = 1.000.

- $\mathrm{IG}(\text{Wind}) = 0.940 - (8/14)(0.811) - (6/14)(1.000) = 0.940 - 0.892 = \mathbf{0.048}$.

Table 1.17 The Play Tennis dataset (14 examples, 4 categorical features).

Day	Outlook	Temperature	Humidity	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Split on Temperature. Hot 2/2, Mild 4/2, Cool 3/1 (Yes/No) → IG = **0.029**.

Outlook has the highest gain and becomes the root. The Overcast branch is pure (always Yes) and becomes a leaf; the procedure recurses on Sunny (where Humidity separates perfectly) and Rain (where Wind separates perfectly).

Gini check. Gini(root) $= 1 - (9/14)^2 - (5/14)^2 = 0.459$. For Outlook: Gini(Sunny) = Gini(Rain) $= 1 - 0.4^2 - 0.6^2 = 0.48$ and Gini(Overcast) $= 0$, so the weighted Gini $= (10/14)(0.48) = 0.343$ and the decrease is 0.116 — again the largest of the four attributes.

1.18.5 Overfitting, Pruning and Practical Considerations

A tree grown until every leaf is pure fits the training data perfectly and almost always overfits, particularly with noisy labels. Two remedies exist. *Pre-pruning* (early stopping) halts growth using the stopping parameters of Algorithm 1.4 — maximum depth, minimum samples per split or per leaf, or minimum impurity decrease. *Post-pruning* grows a full tree and then removes subtrees that do not improve validation performance. CART's *cost-complexity pruning* minimises $R_\alpha(T) = R(T) + \alpha|T|$, where $R(T)$ is the training error, $|T|$ the number of leaves and $\alpha \geq 0$ a complexity parameter chosen by cross-validation; increasing α yields a nested sequence of smaller trees. Beyond overfitting, single trees have *high variance* — a small change in the data can change the root split and hence the whole tree — and their axis-aligned, piecewise-constant boundaries approximate diagonal or smooth boundaries poorly. Both weaknesses are addressed by the ensemble methods of the following sections. Listing 1.9 trains and prunes a tree in scikit-learn.

Listing 1.9 Training, pruning and printing a decision tree.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, export_text

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target,
                                          stratify=data.target, random_state=0)

# Cost-complexity pruning path: candidate alphas from the fully grown tree
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(X_tr, y_tr)
search = GridSearchCV(DecisionTreeClassifier(criterion="gini", random_state=0),
                      {"ccp_alpha": path.ccp_alphas}, cv=5).fit(X_tr, y_tr)
tree = search.best_estimator_
print("best alpha:", round(search.best_params_["ccp_alpha"], 4),
      "leaves:", tree.get_n_leaves(), "test acc:", round(tree.score(X_te, y_te), 3))
print(export_text(tree, feature_names=list(data.feature_names), max_depth=3))
```

Key Point | Key Points

- Trees partition the feature space with axis-aligned splits chosen greedily to maximise impurity reduction.
- Entropy/information gain (ID3, C4.5) and Gini impurity (CART) usually give similar trees; gain ratio corrects the bias of information gain towards many-valued attributes.
- Unpruned trees overfit; control depth or use cost-complexity pruning.
- Trees are interpretable and need no scaling, but have high variance — the motivation for forests.

1.19 Random Forests

A random forest (Breiman, 2001) is an ensemble of decision trees, each trained on a different random sample of the data and a different random selection of features, whose predictions are combined by majority vote (classification) or averaging (regression). It retains the strengths of trees — no scaling, mixed feature types, non-linearity, interactions — while greatly reducing their variance. With default settings it is often within a few per cent of the best achievable accuracy, which has made it one of the most popular general-purpose learners.

1.19.1 Two Sources of Randomness

Random forests inject randomness in two ways. First, each tree is trained on a *bootstrap sample* — n examples drawn with replacement from the n training examples —

a technique called *bagging* that is described more generally in the section on bagging, boosting and stacking. Second, at every split, only a random subset of m of the d features is considered as candidates (*feature bagging* or the *random subspace* method). Typical defaults are $m = \sqrt{d}$ for classification and $m = d/3$ for regression. Trees are usually grown deep and left unpruned, so each has low bias but high variance; averaging then reduces the variance.

Why does the second source of randomness help? Averaging B identically distributed predictors, each with variance σ^2 and pairwise correlation ρ, gives the variance in Equation (1.24). As B grows the second term vanishes, but the first, $\rho\sigma^2$, remains: averaging cannot remove variance shared by all trees. If one feature is very strong, every bagged tree splits on it at the root and the trees are highly correlated. Restricting each split to a random subset of features *decorrelates* the trees, lowering ρ and hence the variance of the ensemble, at the cost of a small increase in the bias of each tree. The procedure is given in Algorithm 1.5.

$$\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2 \tag{1.24}$$

where T_b is the b-th tree, σ^2 the variance of a single tree's prediction and ρ the average correlation between trees.

Algorithm 1.5 Random Forest

Input: Training set $S = \{(x_i, y_i)\}_{i=1}^{n}$ with d features; number of trees B; features per split m; tree stopping rule (e.g. min_samples_leaf)
Output: Ensemble $\{T_1, \ldots, T_B\}$; prediction function

```
for b = 1 to B do
    draw bootstrap sample S_b of size n from S with replacement
    grow tree T_b on S_b:
        at each node, select m features at random from the d features
        choose the best split among those m features only
        split until the stopping rule is met (no pruning)
    record out-of-bag set OOB_b ← S \ S_b
predict for new x:
    classification: ŷ ← majority vote of T_1(x), ..., T_B(x)
    regression: ŷ ← (1/B) Σ_b T_b(x)
return {T_b}
```

1.19.2 Out-of-Bag Error and Feature Importance

The probability that a particular example is *not* selected in a bootstrap sample of size n is $(1 - 1/n)^n \approx e^{-1} \approx 0.368$, so each tree leaves out about 37% of the training data. These

out-of-bag (OOB) examples act as a built-in validation set: each training example can be predicted by the roughly B/3 trees that did not see it, and the resulting *OOB error* is a nearly unbiased estimate of the test error, available at no extra cost. Random forests also provide measures of *feature importance*. *Mean decrease in impurity* (MDI) sums the impurity reduction contributed by each feature over all splits and trees; it is cheap but biased toward continuous and high-cardinality features. *Permutation importance* measures how much the OOB or validation score drops when a feature's values are randomly shuffled; it is more reliable but slower, and can understate the importance of correlated features. Figure 1.21 shows how test error falls and stabilises as trees are added.

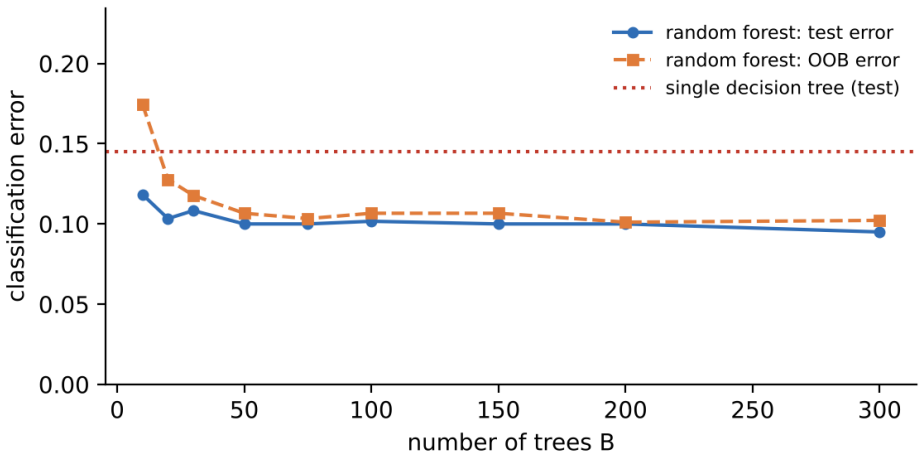


Figure 1.21 Test and out-of-bag error of a random forest as the number of trees grows, compared with a single fully grown decision tree; error stabilises after a few dozen trees and adding more does not cause overfitting.

E Worked Example 1.13 | Voting and Bootstrap Arithmetic

- **Bootstrap coverage.** With n = 10 training examples, P(an example is never drawn) = $(1 - 1/10)^{10} = 0.9^{10} \approx 0.349$; with n = 1000, $(0.999)^{1000} \approx 0.368 \approx e^{-1}$. On average a bootstrap sample contains about 63.2% of the distinct examples.
- **Voting.** Five trees predict (Yes, No, Yes, Yes, No) for a new customer; the forest predicts Yes with an estimated probability of 3/5 = 0.6.
- **Variance reduction.** Suppose each tree has prediction variance $\sigma^2 = 4$. With B = 100 independent trees ($\rho = 0$) the ensemble variance is 4/100 = 0.04; with $\rho = 0.5$ it is $0.5 \times 4 + 0.5 \times 4/100 = 2.02$. Decorrelation matters far more than the number of trees.

1.19.3 Hyperparameters and Practical Use

Random forests are notably forgiving of hyperparameter choices; the main ones are listed in Table 1.18. More trees never increase the expected error — they only cost computation — so B is set as large as the budget allows, typically a few hundred. The number of candidate features m is the most influential setting. Related methods include *extremely randomised trees* (Extra-Trees, Geurts et al., 2006), which also choose split thresholds at random, further reducing variance and training time. The main drawbacks of forests are their size and prediction latency, a loss of the single tree's interpretability, and an inability to extrapolate beyond the range of training targets in regression.

Table 1.18 Key random-forest hyperparameters (scikit-learn names).

Hyperparameter	Meaning	Typical values / advice
n_estimators	Number of trees B	200–1000; more is never worse, only slower
max_features	Features considered per split (m)	'sqrt' (classification), 1/3 of d (regression); tune
max_depth	Maximum tree depth	None (fully grown) by default; limit for noisy data
min_samples_leaf	Minimum examples per leaf	1–5 for classification, 5 for regression
bootstrap / max_samples	Use bootstrap; sample size per tree	True; smaller samples speed training
class_weight	Reweight classes	'balanced' for imbalanced data
oob_score	Compute out-of-bag estimate	True for a free validation estimate

1.20 Support Vector Machines (SVM)

Support vector machines, developed by Vapnik and colleagues and given their modern soft-margin form by Cortes and Vapnik (1995), were the dominant classification method of the late 1990s and 2000s and remain an excellent choice for small and medium-sized datasets, especially in high dimensions. Their central idea is geometric: among all hyperplanes that separate two classes, choose the one that is farthest from the nearest training points of either class. Combined with the *kernel trick*, which allows linear methods to learn non-linear boundaries, this yields a powerful and theoretically well-founded classifier.

1.20.1 The Maximum-Margin Hyperplane

Consider binary labels $y_i \in \{-1, +1\}$ and a linear classifier $\hat{y} = \operatorname{sign}(w^T x + b)$. If the data are linearly separable, infinitely many hyperplanes classify them perfectly. The distance from a point x_i to the hyperplane $w^T x + b = 0$ is $|w^T x_i + b| \,/\, \|w\|$. Because w and b can be rescaled freely, we fix the scale by requiring $y_i(w^T x_i + b) \geq 1$ for all i, with equality for the closest points. The *margin* — the width of the empty band between the classes — is then $2/\|w\|$, as illustrated in Figure 1.22. Maximising the margin is equivalent to minimising $\tfrac{1}{2}\|w\|^2$, giving the *hard-margin* SVM in Equation (1.25). The points that lie exactly on the margin boundaries are the *support vectors*; they alone determine the solution — removing any other point leaves the hyperplane unchanged. Intuitively, a wide margin is robust to small perturbations of the data, and statistical learning theory shows that large-margin classifiers have low effective capacity regardless of the dimension.

$$\min_{\mathbf{w},\, b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \ldots, n \tag{1.25}$$

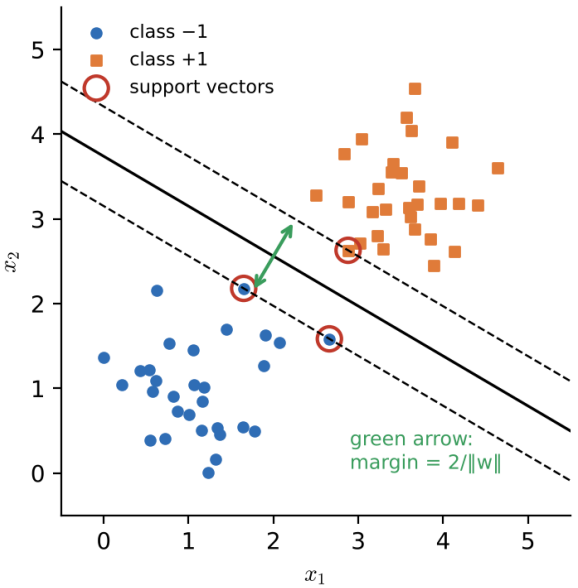


Figure 1.22 A linear soft-margin SVM: the solid line is the decision boundary $w^T x + b = 0$, dashed lines are the margin boundaries $w^T x + b = \pm 1$, and circled points are support vectors.

1.20.2 Soft Margin and Hinge Loss

Real data are rarely perfectly separable, and even when they are, insisting on separation makes the boundary hostage to outliers. The *soft-margin* SVM introduces *slack*

variables $\xi_i \geq 0$ that allow point i to violate the margin by ξ_i, and penalises the total slack, Equation (1.26). The hyperparameter C controls the trade-off: large C penalises violations heavily, giving a narrow margin that fits the training data closely (low bias, high variance); small C tolerates violations, giving a wide margin and a smoother boundary (higher bias, lower variance). At the optimum, $\xi_i = \max(0, 1 - y_i(w^T x_i + b))$, so the problem can be rewritten without constraints as the minimisation of the *hinge loss* plus an L2 penalty — the second line of Equation (1.26). Seen this way, the SVM is simply a regularised linear model with a particular loss. The hinge loss is zero for points correctly classified beyond the margin, so only points on or inside the margin influence the solution — the source of the support vectors' special status.

$$\begin{aligned} &\min_{\mathbf{w},\, b,\, \xi} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s. t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \\ &\Leftrightarrow \ \min_{\mathbf{w},\, b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\max\left(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right) \end{aligned} \tag{1.26}$$

where ξ_i are slack variables and $C > 0$ is the penalty on margin violations.

For large linear problems the unconstrained hinge-loss form can be optimised directly by stochastic sub-gradient descent, as in the Pegasos algorithm (Shalev-Shwartz et al., 2007) summarised in Algorithm 1.6; this is how linear SVMs are trained on millions of examples.

Algorithm 1.6 Linear SVM by Stochastic Sub-gradient Descent (Pegasos)

Input: Training set $\{(x_i, y_i)\}_{i=1}^{n}$ with $y_i \in \{-1, +1\}$; regularisation $\lambda > 0$ ($\lambda = 1/(nC)$); iterations T

Output: Weight vector w (bias absorbed via a constant feature)

```
w ← 0
for t = 1 to T do
    choose i uniformly at random from {1, ..., n}
    η_t ← 1 / (λ t)
    if y_i w^T x_i < 1 then                    ▷ margin violated: hinge active
        w ← (1 − η_t λ) w + η_t y_i x_i
    else
        w ← (1 − η_t λ) w                      ▷ only the regulariser acts
return w
```

1.20.3 The Dual Problem and the Kernel Trick

Applying Lagrange multipliers $\alpha_i \geq 0$ to the constraints of Equation (1.26) and eliminating w and b yields the *dual problem*, Equation (1.27). Its solution gives $w = \Sigma_i \alpha_i y_i x_i$,

and $\alpha_i > 0$ only for support vectors. The dual is a convex quadratic programme in n variables, solved efficiently by *sequential minimal optimisation* (SMO, Platt, 1998), the algorithm behind LIBSVM. The crucial observation is that the data appear in the dual — and in the resulting decision function — *only through inner products* $x_i^T x_j$.

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, K(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s. t.} \quad 0 \le \alpha_i \le C, \;\; \sum_i \alpha_i y_i = 0$$

$$f(\mathbf{x}) = \operatorname{sign}\left(\sum_{i \in SV} \alpha_i y_i \, K(\mathbf{x}_i, \mathbf{x}) + b \right) \tag{1.27}$$

where α_i are Lagrange multipliers, SV is the set of support vectors ($\alpha_i > 0$) and $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$ is a kernel function ($K(x_i, x_j) = x_i^T x_j$ for the linear SVM).

Suppose we map each input into a higher-dimensional feature space with a non-linear map $\varphi(x)$, where the classes become linearly separable, and fit a linear SVM there. We would need inner products $\varphi(x_i)^T \varphi(x_j)$, which might be extremely expensive — or, for some maps, infinite-dimensional. The *kernel trick* replaces them by a *kernel function* $K(x_i, x_j)$ that computes the inner product directly from the original inputs. Any symmetric function whose Gram matrix is positive semi-definite for all inputs (Mercer's condition) corresponds to some feature map. The common kernels are listed in Table 1.19. The *radial basis function* (RBF, Gaussian) kernel corresponds to an infinite-dimensional feature space and is the default choice; its parameter γ sets the width of influence of each support vector — large γ yields wiggly boundaries that can overfit, small γ yields smooth ones. C and γ must be tuned jointly, typically by grid search over logarithmic ranges, as Figure 1.23 illustrates.

Table 1.19 Common SVM kernel functions.

Kernel	K(x, z)	Hyperparameters	Notes
Linear	$x^T z$	C	Fast; best for high-dimensional sparse data (text)
Polynomial	$(\gamma\, x^T z + r)^p$	C, degree p, γ, r	Feature interactions up to degree p
RBF (Gaussian)	$\exp(-\gamma \lVert x - z \rVert^2)$	C, γ	Default non-linear kernel; requires scaling
Sigmoid	$\tanh(\gamma\, x^T z + r)$	C, γ, r	Not always positive semi-definite; rarely used

E Worked Example 1.14 | Hinge Loss and a Kernel Evaluation

A linear SVM has $w = (2, -1)$ and $b = -1$. Evaluate three training points:

- $x = (2, 1)$, $y = +1$: score $= 2 \cdot 2 - 1 \cdot 1 - 1 = 2$; margin $y \cdot \text{score} = 2 \ge 1 \rightarrow$ hinge loss 0 (not a support vector).
- $x = (1, 0.5)$, $y = +1$: score $= 2 - 0.5 - 1 = 0.5$; margin $0.5 \rightarrow$ hinge loss $1 - 0.5 = 0.5$ (inside the margin).

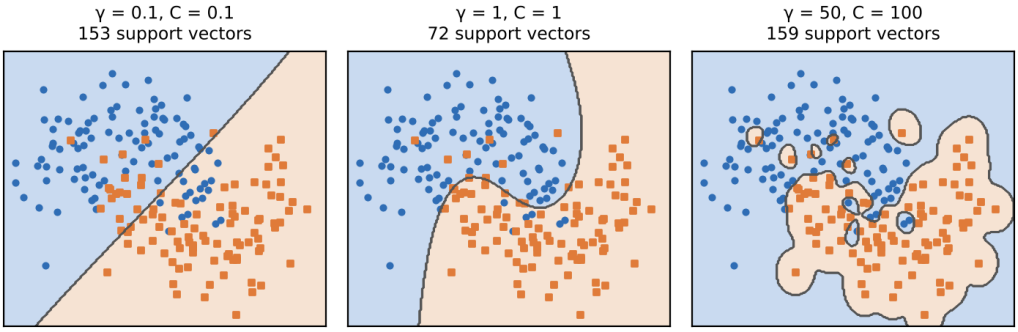


Figure 1.23 Effect of the RBF-kernel parameters on an SVM's decision boundary: small γ and C give smooth, underfitting boundaries, while large values give complex boundaries that overfit.

- $x = (1, 2)$, $y = +1$: score $= 2 - 2 - 1 = -1$; margin $-1 \rightarrow$ hinge loss 2 (misclassified).
- The margin width is $2/\|w\| = 2/\sqrt{5} \approx 0.894$; with $C = 1$ the objective is $\tfrac{1}{2}\cdot 5 + 1\cdot(0 + 0.5 + 2) = 5.0$.

Kernel trick check. For $x = (1, 2)$, $z = (3, 1)$ and the polynomial kernel $K(x, z) = (x^{T}z)^{2}$: $x^{T}z = 3 + 2 = 5$, so $K = 25$. The explicit map $\varphi(x) = (x_1^{2}, \sqrt{2}\cdot x_1 x_2, x_2^{2})$ gives $\varphi(x) = (1, 2\sqrt{2}, 4)$, $\varphi(z) = (9, 3\sqrt{2}, 1)$, and $\varphi(x)^{T}\varphi(z) = 9 + 12 + 4 = 25$ — identical, without ever forming φ.

Beyond binary classification, SVMs handle multiple classes through one-vs-one or one-vs-rest decompositions, regression through *support vector regression* (SVR), which uses an ε-insensitive loss that ignores errors smaller than ε, and novelty detection through the one-class SVM. Their main limitations are training time that scales between $O(n^2)$ and $O(n^3)$ for kernel SVMs, making them impractical beyond a few hundred thousand examples; the need for feature scaling and careful tuning of C and γ; and the absence of native probability outputs (Platt scaling fits a sigmoid to the scores when probabilities are needed). Listing 1.10 tunes an RBF SVM by cross-validated grid search.

Listing 1.10 Tuning an RBF-kernel SVM with a scaled pipeline and grid search.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)

pipe = Pipeline([("scale", StandardScaler()), ("svm", SVC(kernel="rbf"))])
grid = {"svm__C": [0.1, 1, 10, 100], "svm__gamma": [0.001, 0.01, 0.1, 1]}
search = GridSearchCV(pipe, grid, cv=5, scoring="accuracy", n_jobs=-1).fit(X_tr, y_tr)

best = search.best_estimator_
print("best params:", search.best_params_)
```

```
print("support vectors per class:", best["svm"].n_support_)
print("test accuracy:", round(best.score(X_te, y_te), 3))
```

1.21 k-Nearest Neighbors (k-NN)

The k-nearest neighbours algorithm is perhaps the simplest of all learning methods: to classify a new point, find the k training points closest to it and take a majority vote of their labels; for regression, average their target values. It embodies the inductive bias that similar inputs have similar outputs. k-NN is a *non-parametric*, *instance-based* or *lazy* learner: training consists merely of storing the data, and all computation is deferred to prediction time. Despite its simplicity it has attractive theory — Cover and Hart (1967) showed that, as n → ∞, the error of the 1-nearest-neighbour rule is at most twice the Bayes-optimal error.

1.21.1 The Algorithm and Distance Metrics

Algorithm 1.7 gives the procedure. The notion of "nearest" requires a distance metric, the most common being the *Minkowski* family in Equation (1.28). With p = 2 it is the familiar Euclidean distance; with p = 1 the Manhattan (city-block) distance, which is less influenced by a single large coordinate difference. Other choices include cosine distance for text and other high-dimensional sparse vectors, Hamming distance for binary or categorical features, and the Mahalanobis distance, which accounts for correlations between features. Because distances are dominated by features with large ranges, **features must be scaled** before applying k-NN.

$$d_p(\mathbf{x},\mathbf{z}) = \left(\sum_{j=1}^{d} |x_j - z_j|^p\right)^{1/p}$$

$$\hat{y}(\mathbf{x}) = \arg\max_{c} \sum_{i \in N_k(\mathbf{x})} w_i \, \mathbf{1}[y_i = c], \qquad w_i = 1 \;\text{ or }\; w_i = \frac{1}{d(\mathbf{x},\mathbf{x}_i)} \tag{1.28}$$

where $N_k(x)$ is the set of the k training points nearest to x, $1[\cdot]$ is the indicator function and w_i is a uniform or distance-based vote weight.

Algorithm 1.7 k-Nearest Neighbours Prediction

Input: Training set $\{(x_i, y_i)\}_{i=1}^{n}$ (scaled features); query point x; number of neighbours k; distance d; weighting scheme
Output: Predicted label (or value) $\hat{y}$

```
for i = 1 to n do
    d_i ← d(x, x_i)
N ← indices of the k smallest d_i                ▷ use a k-d tree or ball tree for speed
if classification then
    for each class c: score(c) ← Σ_{i∈N} w_i · 1[y_i = c]
    ŷ ← argmax_c score(c)                        ▷ ties broken by nearest neighbour
else
    ŷ ← Σ_{i∈N} w_i y_i / Σ_{i∈N} w_i
return ŷ
```

1.21.2 Choosing k

The number of neighbours k controls the bias–variance trade-off directly. With $k = 1$ the decision boundary follows every training point, including noisy ones: training error is zero and variance is high. As k increases the boundary becomes smoother, variance falls and bias rises; with $k = n$ every query receives the majority class. Figure 1.24 shows the effect. k is chosen by cross-validation, commonly within the range 3 to $\sqrt{n}$; an odd k avoids ties in binary problems. *Distance weighting*, which gives closer neighbours larger votes, reduces sensitivity to the exact choice of k.

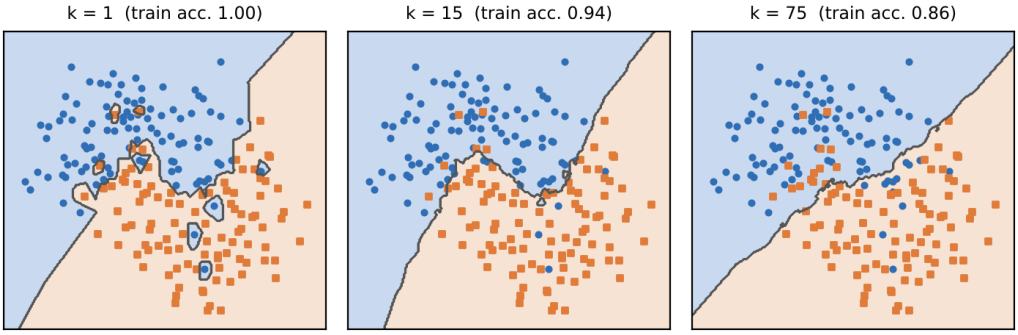


Figure 1.24 k-NN decision boundaries for $k = 1$, 15 and 75 on the same 200-point dataset: small k overfits noise; large k oversmooths.

E Worked Example 1.15 | **Classifying with k = 3**

Training data (two scaled features → class): A (1, 1) → Red; B (2, 1) → Red; C (4, 3) → Blue; D (5, 4) → Blue; E (3, 2) → Blue; F (1, 3) → Red. Query $x = (2.5, 2)$.

- Euclidean distances: A $\sqrt{(1.5^2 + 1^2)} = \sqrt{3.25} \approx 1.80$; B $\sqrt{(0.5^2 + 1^2)} = \sqrt{1.25} \approx 1.12$; C $\sqrt{(1.5^2 + 1^2)} \approx 1.80$; D $\sqrt{(2.5^2 + 2^2)} = \sqrt{10.25} \approx 3.20$; E $\sqrt{(0.5^2 + 0^2)} = 0.50$; F $\sqrt{(1.5^2 + 1^2)} \approx 1.80$.
- The nearest is E (0.50), then B (1.12), then a three-way tie at 1.80 between A, C and F.
- Breaking the tie by index order takes A: neighbours {E Blue, B Red, A Red} → majority **Red** (2 vs 1).
- With distance weighting $1/d$: Blue $1/0.50 = 2.00$; Red $1/1.12 + 1/1.80 = 0.89 + 0.56 = 1.45$ → **Blue**.

The tie and the disagreement between schemes show why distance weighting and an appropriate k matter near class boundaries.

1.21.3 Computational Cost and the Curse of Dimensionality

A naive k-NN query costs $O(nd)$ time, which is prohibitive for large datasets or real-time use. Spatial index structures such as *k-d trees* and *ball trees* reduce query time to roughly $O(\log n)$ in low dimensions, and *approximate nearest-neighbour* methods based on locality-sensitive hashing or proximity graphs (e.g. HNSW) scale to billions of vectors and underpin modern vector databases. A more fundamental problem is the curse of dimensionality: in high-dimensional spaces the distances from a query to its nearest and farthest neighbours become nearly equal, so "nearest" loses meaning, and irrelevant features add noise to every distance. Dimensionality reduction, feature selection or metric learning are therefore often applied first. k-NN remains valuable as a baseline, for recommendation and retrieval, for imputation of missing values, and whenever a transparent, example-based justification ("these similar cases had outcome Y") is desirable.

Table 1.20 Strengths and weaknesses of k-NN.

Strengths	Weaknesses
No training phase; new data are added instantly	Prediction is slow and memory-hungry for large n
Naturally multiclass; flexible non-linear boundaries	Sensitive to feature scaling and irrelevant features
Few hyperparameters (k, metric, weighting)	Degrades in high dimensions (curse of dimensionality)
Example-based explanations	Sensitive to class imbalance (majority dominates votes)

1.22 Naive Bayes

Naive Bayes classifiers are *generative* probabilistic models: rather than learning a decision boundary directly, they model how each class generates its features and then use Bayes' theorem to infer the most probable class for a new instance. Their defining simplification — that features are conditionally independent given the class — is almost never exactly true, yet naive Bayes is fast, needs little data, scales to enormous numbers of features and works remarkably well in practice, especially for text classification such as spam filtering, where it was popularised in the late 1990s.

1.22.1 Bayes' Theorem and the Naive Assumption

For a class y and feature vector $x = (x_1, \dots, x_d)$, Bayes' theorem gives the *posterior* probability of the class in terms of the *likelihood* $P(x \mid y)$, the *prior* $P(y)$ and the *evidence* $P(x)$, as in the first line of Equation (1.29). Estimating the full joint likelihood $P(x_1, \dots, x_d \mid y)$ is infeasible: with d binary features it has $2^d - 1$ free parameters per class. The *naive* assumption factorises it into a product of one-dimensional terms, reducing the parameter count to d per class. Because the evidence $P(x)$ is the same for every class, classification simply chooses the class with the largest numerator. In practice the computation is done in log space, summing log-probabilities, to avoid numerical underflow when multiplying many small numbers.

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)\,P(y)}{P(\mathbf{x})} \;\propto\; P(y)\prod_{j=1}^{d} P(x_j \mid y)$$
$$\hat{y} = \arg\max_{c} \left[\log P(y=c) + \sum_{j=1}^{d} \log P(x_j \mid y=c)\right] \tag{1.29}$$

where $P(y = c)$ is the class prior and $P(x_j \mid y = c)$ the class-conditional distribution of feature j.

1.22.2 Variants of Naive Bayes

The variants differ in the distribution assumed for $P(x_j \mid y)$, as summarised in Table 1.21. *Gaussian* naive Bayes models each continuous feature as normally distributed within each class, with a class-specific mean and variance estimated from the training data; its density is given in Equation (1.30). *Multinomial* naive Bayes models count data, such as how many times each word appears in a document. *Bernoulli* naive Bayes models binary indicators, such as whether each word appears at all, and explicitly penalises the absence of features. *Complement* naive Bayes is an adaptation of the multinomial model that is more robust for imbalanced text classes, and *categorical* naive Bayes handles nominal features directly.

$$P(x_j \mid y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right) \tag{1.30}$$

where μ_jc and σ²_jc are the mean and variance of feature j among training examples of class c.

Table 1.21 Naive Bayes variants.

Variant	Feature type	Model for $P(x_j \mid y)$	Typical application
Gaussian	Continuous	Normal with class-specific μ, σ^2	Sensor or measurement data
Multinomial	Counts	Multinomial word frequencies	Document classification (bag of words)
Bernoulli	Binary	Bernoulli per feature	Short texts; presence/absence features
Complement	Counts	Estimated from complement of each class	Imbalanced text classification
Categorical	Nominal	Categorical distribution per feature	Survey or categorical tabular data

1.22.3 Laplace Smoothing

A practical difficulty arises when a feature value never co-occurs with a class in the training data. Its estimated probability is then zero, and because the posterior is a product, that single zero eliminates the class regardless of all other evidence. *Additive* or *Laplace smoothing* adds a pseudo-count α ($\alpha = 1$ for Laplace, $0 < \alpha < 1$ for Lidstone) to every count, as in Equation (1.31) for the multinomial model. Smoothing can be interpreted as placing a uniform Dirichlet prior on the probabilities, and α is a hyper-parameter that can be tuned by cross-validation.

$$\hat{P}(w \mid y = c) = \frac{\text{count}(w, c) + \alpha}{\sum_{w'} \text{count}(w', c) + \alpha\,|V|} \tag{1.31}$$

where count(w, c) is the number of occurrences of word w in training documents of class c, and |V| is the vocabulary size.

E Worked Example 1.16 | Multinomial Naive Bayes for Spam

Training corpus. Spam: “win money now”, “win prize”. Ham: “meeting now”, “project meeting notes”. Vocabulary V = {win, money, now, prize, meeting, project, notes}, |V| = 7. Priors P(spam) = P(ham) = 0.5.

- Word counts. Spam (5 tokens): win 2, money 1, now 1, prize 1. Ham (5 tokens): meeting 2, now 1, project 1, notes 1.
- With $\alpha = 1$ the denominators are 5 + 7 = 12 for each class.
- Classify “win meeting now”:
- Spam: P(win|s) = 3/12, P(meeting|s) = 1/12, P(now|s) = 2/12 → 0.5 × (3·1·2)/12^3 = 0.5 × 6/1728 = 0.001736.

- Ham: $P(\text{win}|\text{h}) = 1/12$, $P(\text{meeting}|\text{h}) = 3/12$, $P(\text{now}|\text{h}) = 2/12 \to 0.5 \times (1{\cdot}3{\cdot}2)/12^3 = 0.001736$.
- The scores tie — the message is equally consistent with both classes. Classify "win money" instead: spam $0.5 \times (3{\cdot}2)/144 = 0.0208$; ham $0.5 \times (1{\cdot}1)/144 = 0.00347$. Normalising, $P(\text{spam} \mid \text{x}) = 0.0208/(0.0208 + 0.00347) \approx \mathbf{0.857}$.

Without smoothing, $P(\text{money} \mid \text{ham}) = 0/5$ would have made the ham score exactly zero.

1.22.4 Strengths, Weaknesses and Use

Naive Bayes trains in a single pass over the data ($O(nd)$), predicts in $O(Kd)$ time, handles thousands or millions of sparse features, copes with missing features by simply omitting their terms, and updates incrementally as new data arrive. Its classification accuracy is often competitive even when the independence assumption is clearly violated, because classification depends only on which class scores highest, not on the exact probabilities (Domingos and Pazzani, 1997). The flip side is that its *probability estimates* are poorly calibrated — correlated features are counted repeatedly, pushing posteriors towards 0 and 1 — so calibration is advisable when probabilities matter. With Gaussian class-conditionals and shared variances its decision boundary is linear; with class-specific variances it is quadratic. Naive Bayes is closely related to logistic regression: the two form a classic generative–discriminative pair, with naive Bayes converging faster with few examples and logistic regression achieving lower asymptotic error with many (Ng and Jordan, 2002). Listing 1.11 builds a text classifier.

Listing 1.11 A multinomial naive Bayes text classifier.

```python
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report

cats = ["sci.space", "rec.sport.hockey", "comp.graphics"]
train = fetch_20newsgroups(subset="train", categories=cats, remove=("headers",
    "footers"))
test = fetch_20newsgroups(subset="test", categories=cats, remove=("headers",
    "footers"))

# alpha is the additive (Laplace/Lidstone) smoothing parameter
clf = make_pipeline(TfidfVectorizer(stop_words="english", min_df=2),
    MultinomialNB(alpha=0.1))
clf.fit(train.data, train.target)
print(classification_report(test.target, clf.predict(test.data),
    target_names=test.target_names))
print(clf.predict(["the shuttle reached orbit", "a great goal in overtime"]))
```

1.23 Ensemble Learning

Ensemble learning combines the predictions of several models — the *base learners* — to obtain a prediction that is better than any of them individually. The idea has a long pedigree: Francis Galton observed in 1907 that the median of 787 visitors' guesses of an ox's weight at a country fair was within 1% of the truth, more accurate than most individual guesses. In machine learning, ensembles win most competitions on tabular data and are the default choice in many production systems. Random forests, introduced earlier, are one example; this section explains why ensembles work, and the next describes the main construction methods.

1.23.1 Why Ensembles Work

Ensembles help for three reasons, identified by Dietterich (2000). *Statistically*, with limited data many hypotheses fit the training set equally well; averaging them reduces the risk of choosing a poor one. *Computationally*, learning algorithms such as greedy tree induction and gradient descent can get stuck in local optima; combining runs from different starting points gives a better approximation of the best hypothesis. *Representationally*, the true function may lie outside the hypothesis space of any single learner, but a weighted combination — for example, of many axis-aligned trees — can approximate a diagonal or curved boundary.

The essential ingredient is *diversity*: combining identical models achieves nothing. If B classifiers each have error rate $\varepsilon < 0.5$ and make *independent* errors, the majority vote is wrong only when more than half of them are wrong, and that probability, given by the binomial tail in Equation (1.32), falls rapidly as B grows. Figure 1.25 plots it. In reality errors are correlated, because the base learners see similar data, so the gains are smaller; ensemble methods are, in large part, strategies for *creating diversity* — different data samples, different features, different algorithms, or sequential focus on different examples.

$$P(\text{ensemble error}) = \sum_{k=\lfloor B/2 \rfloor + 1}^{B} \binom{B}{k} \varepsilon^{k} (1-\varepsilon)^{B-k} \tag{1.32}$$

where B is the (odd) number of independent base classifiers and ε the error rate of each.

E Worked Example 1.17 | Majority Vote of Three Classifiers

Three independent classifiers each have error $\varepsilon = 0.3$. The majority vote errs when at least two are wrong:

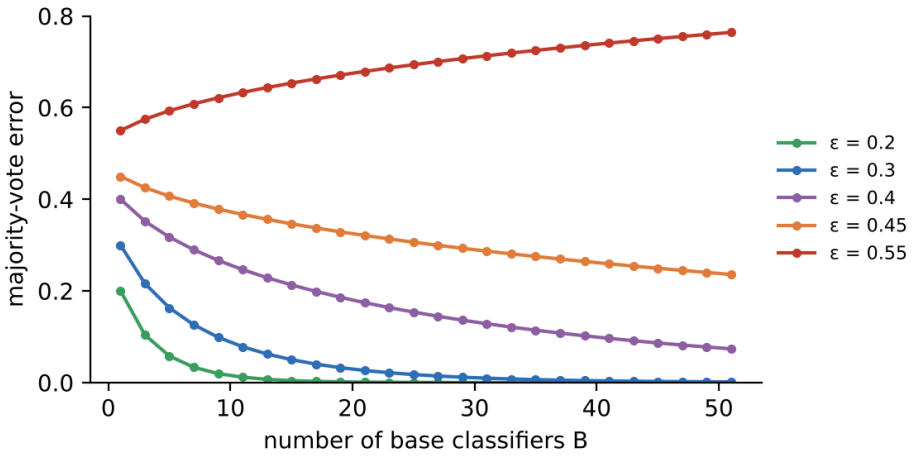


Figure 1.25 Error of a majority vote of B independent classifiers, each with error rate ε. Ensembles help when $\varepsilon < 0.5$ and hurt when $\varepsilon > 0.5$; real gains are smaller because errors are correlated.

- $P(\text{exactly 2 wrong}) = 3 \times 0.3^2 \times 0.7 = 3 \times 0.09 \times 0.7 = 0.189$.
- $P(\text{all 3 wrong}) = 0.3^3 = 0.027$.
- Ensemble error $= 0.189 + 0.027 =$ **0.216**, versus 0.3 for each member — a 28% relative reduction.

With five such classifiers: $P(\geq 3 \text{ wrong}) = 10(0.3^3)(0.7^2) + 5(0.3^4)(0.7) + 0.3^5 = 0.1323 + 0.0284 + 0.0024 =$ **0.163**. If the classifiers were perfectly correlated, the ensemble error would remain 0.3.

1.23.2 Combining Predictions and Families of Ensembles

Predictions can be combined by *hard voting* (majority of predicted labels), *soft voting* (averaging predicted class probabilities, which usually works better when the probabilities are reasonably calibrated), *weighted* averaging (giving better models more influence), or by a *learned* combiner, as in stacking. Ensembles fall into the families compared in Table 1.22. *Bagging* trains base learners independently and in parallel on bootstrap samples and mainly reduces variance. *Boosting* trains them sequentially, each focusing on the errors of its predecessors, and mainly reduces bias. *Stacking* trains a meta-model to combine heterogeneous base learners. These are developed in the next section.

Listing 1.12 Hard and soft voting ensembles of heterogeneous classifiers.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import VotingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
```

Table 1.22 The three main families of ensemble methods.

Aspect	Bagging	Boosting	Stacking
Training	Parallel, independent	Sequential, dependent	Base learners in parallel, then meta-learner
Diversity from	Bootstrap samples (and feature subsets)	Reweighting or fitting residuals	Different algorithms
Base learners	Strong, low-bias (deep trees)	Weak, high-bias (stumps, shallow trees)	Any, typically strong and diverse
Combination	Uniform vote or average	Weighted sum	Learned by a meta-model
Mainly reduces	Variance	Bias (and variance)	Both
Examples	Bagged trees, random forest	AdaBoost, gradient boosting, XGBoost	Stacked generalisation, blending
Overfitting risk	Low	Higher; needs early stopping	Moderate; needs out-of-fold predictions

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
members = [("lr", make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
           ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(7))),
           ("rf", RandomForestClassifier(n_estimators=300, random_state=0))]

for name, model in members + [("hard vote", VotingClassifier(members, voting="hard")),
                              ("soft vote", VotingClassifier(members, voting="soft"))]:
    print(f"{name:10s} CV accuracy = {cross_val_score(model, X, y, cv=5).mean():.3f}")
```

1.24 Bagging, Boosting, and Stacking

This section presents the three principal ensemble strategies in detail: bagging, which averages models trained on resampled data; boosting, which builds a strong model from a sequence of weak ones; and stacking, which learns how to combine diverse models. Particular attention is given to AdaBoost and gradient boosting, and to the modern gradient-boosting libraries XGBoost, LightGBM and CatBoost, which are the most widely used tools for structured, tabular data.

1.24.1 Bagging

Bagging (bootstrap aggregating, Breiman, 1996) trains B copies of a base learner, each on a bootstrap sample of the training set, and averages their predictions (or takes a majority vote). Because each bootstrap sample omits about 37% of the examples and duplicates others, the models differ, and averaging reduces variance according to Equation

(1.24). Bagging is most effective for *unstable*, low-bias, high-variance learners — deep decision trees above all — whose predictions change substantially with small perturbations of the data; it does little for stable learners such as linear regression or k-NN with large k. Bagging parallelises trivially and provides out-of-bag error estimates. The random forest is bagging of trees with added feature randomisation; variants include *pasting* (sampling without replacement) and *random patches* (sampling both rows and columns).

1.24.2 Boosting and AdaBoost

Boosting answers a question posed by Kearns and Valiant: can a collection of *weak learners* — classifiers only slightly better than random guessing — be combined into an arbitrarily accurate *strong learner*? Schapire (1990) proved that it can, and *AdaBoost* (adaptive boosting, Freund and Schapire, 1997) made the idea practical. AdaBoost trains weak learners — typically *decision stumps*, trees with a single split — sequentially. It maintains a weight on each training example; after each round, the weights of misclassified examples are increased so that the next learner concentrates on the hard cases. Each learner receives a vote weight α_m that grows as its weighted error ε_m shrinks. The final classifier is a weighted vote, as given in Equation (1.33) and Algorithm 1.8; Figure 1.26 illustrates the sequential reweighting.

$$\varepsilon_m = \sum_{i=1}^{n} w_i \, \mathbf{1}[h_m(\mathbf{x}_i) \neq y_i], \qquad \alpha_m = \tfrac{1}{2}\ln\tfrac{1-\varepsilon_m}{\varepsilon_m}$$
$$w_i \leftarrow \frac{w_i \exp\left(-\alpha_m \, y_i \, h_m(\mathbf{x}_i)\right)}{Z_m}, \qquad H(\mathbf{x}) = \operatorname{sign}\left(\sum_{m=1}^{M} \alpha_m h_m(\mathbf{x})\right) \tag{1.33}$$

where $y_i, h_m(x_i) \in \{-1, +1\}$, the weights w_i sum to one, and Z_m is the normalising constant.

Algorithm 1.8 AdaBoost (Discrete, Binary)

Input: Training set $\{(x_i, y_i)\}_{i=1}^{n}$ with $y_i \in \{-1, +1\}$; weak learning algorithm; number of rounds M

Output: Strong classifier $H(x) = \operatorname{sign}(\Sigma_m \alpha_m h_m(x))$

initialise weights $w_i \leftarrow 1/n$ **for** $i = 1, \ldots, n$
for $m = 1$ **to** M **do**
  fit weak learner h_m **to** the training data using weights w
  $\varepsilon_m \leftarrow \Sigma_i w_i \cdot 1[h_m(x_i) \neq y_i]$ ▷ *weighted error*
  if $\varepsilon_m \geq 0.5$ **then**
    break ▷ *no better than chance*
  $\alpha_m \leftarrow \tfrac{1}{2} \ln((1 - \varepsilon_m) / \varepsilon_m)$ ▷ *learner's vote weight*
  for $i = 1$ **to** n **do**

$w_i \leftarrow w_i \cdot \exp(-\alpha_m\, y_i\, h_m(x_i))$ ▷ *up-weight mistakes*
normalise w so that $\sum_i w_i = 1$
return $H(x) = \operatorname{sign}(\sum_m \alpha_m h_m(x))$

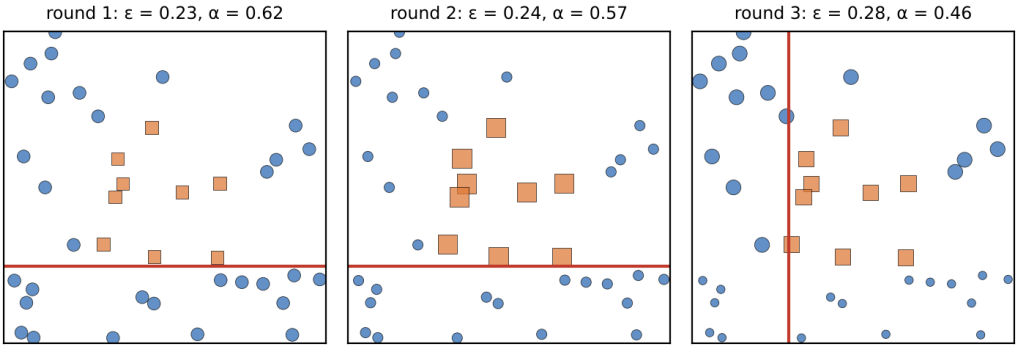


Figure 1.26 Boosting builds learners sequentially: each weak learner is trained on reweighted data emphasising the previous learners' mistakes (marker size $\propto$ weight), and the final model is a weighted vote.

E Worked Example 1.18 | One Round of AdaBoost

Ten training examples start with weights $w_i = 0.1$. The first stump misclassifies examples 3, 7 and 9.

- Weighted error $\varepsilon_1 = 0.1 + 0.1 + 0.1 = 0.3$.
- Vote weight $\alpha_1 = \tfrac{1}{2}\ln(0.7/0.3) = \tfrac{1}{2}\ln(2.333) = \tfrac{1}{2} \times 0.847 = \mathbf{0.424}$.
- Misclassified examples: $0.1 \times e^{0.424} = 0.1 \times 1.528 = 0.1528$. Correct examples: $0.1 \times e^{-0.424} = 0.1 \times 0.655 = 0.0655$.
- $Z_1 = 3 \times 0.1528 + 7 \times 0.0655 = 0.458 + 0.458 = 0.917$.
- New weights: misclassified $0.1528/0.917 = \mathbf{0.167}$; correct $0.0655/0.917 = \mathbf{0.0714}$.

After reweighting, the three mistakes carry half of the total weight ($3 \times 0.167 = 0.5$), so the previous stump now has weighted error exactly 0.5 — the next stump is forced to find a different split.

AdaBoost can be shown to perform stagewise minimisation of the *exponential loss* $\sum_i \exp(-y_i F(x_i))$, where F is the additive model (Friedman, Hastie and Tibshirani, 2000). This view explains both its power and its main weakness: the exponential loss grows very fast for badly misclassified points, so AdaBoost is sensitive to label noise and outliers. It also leads directly to the more general framework of gradient boosting.

1.24.3 Gradient Boosting

Gradient boosting (Friedman, 2001) generalises boosting to any differentiable loss function. It builds an additive model $F_M(x) = F_0(x) + \eta \Sigma_m h_m(x)$ in a forward stagewise manner, where each new base learner h_m — usually a small regression tree with 3 to 8 levels — is fitted to the *negative gradient* of the loss with respect to the current predictions, Equation (1.34). These negative gradients are called *pseudo-residuals*. For the squared loss $\tfrac{1}{2}(y - F)^2$, the negative gradient is simply the ordinary residual $y - F$, so each tree literally fits the errors of the ensemble so far. For other losses — absolute error, Huber, logistic loss for classification, Poisson for counts — the same machinery applies with different pseudo-residuals. Gradient boosting can thus be viewed as gradient descent in *function space*: each tree is a step in the direction that most decreases the loss. The full procedure is Algorithm 1.9, and Figure 1.27 shows the fit improving stage by stage.

$$r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F = F_{m-1}}$$
$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}), \qquad 0 < \eta \le 1 \tag{1.34}$$

where r_{im} is the pseudo-residual of example i at stage m, h_m a regression tree fitted to $\{(x_i, r_{im})\}$, and η the learning rate (shrinkage).

Algorithm 1.9 Gradient Boosting with Regression Trees

Input: Training set $\{(x_i, y_i)\}_{i=1}^{n}$; differentiable loss $L(y, F)$; number of stages M; learning rate η; tree depth J; optional subsample fraction s

Output: Additive model $F_M(x)$

$F_0(x) \leftarrow \arg\min_\gamma \Sigma_i L(y_i, \gamma)$ ▷ *e.g. mean of y for squared loss*
for $m = 1$ **to** M **do**
  optionally draw a random subsample of size $s \cdot n$ ▷ *stochastic gradient boosting*
  for $i = 1$ **to** n **do**
    $r_{im} \leftarrow -\partial L(y_i, F_{m-1}(x_i)) \,/\, \partial F_{m-1}(x_i)$ ▷ *pseudo-residuals*
  fit a regression tree h_m with at most J levels **to** $\{(x_i, r_{im})\}$
  for each leaf region R_{jm} of h_m **do**
    $\gamma_jm \leftarrow \arg\min_\gamma \Sigma_\{x_i \in R_{jm}\} L(y_i, F_{m-1}(x_i) + \gamma)$ ▷ *optimal leaf value*
  $F_m(x) \leftarrow F_{m-1}(x) + \eta \Sigma_j \gamma_jm \cdot 1[x \in R_{jm}]$
  if validation loss has **not** improved **for** p rounds **then**
    break ▷ *early stopping*
return F_M

Gradient boosting is powerful but can overfit, because each stage reduces training error further. It is regularised by four interacting mechanisms: *shrinkage* (a small learning rate η, typically 0.01–0.1, with correspondingly more trees), *shallow trees* (limiting in-

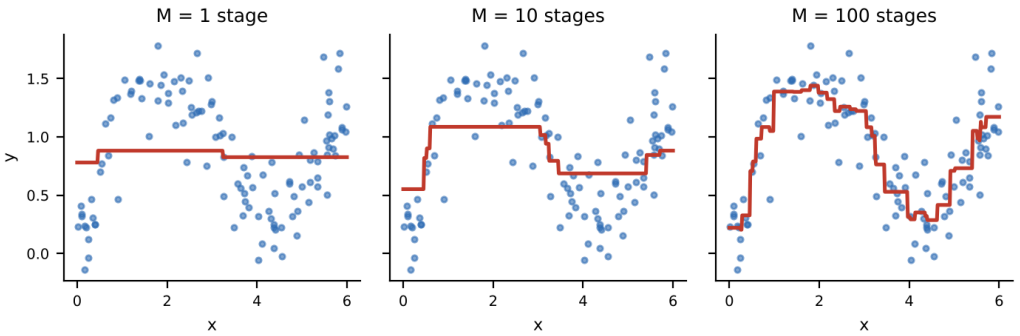


Figure 1.27 Gradient boosting for regression with depth-2 trees and learning rate 0.1: the ensemble prediction (red) after 1, 10 and 100 stages; each stage adds a small tree fitted to the current residuals.

teraction order), *subsampling* rows and columns at each stage (Friedman's stochastic gradient boosting, which also speeds training) and, above all, *early stopping* on a validation set, which chooses M automatically. A small learning rate with early stopping is the most reliable recipe.

E Worked Example 1.19 | Two Stages of Gradient Boosting (Squared Loss)

Targets $y = (3, 5, 9, 11)$ for $x = (1, 2, 3, 4)$; learning rate $\eta = 0.5$; base learners are stumps.

- **Stage 0.** $F_0 = \text{mean}(y) = 7$. Residuals $r = y - F_0 = (-4, -2, 2, 4)$.
- **Stage 1.** The best stump splits at $x \le 2.5$ with leaf values $\text{mean}(-4, -2) = -3$ and $\text{mean}(2, 4) = 3$. $F_1 = 7 + 0.5 \times (-3, -3, 3, 3) = (5.5, 5.5, 8.5, 8.5)$. New residuals $(-2.5, -0.5, 0.5, 2.5)$; MSE falls from $(16 + 4 + 4 + 16)/4 = 10$ to $(6.25 + 0.25 + 0.25 + 6.25)/4 = 3.25$.
- **Stage 2.** The best stump on the new residuals again splits at $x \le 2.5$ giving leaf values -1.5 and 1.5. $F_2 = (4.75, 4.75, 9.25, 9.25)$; residuals $(-1.75, 0.25, -0.25, 1.75)$; MSE $= (3.0625 + 0.0625 + 0.0625 + 3.0625)/4 = 1.5625$.

Further stages would split at $x \le 1.5$ or $x \le 3.5$ to fit the remaining within-group differences. The learning rate halves each correction, trading more stages for a smoother, better-generalising fit.

1.24.4 XGBoost, LightGBM and CatBoost

Three open-source libraries have made gradient boosting fast, scalable and dominant on tabular data; they are compared in Table 1.23. **XGBoost** (Chen and Guestrin, 2016) adds an explicit regularisation term on tree complexity to the objective, Equation (1.35), and uses a second-order Taylor expansion of the loss — gradients g_i and Hessians h_i — to derive the optimal leaf weight and a closed-form split-gain score. It also offers sparsity-aware handling of missing values, weighted quantile sketches for approximate splits, and efficient parallel and out-of-core computation. **LightGBM** (Ke et al., 2017) achieves large speed-ups through histogram-based split finding,

leaf-wise (best-first) tree growth instead of level-wise growth, *gradient-based one-side sampling* (GOSS), which keeps examples with large gradients and subsamples the rest, and *exclusive feature bundling* for sparse features. **CatBoost** (Prokhorenkova et al., 2018) focuses on categorical features, which it encodes with *ordered target statistics* computed only from preceding examples in a random permutation, and uses *ordered boosting* to avoid the prediction shift caused by reusing the same data for residuals and fitting; it builds *symmetric* (*oblivious*) trees that are fast to evaluate and robust with default settings.

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} L\big(y_i, \hat{y}_i^{(m-1)} + h_m(\mathbf{x}_i)\big) + \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$
$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{1.35}$$

where T is the number of leaves, w_j the weight (output) of leaf j, I_j the examples in leaf j, g_i and h_i the first and second derivatives of the loss at the current prediction, and γ, λ regularisation parameters.

Table 1.23 Comparison of the major gradient-boosting libraries.

Feature	XGBoost	LightGBM	CatBoost
Introduced	Chen & Guestrin, 2016	Ke et al. (Microsoft), 2017	Prokhorenkova et al. (Yandex), 2018
Tree growth	Level-wise (depth-wise) by default	Leaf-wise (best-first)	Symmetric (oblivious) trees
Split finding	Exact or histogram / quantile sketch	Histogram-based; GOSS sampling	Histogram-based
Categorical features	Native support in recent versions; often one-hot	Native (optimal partitioning)	Native, ordered target statistics
Missing values	Learned default direction	Learned default direction	Handled natively
Regularisation	L1/L2 on leaf weights, γ per leaf	L1/L2, min data per leaf, max leaves	L2, ordered boosting
Strengths	Mature, accurate, wide ecosystem	Fastest on large data; low memory	Best defaults; categorical data
Watch out for	Many hyperparameters	Overfitting on small data (limit num_leaves)	Slower training on CPU

1.24.5 Stacking

Stacking or *stacked generalisation* (Wolpert, 1992) combines heterogeneous base learners — say, a regularised logistic regression, a random forest, a gradient-boosting model and a k-NN classifier — by training a *meta-learner* on their predictions, as shown in Figure 1.28. The meta-learner learns which base model to trust in which circumstances.

The critical detail is that the meta-learner must be trained on predictions for examples the base models did *not* see during training; otherwise it would learn to trust whichever model overfits most. The standard procedure uses *k-fold out-of-fold predictions*: the training set is split into k folds, each base model is trained on k − 1 folds and predicts the held-out fold, and the assembled out-of-fold predictions form the meta-learner's training features. The base models are then refitted on the full training set to produce test-time inputs. *Blending* is a simpler variant that uses a single hold-out set. A simple, regularised meta-learner such as logistic or ridge regression is usually best.

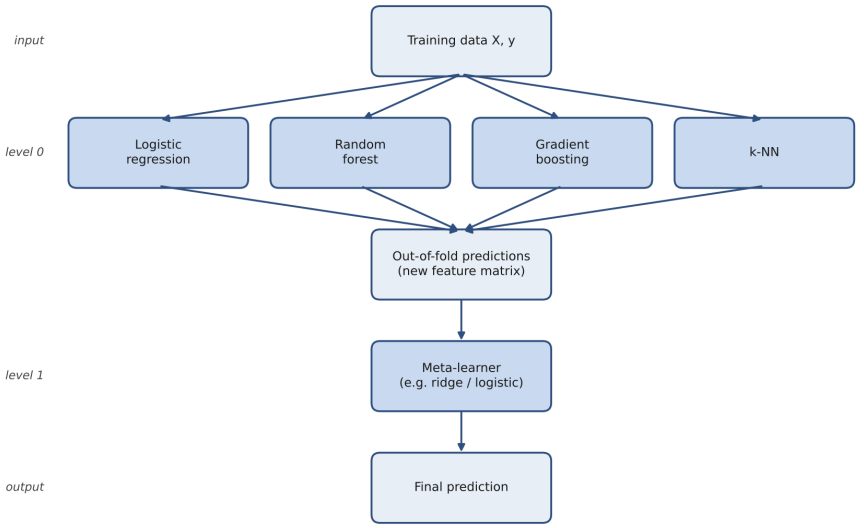


Figure 1.28 Stacking: diverse level-0 base models produce out-of-fold predictions, which become the input features of a level-1 meta-learner that outputs the final prediction.

Listing 1.13 AdaBoost, gradient boosting with early stopping, and stacking in scikit-learn.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import (AdaBoostClassifier, HistGradientBoostingClassifier,
                              RandomForestClassifier, StackingClassifier)
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, stratify=y, random_state=0)

ada = AdaBoostClassifier(n_estimators=200, learning_rate=0.5, random_state=0)   # stumps by default
gbm = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=1000, early_stopping=True,
```

```
                                    validation_fraction=0.15, n_iter_no_change=20,
                                    ↪  random_state=0)
stack = StackingClassifier(
    estimators=[("rf", RandomForestClassifier(n_estimators=300, random_state=0)),
                ("gbm", gbm),
                ("lr", make_pipeline(StandardScaler(),
                ↪  LogisticRegression(max_iter=1000)))],
    final_estimator=LogisticRegression(), cv=5)          # out-of-fold meta-features

for name, m in [("AdaBoost", ada), ("Gradient boosting", gbm), ("Stacking", stack)]:
    m.fit(X_tr, y_tr)
    print(f"{name:18s} test accuracy = {m.score(X_te, y_te):.3f}")
print("boosting stages used:", gbm.n_iter_)
# XGBoost / LightGBM / CatBoost offer the same fit/predict API:
#   xgboost.XGBClassifier, lightgbm.LGBMClassifier, catboost.CatBoostClassifier
```

C Case Study 1.2 | Ensembles in the Netflix Prize

From 2006 to 2009 Netflix offered US$1 million to any team that improved the root mean squared error of its film-rating predictor by 10%. The winning team, BellKor's Pragmatic Chaos, blended the predictions of more than a hundred models — matrix factorisations, neighbourhood methods, restricted Boltzmann machines and others — using gradient-boosted decision trees and linear blending. The runner-up achieved the same test score using a similar ensemble strategy. Two lessons endure: combining diverse models produces gains that no single model achieves; and the operational cost of maintaining a very large ensemble can outweigh small accuracy gains — Netflix reported that it did not deploy the full winning blend in production.

K Key Point | Key Points

- Bagging averages independently trained, high-variance models to reduce variance; random forests add feature randomisation to decorrelate the trees.
- Boosting fits weak learners sequentially to the mistakes of the ensemble: AdaBoost reweights examples; gradient boosting fits trees to the negative gradient of any differentiable loss.
- Regularise boosting with a small learning rate, shallow trees, subsampling and early stopping.
- XGBoost, LightGBM and CatBoost are fast, regularised gradient-boosting implementations and the default choice for tabular data.
- Stacking learns how to combine diverse models; always train the meta-learner on out-of-fold predictions.

1.25 Unsupervised Learning Algorithms (overview)

The supervised methods studied so far in this chapter — linear and logistic regression, decision trees, support vector machines and their ensembles — all learn a mapping from inputs to a known target. In many real projects, however, no target exists. A retailer holds millions of transaction records but nobody has labelled customers as "bargain hunters" or "loyal regulars"; a hospital stores thousands of gene-expression profiles without a ready-made disease taxonomy; a network operator logs billions of packets of which only a handful are ever confirmed as attacks. **Unsupervised learning** is the family of methods that extract structure from such unlabelled data.

D Definition 1.3 | Unsupervised Learning

Unsupervised learning is the task of inferring useful structure from a dataset $D = \{x_1, \ldots, x_n\}$ of feature vectors without associated target values. The structure may be a partition into groups (clustering), a low-dimensional representation (dimensionality reduction), an estimate of the data density $p(x)$, a set of co-occurrence rules (association rule mining), or a score of how unusual each point is (anomaly detection).

Because there is no ground truth, the learning objective must be defined from the data alone. Typical objectives are *compactness* (points in the same group should be close), *reconstruction* (a compressed representation should allow the input to be rebuilt), *likelihood* (a probabilistic model should assign high probability to observed data) and *frequency* (patterns that occur often are interesting). Table 1.24 summarises the main families covered in the following sections, together with the question each answers and a representative algorithm.

Table 1.24 Main families of unsupervised learning and the questions they answer.

Family	Question answered	Typical output	Representative methods
Clustering	Which observations belong together?	Cluster label per point	k-means, hierarchical, DBSCAN, GMM
Dimensionality reduction	Can the data be described with fewer variables?	Low-dimensional embedding	PCA, t-SNE, UMAP, autoencoders
Density estimation	How is the data distributed?	Estimated $p(x)$	Kernel density estimation, GMM
Association rule learning	Which items occur together?	Rules $X \rightarrow Y$ with support and confidence	Apriori, FP-Growth
Anomaly detection	Which observations are unusual?	Outlier score or flag	Isolation Forest, LOF, One-Class SVM

1.25.1 Why Unsupervised Learning Matters

Unlabelled data is cheap and abundant; labels are expensive, slow and sometimes impossible to obtain. Unsupervised methods therefore play three roles in practice. First,

they are **exploratory tools**: clustering and projection reveal segments, trends and errors that guide later modelling, extending the exploratory data analysis discussed earlier in this chapter. Second, they act as **preprocessing steps** for supervised learning: PCA removes redundant features, cluster memberships become new features, and anomaly detectors clean training sets. Third, they are **end products** in their own right — a fraud-alert system, a recommender built on co-purchase rules, or a customer-segmentation dashboard may involve no supervised model at all.

Figure 1.29 contrasts the same two-dimensional data seen with and without labels. In the supervised view, the analyst knows the class of each point and seeks a decision boundary. In the unsupervised view only the raw points are available, and an algorithm must discover that three groups exist — and decide where they begin and end.

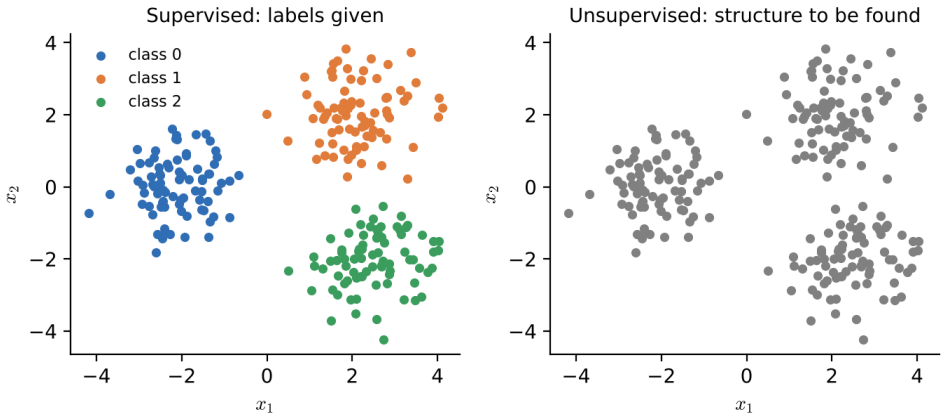


Figure 1.29 The same data viewed through supervised (left, labels known) and unsupervised (right, labels unknown) lenses; the unsupervised learner must discover the three groups from geometry alone.

1.25.2 Evaluating Without Labels

The absence of a target makes evaluation the hardest part of unsupervised learning. Three strategies are used. **Internal criteria** score a result using only the data — for example, the within-cluster sum of squares or the silhouette coefficient introduced in the k-means section. **External criteria** compare the result with labels that were withheld or that exist for a small subset, using measures such as the adjusted Rand index or normalised mutual information. **Task-based evaluation** measures whether the unsupervised output improves a downstream objective — whether PCA features raise classifier accuracy, or whether discovered customer segments respond differently to a marketing campaign. In practice all three should be combined with domain judgement, because an algorithm will always return *some* structure, even when the data contain

none.

> **Key Point | Unsupervised Results Are Hypotheses**
>
> A clustering or projection is a hypothesis about the data, not a fact. Always check stability (does the result change with a different random seed or subsample?), sensitivity to preprocessing (scaling, feature choice) and interpretability with a domain expert before acting on it.

1.26 Clustering Techniques

Clustering partitions a dataset into groups, called clusters, such that observations within a cluster are more similar to one another than to observations in other clusters. The definition sounds simple, yet it hides two choices that determine everything else: *what similarity means* and *what shape a cluster may take*. Different algorithms encode different answers, and no single algorithm is best for all data — a consequence of what Kleinberg (2002) formalised as an impossibility theorem for clustering: no clustering function can simultaneously satisfy three intuitive axioms of scale invariance, richness and consistency.

1.26.1 Measuring Similarity

Most clustering algorithms rely on a distance function $d(x_i, x_j)$. The general **Minkowski distance** of order p is given in Equation (1.36); $p = 2$ gives the familiar Euclidean distance and $p = 1$ the Manhattan (city-block) distance. For text and other high-dimensional sparse vectors, the angle between vectors is often more meaningful than their length, which motivates the cosine distance of Equation (1.37). Categorical attributes call for measures such as the Hamming distance or the Gower coefficient, which handles mixed numerical and categorical features.

$$d_p(\mathbf{x}_i, \mathbf{x}_j) = \left(\sum_{k=1}^{d} |x_{ik} - x_{jk}|^p \right)^{1/p} \tag{1.36}$$

where d is the number of features and $p \geq 1$ is the order of the distance.

$$d_{\cos}(\mathbf{x}_i, \mathbf{x}_j) = 1 - \frac{\mathbf{x}_i^\top \mathbf{x}_j}{\|\mathbf{x}_i\| \, \|\mathbf{x}_j\|} \tag{1.37}$$

> **Note | Scale Before You Cluster**
>
> Distance-based clustering is extremely sensitive to feature scale. If annual income is measured in rupees and age in years, income differences will dominate every distance and age will be ignored. Apply the standardisation or min–max scaling techniques described earlier in this chapter before clustering, unless the raw units are genuinely comparable.

1.26.2 Families of Clustering Algorithms

Clustering algorithms fall into five broad families, summarised in Table 1.25. **Partitional** methods such as k-means divide the data directly into k non-overlapping groups by optimising an objective. **Hierarchical** methods build a tree of nested clusters that can be cut at any level. **Density-based** methods such as DBSCAN define clusters as dense regions separated by sparse ones, allowing arbitrary shapes and explicit noise. **Model-based** methods, notably Gaussian mixture models (GMMs) fitted with the expectation–maximisation (EM) algorithm, assume the data are generated by a mixture of probability distributions and assign each point a *soft* membership probability. **Graph-based** methods such as spectral clustering build a similarity graph and partition it using the eigenvectors of its Laplacian, which lets them find non-convex clusters.

Table 1.25 Comparison of the main clustering families.

Family	Cluster shape	Needs k?	Handles noise?	Typical complexity
Partitional (k-means)	Convex, roughly spherical	Yes	No	$O(nkdT)$
Hierarchical (agglomerative)	Depends on linkage	No (cut tree)	Poorly	$O(n^2 \log n)$ to $O(n^3)$
Density-based (DBSCAN)	Arbitrary	No	Yes, labels noise	$O(n \log n)$ with index
Model-based (GMM)	Ellipsoidal	Yes	Partly	$O(nk\, d^2 T)$
Spectral	Arbitrary (graph-connected)	Yes	No	$O(n^3)$ eigen-decomposition

The practical consequence of these assumptions is visible in Figure 1.30, which applies four algorithms to two synthetic datasets. On well-separated blobs all methods succeed. On the interleaved "two moons" data, k-means and the Gaussian mixture cut each moon in half because they assume convex clusters, whereas single-linkage agglomerative clustering and DBSCAN follow the curved shapes.

1.26.3 Choosing a Clustering Method

Selecting an algorithm is a matter of matching its assumptions to the data and the question. If the number of groups is known and clusters are compact, k-means is fast

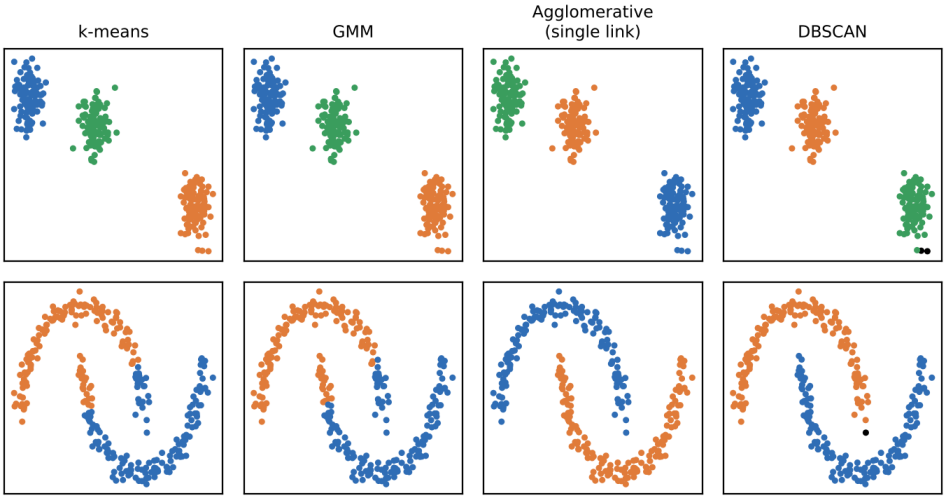


Figure 1.30 Four clustering algorithms on blob-shaped (top) and moon-shaped (bottom) data. Methods assuming convex clusters fail on the moons; density- and connectivity-based methods succeed.

and adequate. If a taxonomy or multi-level grouping is wanted, hierarchical clustering provides it naturally. If clusters have irregular shapes or the data contain noise, density-based methods are preferable. If uncertainty in membership matters — for example, a customer who is 60% "commuter" and 40% "leisure traveller" — a probabilistic mixture model is appropriate. The following three sections examine the three most widely used methods in detail.

1.27 k-Means Clustering

k-means is the most widely used clustering algorithm, valued for its simplicity, speed and interpretability. Its origins lie in signal quantisation: Stuart Lloyd devised the iterative procedure at Bell Labs in 1957 (published in 1982), and James MacQueen coined the name "k-means" in 1967. The algorithm represents each cluster by its *centroid* — the mean of its members — and assigns every point to the nearest centroid.

1.27.1 Objective Function

Given n points $x_1, \ldots, x_n \in \mathbb{R}^d$ and a chosen number of clusters k, k-means seeks a partition $C_1, \ldots, C_k$ and centroids $\mu_1, \ldots, \mu_k$ that minimise the **within-cluster sum of squares** (WCSS), also called *inertia*, defined in Equation (1.38). The objective rewards compact clusters: every point contributes the squared Euclidean distance to its own centroid.

$$J(C, \mu) = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \|\mathbf{x}_i - \mu_j\|^2 \tag{1.38}$$

where C_j is the set of points in cluster j and $\mu_j \in \mathbb{R}^d$ is its centroid.

Minimising Equation (1.38) exactly is NP-hard even for $k = 2$ in general dimension, so k-means uses a heuristic: alternate between optimising the assignments with centroids fixed and optimising the centroids with assignments fixed. Each sub-problem has a closed-form solution, given in Equation (1.39). With centroids fixed, the best assignment sends each point to the nearest centroid; with assignments fixed, setting the gradient of J with respect to μ_j to zero shows that the best centroid is the arithmetic mean of the cluster's members.

$$\begin{aligned} c_i &= \arg\min_{j \in \{1,\dots,k\}} \|\mathbf{x}_i - \mu_j\|^2 \quad \text{(assignment)} \\ \mu_j &= \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i \quad \text{(update)} \end{aligned} \tag{1.39}$$

where c_i is the cluster index of point i and $|C_j|$ is the number of points in cluster j.

1.27.2 Lloyd's Algorithm

Alternating the two steps of Equation (1.39) gives **Lloyd's algorithm**, set out in Algorithm 1.10. Neither step can increase J, and there are only finitely many partitions, so the algorithm is guaranteed to converge — but only to a *local* minimum that depends on the initial centroids. Figure 1.31 shows the algorithm at work: poorly placed initial centroids migrate towards the centres of the three groups within a few iterations.

Algorithm 1.10 Lloyd's Algorithm for k-Means

Input: Data $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$; number of clusters k; tolerance ε; maximum iterations T
Output: Cluster assignments $c_1, \dots, c_n$ and centroids $\mu_1, \dots, \mu_k$

initialise $\mu_1, \dots, \mu_k$ (randomly chosen data points **or** k-means++)
for t = 1 **to** T **do**
  for i = 1 **to** n **do** ▷ *assignment step*
    $c_i \leftarrow \arg\min_j \|x_i - \mu_j\|^2$
  for j = 1 **to** k **do** ▷ *update step*
    $C_j \leftarrow \{x_i : c_i = j\}$
    if C_j is empty **then** re-seed μ_j with the point farthest from its centroid
    else $\mu_j \leftarrow (1/|C_j|) \sum_{x_i \in C_j} x_i$
  if no assignment changed **or** decrease **in** J < ε **then break**
return c, μ

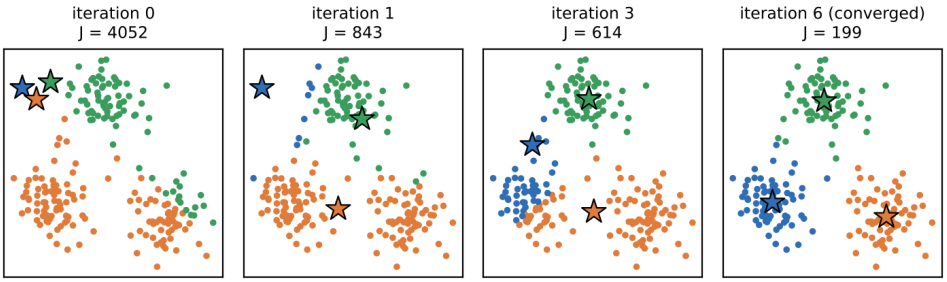


Figure 1.31 Lloyd's algorithm on three groups: centroids (stars) start bunched in one corner and migrate to the group centres over successive iterations, while the inertia J falls monotonically until convergence.

E Worked Example 1.20 | k-Means by Hand

Cluster the one-dimensional data $\{1, 2, 4, 7, 8, 10\}$ into $k = 2$ groups, starting from $\mu_1 = 1$ and $\mu_2 = 4$.

- **Iteration 1, assignment:** 1 and 2 are closer to μ_1; 4, 7, 8, 10 are closer to μ_2.
- **Iteration 1, update:** $\mu_1 = (1 + 2)/2 = 1.5$; $\mu_2 = (4 + 7 + 8 + 10)/4 = 7.25$.
- **Iteration 2, assignment:** point 4 is now 2.5 from μ_1 and 3.25 from μ_2, so it moves to cluster 1.
- **Iteration 2, update:** $\mu_1 = (1 + 2 + 4)/3 \approx 2.33$; $\mu_2 = (7 + 8 + 10)/3 \approx 8.33$.
- **Iteration 3:** no assignment changes, so the algorithm has converged.

Final inertia: cluster 1 contributes $(1-2.33)^2 + (2-2.33)^2 + (4-2.33)^2 \approx 1.78 + 0.11 + 2.78 = 4.67$; cluster 2 contributes $(7-8.33)^2 + (8-8.33)^2 + (10-8.33)^2 \approx 4.67$; so $J \approx 9.33$.

1.27.3 Initialisation with k-Means++

Because Lloyd's algorithm only finds a local optimum, initialisation matters. Choosing k random data points can place two centroids in the same natural group and none in another, yielding a poor solution. **k-means++** (Arthur and Vassilvitskii, 2007) spreads the initial centroids out: the first is drawn uniformly at random and each subsequent centroid is drawn with probability proportional to the squared distance from the nearest centroid already chosen, as in Equation (1.40). The procedure is given in Algorithm 1.11. It is provably $O(\log k)$-competitive with the optimal WCSS in expectation, and in practice it reduces both the final inertia and the number of Lloyd iterations. Scikit-learn uses it by default and additionally runs several initialisations (`n_init`), keeping the best.

$$P(\mathbf{x}_i \text{ chosen}) = \frac{D(\mathbf{x}_i)^2}{\sum_{l=1}^{n} D(\mathbf{x}_l)^2} \tag{1.40}$$

where $D(x_i)$ is the distance from x_i to the nearest centroid selected so far.

Algorithm 1.11 k-Means++ Seeding

Input: Data $X = \{x_1, \ldots, x_n\}$; number of clusters k
Output: Initial centroids $\mu_1, \ldots, \mu_k$

```
choose μ_1 uniformly at random from X
for j = 2 to k do
    for i = 1 to n do
        D(x_i) ← min over already chosen centroids μ of ‖x_i − μ‖
    sample μ_j from X with probability D(x_i)^2 / Σ_l D(x_l)^2
return μ_1, ..., μ_k
```

1.27.4 Choosing k: the Elbow Method and the Silhouette Coefficient

k-means requires the number of clusters in advance. Since WCSS always decreases as k grows — reaching zero when every point is its own cluster — it cannot be minimised directly over k. The **elbow method** plots WCSS against k and looks for the point at which further increases yield diminishing returns. The **silhouette coefficient** (Rousseeuw, 1987) offers a more principled internal criterion. For each point i, let $a(i)$ be the mean distance to the other members of its own cluster and $b(i)$ the mean distance to the members of the nearest other cluster. The silhouette of point i is defined in Equation (1.41); it ranges from -1 (probably misassigned) through 0 (on a boundary) to $+1$ (well clustered). The average silhouette over all points is maximised to select k.

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}} \tag{1.41}$$

where $a(i)$ is the mean intra-cluster distance of point i and $b(i)$ is its mean distance to the nearest neighbouring cluster.

Figure 1.32 applies both criteria to data generated from four groups. The elbow is visible at $k = 4$, and the mean silhouette peaks at the same value. In real data the two criteria often disagree or the elbow is indistinct; the gap statistic (Tibshirani, Walther and Hastie, 2001) and information criteria for Gaussian mixtures are useful additional tools, and domain interpretability should have the final word.

1.27.5 Implementation, Variants and Limitations

Listing 1.14 shows a typical scikit-learn workflow: scale the features, sweep k, inspect the silhouette and fit the final model. Note the use of a pipeline so that scaling is applied consistently to new data.

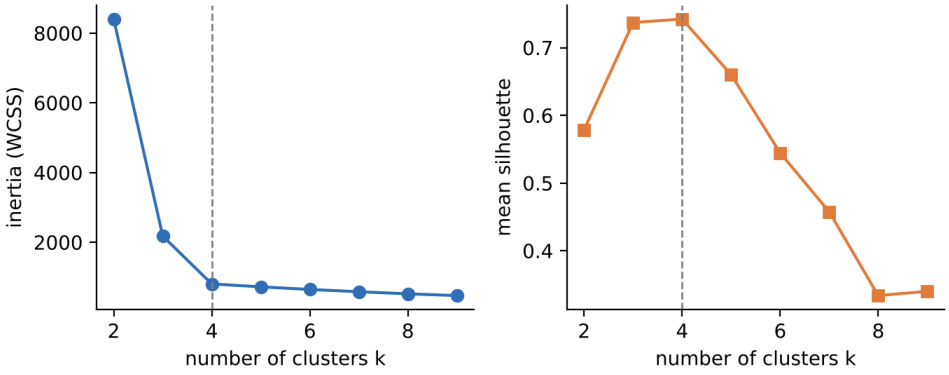


Figure 1.32 Choosing k for data drawn from four groups: the elbow in the inertia curve (left) and the peak of the mean silhouette coefficient (right) both indicate k = 4.

Listing 1.14 Selecting k with the silhouette score and fitting k-means in scikit-learn.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X, _ = make_blobs(n_samples=600, centers=4, random_state=42)

scores = {}
for k in range(2, 9):
    model = make_pipeline(StandardScaler(),
                          KMeans(n_clusters=k, init="k-means++", n_init=10,
                          ↪  random_state=0))
    labels = model.fit_predict(X)
    Xs = model[:-1].transform(X)            # scaled data used by k-means
    scores[k] = silhouette_score(Xs, labels)

best_k = max(scores, key=scores.get)
final = make_pipeline(StandardScaler(), KMeans(n_clusters=best_k, n_init=10,
↪  random_state=0)).fit(X)
print("best k:", best_k, "silhouette:", round(scores[best_k], 3))
print("centroids (scaled space):\n", final[-1].cluster_centers_)
print("cluster of a new point:", final.predict([[0.0, 2.0]]))
```

Several variants address specific weaknesses. **Mini-batch k-means** updates centroids from small random batches and scales to millions of points. **k-medoids** (PAM) restricts centres to actual data points and can use any dissimilarity, making it more robust to outliers. **Kernel k-means** applies the kernel trick, introduced with support vector machines, to find non-linear cluster boundaries. Table 1.26 summarises the strengths and limitations of the standard algorithm.

Table 1.26 Strengths and limitations of k-means.

Strengths	Limitations
Simple, fast: linear in n per iteration	k must be specified in advance
Scales to large datasets (mini-batch variant)	Assumes convex, similarly sized, isotropic clusters
Centroids are interpretable prototypes	Sensitive to initialisation and to outliers
Guaranteed convergence	Converges to a local optimum only
Easy to assign new points to clusters	Requires numerical features and meaningful Euclidean distance

1.28 Hierarchical Clustering

Partitional methods return a single flat grouping. Many domains, however, are naturally hierarchical: species form genera and families, documents form topics and subtopics, and customers form segments within segments. **Hierarchical clustering** produces a nested sequence of partitions that captures this multi-level structure and does not require the number of clusters to be fixed in advance.

1.28.1 Agglomerative and Divisive Strategies

There are two strategies. **Agglomerative** (bottom-up) clustering starts with every point as its own cluster and repeatedly merges the two closest clusters until a single cluster remains. **Divisive** (top-down) clustering starts with all points in one cluster and recursively splits it; the DIANA algorithm and bisecting k-means are examples. Agglomerative clustering is far more common in practice, and its general form is given in Algorithm 1.12.

Algorithm 1.12 Agglomerative Hierarchical Clustering

Input: Data $X = \{x_1, \ldots, x_n\}$; distance function d; linkage criterion L
Output: Merge history (dendrogram) of $n - 1$ merges with their heights

create n singleton clusters $C_i = \{x_i\}$
compute the $n \times n$ matrix of pairwise cluster distances
for $m = 1$ **to** $n - 1$ **do**
    find the pair (C_a, C_b) with the smallest linkage distance $L(C_a, C_b)$
    merge them into $C_{new} = C_a \cup C_b$; record $(a, b, L(C_a, C_b))$
    remove C_a **and** C_b; compute $L(C_{new}, C_j)$ **for** every remaining cluster C_j
return the list of merges

1.28.2 Linkage Criteria

The behaviour of agglomerative clustering is governed by the **linkage criterion**, which defines the distance between two clusters A and B in terms of point-wise distances. The four standard criteria are given in Equation (1.42). **Single linkage** uses the closest pair and can follow elongated, chain-like shapes but suffers from *chaining*, where a thin bridge of points merges two otherwise distinct groups. **Complete linkage** uses the farthest pair and produces compact clusters of similar diameter. **Average linkage** (UPGMA) is a compromise. **Ward's method** (Ward, 1963) merges the pair whose union causes the smallest increase in total within-cluster variance, and therefore optimises the same objective as k-means in a greedy, hierarchical way.

$$
\begin{aligned}
L_{\text{single}}(A,B) &= \min_{\mathbf{a}\in A,\ \mathbf{b}\in B} d(\mathbf{a},\mathbf{b}) \\
L_{\text{complete}}(A,B) &= \max_{\mathbf{a}\in A,\ \mathbf{b}\in B} d(\mathbf{a},\mathbf{b}) \\
L_{\text{average}}(A,B) &= \frac{1}{|A|\,|B|}\sum_{\mathbf{a}\in A}\sum_{\mathbf{b}\in B} d(\mathbf{a},\mathbf{b}) \\
L_{\text{Ward}}(A,B) &= \frac{|A|\,|B|}{|A|+|B|}\,\lVert \mu_A - \mu_B \rVert^2
\end{aligned}
\tag{1.42}
$$

where μ_A and μ_B are the centroids of clusters A and B.

Table 1.27 Properties of common linkage criteria.

Linkage	Cluster shape favoured	Outlier sensitivity	Typical use
Single	Elongated, chained	Low for merging, prone to chaining	Detecting non-convex or filamentary structure
Complete	Compact, equal diameter	High (max distance)	Well-separated compact groups
Average	Intermediate	Moderate	Phylogenetics (UPGMA), general use
Ward	Spherical, similar size	Moderate	Default for numerical data; mirrors k-means

E Worked Example 1.21 | Single vs Complete Linkage

Four points lie on a line at A = 0, B = 1, C = 4 and D = 6. Pairwise distances: AB = 1, AC = 4, AD = 6, BC = 3, BD = 5, CD = 2.

- **Merge 1 (both linkages):** the smallest distance is AB = 1, so {A, B} is formed at height 1.
- **Merge 2:** single linkage gives d({A,B}, C) = min(4, 3) = 3 and d({A,B}, D) = min(6, 5) = 5; complete linkage gives max(4, 3) = 4 and max(6, 5) = 6. In both cases CD = 2 is smallest, so {C, D} is formed at height 2.
- **Merge 3:** single linkage joins {A,B} and {C,D} at min(AC, AD, BC, BD) = 3; complete

linkage at max = 6; average linkage at $(4 + 6 + 3 + 5)/4 = 4.5$.

The tree topology is identical here, but the merge heights differ — and in the dendrogram those heights determine where a horizontal cut produces two clusters.

1.28.3 The Dendrogram

The result of hierarchical clustering is displayed as a **dendrogram**, a tree in which leaves are data points and the height of each junction equals the linkage distance at which two clusters merged. Cutting the dendrogram with a horizontal line at height h yields a flat clustering whose clusters are the subtrees below the cut. A long vertical gap between successive merges suggests a natural number of clusters. Figure 1.33 shows a dendrogram built with Ward linkage for twenty points drawn from three groups; a cut at height 6 recovers the three groups.

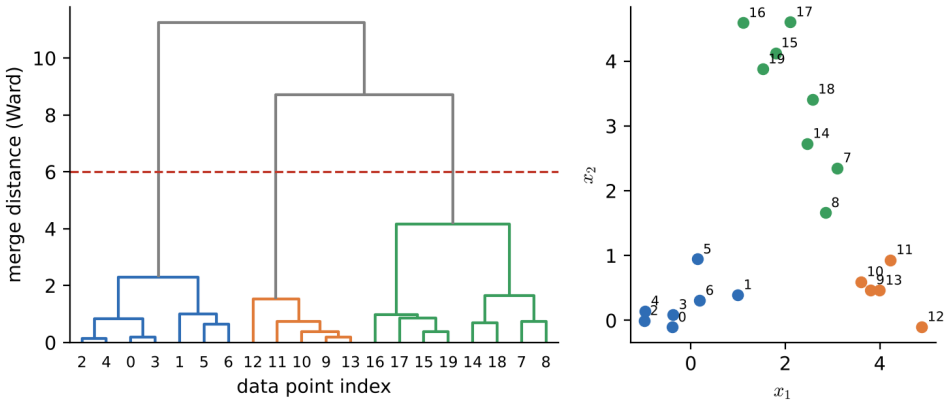


Figure 1.33 Ward-linkage dendrogram of 20 points from three groups (left) and the resulting clusters (right). The dashed cut at height 6 crosses three branches, giving three clusters.

1.28.4 Implementation and Practical Considerations

SciPy provides efficient agglomerative clustering and dendrogram plotting, as illustrated in Listing 1.15; scikit-learn's `AgglomerativeClustering` offers the same capability within its estimator API, including connectivity constraints that restrict merges to spatial neighbours (useful for image segmentation). The standard algorithm needs $O(n^2)$ memory for the distance matrix and between $O(n^2 \log n)$ and $O(n^3)$ time, so it is practical for up to a few tens of thousands of points. For larger datasets, BIRCH (Zhang, Ramakrishnan and Livny, 1996) summarises the data in a compact clustering-feature tree before clustering, and HDBSCAN combines hierarchical and density-based ideas.

Listing 1.15 Agglomerative clustering, dendrogram and flat cut with SciPy.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import pdist
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

X = StandardScaler().fit_transform(load_iris().data)

D = pdist(X, metric="euclidean")          # condensed distance matrix
Z = linkage(D, method="average")          # try "single", "complete", "ward" (Ward
↪  needs raw X)

plt.figure(figsize=(8, 3))
dendrogram(Z, truncate_mode="lastp", p=30)   # show only the last 30 merges
plt.ylabel("average-linkage distance")
plt.show()

labels = fcluster(Z, t=3, criterion="maxclust")  # cut to obtain exactly 3 clusters
print(np.bincount(labels))
```

Practice Tip | Reading a Dendrogram Correctly

The horizontal order of leaves in a dendrogram is partly arbitrary — any branch can be flipped without changing the tree. Judge similarity only by the height at which two points or clusters join, never by how close their leaves appear along the horizontal axis.

1.29 DBSCAN and Density-Based Clustering

k-means and Ward-linkage clustering assume that clusters are compact blobs, and they force every point into some cluster. Real data often violate both assumptions: GPS traces of vehicles form winding roads, astronomical surveys reveal filaments of galaxies, and sensor logs contain stray readings that belong to no group. **Density-based clustering** defines a cluster as a region of high point density separated from other such regions by areas of low density. Points in sparse areas are labelled *noise* rather than assigned to a cluster.

1.29.1 Core, Border and Noise Points

The most influential density-based algorithm is **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise), proposed by Ester, Kriegel, Sander and Xu in 1996. It has two hyperparameters: a radius ε (`eps`) and a minimum count `minPts`. The

ε-neighbourhood of a point is defined in Equation (1.43). Using it, DBSCAN classifies every point into one of three types, defined below and illustrated in Figure 1.34.

$$N_\varepsilon(\mathbf{x}_i) = \{\mathbf{x}_j \in X : d(\mathbf{x}_i, \mathbf{x}_j) \le \varepsilon\} \tag{1.43}$$

where $\varepsilon > 0$ is the neighbourhood radius and d is a distance function (usually Euclidean). The neighbourhood includes $\mathbf{x}_i$ itself.

Definition 1.4 | DBSCAN Point Types and Reachability

- A **core point** has at least minPts points in its ε-neighbourhood: $|N_\varepsilon(x)| \ge \text{minPts}$.
- A **border point** is not a core point but lies within the ε-neighbourhood of some core point.
- A **noise point** is neither core nor border.
- A point q is **directly density-reachable** from a core point p if $q \in N_\varepsilon(p)$. q is **density-reachable** from p if there is a chain of core points $p = p_1, p_2, \ldots, p_m, q$ with each step directly density-reachable.
- Two points are **density-connected** if both are density-reachable from a common core point. A cluster is a maximal set of density-connected points.

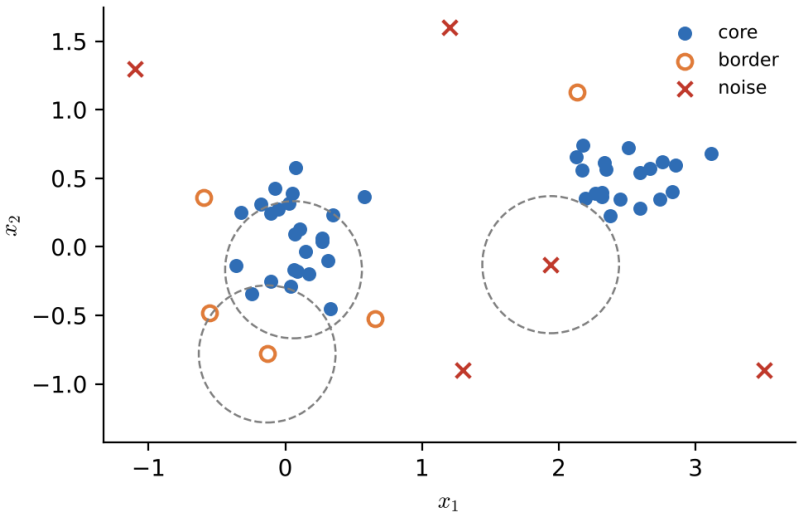


Figure 1.34 DBSCAN point types for $\varepsilon = 0.5$ and $\text{minPts} = 4$: core points (filled) have at least four neighbours within ε, border points (open) lie within ε of a core point, and noise points (crosses) are isolated. Circles show ε-neighbourhoods of selected points.

1.29.2 The DBSCAN Algorithm

DBSCAN grows clusters outward from core points, as described in Algorithm 1.13. Starting from an unvisited core point, it creates a new cluster and repeatedly adds all

points in the ε-neighbourhoods of the cluster's core points. Border points are absorbed but do not extend the cluster further. Points never reached from any core point remain noise. The result is deterministic for core and noise points; a border point that lies within ε of core points in two different clusters is assigned to whichever cluster reaches it first, which depends on processing order.

Algorithm 1.13 DBSCAN

Input: Data $X = \{x_1, \ldots, x_n\}$; radius ε; minimum neighbours minPts
Output: Cluster label for every point (-1 denotes noise)

```
label(x_i) ← undefined for all i; C ← 0
for each point p in X do
    if label(p) is defined then continue
    N ← RangeQuery(p, ε)
    if |N| < minPts then label(p) ← −1; continue          ▷ provisional noise
    C ← C + 1; label(p) ← C
    S ← N \ {p}                                           ▷ seed set to expand
    for each q in S do
        if label(q) = −1 then label(q) ← C                ▷ noise becomes border
        if label(q) is defined then continue
        label(q) ← C
        N_q ← RangeQuery(q, ε)
        if |N_q| ≥ minPts then S ← S ∪ N_q                ▷ q is core: expand
return labels
```

Each point triggers one range query. With a spatial index such as a k-d tree or ball tree, a range query costs roughly $O(\log n)$ in low dimensions, giving an overall $O(n \log n)$ average running time; without an index, or in high dimensions where indexes degrade, the cost is $O(n^2)$.

1.29.3 Choosing ε and minPts

A common heuristic for minPts is $2d$, where d is the number of features, with a minimum of about 4 for two-dimensional data; larger values make the algorithm more robust to noise. For ε, the **k-distance plot** is the standard tool: compute for each point the distance to its k-th nearest neighbour ($k = \text{minPts} - 1$), sort these values in ascending order and plot them. Points inside clusters have small k-distances, while noise points have large ones, so the curve shows a "knee" where it bends sharply upward. The ε at the knee separates the two regimes. Listing 1.16 applies this procedure.

Listing 1.16 Choosing ε with a k-distance plot and running DBSCAN.

```python
import numpy as np
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

X, _ = make_moons(n_samples=500, noise=0.07, random_state=0)
X = StandardScaler().fit_transform(X)

min_pts = 5
nn = NearestNeighbors(n_neighbors=min_pts).fit(X)
dist, _ = nn.kneighbors(X)                  # column 0 is the point itself
k_dist = np.sort(dist[:, -1])               # distance to (minPts-1)-th neighbour
eps = np.quantile(k_dist, 0.95)             # or read the knee from a plot of k_dist

db = DBSCAN(eps=eps, min_samples=min_pts).fit(X)
labels = db.labels_
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print(f"eps={eps:.3f}  clusters={n_clusters}  noise points={np.sum(labels == -1)}")
print("core points:", len(db.core_sample_indices_))
```

1.29.4 Strengths, Limitations and Extensions

Table 1.28 summarises DBSCAN's properties. Its main weakness is the single global density threshold: when clusters have very different densities, no single ε separates them all. **OPTICS** (Ankerst et al., 1999) addresses this by computing an ordering of points together with a *reachability distance*, from which clusters at all density levels can be extracted. **HDBSCAN** (Campello, Moulavi and Sander, 2013) builds a hierarchy of DBSCAN solutions across all ε values and selects the most stable clusters, requiring only a minimum cluster size; it is now often the recommended default for exploratory density-based clustering. **Mean shift** takes a different route, moving each point uphill on a kernel density estimate until it reaches a mode, and defining clusters as the basins of attraction of these modes.

Table 1.28 Strengths and limitations of DBSCAN.

Strengths	Limitations
Finds clusters of arbitrary shape	Struggles when cluster densities differ greatly
Number of clusters is not required	Sensitive to the choice of ε and minPts
Explicitly identifies noise and outliers	Distance becomes less meaningful in high dimensions
Deterministic for core and noise points	Border-point assignment depends on processing order
Efficient with spatial indexing	Requires feature scaling for meaningful ε

Case Study 1.3 | Density-Based Clustering of GPS Data

Ride-hailing and logistics companies collect millions of GPS pings from drivers. Clustering the locations where vehicles stop for long periods with DBSCAN (using the haversine distance on latitude–longitude) reveals popular pick-up points, depots and congestion hotspots, whatever their shape. Isolated pings caused by GPS errors are naturally labelled as noise instead of distorting the clusters — something k-means cannot do.

1.30 Dimensionality Reduction

Modern datasets frequently describe each observation with hundreds or thousands of features: pixel intensities of an image, expression levels of 20,000 genes, or term frequencies over a large vocabulary. **Dimensionality reduction** maps data from a high-dimensional space $\mathbb{R}^{\mathrm{d}}$ to a lower-dimensional space $\mathbb{R}^{\mathrm{m}}$ ($\mathrm{m} \ll \mathrm{d}$) while preserving as much of the relevant structure as possible. It is used to visualise data in two or three dimensions, to compress data, to remove noise and redundancy, and to make subsequent learning faster and more reliable.

1.30.1 The Curse of Dimensionality

The phrase **curse of dimensionality**, coined by Richard Bellman in 1957, refers to the collection of phenomena that make high-dimensional spaces counter-intuitive and data-hungry. Three are especially important for machine learning. First, *volume grows exponentially*: to cover the unit hypercube $[0,1]^{\mathrm{d}}$ with a grid of spacing 0.1 needs 10^{d} cells, so a fixed sample becomes ever sparser as d grows. Second, *most of the volume lies near the surface*: the fraction of a unit hypercube's volume that lies within a thin shell of width δ at its boundary is $1-(1-2\delta)^{\mathrm{d}}$, which approaches 1 as d increases. Third, and most damaging for distance-based methods such as k-NN and k-means, *distances concentrate*: the ratio between the farthest and nearest neighbour distances tends to 1, as expressed in Equation (1.44), so the notion of a "nearest" neighbour loses meaning.

$$\lim_{d\to\infty} \frac{\max_j d(\mathbf{q},\mathbf{x}_j) - \min_j d(\mathbf{q},\mathbf{x}_j)}{\min_j d(\mathbf{q},\mathbf{x}_j)} = 0 \tag{1.44}$$

where q is a query point and $\mathrm{x_j}$ are data points drawn independently from a broad class of distributions (Beyer et al., 1999).

Figure 1.35 demonstrates distance concentration empirically. For uniformly random points, the relative contrast between the farthest and nearest neighbour of a query point collapses as the dimension rises from 2 to 1,000. Fortunately, real data rarely fill

their ambient space uniformly. The **manifold hypothesis** states that high-dimensional real-world data tend to lie on or near a manifold of much lower intrinsic dimension — images of a face, for example, vary mainly with a few factors such as pose, lighting and expression. Dimensionality reduction aims to discover such low-dimensional coordinates.

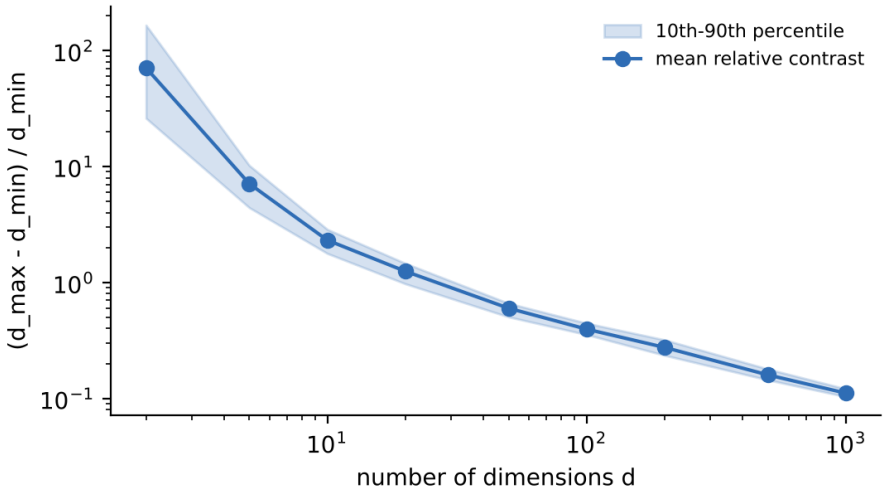


Figure 1.35 Distance concentration: for 500 uniformly random points, the relative contrast $(d_{max} - d_{min})/d_{min}$ between a query's farthest and nearest neighbour shrinks rapidly as dimension grows.

1.30.2 Feature Selection versus Feature Extraction

There are two ways to reduce dimension. **Feature selection** keeps a subset of the original features (using filter, wrapper or embedded methods such as L1 regularisation, discussed earlier in this chapter), preserving their interpretability. **Feature extraction** constructs new features as functions of all the original ones. Extraction methods are either *linear*, producing new features that are weighted sums of the originals, or *nonlinear*, able to unfold curved manifolds.

1.30.3 Linear and Non-Linear Methods

Principal component analysis (PCA), treated in depth in the next section, finds orthogonal directions of maximum variance. **Linear discriminant analysis** (LDA) is a supervised linear method that instead maximises the separation between known classes relative to within-class scatter, yielding at most (number of classes $-$ 1) components. **Independent component analysis** (ICA) seeks statistically independent, non-

Gaussian components and is used to separate mixed signals, as in the classic "cocktail-party" problem. **Non-negative matrix factorisation** (NMF) constrains components and weights to be non-negative, giving parts-based representations of images and topics in text. **Random projection**, justified by the Johnson–Lindenstrauss lemma, multiplies the data by a random matrix and approximately preserves pairwise distances at very low cost.

Non-linear methods, often called *manifold learning*, include classical techniques such as **kernel PCA** (PCA in a kernel-induced feature space), **Isomap** (Tenenbaum, de Silva and Langford, 2000), which preserves geodesic distances along a neighbourhood graph, and **locally linear embedding** (Roweis and Saul, 2000), which preserves local linear reconstructions. Two methods dominate modern visualisation. **t-distributed stochastic neighbour embedding** (t-SNE; van der Maaten and Hinton, 2008) converts pairwise distances into neighbour probabilities in both the original and embedded space and minimises the Kullback–Leibler divergence between them, as in Equation (1.45). Using a heavy-tailed Student-t distribution in the low-dimensional space alleviates the "crowding problem" and produces well-separated clusters. **UMAP** (uniform manifold approximation and projection; McInnes, Healy and Melville, 2018) builds a fuzzy nearest-neighbour graph grounded in Riemannian geometry and topology, then optimises a low-dimensional layout with a cross-entropy objective. UMAP is usually faster than t-SNE, scales to millions of points, preserves more global structure and can embed new points.

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2 / 2\sigma_i^2)}, \qquad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}, \qquad C = \mathrm{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{1.45}$$

where $\mathbf{y}_i$ is the low-dimensional embedding of $\mathbf{x}_i$ and σ_i is chosen so that the conditional distribution has a user-specified perplexity (typically 5–50).

Figure 1.36 contrasts a linear and a non-linear method on the 8 × 8 handwritten digits dataset (64 dimensions). PCA's two leading components separate some digits but leave many overlapping; t-SNE reveals ten well-separated groups. Table 1.29 compares the main methods.

> **Note | Interpreting t-SNE and UMAP Plots**
>
> Cluster sizes and inter-cluster distances in t-SNE plots are largely meaningless, and the picture can change markedly with perplexity or random seed. Use these embeddings to *explore* and *visualise*, not as input features for distance-based models or as evidence of a specific number of clusters. Always try several perplexity (t-SNE) or n_neighbors (UMAP) values.

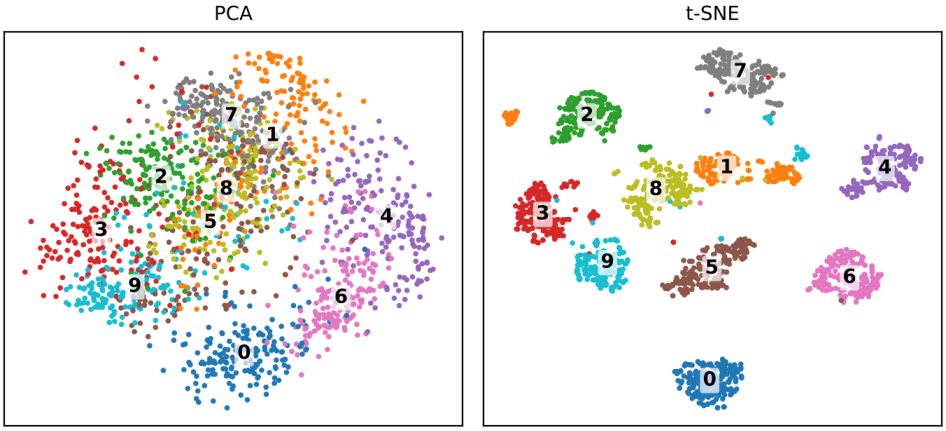


Figure 1.36 Two-dimensional embeddings of the 64-dimensional handwritten digits data: PCA (left) preserves global variance but overlaps classes, whereas t-SNE (right) separates the ten digits into distinct clusters. Colours show the true digit, which neither method used.

Table 1.29 Comparison of common dimensionality-reduction methods.

Method	Type	Preserves	New points?	Main use
PCA	Linear, unsupervised	Global variance	Yes	Compression, preprocessing, denoising
LDA	Linear, supervised	Class separability	Yes	Preprocessing for classification
ICA	Linear, unsupervised	Statistical independence	Yes	Blind source separation
Kernel PCA	Non-linear	Variance in feature space	Yes (approx.)	Non-linear feature extraction
Isomap	Non-linear	Geodesic distances	Limited	Manifold unfolding
t-SNE	Non-linear	Local neighbourhoods	No (standard)	2-D/3-D visualisation
UMAP	Non-linear	Local + some global structure	Yes	Visualisation, general embedding
Autoencoder	Non-linear (neural)	Reconstruction ability	Yes	Representation learning (Chapter 2)

1.31 Principal Component Analysis (PCA)

Principal component analysis is the most widely used dimensionality-reduction technique. Introduced by Karl Pearson in 1901 as the problem of finding "lines and planes of closest fit" and developed independently by Harold Hotelling in 1933, PCA finds a new orthogonal coordinate system whose axes — the **principal components** — are ordered by the amount of variance in the data they capture. Keeping only the first few components gives a low-dimensional representation that retains most of the variability.

1.31.1 Intuition

Consider two strongly correlated features such as height and weight. The data form an elongated cloud lying along a diagonal. Describing each point by its position *along* the diagonal captures most of the information; its position *across* the diagonal is a small residual. PCA formalises this: the first principal component is the direction along which the projected data have maximum variance; the second is the direction of maximum remaining variance orthogonal to the first; and so on. Equivalently, the first m components span the m-dimensional subspace that minimises the mean squared reconstruction error when the data are projected onto it. Figure 1.37 shows the two principal axes of a correlated cloud.

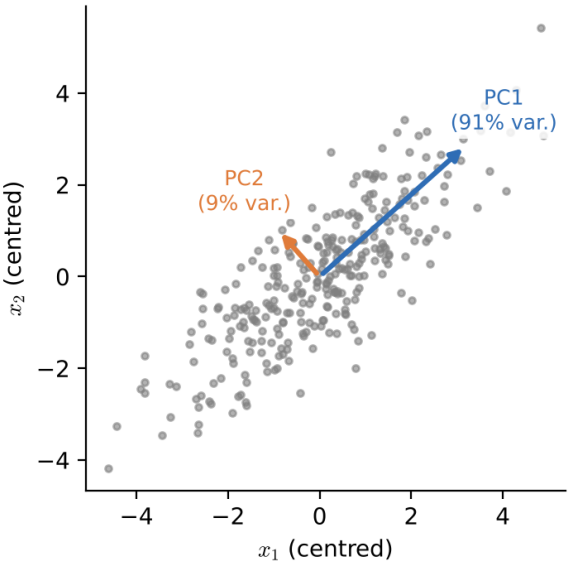


Figure 1.37 Principal components of correlated two-dimensional data. Arrows show PC1 and PC2 scaled by two standard deviations along each axis; PC1 captures the direction of greatest spread.

1.31.2 Derivation via Eigen-Decomposition of the Covariance Matrix

Let $X \in \mathbb{R}^{nxd}$ be the data matrix whose rows are observations, and assume each column has been centred to have zero mean (and usually scaled to unit variance). The sample covariance matrix is defined in Equation (1.46). Projecting the data onto a unit vector w gives scores $z = Xw$, whose sample variance is $w^T Sw$. The first principal component is therefore the solution of the constrained optimisation problem in Equation (1.47).

$$S = \frac{1}{n-1} X^{\top} X \in \mathbb{R}^{d \times d} \tag{1.46}$$

where X is the column-centred n × d data matrix; S is symmetric and positive semi-definite.

$$\begin{aligned} \mathbf{w}_1 &= \arg\max_{\mathbf{w}} \ \mathbf{w}^{\top} S \mathbf{w} \quad \text{subject to} \quad \mathbf{w}^{\top}\mathbf{w} = 1 \\ \mathcal{L}(\mathbf{w}, \lambda) &= \mathbf{w}^{\top} S \mathbf{w} - \lambda(\mathbf{w}^{\top}\mathbf{w} - 1) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}} &= 2S\mathbf{w} - 2\lambda\mathbf{w} = 0 \ \Rightarrow \ S\mathbf{w} = \lambda\mathbf{w} \end{aligned} \tag{1.47}$$

where λ is a Lagrange multiplier enforcing the unit-length constraint.

The stationarity condition $Sw = \lambda w$ states that w must be an **eigenvector** of S with eigenvalue λ. Substituting back, the projected variance is $w^{T}Sw = \lambda w^{T}w = \lambda$, so the variance is maximised by choosing the eigenvector with the *largest* eigenvalue. Repeating the argument with the additional constraint that w_2 be orthogonal to w_1 shows that the second principal component is the eigenvector with the second-largest eigenvalue, and so on. Because S is symmetric, its eigenvectors are orthogonal and its eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d \geq 0$ are real and non-negative, giving the spectral decomposition of Equation (1.48). Projecting onto the first m eigenvectors yields the reduced representation in Equation (1.49).

$$S = W \Lambda W^{\top}, \quad W = [\mathbf{w}_1, \ldots, \mathbf{w}_d], \quad \Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_d) \tag{1.48}$$

$$\begin{aligned} Z &= X W_m \in \mathbb{R}^{n \times m} \\ \hat{X} &= Z W_m^{\top} + \mathbf{1}\bar{\mathbf{x}}^{\top} \end{aligned} \tag{1.49}$$

where W_m contains the first m eigenvectors as columns, Z are the component scores, $\hat{X}$ is the reconstruction and $\bar{x}$ the vector of feature means.

In practice PCA is computed with the **singular value decomposition** (SVD) of the centred data matrix, $X = U\Sigma V^{T}$, rather than by forming S explicitly; this is numerically more stable. The right singular vectors V are the principal directions and the eigenvalues are $\lambda_j = \sigma_j^2/(n-1)$. The complete procedure is summarised in Algorithm 1.14.

Algorithm 1.14 Principal Component Analysis

Input: Data matrix $X \in \mathbb{R}^{n \times d}$; number of components m (or variance threshold τ)
Output: Components $W_m \in \mathbb{R}^{d \times m}$, scores $Z \in \mathbb{R}^{n \times m}$, explained-variance ratios

1 $\bar{x} \leftarrow$ column means of X; $X \leftarrow X - 1\bar{x}^{T}$ ▷ *centre*
2 optionally divide **each** column by its standard deviation ▷ *standardise*
3 compute the SVD $X = U\Sigma V^{T}$ (**or** eigen-decompose $S = X^{T}X/(n-1)$)

$\lambda_j \leftarrow \sigma_j^2/(n-1)$ **for** $j = 1, \ldots, d$ ▷ *sorted descending*
if τ is given **then** $m \leftarrow$ smallest m with $\sum_{j \le m} \lambda_j \,/\, \sum_j \lambda_j \ge \tau$
$W_m \leftarrow$ first m columns of V
$Z \leftarrow X\, W_m$
return W_m, Z, $(\lambda_1, \ldots, \lambda_m)/\sum_j \lambda_j$

1.31.3 Explained Variance and Choosing the Number of Components

The proportion of total variance captured by component j is its **explained variance ratio**, and the cumulative ratio for the first m components is given in Equation (1.50). Common rules for choosing m are to keep enough components to explain a target proportion such as 90% or 95% of the variance; to look for an elbow in the **scree plot** of eigenvalues against component index; or, for standardised data, Kaiser's rule of keeping components with eigenvalue greater than 1. When PCA is a preprocessing step for a supervised model, m is best treated as a hyperparameter and tuned by cross-validation. Figure 1.38 shows a scree plot and cumulative explained variance for the 30-feature breast-cancer dataset, where five components capture about 85% of the variance.

$$\mathrm{EVR}_j = \frac{\lambda_j}{\sum_{l=1}^{d} \lambda_l}, \qquad \mathrm{CEV}(m) = \frac{\sum_{j=1}^{m} \lambda_j}{\sum_{l=1}^{d} \lambda_l} \tag{1.50}$$

1.31.4 Worked Example

E Worked Example 1.22 | PCA of Five Two-Dimensional Points

Data points: $(2,1)$, $(3,4)$, $(4,3)$, $(5,6)$, $(6,6)$.

- **Centre.** The mean is $(4,4)$. Centred data: $(-2,-3)$, $(-1,0)$, $(0,-1)$, $(1,2)$, $(2,2)$.
- **Covariance ($n-1=4$).** $\mathrm{Var}(x_1) = (4+1+0+1+4)/4 = 2.5$; $\mathrm{Var}(x_2) = (9+0+1+4+4)/4 = 4.5$; $\mathrm{Cov}(x_1,x_2) = (6+0+0+2+4)/4 = 3.0$. So $S = [[2.5, 3.0], [3.0, 4.5]]$.
- **Eigenvalues.** Solve $\det(S - \lambda I) = 0$: $\lambda^2 - 7\lambda + (2.5 \times 4.5 - 9) = \lambda^2 - 7\lambda + 2.25 = 0$, so $\lambda = (7 \pm \sqrt{40})/2$, giving $\lambda_1 \approx 6.662$ and $\lambda_2 \approx 0.338$.
- **First eigenvector.** From $(2.5 - 6.662)w_1 + 3w_2 = 0$, $w_2 \approx 1.387 w_1$. Normalising gives $w_1 \approx (0.585, 0.811)$.
- **Explained variance.** PC1 explains $6.662/7 \approx 95.2\%$ of the total variance; PC2 explains 4.8%.
- **Scores on PC1.** $z = 0.585x_1 + 0.811x_2$ for the centred points: $(-2,-3) \to -3.60$; $(-1,0) \to -0.58$; $(0,-1) \to -0.81$; $(1,2) \to 2.21$; $(2,2) \to 2.79$.

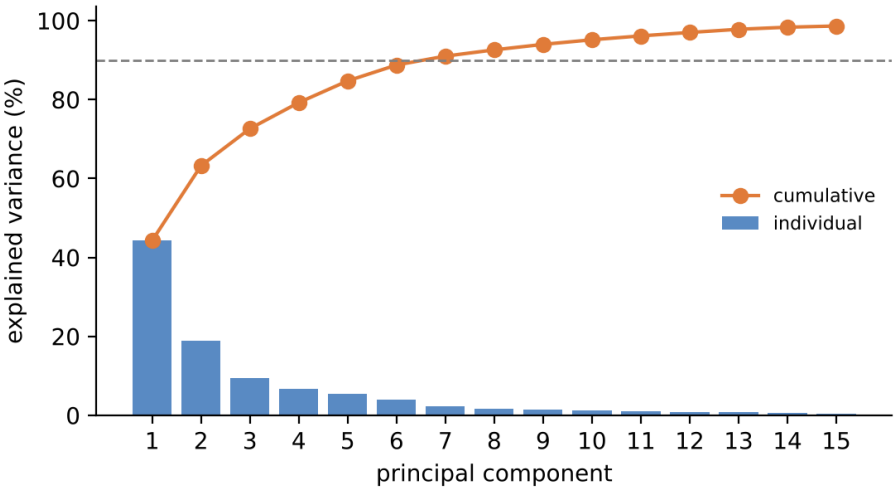


Figure 1.38 Scree plot (bars, individual explained variance) and cumulative explained variance (line) for the standardised 30-feature Wisconsin breast-cancer data. The dashed line marks the 90% threshold.

> Reducing to one dimension therefore retains over 95% of the variability, and the one-dimensional scores preserve the ordering of the points along the main trend.

Listing 1.17 performs PCA with scikit-learn. Passing a fraction to `n_components` selects the smallest number of components reaching that cumulative explained variance.

Listing 1.17 PCA with scikit-learn: standardise, retain 95% of variance, and reconstruct.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = load_breast_cancer().data                # 569 samples x 30 features
scaler = StandardScaler().fit(X)
Xs = scaler.transform(X)

pca = PCA(n_components=0.95, svd_solver="full")   # keep 95% of variance
Z = pca.fit_transform(Xs)
print("components kept:", pca.n_components_)
print("explained variance ratio:", np.round(pca.explained_variance_ratio_, 3))

# Loadings: contribution of each original feature to PC1
top = np.argsort(np.abs(pca.components_[0]))[::-1][:5]
print("features loading most on PC1:", top)

X_rec = pca.inverse_transform(Z)             # back to standardised space
print("mean squared reconstruction error:", np.mean((Xs - X_rec) ** 2).round(4))
```

1.31.5 Practical Considerations and Limitations

- **Standardise first** when features are measured in different units; otherwise the component directions are dominated by high-variance features.
- **Linearity:** PCA captures only linear correlations; curved manifolds need kernel PCA, UMAP or autoencoders.
- **Variance is not relevance:** a low-variance direction can be the one that separates classes, so PCA before classification can discard useful information. Supervised alternatives such as LDA may help.
- **Interpretability:** each component mixes all original features; inspect the *loadings* (component weights) to interpret them, or use sparse PCA for components with few non-zero weights.
- **Outliers** inflate variance and can rotate components; robust PCA variants mitigate this.
- **Fit on training data only:** to avoid leakage, learn the mean, scaling and components on the training split and apply the same transformation to validation and test data.

> **C Case Study 1.4 | Eigenfaces**
>
> Turk and Pentland (1991) applied PCA to aligned face images, each flattened into a vector of thousands of pixels. The leading principal components — visualised as ghostly "eigenfaces" — captured lighting and facial structure, and each face could be represented by a few dozen component scores. Recognition was then performed by nearest-neighbour matching in this compact space. Although modern face recognition uses deep networks, eigenfaces remain a classic illustration of PCA as compression and feature extraction.

1.32 Association Rule Learning

Association rule learning discovers interesting relationships between variables in large transactional databases. Its best-known application is **market-basket analysis**: from the contents of supermarket checkouts, find rules such as "customers who buy bread and butter also tend to buy milk". The framework was introduced by Agrawal, Imieliński and Swami in 1993, and the same ideas apply to web click-streams, medical diagnoses that co-occur, software modules changed together, and many other settings where data consist of sets of items.

1.32.1 Itemsets, Rules and Interestingness Measures

Let $I = \{i_1, \ldots, i_m\}$ be a set of items and $T = \{t_1, \ldots, t_n\}$ a database of transactions, each a subset of I. An **itemset** is any subset $X \subseteq I$; a k-itemset contains k items. An **association rule** is an implication $X \Rightarrow Y$ where X and Y are disjoint itemsets, called the *antecedent* and *consequent*. Three measures quantify a rule's interest, defined in Equation (1.51). **Support** is the fraction of transactions containing an itemset, measuring how frequent a pattern is. **Confidence** estimates the conditional probability $P(Y \mid X)$: of the transactions containing X, the fraction that also contain Y. **Lift** compares confidence with the baseline frequency of Y: a lift greater than 1 indicates that X and Y occur together more often than expected under independence, a lift of 1 indicates independence, and less than 1 indicates negative association.

$$\begin{aligned} \operatorname{supp}(X) &= \frac{|\{t \in T : X \subseteq t\}|}{|T|} \\ \operatorname{conf}(X \Rightarrow Y) &= \frac{\operatorname{supp}(X \cup Y)}{\operatorname{supp}(X)} \\ \operatorname{lift}(X \Rightarrow Y) &= \frac{\operatorname{conf}(X \Rightarrow Y)}{\operatorname{supp}(Y)} = \frac{\operatorname{supp}(X \cup Y)}{\operatorname{supp}(X)\operatorname{supp}(Y)} \end{aligned} \tag{1.51}$$

where $|T|$ is the number of transactions and $X \cup Y$ denotes transactions containing all items of both X and Y.

Other measures are also used. *Leverage* $\operatorname{supp}(X \cup Y) - \operatorname{supp}(X)\operatorname{supp}(Y)$ measures the absolute difference from independence, and *conviction* $(1 - \operatorname{supp}(Y))/(1 - \operatorname{conf}(X \Rightarrow Y))$ measures how much more often the rule would be wrong if X and Y were independent. The mining task is usually stated as: find all rules with support $\geq$ minsup and confidence $\geq$ minconf. It decomposes into two steps — (1) find all **frequent itemsets** with support $\geq$ minsup, and (2) generate high-confidence rules from them. The first step is the computationally hard one, because a set of m items has $2^m - 1$ non-empty subsets.

1.32.2 Worked Market-Basket Example

Table 1.30 lists eight transactions from a small grocery store. We will use it to illustrate the measures and the Apriori algorithm with minimum support 0.375 (three of eight transactions) and minimum confidence 0.7.

Worked Example 1.23 | Support, Confidence and Lift

From Table 1.30: beer appears in 4 transactions (T2, T3, T4, T7), diapers in 6, milk in 6, and {diapers, beer} in 4.

Table 1.30 Eight market-basket transactions used in the worked examples.

TID	Items purchased
T1	bread, milk
T2	bread, diapers, beer, eggs
T3	milk, diapers, beer, cola
T4	bread, milk, diapers, beer
T5	bread, milk, diapers, cola
T6	bread, milk, eggs
T7	milk, diapers, beer
T8	bread, diapers

- $\text{supp}(\{\text{diapers, beer}\}) = 4/8 = 0.50$.
- $\text{conf}(\{\text{diapers}\} \Rightarrow \{\text{beer}\}) = 0.50/0.75 = 0.667$; $\text{conf}(\{\text{beer}\} \Rightarrow \{\text{diapers}\}) = 0.50/0.50 = 1.0$.
- $\text{lift}(\{\text{diapers}\} \Rightarrow \{\text{beer}\}) = 0.667/0.50 = 1.33$ (positive association).
- For $\{\text{milk, diapers}\} \Rightarrow \{\text{beer}\}$: $\text{supp}(\{\text{milk, diapers, beer}\}) = 3/8 = 0.375$ (T3, T4, T7); $\text{supp}(\{\text{milk, diapers}\}) = 4/8$; confidence $= 0.375/0.5 = 0.75$; lift $= 0.75/0.5 = 1.5$.
- By contrast, $\text{conf}(\{\text{bread}\} \Rightarrow \{\text{milk}\}) = 0.5/0.75 = 0.667$ but lift $= 0.667/0.75 = 0.89 < 1$: although four customers bought both, bread buyers are slightly *less* likely than average to buy milk.

The last case shows why confidence alone can mislead when the consequent is very common.

1.32.3 The Apriori Algorithm

Apriori (Agrawal and Srikant, 1994) makes frequent-itemset mining tractable using the **downward closure** (anti-monotone) property: every subset of a frequent itemset must itself be frequent; equivalently, if an itemset is infrequent, all its supersets are infrequent. Apriori therefore searches level by level. It finds frequent 1-itemsets, joins them to form candidate 2-itemsets, prunes candidates that have an infrequent subset, counts the survivors in one scan of the database, and continues until no new frequent itemsets appear. Algorithm 1.15 gives the procedure and Figure 1.39 shows the pruning on the itemset lattice.

Algorithm 1.15 Apriori Frequent-Itemset Mining

Input: Transaction database T; minimum support minsup
Output: All frequent itemsets L

```
L_1 ← {1-itemsets with support ≥ minsup}                ▷ one database scan
k ← 2
while L_{k−1} is not empty do
    C_k ← join pairs in L_{k−1} sharing their first k−2 items   ▷ candidate generation
```

remove from C_k any candidate with a $(k-1)$-subset **not in** L_{k-1} ▷ *prune*
for each transaction t **in** T **do** ▷ *one database scan*
  for each candidate c **in** C_k with $c \subseteq t$ **do** $\text{count}(c) \leftarrow \text{count}(c) + 1$
$L_k \leftarrow \{c \in C_k : \text{count}(c)/|T| \geq \text{minsup}\}$
$k \leftarrow k + 1$
return $L = L_1 \cup L_2 \cup \ldots \cup L_{k-1}$

E Worked Example 1.24 | Apriori on the Basket Data (minsup = 0.375)

- **Level 1.** Counts: bread 6, milk 6, diapers 6, beer 4, cola 2, eggs 2. Cola and eggs are pruned; $L_1 = \{\text{bread, milk, diapers, beer}\}$.
- **Level 2.** Six candidate pairs. Counts: {bread, milk} 4, {bread, diapers} 4, {milk, diapers} 4, {diapers, beer} 4, {milk, beer} 3, {bread, beer} 2. {bread, beer} is pruned; five pairs are frequent.
- **Level 3.** Of the four possible triples, {bread, milk, beer} and {bread, diapers, beer} contain the infrequent pair {bread, beer}, so they are never counted. Only two candidates survive: {bread, milk, diapers} with count 2 (infrequent) and {milk, diapers, beer} with count 3 (frequent).
- **Level 4.** No candidates can be formed, so the search stops.
- **Rules (minconf = 0.7)** from the frequent itemsets include {beer} ⇒ {diapers} (conf 1.0, lift 1.33), {milk, beer} ⇒ {diapers} (conf 1.0, lift 1.33), {milk, diapers} ⇒ {beer} (conf 0.75, lift 1.5) and {beer} ⇒ {milk} (conf 0.75, lift 1.0, uninteresting despite high confidence).

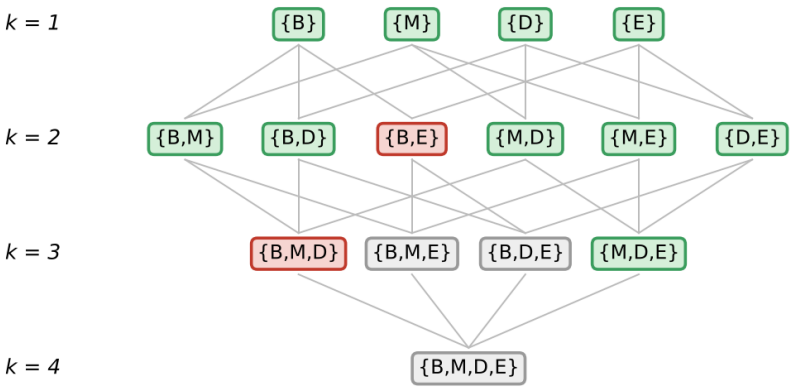


Figure 1.39 Itemset lattice for four frequent items (B = bread, M = milk, D = diapers, E = beer) in the basket example. Once {B, E} is found infrequent (red), every superset is pruned (grey) without counting; green itemsets are frequent.

1.32.4 FP-Growth and Practical Use

Apriori's weakness is its repeated database scans and the potentially huge number of candidates. **FP-Growth** (Han, Pei and Yin, 2000) avoids candidate generation altogether. It scans the database twice: once to count item frequencies, and once to insert every transaction — with infrequent items removed and the rest sorted by descending frequency — into a compact prefix tree called the **FP-tree**, in which transactions sharing a prefix share a path. Frequent itemsets are then mined recursively by building, for each item, a *conditional pattern base* (the prefix paths ending in that item) and a smaller conditional FP-tree. FP-Growth is typically an order of magnitude faster than Apriori on dense data. A third classic, **Eclat** (Zaki, 2000), stores for each itemset the list of transaction IDs containing it and computes supports by intersecting these lists. Table 1.31 compares the three and Listing 1.18 runs FP-Growth with the `mlxtend` library.

Table 1.31 Comparison of frequent-itemset mining algorithms.

Algorithm	Search strategy	Data representation	Database scans	Best suited to
Apriori	Breadth-first, candidate generation	Horizontal (transactions)	One per level	Sparse data, teaching, small itemsets
FP-Growth	Divide and conquer, no candidates	Compressed FP-tree	Two	Dense data, long patterns
Eclat	Depth-first, set intersection	Vertical (TID lists)	One (to build lists)	Medium-sized data fitting in memory

Listing 1.18 Frequent itemsets and association rules with mlxtend.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, association_rules

transactions = [["bread", "milk"], ["bread", "diapers", "beer", "eggs"],
                ["milk", "diapers", "beer", "cola"], ["bread", "milk", "diapers",
                ↪  "beer"],
                ["bread", "milk", "diapers", "cola"], ["bread", "milk", "eggs"],
                ["milk", "diapers", "beer"], ["bread", "diapers"]]

te = TransactionEncoder()
onehot = pd.DataFrame(te.fit(transactions).transform(transactions),
↪  columns=te.columns_)

itemsets = fpgrowth(onehot, min_support=0.375, use_colnames=True)
rules = association_rules(itemsets, metric="confidence", min_threshold=0.7)
rules = rules[rules["lift"] > 1].sort_values("lift", ascending=False)
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

> **T Practice Tip | Making Association Rules Useful**
>
> Mining typically produces thousands of rules, most obvious or redundant. Filter by lift (> 1) as well as confidence, remove rules whose consequent is a near-universal item, prefer rules with short antecedents, and validate promising rules on a later time period before acting on them. Remember that association rules express co-occurrence, not causation.

1.33 Anomaly and Outlier Detection

An **anomaly** (or outlier) is an observation that deviates so markedly from the rest of the data that it appears to have been generated by a different mechanism — a definition due to Hawkins (1980). Anomalies can be errors to be removed, such as a sensor reading of -999 or a patient age of 250, or they can be the very events of interest: a fraudulent card transaction, a network intrusion, a failing turbine bearing or a rare disease. **Anomaly detection** is the task of identifying such observations, usually without (or with very few) labelled examples of anomalies.

1.33.1 Types of Anomalies and Problem Settings

Three types are commonly distinguished. A **point anomaly** is a single observation that is unusual on its own, such as a transaction of an exceptionally large amount. A **contextual anomaly** is unusual only in a particular context — a temperature of 30 °C is normal in Jaffna in May but anomalous in London in January. A **collective anomaly** is a group of observations that is unusual together although each member may look normal, such as a sequence of small transfers that together constitute money laundering. The learning setting also varies: *supervised* detection treats the task as highly imbalanced classification; *semi-supervised* (or novelty) detection trains on clean normal data only and flags deviations; *unsupervised* (or outlier) detection works on contaminated, unlabelled data and assumes anomalies are rare and different.

1.33.2 Statistical Methods: z-Score and IQR

The simplest detectors apply to individual numerical features. The **z-score** of Equation (1.52) measures how many standard deviations an observation lies from the mean; values with $|z| > 3$ are traditionally flagged, since under a normal distribution only about 0.27% of observations lie beyond three standard deviations. Because the mean and standard deviation are themselves inflated by outliers, a **robust z-score** replaces them with the median and the median absolute deviation (MAD), also shown in Equation (1.52). The **interquartile range (IQR) rule**, which underlies Tukey's box plot, makes

no distributional assumption: an observation is an outlier if it lies outside the fences of Equation (1.53).

$$z_i = \frac{x_i - \bar{x}}{s}$$
$$z_i^{\text{robust}} = \frac{x_i - \text{median}(x)}{1.4826\,\text{MAD}}, \qquad \text{MAD} = \text{median}(|x_i - \text{median}(x)|) \tag{1.52}$$

where $\bar{x}$ and s are the sample mean and standard deviation; the constant 1.4826 makes the MAD a consistent estimator of the standard deviation for normal data.

$$x_i < Q_1 - 1.5\,\text{IQR} \quad \text{or} \quad x_i > Q_3 + 1.5\,\text{IQR}, \qquad \text{IQR} = Q_3 - Q_1 \tag{1.53}$$

where Q_1 and Q_3 are the first and third quartiles; a multiplier of 3 instead of 1.5 identifies 'extreme' outliers.

E Worked Example 1.25 | z-Score versus IQR

Daily sales (units): 12, 14, 15, 15, 16, 17, 18, 19, 20, 60.

- **z-score:** mean $\bar{x} = 20.6$, sample standard deviation $s \approx 14.05$. For 60: $z = (60 - 20.6)/14.05 \approx 2.80 < 3$, so the z-score rule does *not* flag it — the outlier has inflated s and masked itself.
- **IQR:** using the median-of-halves method, $Q_1 = 15$ and $Q_3 = 19$, so IQR = 4. Fences: $15 - 6 = 9$ and $19 + 6 = 25$. The value $60 > 25$ is flagged.
- **Robust z:** median = 16.5; absolute deviations 4.5, 2.5, 1.5, 1.5, 0.5, 0.5, 1.5, 2.5, 3.5, 43.5 have median 2.0, so $z = (60 - 16.5)/(1.4826 \times 2.0) \approx 14.7$, an unmistakable outlier.

Robust statistics are therefore preferred whenever outliers may be present in the data used to estimate the thresholds.

Univariate rules miss anomalies that are unusual only in combination — a height of 150 cm and a weight of 90 kg are each plausible, but together are rare. The multivariate analogue of the z-score is the **Mahalanobis distance**, $\sqrt{((x-\mu)^T\Sigma^{-1}(x-\mu))}$, which accounts for correlations between features. For complex, high-dimensional or multimodal data, the machine-learning detectors below are needed.

1.33.3 Isolation Forest

Isolation Forest (Liu, Ting and Zhou, 2008) inverts the usual approach: rather than profiling normal data, it isolates anomalies directly. It builds an ensemble of random binary trees, each grown on a small subsample (typically 256 points) by repeatedly choosing a random feature and a random split value between that feature's minimum and maximum, until every point is isolated in its own leaf. Anomalies, being few and different, are isolated after only a few splits and therefore have short paths from the root; normal points lying in dense regions need many splits. The anomaly score of Equation (1.54) normalises the average path length $E[h(x)]$ by $c(\psi)$, the average path

length of an unsuccessful search in a binary search tree of ψ points. Scores close to 1 indicate anomalies; scores well below 0.5 indicate normal points. The procedure is given in Algorithm 1.16.

$$s(\mathbf{x}, \psi) = 2^{-E[h(\mathbf{x})]\,/\,c(\psi)}$$
$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}, \quad H(i) \approx \ln i + 0.5772 \tag{1.54}$$

where $h(x)$ is the path length of x in one tree, $E[\cdot]$ averages over trees, ψ is the subsample size and $H(i)$ is the harmonic number.

Algorithm 1.16 Isolation Forest

Input: Data X; number of trees t (e.g. 100); subsample size ψ (e.g. 256); contamination rate or threshold
Output: Anomaly score $s(x)$ for every point

```
for b = 1 to t do
    X_b ← random subsample of ψ points from X
    T_b ← iTree(X_b, depth = 0, limit = ⌈log2 ψ⌉)
for each point x do
    E[h(x)] ← mean over b of PathLength(x, T_b)
    s(x) ← 2^(−E[h(x)] / c(ψ))
return s

function iTree(S, depth, limit):
    if |S| ≤ 1 or depth ≥ limit then return Leaf(size = |S|)
    choose feature q uniformly at random
    choose split p uniformly in [min_q(S), max_q(S)]
    return Node(q, p, iTree({x ∈ S : x_q < p}, depth+1, limit), iTree({x ∈ S : x_q ≥ p},
        depth+1, limit))
```

1.33.4 Local Outlier Factor and One-Class SVM

Local Outlier Factor (LOF; Breunig et al., 2000) compares the local density of a point with the densities of its neighbours, so it can detect points that are outliers *relative to their neighbourhood* even when clusters have different densities. The local reachability density of a point is the inverse of the average reachability distance to its k nearest neighbours, and the LOF is the ratio defined in Equation (1.55). LOF ≈ 1 means the point is as dense as its neighbours; LOF substantially greater than 1 marks an outlier.

$$\mathrm{reach}_k(\mathbf{x},\mathbf{o}) = \max\left\{d_k(\mathbf{o}),\ d(\mathbf{x},\mathbf{o})\right\}, \quad \mathrm{lrd}_k(\mathbf{x}) = \left(\frac{1}{k}\sum_{\mathbf{o}\in N_k(\mathbf{x})}\mathrm{reach}_k(\mathbf{x},\mathbf{o})\right)^{-1}$$
$$\mathrm{LOF}_k(\mathbf{x}) = \frac{1}{k}\sum_{\mathbf{o}\in N_k(\mathbf{x})}\frac{\mathrm{lrd}_k(\mathbf{o})}{\mathrm{lrd}_k(\mathbf{x})} \tag{1.55}$$

where $N_k(x)$ is the set of k nearest neighbours of x and $d_k(o)$ is the distance from o to its own k-th nearest neighbour.

The **One-Class SVM** (Schölkopf et al., 2001) adapts the support vector machine to the novelty-detection setting. Using a kernel (usually RBF), it maps data into a feature space and finds a hyperplane that separates the training data from the origin with maximum margin; in input space this corresponds to a tight boundary enclosing the bulk of the data. The hyperparameter $\nu \in (0,\,1]$ is an upper bound on the fraction of training points treated as outliers and a lower bound on the fraction of support vectors. One-Class SVM works well when trained on clean normal data but is sensitive to the kernel width and scales poorly beyond tens of thousands of points. Figure 1.40 compares the decision regions of the three detectors on the same data, and Table 1.32 summarises their properties.

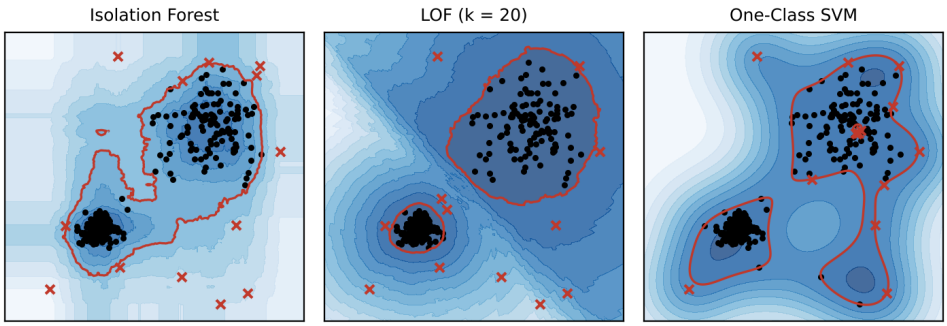


Figure 1.40 Isolation Forest, LOF and One-Class SVM on two normal clusters of different density plus scattered anomalies. Shaded contours show each method's normality score; red crosses are points flagged as anomalies (5% contamination).

Listing 1.19 applies Isolation Forest and LOF with scikit-learn. Because labelled anomalies are scarce, evaluation usually relies on a small labelled sample and on precision-oriented metrics such as precision at k or the area under the precision–recall curve, discussed later in this chapter.

Listing 1.19 Unsupervised anomaly detection with Isolation Forest and LOF.

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import average_precision_score
```

Table 1.32 Comparison of anomaly-detection methods.

Method	Principle	Strengths	Weaknesses
z-score / robust z	Distance from centre in SD (or MAD) units	Simple, fast, interpretable	Univariate; assumes unimodal data
IQR rule	Quartile-based fences	Distribution-free, robust	Univariate only
Mahalanobis distance	Correlation-aware distance from mean	Multivariate, captures correlations	Assumes elliptical, unimodal data
Isolation Forest	Anomalies isolated by few random splits	Fast, scales to large high-dimensional data	Axis-parallel splits; global view
LOF	Local density relative to neighbours	Handles clusters of varying density	Choice of k; $O(n^2)$ without index
One-Class SVM	Kernel boundary around normal data	Flexible non-linear boundary	Sensitive to γ, ν; slow on large n
Autoencoder	High reconstruction error	Images, sequences, complex data	Needs much data and tuning (Chapter 2)

```

rng = np.random.default_rng(0)
X_normal = rng.normal(0, 1, size=(1000, 5))
X_anom = rng.uniform(-6, 6, size=(30, 5))
X = np.vstack([X_normal, X_anom])
y = np.r_[np.zeros(1000), np.ones(30)]          # 1 = anomaly (used only to evaluate)

iso = IsolationForest(n_estimators=200, max_samples=256, contamination="auto",
    random_state=0).fit(X)
iso_score = -iso.score_samples(X)               # higher = more anomalous

lof = LocalOutlierFactor(n_neighbors=20)
lof.fit_predict(X)
lof_score = -lof.negative_outlier_factor_

for name, s in [("Isolation Forest", iso_score), ("LOF", lof_score)]:
    print(f"{name:17s} average precision = {average_precision_score(y, s):.3f}")
```

Case Study 1.5 | Credit-Card Fraud Detection

Card issuers process billions of transactions, of which typically well under 1% are fraudulent. Production systems combine unsupervised detectors (Isolation Forest or autoencoders that flag novel spending patterns) with supervised gradient-boosted classifiers trained on confirmed fraud, plus rule-based contextual checks (a purchase in a new country minutes after a local one). Flagged transactions are reviewed by analysts, whose decisions become new labels — an example of human-in-the-loop, semi-supervised learning.

1.34 Semi-Supervised and Self-Supervised Learning Methods

The overview of learning paradigms earlier in this chapter introduced semi-supervised and self-supervised learning as middle grounds between supervised and unsupervised learning. This section examines the main *methods* in detail. Both paradigms exploit the abundance of unlabelled data: semi-supervised learning uses a small labelled set $D_l = \{(x_i, y_i)\}_{i=1}^{l}$ together with a large unlabelled set $D_u = \{x_j\}_{j=l+1}^{l+u}$ ($u \gg l$) to train a better predictor than D_l alone allows; self-supervised learning creates its own supervisory signal from unlabelled data to learn representations that are later transferred to downstream tasks.

1.34.1 Assumptions Behind Semi-Supervised Learning

Unlabelled data can help only if the input distribution $p(x)$ carries information about $p(y \mid x)$. Three assumptions make this link explicit. The **smoothness assumption** states that points close together in a high-density region should have the same label. The **cluster assumption** states that data form clusters and points in the same cluster probably share a label — equivalently, decision boundaries should pass through low-density regions, an idea exploited by transductive SVMs. The **manifold assumption** states that the data lie near a low-dimensional manifold, on which learning is easier. When these assumptions fail, unlabelled data can actually hurt performance, so semi-supervised methods must always be compared against a purely supervised baseline.

1.34.2 Self-Training and Pseudo-Labelling

Self-training is the oldest and simplest semi-supervised method (Yarowsky, 1995, used it for word-sense disambiguation). A base classifier is trained on the labelled data, used to predict the unlabelled data, and the most confident predictions are added to the training set as if they were true labels; the process repeats, as described in Algorithm 1.17. Its risk is **confirmation bias**: early mistakes are reinforced. High confidence thresholds, class-balanced selection and the ability to remove labels later reduce this risk.

Algorithm 1.17 Self-Training with Confidence Threshold

Input: Labelled set D_l; unlabelled set D_u; base classifier f; threshold τ (e.g. 0.95); maximum rounds R

Output: Trained classifier f

```
train f on D_l
for r = 1 to R do
    for each x ∈ D_u compute p(y | x) ← f(x)
    S ← {(x, ŷ) : x ∈ D_u, max_y p(y | x) ≥ τ, ŷ = argmax_y p(y | x)}
```

```
    if S is empty then break
    D_l ← D_l ∪ S; D_u ← D_u \ {x : (x, ŷ) ∈ S}
    retrain f on D_l
return f
```

Pseudo-labelling (Lee, 2013) is the deep-learning form of self-training. A neural network is trained on a combined loss, Equation (1.56), in which unlabelled examples are assigned their currently predicted class as a hard target and weighted by a coefficient $\alpha(t)$ that is increased gradually so that the network first learns from clean labels. Modern methods such as **FixMatch** (Sohn et al., 2020) combine pseudo-labelling with **consistency regularisation**: a pseudo-label is generated from a weakly augmented image (for example, a flip) and accepted only if its confidence exceeds a threshold, and the network is then trained to predict that label on a strongly augmented version of the same image. With only 40 labelled images FixMatch reached roughly 89% accuracy on CIFAR-10.

$$\mathcal{L} = \frac{1}{l}\sum_{i=1}^{l} \ell\big(y_i, f(\mathbf{x}_i)\big) + \alpha(t)\,\frac{1}{u}\sum_{j=1}^{u} \ell\big(\hat{y}_j, f(\mathbf{x}_j)\big) \tag{1.56}$$

where ℓ is the cross-entropy loss, $\hat{y}_j$ is the pseudo-label of unlabelled example x_j and $\alpha(t)$ is a weight that ramps up with training epoch t.

1.34.3 Co-Training

Co-training (Blum and Mitchell, 1998) applies when each example has two *views* — two disjoint feature sets that are each sufficient to predict the label and are conditionally independent given the label. The classic example is web-page classification, where one view is the page's own text and the other is the anchor text of hyperlinks pointing to it. Two classifiers are trained, one per view; each labels the unlabelled examples on which it is most confident, and those examples are added to the *other* classifier's training set. Because the views are different, each classifier teaches the other things it could not learn alone, reducing confirmation bias. Variants such as tri-training relax the requirement of natural views by using bootstrap samples or different learning algorithms.

1.34.4 Graph-Based Label Propagation

Label propagation (Zhu and Ghahramani, 2002) turns the smoothness assumption into an algorithm. It builds a graph whose nodes are all labelled and unlabelled points and whose edge weights measure similarity, for example $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$ or

k-nearest-neighbour connectivity. Labels then flow along edges: each node repeatedly takes the weighted average of its neighbours' label distributions while labelled nodes are clamped to their known labels. Writing D for the diagonal degree matrix and F for the n × C matrix of label scores, the iteration and its closed-form fixed point (in the *label spreading* variant of Zhou et al., 2004) are given in Equation (1.57). Figure 1.41 shows how two labelled points suffice to label two interleaved moons, whereas a supervised classifier trained on the same two points cuts across both moons with an incorrect boundary.

$$\mathbf{S} = D^{-1/2} W D^{-1/2}, \qquad F^{(t+1)} = \alpha\, \mathbf{S}\, F^{(t)} + (1-\alpha)\, Y$$
$$F^{*} = (1-\alpha)\,(I - \alpha \mathbf{S})^{-1}\, Y \tag{1.57}$$

where Y is the n × C matrix of initial labels (one-hot rows for labelled points, zero rows for unlabelled), $\alpha \in (0, 1)$ balances neighbour information against initial labels, and each point is assigned the class with the largest entry in its row of F^{*}.

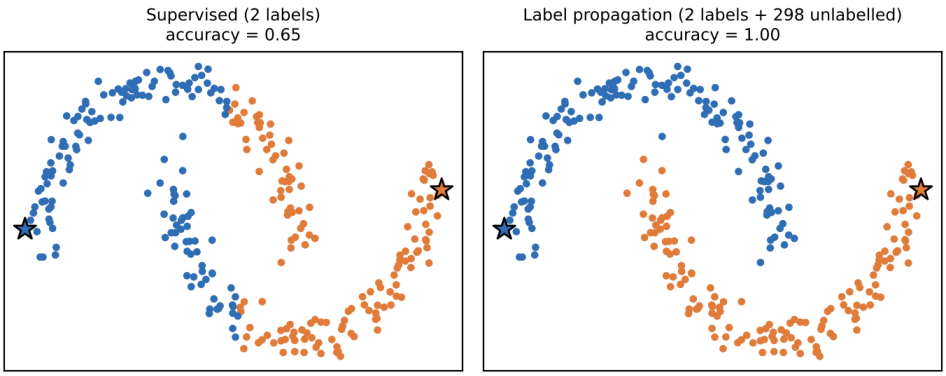


Figure 1.41 Label propagation on the two-moons data with only one labelled point per class (stars). Left: an RBF SVM trained on the two labelled points alone. Right: labels propagated through an RBF similarity graph over all 300 points recover both moons.

Listing 1.20 compares self-training and label spreading in scikit-learn, where unlabelled samples are marked with the label -1.

Listing 1.20 Self-training and label spreading in scikit-learn with 5% labelled data.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.semi_supervised import SelfTrainingClassifier, LabelSpreading

X, y = load_digits(return_X_y=True)
X = X / 16.0
```

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y,
    random_state=0)

rng = np.random.default_rng(0)
y_semi = y_tr.copy()
mask = rng.random(len(y_tr)) > 0.05          # hide 95% of the labels
y_semi[mask] = -1

base = SVC(kernel="rbf", gamma=0.05, probability=True)
sup = base.fit(X_tr[~mask], y_tr[~mask])
self_tr = SelfTrainingClassifier(SVC(kernel="rbf", gamma=0.05, probability=True),
    threshold=0.9).fit(X_tr, y_semi)
spread = LabelSpreading(kernel="knn", n_neighbors=10).fit(X_tr, y_semi)

for name, m in [("supervised only", sup), ("self-training", self_tr), ("label
    spreading", spread)]:
    print(f"{name:16s} test accuracy = {m.score(X_te, y_te):.3f}")
```

1.34.5 Self-Supervised Learning: Pretext Tasks and Contrastive Methods

Self-supervised learning (SSL) removes the need for human labels during pre-training altogether. A **pretext task** is constructed in which the targets are derived automatically from the data; solving it forces the model to learn representations that capture useful structure, which are then fine-tuned on a small labelled **downstream task**. Early pretext tasks for images included predicting the relative position of two patches (Doersch, Gupta and Efros, 2015), solving jigsaw puzzles of shuffled patches, colourising greyscale images and predicting the rotation ($0°$, $90°$, $180°$, $270°$) applied to an image (Gidaris, Singh and Komodakis, 2018). In natural language, **masked language modelling** — hiding 15% of the tokens and predicting them, as in BERT (Devlin et al., 2019) — and **next-token prediction**, as in the GPT family, are pretext tasks so effective that they underpin modern large language models (see Chapter 3).

Contrastive learning became the dominant SSL approach for vision around 2020. Two augmented views of the same image form a *positive pair*; views of different images in the batch are *negatives*. An encoder is trained so that positives have similar embeddings and negatives dissimilar ones, using the InfoNCE (NT-Xent) loss of Equation (1.58). SimCLR (Chen et al., 2020) and MoCo (He et al., 2020) follow this recipe; BYOL and SimSiam showed that negatives can be avoided with asymmetric architectures, and **masked autoencoders** (He et al., 2022) reconstruct randomly masked image patches. Table 1.33 summarises the methods of this section; the deep-learning machinery they rely on is developed in Chapter 2.

$$\ell_{i,j} = -\log \frac{\exp\left(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau\right)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp\left(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau\right)} \tag{1.58}$$

where z_i and z_j are embeddings of two views of the same image, sim is cosine similarity, τ is a temperature, and the sum runs over the 2N views in a batch of N images.

Table 1.33 Semi-supervised and self-supervised methods at a glance.

Method	Paradigm	Key idea	Main risk or requirement
Self-training	Semi-supervised	Add confident predictions as labels	Confirmation bias
Pseudo-labelling / FixMatch	Semi-supervised	Hard pseudo-labels + consistency under augmentation	Good augmentations, calibrated confidence
Co-training	Semi-supervised	Two views teach each other	Two sufficient, independent views
Label propagation / spreading	Semi-supervised (graph)	Labels diffuse over similarity graph	Meaningful graph; $O(n^2)$ memory
Rotation / jigsaw / colourisation	Self-supervised	Predict an automatic transformation	Pretext may not align with downstream task
Masked modelling (BERT, MAE)	Self-supervised	Reconstruct hidden parts of input	Large data and compute
Contrastive (SimCLR, MoCo)	Self-supervised	Pull positive views together, push negatives apart	Large batches or memory banks; augmentation design

Key Point | When to Use Which

With a few hundred labels and a moderate amount of tabular data, try self-training or label spreading against a supervised baseline. With images, text or audio and abundant unlabelled data, start from a self-supervised pre-trained model and fine-tune it — this is now the default route to strong performance with limited labels.

1.35 Reinforcement Learning Fundamentals

Supervised learning learns from examples of correct behaviour; **reinforcement learning** (RL) learns from the *consequences* of behaviour. An **agent** interacts with an **environment** over a sequence of time steps. At each step t it observes a state S_t, chooses an action A_t, and receives a scalar reward R_{t+1} and a new state S_{t+1}, as shown in Figure 1.42. The agent's goal is to learn a behaviour that maximises the cumulative reward it collects over time. There is no teacher saying which action was correct; feedback is evaluative, often delayed, and depends on the agent's own choices. RL has produced landmark results — TD-Gammon's backgammon play (Tesauro, 1995), Atari game-playing from pixels (Mnih et al., 2015) and AlphaGo's defeat of Lee Sedol at Go (Silver

et al., 2016) — and is used today in robotics, recommendation, resource management and the fine-tuning of large language models from human feedback.

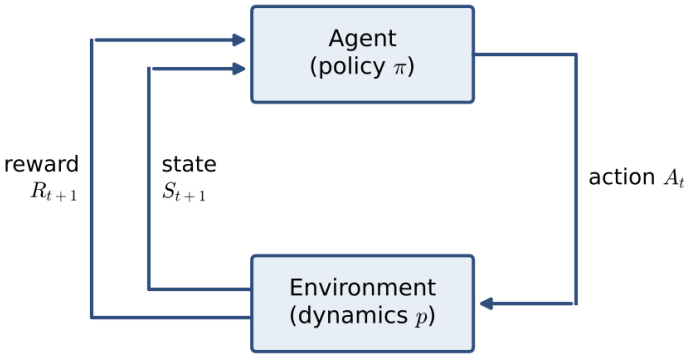


Figure 1.42 The agent–environment interaction loop: at each step the agent observes the state, selects an action according to its policy, and the environment returns a reward and the next state.

1.35.1 Markov Decision Processes

The standard mathematical framework for RL is the **Markov decision process** (MDP), formalised by Bellman in the 1950s. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, \mathrm{p}, \mathrm{r}, \gamma)$ consisting of a set of states $\mathcal{S}$, a set of actions $\mathcal{A}$, a transition function $\mathrm{p}(\mathrm{s}' \mid \mathrm{s}, \mathrm{a})$ giving the probability of moving to state s' after taking action a in state s, a reward function $\mathrm{r}(\mathrm{s}, \mathrm{a})$ (or $\mathrm{r}(\mathrm{s}, \mathrm{a}, \mathrm{s}')$) and a discount factor $\gamma \in [0, 1]$. The **Markov property** requires that the future depends only on the current state and action, not on the earlier history: $\mathrm{P}(\mathrm{S}_{t+1} \mid \mathrm{S}_t, \mathrm{A}_t, \mathrm{S}_{t-1}, \ldots) = \mathrm{P}(\mathrm{S}_{t+1} \mid \mathrm{S}_t, \mathrm{A}_t)$. The agent's objective is to maximise the expected **return**, the discounted sum of future rewards defined in Equation (1.59). Discounting with $\gamma < 1$ keeps the sum finite in continuing tasks and expresses a preference for earlier rewards; γ close to 0 gives a myopic agent and γ close to 1 a far-sighted one.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma G_{t+1} \tag{1.59}$$

where $\gamma \in [0, 1]$ is the discount factor and R_{t+k+1} is the reward received $k+1$ steps after time t.

1.35.2 Policies and Value Functions

A **policy** π defines the agent's behaviour: $\pi(\mathrm{a} \mid \mathrm{s})$ is the probability of choosing action a in state s (a deterministic policy maps each state to a single action). To evaluate how

good a policy is, RL uses **value functions**. The **state-value function** v_π(s) is the expected return when starting in s and following π thereafter; the **action-value function** q_π(s, a) is the expected return when starting in s, taking action a, and following π thereafter. Both are defined in Equation (1.60). A policy π* is **optimal** if its value is at least as great as that of every other policy in every state; all optimal policies share the same optimal value functions v* and q, *and acting greedily with respect to q* — choosing $\operatorname{argmax}_a q^*(s, a)$ — is optimal.

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ q_\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \end{aligned} \tag{1.60}$$

1.35.3 The Bellman Equations

Because the return satisfies the recursion $G_t = R_{t+1} + \gamma G_{t+1}$, value functions satisfy recursive consistency conditions known as the **Bellman equations**. The Bellman *expectation* equation of Equation (1.61) expresses the value of a state under π as the expected immediate reward plus the discounted value of the successor state. The Bellman *optimality* equations of Equation (1.62) replace the average over the policy's actions with a maximum. For a finite MDP with known p and r, these equations can be solved by **dynamic programming**: *policy iteration* alternates policy evaluation (solving Equation (1.61)) with greedy policy improvement, while *value iteration*, given in Algorithm 1.18, repeatedly applies the optimality equation as an update until the values converge.

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big] \tag{1.61}$$

$$\begin{aligned} v_*(s) &= \max_a \sum_{s', r} p(s', r \mid s, a)\big[r + \gamma\, v_*(s')\big] \\ q_*(s, a) &= \sum_{s', r} p(s', r \mid s, a)\Big[r + \gamma \max_{a'} q_*(s', a')\Big] \end{aligned} \tag{1.62}$$

Algorithm 1.18 Value Iteration

Input: Finite MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$; convergence threshold $\theta > 0$
Output: Optimal value function v* (approximately) and greedy policy π*

initialise $v(s) \leftarrow 0$ **for** all $s \in \mathcal{S}$
repeat
  $\Delta \leftarrow 0$
  for each $s \in \mathcal{S}$ **do**

```
        v_old ← v(s)
        v(s) ← max_a Σ_{s',r} p(s', r | s, a) [r + γ v(s')]
        Δ ← max(Δ, |v_old − v(s)|)
until Δ < θ
π*(s) ← argmax_a Σ_{s',r} p(s', r | s, a) [r + γ v(s')] for all s
return v, π*
```

E Worked Example 1.26 | One Bellman Backup

A robot in state s can choose *left* or *right*; $\gamma = 0.9$. Current value estimates: $v(A) = 10$, $v(B) = 2$, $v(C) = 0$.

- *left* leads to A with probability 0.8 (reward 0) and to C with probability 0.2 (reward -5).
- *right* leads to B with probability 1.0 (reward $+4$).
- $q(s, \text{left}) = 0.8 \times (0 + 0.9 \times 10) + 0.2 \times (-5 + 0.9 \times 0) = 7.2 - 1.0 = 6.2$.
- $q(s, \text{right}) = 1.0 \times (4 + 0.9 \times 2) = 5.8$.
- The value-iteration update sets $v(s) \leftarrow \max(6.2, 5.8) = 6.2$ and the greedy action is *left*, even though *right* offers the larger immediate reward — the agent accounts for the long-term value of reaching A.

1.35.4 Exploration versus Exploitation

When p and r are unknown, the agent must learn from experience, and a fundamental dilemma arises: should it **exploit** the action currently believed best, or **explore** other actions that might turn out better? Pure exploitation can lock in a sub-optimal choice; pure exploration wastes reward. The dilemma is studied in its simplest form in the **multi-armed bandit** problem, a single-state MDP. Common strategies include **ε-greedy** (choose a random action with probability ε, otherwise the greedy one, often decaying ε over time), **softmax/Boltzmann** exploration (choose actions with probability proportional to $\exp(q(s, a)/\tau)$), **upper confidence bound** (UCB) selection, which adds a bonus $c\sqrt{(\ln t \,/\, N_t(a))}$ favouring rarely tried actions, and **Thompson sampling**, which samples from a posterior over action values. Figure 1.43 compares ε-greedy settings on a ten-armed bandit, and Table 1.34 places these ideas within the wider taxonomy of RL methods.

1.36 Q-Learning and Deep Q-Networks

Dynamic programming requires a complete model of the environment, which is rarely available. **Temporal-difference (TD) learning** (Sutton, 1988) instead learns directly

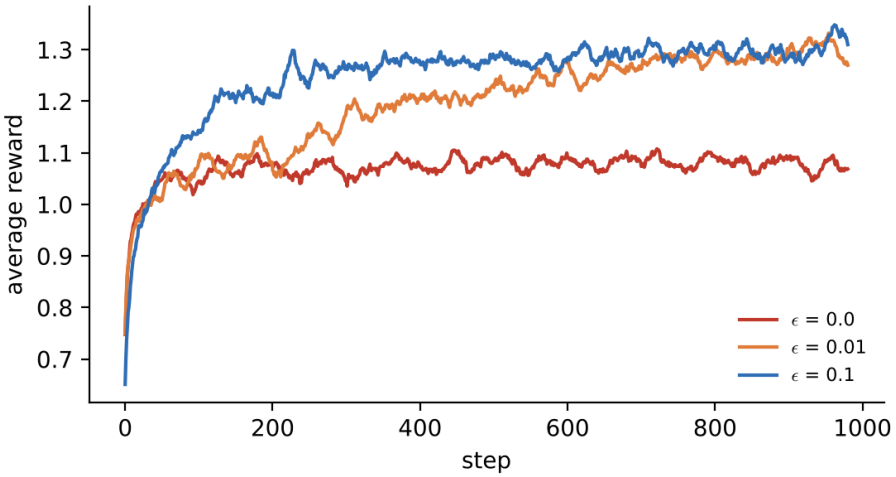


Figure 1.43 Average reward on a ten-armed Gaussian bandit (averaged over 300 runs). A purely greedy agent ($\varepsilon = 0$) gets stuck on a sub-optimal arm; moderate exploration ($\varepsilon = 0.1$) learns fastest, while $\varepsilon = 0.01$ improves more slowly but steadily.

Table 1.34 A taxonomy of reinforcement learning approaches.

Dimension	Option A	Option B
Knowledge of environment	Model-based: learn or use p and r to plan (dynamic programming, MCTS)	Model-free: learn values or policies directly from experience
What is learned	Value-based: learn q(s, a), act greedily (Q-learning, DQN)	Policy-based: learn π directly (REINFORCE, PPO); actor–critic combines both
Data source	On-policy: learn about the policy generating the data (SARSA)	Off-policy: learn about a different (e.g. greedy) policy (Q-learning)
Update target	Monte Carlo: wait for full episode return G_t	Temporal difference: bootstrap from current estimates
Representation	Tabular: one entry per state–action pair	Function approximation: linear models or deep networks

from sampled transitions, updating an estimate towards a target that itself uses current estimates — a technique called **bootstrapping**. **Q-learning**, introduced by Chris Watkins in his 1989 PhD thesis, applies TD learning to the action-value function and is one of the most important algorithms in reinforcement learning.

1.36.1 The Q-Learning Update

Q-learning maintains a table Q(s, a) and, after each transition (s, a, r, s′), moves Q(s, a) towards the **TD target** $r + \gamma \max_{a'} Q(s', a')$, a sample-based version of the Bellman optimality equation for q. *The update rule is given in Equation (1.63), and the full algorithm*

in Algorithm 1.19. Because the target uses the maximum* over next actions regardless of which action the agent actually takes next, Q-learning is **off-policy**: it learns the optimal greedy policy while following an exploratory (for example ε-greedy) behaviour policy. Watkins and Dayan (1992) proved that tabular Q-learning converges to q^* with probability 1 provided every state–action pair is visited infinitely often and the learning rate decays appropriately.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big] \tag{1.63}$$

where $\alpha \in (0, 1]$ is the learning rate and the bracketed quantity is the TD error δ_t.

The closely related on-policy algorithm **SARSA** (named after the quintuple $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$) uses the action actually taken next, $Q(S_{t+1}, A_{t+1})$, in place of the maximum. SARSA therefore learns the value of the exploratory policy it follows, and tends to learn safer paths when exploration is risky — the classic "cliff-walking" example in Sutton and Barto (2018) shows Q-learning learning the optimal path along a cliff edge but falling off more often during training, while SARSA learns a longer, safer route.

Algorithm 1.19 Tabular Q-Learning with ε-Greedy Exploration

Input: Environment; learning rate α; discount γ; exploration rate ε; number of episodes M
Output: Action-value table Q and greedy policy $\pi(s) = \operatorname{argmax}_a Q(s, a)$

initialise $Q(s, a) \leftarrow 0$ **for** all s, a ($Q(\text{terminal}, \cdot) = 0$)
for episode $= 1$ **to** M **do**
  observe initial state s
  while s is **not** terminal **do**
    with probability ε choose a random action a, otherwise $a \leftarrow \operatorname{argmax}_a Q(s, a)$
    take action a; observe reward r **and** next state s'
    $\delta \leftarrow r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ ▷ *TD error*
    $Q(s, a) \leftarrow Q(s, a) + \alpha\, \delta$
    $s \leftarrow s'$
  optionally decay ε
return Q

Worked Example 1.27 | **Two Q-Learning Updates**

Let $\alpha = 0.5$, $\gamma = 0.9$, and all Q-values initially 0 except $Q(s_2, \text{right}) = 4$.

- **Step 1.** In s_1 the agent takes *right*, receives $r = -1$ and reaches s_2. Target $= -1 + 0.9 \times \max(Q(s_2, \text{left}) = 0, Q(s_2, \text{right}) = 4) = -1 + 3.6 = 2.6$. Update: $Q(s_1, \text{right}) \leftarrow 0 + 0.5 \times (2.6 - 0) = 1.3$.
- **Step 2.** In s_2 the agent explores with *left*, receives $r = 0$ and returns to s_1. Target $= 0 + 0.9 \times \max(Q(s_1, \cdot)) = 0.9 \times 1.3 = 1.17$. Update: $Q(s_2, \text{left}) \leftarrow 0 + 0.5 \times 1.17 = 0.585$.

Note how the value of reaching s_2 propagated backwards to s_1 in step 1, and how step 2 used

the value just learned. Repeated over many episodes, such backups spread reward information through the whole state space.

Listing 1.21 implements tabular Q-learning for the *FrozenLake* grid world from the Gymnasium library, in which an agent must cross a slippery frozen lake to a goal without falling into holes.

Listing 1.21 Tabular Q-learning on FrozenLake with Gymnasium.

```
import numpy as np
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=True)
nS, nA = env.observation_space.n, env.action_space.n
Q = np.zeros((nS, nA))
alpha, gamma, eps, episodes = 0.1, 0.99, 1.0, 20_000
rng = np.random.default_rng(0)

for ep in range(episodes):
    s, _ = env.reset(seed=int(rng.integers(1e6)))
    done = False
    while not done:
        a = env.action_space.sample() if rng.random() < eps else int(np.argmax(Q[s]))
        s_next, r, terminated, truncated, _ = env.step(a)
        target = r + gamma * np.max(Q[s_next]) * (not terminated)
        Q[s, a] += alpha * (target - Q[s, a])
        s, done = s_next, terminated or truncated
    eps = max(0.05, eps * 0.9997)               # decay exploration

# Evaluate the greedy policy
wins = 0
for _ in range(1000):
    s, _ = env.reset(); done = False
    while not done:
        s, r, term, trunc, _ = env.step(int(np.argmax(Q[s]))); done = term or trunc
    wins += r
print("success rate of greedy policy:", wins / 1000)
```

1.36.2 From Tables to Function Approximation

Tabular Q-learning needs one entry per state–action pair, which is infeasible when states are images, continuous sensor readings or board positions — Go has more than 10^{170} legal positions. The solution is to approximate the action-value function with a parameterised model $Q(s, a; \theta)$ that generalises across similar states. Naively combining Q-learning with a neural network is unstable, however: Sutton and Barto describe the **deadly triad** of function approximation, bootstrapping and off-policy learning, which together can cause values to diverge. Consecutive samples are also highly correlated, and the target $r + \gamma \max_{a'} Q(s', a'; \theta)$ shifts every time θ is updated.

1.36.3 Deep Q-Networks

The **Deep Q-Network** (DQN) of Mnih et al. (2015) was the first method to learn successful control policies directly from high-dimensional sensory input across many tasks: from raw Atari 2600 screen pixels and the game score alone, a single architecture and set of hyperparameters reached human-level performance on many of 49 games. Its architecture, sketched in Figure 1.44, is a convolutional network (see Chapter 2) that takes a stack of the last four 84×84 greyscale frames and outputs one Q-value per action, so a single forward pass evaluates all actions. Two innovations stabilise learning:

- **Experience replay.** Transitions $(s, a, r, s', \text{done})$ are stored in a large replay buffer (one million transitions in the original work), and the network is trained on random mini-batches drawn from it. This breaks the correlation between consecutive samples and lets each experience be reused many times.
- **Target network.** A separate copy of the network with parameters θ^{-} computes the TD target and is updated to equal θ only every C steps (or slowly, by Polyak averaging). The target therefore stays fixed for a while, turning each phase into a more stable regression problem.

The resulting loss, minimised by stochastic gradient descent on mini-batches from the replay buffer $\mathcal{D}$, is given in Equation (1.64), and the training loop in Algorithm 1.20. In practice the squared error is often replaced by the Huber loss, and rewards are clipped to $[-1, 1]$ to keep gradient magnitudes comparable across games.

$$
\begin{aligned}
y_j &= r_j + \gamma\,(1 - d_j)\max_{a'} Q(s'_j, a'; \theta^{-}) \\
\mathcal{L}(\theta) &= \mathbb{E}_{(s,a,r,s',d)\sim\mathcal{D}}\left[\left(y_j - Q(s_j, a_j; \theta)\right)^2\right]
\end{aligned}
\tag{1.64}
$$

where θ are the online network parameters, θ^{-} the target network parameters and $d_j = 1$ if s'_j is terminal.

Algorithm 1.20 Deep Q-Learning with Experience Replay and Target Network

Input: Environment; Q-network $Q(\cdot, \cdot; \theta)$; replay capacity N; batch size B; target update period C; ε schedule
Output: Trained network parameters θ

initialise replay buffer $\mathcal{D}$ with capacity N; **initialise** θ randomly; $\theta^{-} \leftarrow \theta$
for episode $= 1$ **to** M **do**
  observe initial state s
  for $t = 1$ **to** T **do**
    $a \leftarrow$ random action with probability ε, **else** $\operatorname{argmax}_a Q(s, a; \theta)$
    execute a; observe r, s' **and** terminal flag d

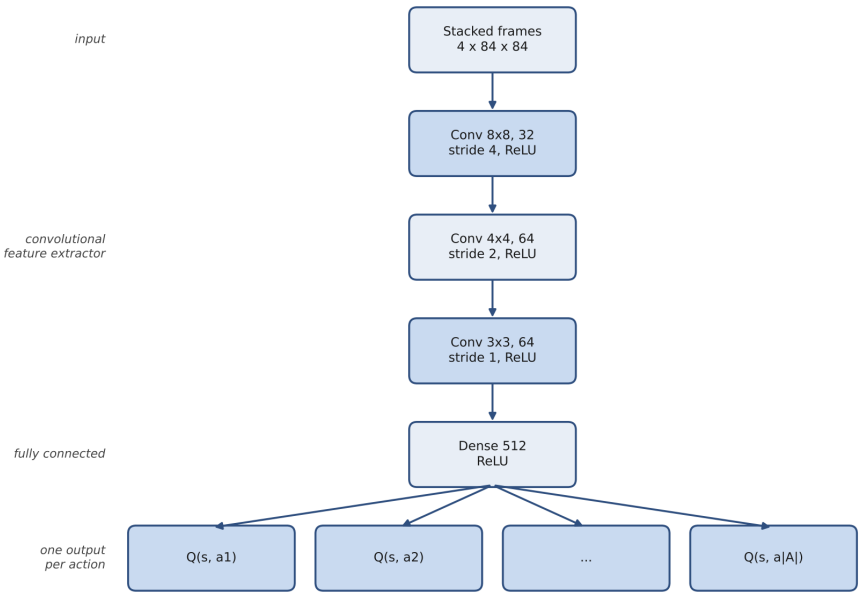


Figure 1.44 DQN architecture and training components: a convolutional Q-network maps stacked frames to one Q-value per action; experience replay and a periodically synchronised target network provide stable regression targets.

store (s, a, r, s', d) **in** $\mathcal{D}$; $s \leftarrow s'$
sample a random mini-batch of B transitions from $\mathcal{D}$
for each transition j: $y_j \leftarrow r_j + \gamma(1 - d_j)\max_{a'} Q(s'_j, a'; \theta^-)$
take a gradient step on $\Sigma_j \left(y_j - Q(s_j, a_j; \theta)\right)^2$ with respect **to** θ
every C steps set $\theta^- \leftarrow \theta$
if d **then break**
return θ

1.36.4 Improvements to DQN

A series of refinements followed, summarised in Table 1.35. **Double DQN** (van Hasselt, Guez and Silver, 2016) reduces the systematic over-estimation caused by taking a maximum over noisy estimates, by selecting the next action with the online network but evaluating it with the target network. **Dueling DQN** splits the network into a state-value stream and an advantage stream. **Prioritised experience replay** samples transitions with large TD error more often. **Rainbow** (Hessel et al., 2018) combined six such extensions and substantially outperformed each alone. For continuous action spaces, where the maximisation over actions is itself hard, policy-gradient and actor–

critic methods such as DDPG, PPO and SAC are used instead; these are covered with deep reinforcement learning in Chapter 3.

Table 1.35 Key extensions of the Deep Q-Network.

Extension	Problem addressed	Key idea
Double DQN	Over-estimation of Q-values	Online net selects a', target net evaluates it
Dueling DQN	Inefficient learning of state values	$Q(s, a) = V(s) + A(s, a) - \text{mean } A(s, \cdot)$
Prioritised replay	Uniform sampling wastes updates	Sample transitions in proportion to |TD error|
Multi-step returns	Slow reward propagation	Bootstrap from n-step return
Distributional RL (C51)	Expected value discards risk information	Learn the full return distribution
Noisy nets	ε-greedy exploration is undirected	Learnable parametric noise in weights
Rainbow	Combining the above	Integrates all six extensions

Case Study 1.6 | Reinforcement Learning for Data-Centre Cooling

In 2016 DeepMind and Google reported that an RL-based control system, trained on historical sensor data from Google data centres, recommended cooling-plant settings that reduced the energy used for cooling by up to 40%. Later versions acted autonomously under safety constraints and human oversight. The project illustrates the practical ingredients of applied RL: a carefully defined reward (energy efficiency), safety limits on actions, learning from logged data before live deployment, and continuous monitoring.

1.37 Model Evaluation and Performance Metrics

A model is useful only if it performs well on data it has not seen. **Model evaluation** is the disciplined process of estimating that future performance, comparing candidate models, and deciding whether a model is fit for deployment. Earlier in this chapter we saw how data are divided into training, validation and test sets and how the bias–variance trade-off governs over- and under-fitting. This section and the next four build on those foundations: they define the metrics used to score predictions, the resampling schemes used to estimate them reliably, and the search procedures used to tune hyperparameters without fooling ourselves.

1.37.1 Choosing a Metric

A metric turns predictions and ground truth into a single number, and every metric embodies a value judgement about which errors matter. In cancer screening, missing a malignant tumour (a false negative) is far costlier than a false alarm; in spam filtering, deleting a legitimate email (a false positive) is worse than letting one spam message through. The metric must therefore be chosen *before* modelling, in consultation with stakeholders, and should reflect the real cost of errors. It is also useful to distinguish the **loss function** optimised during training (for example cross-entropy, chosen for differentiability) from the **evaluation metric** reported to stakeholders (for example recall at a fixed false-positive rate), which need not be differentiable. Table 1.36 offers a starting point for common task types.

Table 1.36 Typical evaluation metrics by task type.

Task	Common metrics	When to prefer
Binary classification (balanced)	Accuracy, ROC-AUC, log-loss	Errors have similar costs
Binary classification (imbalanced)	Precision, recall, F1, PR-AUC	Positive class is rare and important
Multi-class classification	Macro/micro/weighted F1, top-k accuracy	Several classes, possibly imbalanced
Probabilistic prediction	Log-loss, Brier score, calibration curve	Probabilities drive decisions
Regression	MAE, RMSE, R^2, MAPE	Continuous targets; choice depends on outlier cost
Ranking / recommendation	Precision@k, MAP, NDCG	Order of results matters
Clustering	Silhouette, adjusted Rand index, NMI	Internal or external validation

1.37.2 Baselines, Uncertainty and Significance

A metric value is meaningful only in comparison. Every evaluation should report a **baseline**: a majority-class classifier for classification, the mean or last-value predictor for regression, or the currently deployed system. A model with 95% accuracy is unimpressive if 95% of examples belong to one class. Metric estimates are also subject to sampling variability, especially on small test sets. A **confidence interval** can be obtained by bootstrapping the test set — resampling it with replacement many times and recomputing the metric — or, for accuracy, from the normal approximation to the binomial in Equation (1.65). When comparing two models on the same data, paired

tests such as McNemar's test (for classifiers) or the corrected resampled t-test (for cross-validation scores) are more appropriate than comparing intervals.

$$\widehat{\mathrm{acc}} \pm z_{\alpha/2}\sqrt{\frac{\widehat{\mathrm{acc}}\,(1-\widehat{\mathrm{acc}})}{n_{\mathrm{test}}}} \tag{1.65}$$

where n_{test} is the number of test examples and $z_{\alpha/2} = 1.96$ for a 95% interval.

> **Note | The Test Set Is Sacred**
>
> Every decision informed by test-set performance — choosing features, hyperparameters or even which model family to report — leaks information and biases the final estimate upwards. Use validation data or cross-validation for all choices, and touch the test set once, at the end. If it must be reused, report the result as optimistic and, ideally, collect a fresh hold-out set.

> **Key Point | Beyond a Single Number**
>
> Good evaluation reports a primary metric with its uncertainty, a baseline, performance broken down by important subgroups (for fairness and robustness), and an error analysis of representative failures. Offline metrics should finally be confirmed by online evaluation, such as an A/B test, after deployment.

1.38 Classification Metrics: Accuracy, Precision, Recall, F1-Score, and ROC-AUC

Most classification metrics are derived from the **confusion matrix**, which cross-tabulates actual and predicted classes. For a binary problem with a designated *positive* class (for example "disease present"), the four cells are the true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN), arranged as in Table 1.37. A false positive is also called a *Type I error* and a false negative a *Type II error*.

Table 1.37 Layout of the binary confusion matrix.

	Predicted positive	Predicted negative
Actual positive	True positive (TP)	False negative (FN) — Type II error
Actual negative	False positive (FP) — Type I error	True negative (TN)

1.38.1 Accuracy, Precision, Recall and F1

The core metrics are defined in Equation (1.66). **Accuracy** is the fraction of all predictions that are correct. **Precision** (positive predictive value) answers "of the cases predicted positive, how many really are?" **Recall** (sensitivity, true positive rate) answers "of the actual positives, how many did we find?" **Specificity** (true negative rate) is the recall of the negative class. The **F1-score** is the harmonic mean of precision and recall; because the harmonic mean is dominated by the smaller value, F1 is high only when both are high. The more general **F_β-score** weights recall β times as much as precision — F_2 is common when missing positives is costly.

$$\begin{gathered}\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \qquad \text{Precision} = \frac{TP}{TP+FP} \\ \text{Recall} = \text{TPR} = \frac{TP}{TP+FN}, \qquad \text{Specificity} = \frac{TN}{TN+FP}, \qquad \text{FPR} = 1 - \text{Specificity} \\ F_1 = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}, \qquad F_\beta = \frac{(1+\beta^2)\,\text{Precision}\cdot\text{Recall}}{\beta^2\,\text{Precision}+\text{Recall}}\end{gathered} \tag{1.66}$$

E Worked Example 1.28 | Metrics from a Confusion Matrix

A diabetes screening model is evaluated on 200 patients, 50 of whom have diabetes. It produces TP = 40, FN = 10, FP = 20, TN = 130.

- Accuracy = (40 + 130)/200 = 0.850.
- Precision = 40/(40 + 20) = 0.667 — one in three positive alerts is a false alarm.
- Recall = 40/(40 + 10) = 0.800 — the model finds 80% of diabetic patients.
- Specificity = 130/(130 + 20) = 0.867; FPR = 0.133.
- F1 = 2 × 0.667 × 0.800/(0.667 + 0.800) = 1.067/1.467 ≈ 0.727.
- F_2 = 5 × 0.667 × 0.800/(4 × 0.667 + 0.800) = 2.667/3.467 ≈ 0.769 (higher, because recall is the stronger component).

A trivial model predicting "no diabetes" for everyone would score 150/200 = 0.75 accuracy but zero recall — illustrating why accuracy alone is misleading under class imbalance.

For **multi-class** problems, per-class precision, recall and F1 are computed one-versus-rest and then averaged. **Macro-averaging** takes the unweighted mean across classes, treating all classes equally and highlighting performance on rare ones; **weighted averaging** weights each class by its support; **micro-averaging** pools all TP, FP and FN counts before computing the metric, which for single-label multi-class problems equals accuracy. Cohen's kappa and the **Matthews correlation coefficient** (MCC) are single-number summaries that remain informative under imbalance.

1.38.2 Thresholds, ROC Curves and AUC

Most classifiers output a score or probability, and a class is assigned by comparing it with a threshold t (by default 0.5). Lowering t increases recall at the cost of precision; raising it does the opposite. The **receiver operating characteristic** (ROC) curve, which originated in signal detection theory during the Second World War, plots the true positive rate against the false positive rate as t sweeps from $+\infty$ to $-\infty$. A perfect classifier passes through the top-left corner (FPR = 0, TPR = 1); a random classifier follows the diagonal. The **area under the ROC curve** (ROC-AUC) summarises the curve in one number and has an elegant probabilistic interpretation, given in Equation (1.67): it is the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative one. An AUC of 0.5 corresponds to random ranking and 1.0 to perfect ranking. AUC is threshold-independent and insensitive to class proportions, which makes it a good measure of *ranking* quality.

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}\big(\mathrm{FPR}^{-1}(u)\big)\, du = P\big(s(\mathbf{x}^+) > s(\mathbf{x}^-)\big) \tag{1.67}$$

where $s(\cdot)$ is the classifier's score, $\mathbf{x}^+$ a random positive example and $\mathbf{x}^-$ a random negative example (ties count one half).

Under heavy class imbalance, however, ROC curves can look deceptively good, because the FPR denominator (all negatives) is so large that thousands of false positives barely move it. The **precision–recall (PR) curve** plots precision against recall and is far more sensitive to false positives when positives are rare; its baseline is the positive-class prevalence rather than 0.5. Its summary, **average precision** (PR-AUC), is the preferred metric for fraud, rare-disease and information-retrieval problems (Saito and Rehmsmeier, 2015). Figure 1.45 compares ROC and PR curves for two models on an imbalanced dataset with 5% positives.

Figure 1.46 makes the threshold trade-off explicit for the logistic-regression model. The optimal operating point depends on the application's costs: one may choose the threshold that maximises F1, that achieves a required recall, or that minimises expected cost given the costs of false positives and false negatives. Threshold selection must be done on validation data, never on the test set.

1.38.3 Computing Classification Metrics in Practice

Listing 1.22 computes the metrics of this section with scikit-learn. Table 1.38 summarises when each is most informative.

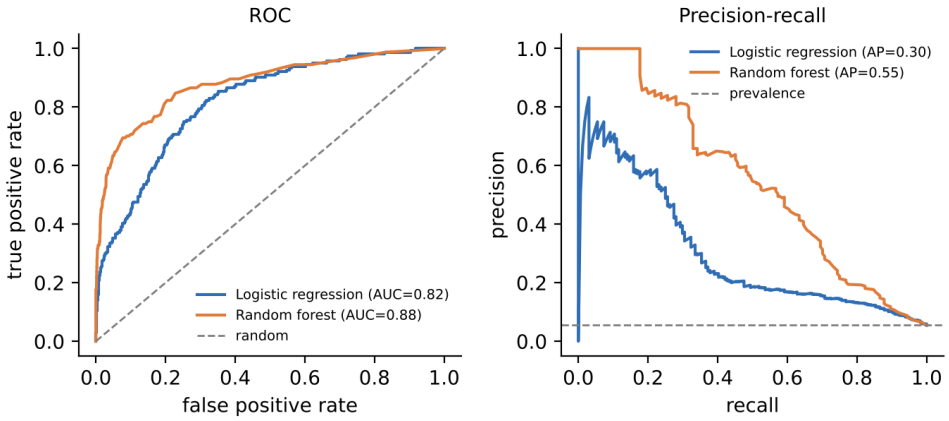


Figure 1.45 ROC curves (left) and precision–recall curves (right) for logistic regression and a random forest on imbalanced data (5% positives). Both ROC curves look strong, but the PR curves reveal much larger differences in precision.

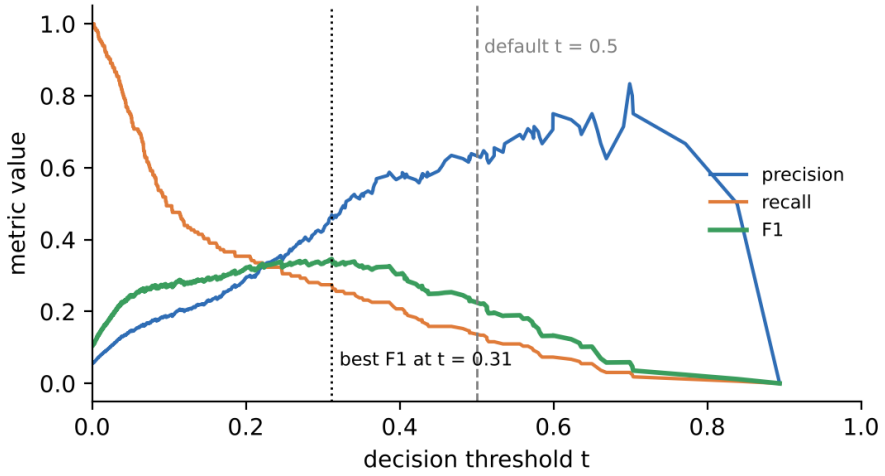


Figure 1.46 Precision, recall and F1 of a logistic-regression model as functions of the decision threshold. The dotted line marks the threshold maximising F1, which is well below the default of 0.5 on this imbalanced problem.

Listing 1.22 Confusion matrix, classification report, ROC-AUC and average precision.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report, roc_auc_score,
                             average_precision_score, matthews_corrcoef, f1_score)

X, y = load_breast_cancer(return_X_y=True)
```

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y,
    random_state=1)

clf = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)).fit(X_tr,
    y_tr)
proba = clf.predict_proba(X_te)[:, 1]
y_pred = (proba >= 0.5).astype(int)

print(confusion_matrix(y_te, y_pred))                # rows = actual, cols = predicted
print(classification_report(y_te, y_pred, digits=3))  # per-class precision, recall, F1
print("ROC-AUC :", round(roc_auc_score(y_te, proba), 3))
print("PR-AUC  :", round(average_precision_score(y_te, proba), 3))
print("MCC     :", round(matthews_corrcoef(y_te, y_pred), 3))
print("macro F1:", round(f1_score(y_te, y_pred, average="macro"), 3))
```

Table 1.38 Summary of classification metrics.

Metric	Question answered	Range	Caveat
Accuracy	What fraction of predictions is correct?	0–1	Misleading under imbalance
Precision	How trustworthy is a positive prediction?	0–1	Ignores false negatives
Recall (sensitivity)	What fraction of positives is found?	0–1	Ignores false positives
Specificity	What fraction of negatives is correctly rejected?	0–1	Ignores the positive class
F1-score	Balance of precision and recall	0–1	Ignores true negatives; equal weighting
MCC	Correlation between prediction and truth	−1 to 1	Less intuitive to stakeholders
ROC-AUC	How well are positives ranked above negatives?	0–1 (0.5 random)	Optimistic under heavy imbalance
PR-AUC (average precision)	Ranking quality focused on positives	prevalence–1	Baseline depends on prevalence
Log-loss	How good are the predicted probabilities?	0–∞	Heavily penalises confident errors

1.39 Regression Metrics: MAE, MSE, RMSE, and R^2

For regression, where the target is continuous, metrics summarise the residuals $e_i = y_i - \hat{y}_i$. The four most widely used are defined in Equation (1.68). The **mean absolute error** (MAE) is the average magnitude of the errors, in the units of the target. The **mean squared error** (MSE) averages squared errors and therefore penalises large errors disproportionately; it is the loss minimised by ordinary least squares. The **root**

mean squared error (RMSE) is the square root of the MSE, restoring the target's units. The **coefficient of determination** R^2 compares the model's squared error with that of a baseline that always predicts the mean $\bar{y}$.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}$$
$$R^2 = 1 - \frac{SS_{\mathrm{res}}}{SS_{\mathrm{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \tag{1.68}$$

where y_i are true values, $\hat{y}_i$ predictions, $\bar{y}$ the mean of the true values, SS_{res} the residual sum of squares and SS_{tot} the total sum of squares.

1.39.1 Interpreting the Metrics

MAE and RMSE are both in the target's units, but they weigh errors differently. Because RMSE squares errors before averaging, RMSE $\geq$ MAE always, and a large gap between them signals that a few large errors dominate. MAE is more robust to outliers and corresponds to predicting the conditional *median*; MSE corresponds to the conditional *mean*. R^2 is unit-free: $R^2 = 1$ is a perfect fit, $R^2 = 0$ means the model is no better than predicting $\bar{y}$, and R^2 can be negative on test data when the model is worse than that baseline. Because training R^2 never decreases when features are added, the **adjusted R^2** of Equation (1.69) penalises model size and is used for comparing linear models with different numbers of predictors.

$$\bar{R}^2 = 1 - (1 - R^2)\,\frac{n-1}{n-p-1} \tag{1.69}$$

where n is the number of observations and p the number of predictors.

E Worked Example 1.29 | Computing Regression Metrics

True values $y = (3, 5, 2, 7, 4)$; predictions $\hat{y} = (2.5, 5, 3, 8, 4)$.

- Residuals $e = (0.5, 0, -1, -1, 0)$; absolute errors $(0.5, 0, 1, 1, 0)$; squared errors $(0.25, 0, 1, 1, 0)$.
- MAE $= 2.5/5 = 0.50$.
- MSE $= 2.25/5 = 0.45$; RMSE $= \sqrt{0.45} \approx 0.671$.
- $\bar{y} = 21/5 = 4.2$; $SS_{\mathrm{tot}} = 1.44 + 0.64 + 4.84 + 7.84 + 0.04 = 14.8$; $SS_{\mathrm{res}} = 2.25$.
- $R^2 = 1 - 2.25/14.8 \approx 0.848$: the model explains about 85% of the variance in y.
- With $p = 1$ predictor, adjusted $R^2 = 1 - 0.152 \times 4/3 \approx 0.797$.

1.39.2 Sensitivity to Outliers and Other Metrics

Figure 1.47 illustrates how a single large error affects each metric differently. Starting from 100 predictions with small Gaussian errors, one residual is gradually increased; RMSE grows almost linearly with the outlier's size, whereas MAE grows much more slowly. Other metrics address specific needs. The **mean absolute percentage error** (MAPE) expresses errors relative to the true value, which business users find intuitive, but it is undefined when $y = 0$ and asymmetric; symmetric MAPE and the mean absolute scaled error (MASE) are alternatives in forecasting. The **Huber loss** is quadratic for small errors and linear for large ones, combining MSE's smoothness with MAE's robustness. RMSLE (root mean squared logarithmic error) is used when relative errors matter and targets span orders of magnitude, such as prices or counts. Table 1.39 compares these metrics.

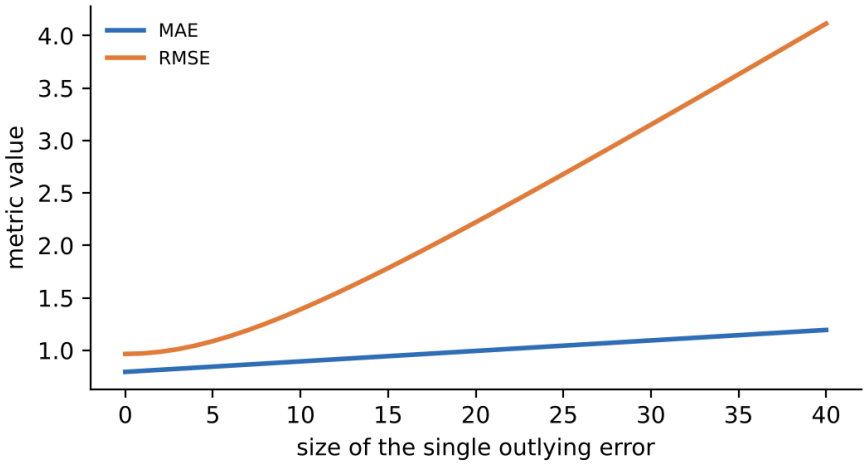


Figure 1.47 Effect of a single growing error on MAE and RMSE for 100 predictions with otherwise small errors: RMSE is far more sensitive to the outlier than MAE.

Listing 1.23 Regression metrics with scikit-learn.

```
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

X, y = fetch_california_housing(return_X_y=True)     # target: median house value
↪  ($100k)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=0)

for name, model in [("mean baseline", DummyRegressor()),
```

Table 1.39 Comparison of regression metrics.

Metric	Units	Outlier sensitivity	Typical use
MAE	Target units	Low	Robust reporting; median-type prediction
MSE	Squared units	High	Training loss; analytical convenience
RMSE	Target units	High	Default reporting when large errors are costly
R^2	Unit-free	High	Proportion of variance explained; model comparison on same data
Adjusted R^2	Unit-free	High	Comparing linear models with different numbers of features
MAPE	Percent	Moderate; undefined at $y = 0$	Business forecasting
RMSLE	Log units	Low for large values	Targets spanning orders of magnitude

```
                    ("gradient boosting",
                    ↪  HistGradientBoostingRegressor(random_state=0))]:
    pred = model.fit(X_tr, y_tr).predict(X_te)
    mae = mean_absolute_error(y_te, pred)
    rmse = np.sqrt(mean_squared_error(y_te, pred))
    print(f"{name:18s} MAE={mae:.3f}  RMSE={rmse:.3f}  R2={r2_score(y_te, pred):.3f}")
```

Practice Tip | Report More Than One Regression Metric

Report MAE and RMSE together (their ratio hints at the error distribution), state R^2 relative to a sensible baseline, and always plot residuals against predicted values: patterns in the residuals reveal non-linearity, heteroscedasticity or subgroups that no single summary number exposes.

1.40 Cross-Validation and Hyperparameter Tuning

A single train/validation split gives a noisy estimate of performance: a different random split might give a noticeably different score, particularly for small datasets. **Cross-validation** (CV) reduces this variance by repeating the split several times and averaging, and it uses the data more efficiently because every observation serves for both training and validation. Cross-validation is the workhorse for both **model assessment** (estimating generalisation performance) and **model selection** (choosing hyperparameters, features or algorithms).

1.40.1 k-Fold and Its Variants

In **k-fold cross-validation** the data are partitioned into k folds of roughly equal size. The model is trained k times, each time on k − 1 folds and validated on the remaining fold, and the k scores are averaged as in Equation (1.70), as illustrated in Figure 1.48. Values of k = 5 or 10 are standard: they offer a good balance between the bias of training on less data and the variance and computational cost of many folds (Kohavi, 1995). The standard deviation of the fold scores should be reported alongside the mean.

$$\mathrm{CV}_{(k)} = \frac{1}{k}\sum_{j=1}^{k} \mathcal{M}\left(f^{(-j)},\, D_j\right) \tag{1.70}$$

where $f^{(-j)}$ is the model trained on all folds except fold j, D_j is the j-th fold and $\mathcal{M}$ is the evaluation metric.

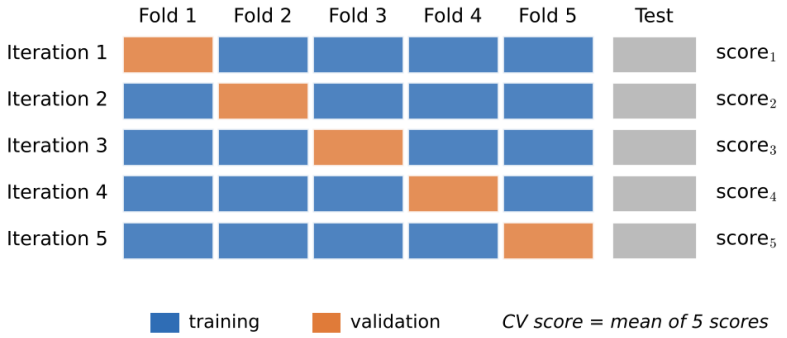


Figure 1.48 Five-fold cross-validation: in each of five iterations a different fold (orange) is held out for validation and the remaining four (blue) are used for training; the final test set (grey) is never used during cross-validation.

Several variants handle special data structures, as summarised in Table 1.40. **Stratified k-fold** preserves the class proportions in every fold, which is essential for imbalanced classification. **Leave-one-out CV** (LOOCV) sets k = n; it is nearly unbiased but expensive and has high variance, and is mainly used for very small datasets or models with closed-form LOO shortcuts (such as ridge regression). **Repeated k-fold** reruns k-fold with different shuffles to reduce variance further. **Group k-fold** keeps all observations from the same group (patient, user, site) in the same fold, preventing leakage when samples are not independent. For temporal data, random splitting would let the model train on the future and predict the past; **time-series split** (forward chaining) instead always trains on an initial segment and validates on the period immediately following, with an optional gap to avoid leakage from autocorrelation.

Table 1.40 Cross-validation schemes and when to use them.

Scheme	How folds are formed	Use when	scikit-learn class
k-fold	Random partition into k folds	Default for i.i.d. regression data	KFold
Stratified k-fold	Class proportions preserved per fold	Classification, especially imbalanced	StratifiedKFold
Leave-one-out	Each single point is a fold (k = n)	Very small datasets	LeaveOneOut
Repeated (stratified) k-fold	k-fold repeated with different shuffles	Small/medium data; stable estimates	RepeatedStratifiedKFold
Group k-fold	Groups never split across folds	Multiple samples per patient/user	GroupKFold
Time-series split	Expanding training window, next block validates	Temporal or sequential data	TimeSeriesSplit

1.40.2 Hyperparameter Search Strategies

Hyperparameters — the regularisation strength of logistic regression, the depth of a decision tree, the number of neighbours in k-NN — are set before training and control model capacity. **Hyperparameter tuning** searches for the configuration with the best cross-validated score. **Grid search** evaluates every combination on a predefined grid; it is exhaustive but its cost grows exponentially with the number of hyperparameters. **Random search** samples configurations from specified distributions. Bergstra and Bengio (2012) showed that random search is usually far more efficient, because typically only a few hyperparameters matter: with the same budget, random search tries many more distinct values of each important hyperparameter, as Figure 1.49 illustrates.

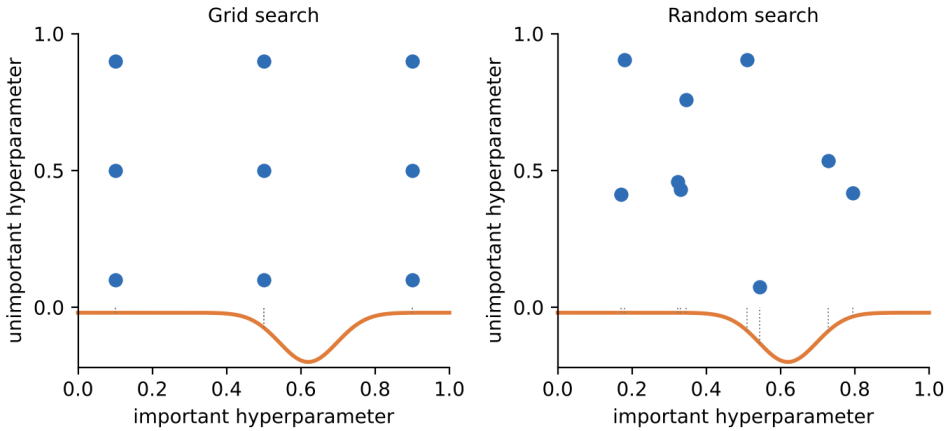


Figure 1.49 Grid search versus random search with nine trials when only one of two hyperparameters matters (Bergstra and Bengio, 2012). The grid tests only three distinct values of the important parameter; random search tests nine.

Bayesian optimisation goes further by learning from previous trials. It fits a proba-

bilistic *surrogate model* — a Gaussian process, or the tree-structured Parzen estimator (TPE) used by Hyperopt and Optuna — to the observed (configuration, score) pairs and uses an **acquisition function** such as expected improvement, defined in Equation (1.71), to choose the next configuration, balancing exploitation of promising regions against exploration of uncertain ones. The procedure is outlined in Algorithm 1.21. Multi-fidelity methods such as **successive halving** and **Hyperband** (Li et al., 2017) save further compute by evaluating many configurations cheaply (few epochs, small subsamples) and allocating more resources only to the most promising.

$$\mathrm{EI}(\lambda) = \mathbb{E}[\max(0,\ f(\lambda) - f^{*})] \tag{1.71}$$

where λ is a hyperparameter configuration, $f(\lambda)$ its (unknown) validation score modelled by the surrogate, and f^* the best score observed so far.

Algorithm 1.21 Bayesian Optimisation for Hyperparameter Tuning

Input: Search space Λ; objective $f(\lambda)$ = cross-validated score; initial design size n_0; budget N

Output: Best configuration λ^*

evaluate f at n_0 random configurations; $H \leftarrow \{(\lambda_i, f(\lambda_i))\}$
for $t = n_0 + 1$ **to** N **do**
  fit surrogate model $p(f \mid \lambda, H)$ (e.g. Gaussian process **or** TPE)
  $\lambda_t \leftarrow \operatorname{argmax}_{\lambda \in \Lambda}$ acquisition$(\lambda; p, H)$ ▷ *e.g. expected improvement*
  $y_t \leftarrow f(\lambda_t)$ ▷ *expensive: train + cross-validate*
  $H \leftarrow H \cup \{(\lambda_t, y_t)\}$
return λ^* = the configuration **in** H with the best score

1.40.3 Nested Cross-Validation and Practical Implementation

If the same CV folds are used both to select hyperparameters and to report performance, the reported score is optimistically biased, because the selection has overfit to those folds. **Nested cross-validation** solves this with two loops: an inner CV loop tunes hyperparameters on each outer training set, and the outer loop estimates the performance of the *whole tuning procedure* on held-out outer folds. Listing 1.24 shows grid and random search with a pipeline (so that preprocessing is refitted inside every fold, avoiding leakage) and nested CV.

Listing 1.24 Grid search, randomised search and nested cross-validation with scikit-learn.

```
from scipy.stats import loguniform
from sklearn.datasets import load_breast_cancer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
from sklearn.model_selection import (StratifiedKFold, GridSearchCV,
                                     RandomizedSearchCV, cross_val_score)

X, y = load_breast_cancer(return_X_y=True)
pipe = Pipeline([("scale", StandardScaler()), ("svc", SVC(kernel="rbf"))])
inner = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
outer = StratifiedKFold(n_splits=5, shuffle=True, random_state=1)

grid = GridSearchCV(pipe, {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [1e-3, 1e-2,
    1e-1]},
                    cv=inner, scoring="roc_auc", n_jobs=-1).fit(X, y)
print("grid best:", grid.best_params_, round(grid.best_score_, 4))

rand = RandomizedSearchCV(pipe, {"svc__C": loguniform(1e-2, 1e3), "svc__gamma":
    loguniform(1e-4, 1)},
                          n_iter=30, cv=inner, scoring="roc_auc", random_state=0,
                          n_jobs=-1).fit(X, y)
print("random best:", rand.best_params_, round(rand.best_score_, 4))

# Nested CV: unbiased estimate of the performance of the tuning procedure itself
nested = cross_val_score(rand, X, y, cv=outer, scoring="roc_auc")
print(f"nested CV ROC-AUC: {nested.mean():.4f} +/- {nested.std():.4f}")
```

Table 1.41 Comparison of hyperparameter search strategies.

Strategy	Uses past results?	Scales to many hyperparameters?	Parallelism	Typical tools
Grid search	No	Poorly (exponential)	Embarrassingly parallel	GridSearchCV
Random search	No	Well	Embarrassingly parallel	RandomizedSearchCV
Bayesian optimisation	Yes (surrogate model)	Moderately (≈ 20 dims)	Limited (sequential)	Optuna, Hyperopt, scikit-optimize
Successive halving / Hyperband	Yes (early stopping)	Well	Good	HalvingGridSearchCV, Ray Tune
Evolutionary / population-based	Yes (selection)	Well	Good	DEAP, PBT in Ray Tune

1.41 Optimization Algorithms and Gradient Descent

Almost every model in this chapter is trained by solving an **optimisation problem**: find parameters θ that minimise an objective $J(\theta)$, usually the average loss over the training data plus a regularisation term, as in Equation (1.72). Some problems have closed-form solutions — ordinary least squares via the normal equations — but most,

including logistic regression, support vector machines with large datasets, and all neural networks, are solved iteratively. This section presents the core iterative methods and the concepts needed to understand their behaviour; the adaptive optimisers used for deep networks (momentum, RMSProp, Adam) are treated in Chapter 2.

$$\theta^{*} = \arg\min_{\theta} \; J(\theta) = \tfrac{1}{n}\sum_{i=1}^{n} \ell(y_i,\, f(\mathbf{x}_i;\theta)) + \lambda\, \Omega(\theta) \tag{1.72}$$

where ℓ is a per-example loss, f the model, Ω a regulariser and $\lambda \geq 0$ its strength.

1.41.1 Convexity

A function J is **convex** if the line segment between any two points on its graph lies on or above the graph, formally $J(\alpha\theta_1 + (1-\alpha)\theta_2) \leq \alpha J(\theta_1) + (1-\alpha)J(\theta_2)$ for all $\alpha \in [0,1]$. For a twice-differentiable function this is equivalent to the Hessian being positive semi-definite everywhere. Convexity matters because *every local minimum of a convex function is a global minimum*, so gradient-based methods with suitable step sizes are guaranteed to find the best solution. The losses of linear regression, logistic regression, ridge and lasso regression and linear SVMs are convex. Neural-network losses are not: they have many local minima and saddle points, although in very high dimensions most local minima found in practice turn out to be of similar quality.

1.41.2 Gradient Descent and Its Variants

Gradient descent, attributed to Cauchy (1847), repeatedly takes a step in the direction of steepest descent, the negative gradient, as in Equation (1.73). The **learning rate** η controls the step size. The three main variants differ in how much data is used to compute each gradient. **Batch gradient descent** uses all n examples, giving the exact gradient but costing a full pass per step. **Stochastic gradient descent** (SGD), rooted in the stochastic approximation method of Robbins and Monro (1951), uses one randomly chosen example per step, giving a cheap but noisy estimate. **Mini-batch gradient descent** uses a small random batch (typically 32–512 examples), combining reduced noise with efficient vectorised computation on GPUs; it is the default in modern machine learning and is what practitioners usually mean by "SGD". The generic procedure is given in Algorithm 1.22, and Figure 1.50 compares the trajectories.

$$\begin{aligned} \theta_{t+1} &= \theta_t - \eta\, \nabla_\theta J(\theta_t) \quad \text{(batch)} \\ \theta_{t+1} &= \theta_t - \eta\, \tfrac{1}{|B_t|}\sum_{i \in B_t} \nabla_\theta\, \ell(y_i,\, f(\mathbf{x}_i;\theta_t)) \quad \text{(mini-batch)} \end{aligned} \tag{1.73}$$

where $\eta > 0$ is the learning rate and B_t is the mini-batch of example indices at step t ($|B_t| = 1$ gives SGD).

Algorithm 1.22 Mini-Batch Stochastic Gradient Descent

Input: Training data $\{(x_i, y_i)\}_{i=1}^{n}$; loss ℓ; initial parameters θ_0; learning-rate schedule η_t; batch size b; epochs E
Output: Parameters θ

$\theta \leftarrow \theta_0$; $t \leftarrow 0$
for epoch $= 1$ **to** E **do**
  randomly shuffle the training indices
  for each consecutive block B of b indices **do**
    $g \leftarrow (1/b) \sum_{i \in B} \nabla_\theta \ell(y_i, f(x_i; \theta))$ ▷ *stochastic gradient estimate*
    $\theta \leftarrow \theta - \eta_t\, g$
    $t \leftarrow t + 1$
  optionally evaluate on validation data; stop early **if** no improvement
return θ

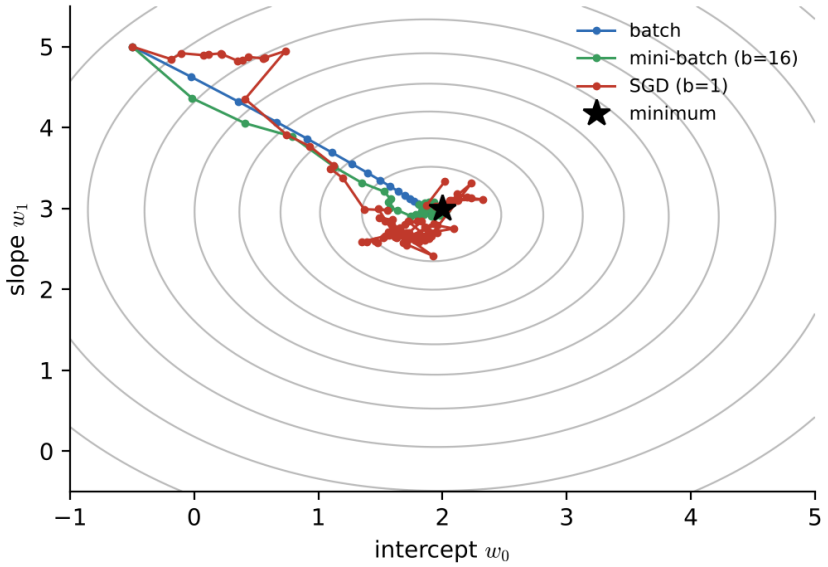


Figure 1.50 Optimisation paths on the convex least-squares loss of a two-parameter linear model: batch gradient descent moves smoothly; mini-batch ($b = 16$) and stochastic ($b = 1$) gradient descent follow noisier but much cheaper paths.

1.41.3 The Learning Rate

The learning rate is the most important hyperparameter of gradient-based training. If η is too small, progress is painfully slow; if too large, the iterates overshoot the minimum,

oscillate or diverge. For a convex quadratic objective, gradient descent converges only if $\eta < 2/L$, where L is the largest eigenvalue of the Hessian (the Lipschitz constant of the gradient). Figure 1.51 shows the three regimes on a simple quadratic. Because SGD's gradient noise does not vanish at the minimum, convergence also requires the learning rate to decrease over time; Robbins and Monro's conditions $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$ are satisfied, for example, by $\eta_t \propto 1/t$. In practice, step decay, exponential decay, cosine annealing and short warm-up phases are common learning-rate schedules.

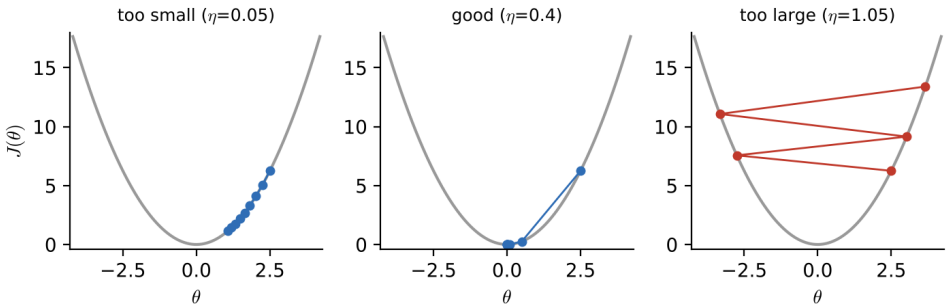


Figure 1.51 Effect of the learning rate on gradient descent for $J(\theta) = \theta^2$: a small rate converges slowly, a well-chosen rate converges quickly, and a rate above the stability limit ($\eta > 1$ here) diverges.

1.41.4 Second-Order Methods: Newton's Method

Gradient descent uses only first-order (slope) information. **Newton's method** also uses curvature, through the Hessian matrix $H = \nabla^2 J(\theta)$. It minimises a local quadratic approximation of J at each step, giving the update of Equation (1.74). Near a minimum, Newton's method converges *quadratically* — the number of correct digits roughly doubles per iteration — and it needs no learning rate for quadratic objectives, which it minimises in a single step. Logistic regression is classically fitted this way, where the method is known as iteratively reweighted least squares (IRLS). The cost, however, is forming and inverting a $d \times d$ Hessian, $O(d^2)$ memory and $O(d^3)$ time per step, which is prohibitive for models with millions of parameters. **Quasi-Newton** methods such as BFGS and its limited-memory variant **L-BFGS** build an approximation to the inverse Hessian from successive gradients and are the default solvers for many medium-sized convex problems, including scikit-learn's `LogisticRegression`.

$$\theta_{t+1} = \theta_t - H(\theta_t)^{-1}\,\nabla_\theta J(\theta_t), \qquad H_{jk} = \frac{\partial^2 J}{\partial\theta_j\,\partial\theta_k} \tag{1.74}$$

E Worked Example 1.30 | Gradient Descent versus Newton's Method

Minimise $J(\theta) = (\theta - 3)^2 + 1$, starting at $\theta_0 = 0$. The gradient is $J'(\theta) = 2(\theta - 3)$ and the second derivative $J'' = 2$.

- **Gradient descent, $\eta = 0.25$:** $\theta_1 = 0 - 0.25 \times (-6) = 1.5$; $\theta_2 = 1.5 - 0.25 \times (-3) = 2.25$; $\theta_3 = 2.625$; $\theta_4 = 2.8125$. The distance to the minimum halves at each step.
- **Newton's method:** $\theta_1 = 0 - (-6)/2 = 3$, the exact minimum in one step, because J is quadratic.

For non-quadratic objectives Newton's method needs several steps, but still far fewer than gradient descent near the solution.

Table 1.42 Comparison of optimisation methods for machine learning.

Method	Information used	Cost per step	Convergence (convex case)	Typical use
Batch gradient descent	Exact gradient	$O(nd)$	Linear (strongly convex)	Small datasets, teaching
SGD	Gradient of one example	$O(d)$	Sublinear, noisy	Online and very large-scale learning
Mini-batch SGD	Gradient of a batch	$O(bd)$	Sublinear, less noisy	Neural networks (with momentum/Adam)
Newton's method	Gradient + Hessian	$O(nd^2 + d^3)$	Quadratic near optimum	Small d, e.g. logistic regression (IRLS)
L-BFGS	Gradient history	$O(nd + md)$	Superlinear	Medium-sized smooth convex problems
Coordinate descent	One coordinate at a time	$O(n)$ per coordinate	Linear for many problems	Lasso, elastic net (glmnet, scikit-learn)

T Practice Tip | Practical Advice for Gradient-Based Training

- Standardise features: poorly scaled features create elongated loss contours that slow gradient descent.
- Find a good learning rate by trying values on a logarithmic grid (1e−4 ... 1) or with a learning-rate range test.
- Monitor both training and validation loss; use early stopping to regularise.
- Shuffle data each epoch so that mini-batches are representative.
- Verify analytic gradients against finite differences when implementing a new model.

1.42 Explainable and Interpretable Machine Learning

As machine learning moves into credit scoring, medical diagnosis, hiring and criminal justice, it is no longer enough for a model to be accurate; stakeholders must be able

to understand, trust and contest its decisions. Regulators increasingly require this: the EU General Data Protection Regulation (2016) gives individuals rights concerning automated decision-making, and the EU AI Act (2024) imposes transparency obligations on high-risk systems. **Explainable and interpretable machine learning** (often called XAI) provides methods to make model behaviour understandable to humans.

Definition 1.5 | Interpretability and Explainability

Interpretability is the degree to which a human can understand the cause of a model's decision, usually by inspecting the model itself (for example the coefficients of a linear model or the rules of a shallow tree). **Explainability** refers to methods that produce human-understandable explanations of a model's behaviour, often *post hoc* for models that are not intrinsically interpretable. The two terms are frequently used interchangeably.

1.42.1 A Taxonomy of Explanation Methods

Explanation methods can be classified along three axes. **Intrinsic** interpretability comes from using a transparent model — linear and logistic regression, short decision trees, rule lists or generalised additive models — whereas **post-hoc** methods explain an already trained, possibly opaque model. **Model-specific** methods exploit a model's internals (tree-based impurity importance, gradients of a neural network), whereas **model-agnostic** methods treat the model as a black box and only query its predictions. **Global** explanations describe overall model behaviour (which features matter on average), while **local** explanations account for a single prediction (why *this* loan was refused). Table 1.43 positions the main methods on these axes. Rudin (2019) argues that for high-stakes decisions, intrinsically interpretable models should be preferred wherever they achieve comparable accuracy, rather than explaining black boxes after the fact.

1.42.2 Feature Importance and Partial Dependence

Permutation importance (introduced by Breiman (2001) for random forests and generalised by Fisher, Rudin and Dominici, 2019) measures how much a model's validation score drops when the values of one feature are randomly shuffled, breaking its relationship with the target, as in Equation (1.75). Unlike the impurity-based importance built into tree ensembles, it is computed on held-out data and is not biased towards high-cardinality features, but it can understate the importance of correlated features, since the model may fall back on a correlated partner.

$$\mathrm{PI}_j = s - \frac{1}{K}\sum_{k=1}^{K} s_{k,j} \tag{1.75}$$

Table 1.43 Common explanation methods and their properties.

Method	Scope	Model dependence	Output
Coefficients / tree rules	Global	Intrinsic	Weights or decision paths
Impurity-based importance	Global	Tree ensembles only	Importance score per feature
Permutation importance	Global	Model-agnostic	Drop in score when a feature is shuffled
Partial dependence plot (PDP)	Global	Model-agnostic	Average effect curve of one or two features
Individual conditional expectation (ICE)	Local/global	Model-agnostic	One effect curve per instance
LIME	Local	Model-agnostic	Sparse linear surrogate around one instance
SHAP	Local (aggregates to global)	Agnostic; fast exact versions for trees	Additive contribution per feature
Counterfactual explanations	Local	Model-agnostic	Minimal change that flips the decision

where s is the model's score on validation data and $s_{k,j}$ the score after the k-th random permutation of feature j.

Importance tells us *which* features matter but not *how*. A **partial dependence plot** (PDP; Friedman, 2001) shows the average predicted outcome as one feature is varied while the others keep their observed values, as defined in Equation (1.76). **Individual conditional expectation** (ICE) curves plot the same quantity separately for each instance, revealing heterogeneous effects that the average can hide. Figure 1.52 shows the PDP with ICE curves for a gradient-boosted model of simulated house prices in which house age raises the price of coastal homes but lowers that of inland ones.

$$\hat{f}_S(x_S) = \frac{1}{n}\sum_{i=1}^{n} \hat{f}\left(x_S,\ \mathbf{x}_C^{(i)}\right) \tag{1.76}$$

where S is the feature (or pair of features) of interest, C the remaining features and $x_C^{(i)}$ their observed values for instance i.

1.42.3 SHAP: Shapley Additive Explanations

SHAP (Lundberg and Lee, 2017) grounds local explanations in cooperative game theory. It treats the features of an instance as players in a game whose payout is the model's prediction, and assigns each feature its **Shapley value** (Shapley, 1953): the average marginal contribution of that feature over all possible orders in which features could be added, given in Equation (1.77). Shapley values are the unique attribution satisfying desirable axioms — *efficiency* (contributions sum to the difference between the prediction and the average prediction), *symmetry*, *dummy* (a feature that never changes

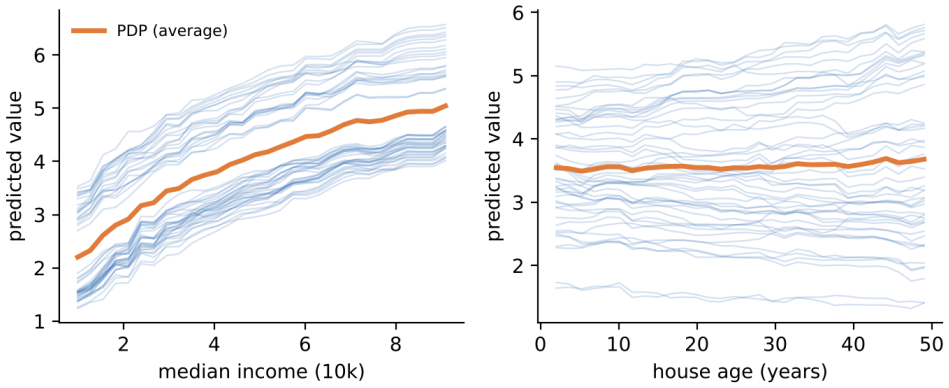


Figure 1.52 Partial dependence (thick line) and individual conditional expectation curves (thin lines, 50 houses) for income and house age in a gradient-boosted model of synthetic house prices. The income effect is uniform and concave; for age, ICE curves fan out in two directions (coastal vs inland homes), a heterogeneity the flat average PDP hides.

the output gets zero) and *additivity*. The resulting explanation is additive: $f(x) = \varphi_0 + \Sigma_j \varphi_j$, where φ_0 is the expected model output. Exact computation is exponential in the number of features, but **TreeSHAP** computes exact values for tree ensembles in polynomial time, and **KernelSHAP** approximates them for any model by weighted linear regression on sampled coalitions. Aggregating absolute SHAP values over many instances yields a global importance ranking, and SHAP summary (beeswarm) plots show both importance and direction of effect.

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{j\}) - v(S) \right] \tag{1.77}$$

where F is the set of all features, S a subset (coalition) not containing j, and $v(S)$ the expected model output when only the features in S are known.

E Worked Example 1.31 | Shapley Values for Two Features

A model predicts a loan default risk. The average prediction over the data is $v(\emptyset) = 0.20$. Knowing only income gives $v(\{\text{income}\}) = 0.30$; knowing only debt gives $v(\{\text{debt}\}) = 0.35$; knowing both gives the actual prediction $v(\{\text{income, debt}\}) = 0.50$.

- Orders: (income, debt) gives income a marginal contribution of $0.30 - 0.20 = 0.10$ and debt $0.50 - 0.30 = 0.20$. (debt, income) gives debt $0.35 - 0.20 = 0.15$ and income $0.50 - 0.35 = 0.15$.
- $\varphi_\text{income} = (0.10 + 0.15)/2 = 0.125$; $\varphi_\text{debt} = (0.20 + 0.15)/2 = 0.175$.
- Check (efficiency): $0.20 + 0.125 + 0.175 = 0.50$, the model's prediction.

1.42.4 LIME: Local Interpretable Model-Agnostic Explanations

LIME (Ribeiro, Singh and Guestrin, 2016) explains a single prediction by fitting a simple, interpretable surrogate model — usually a sparse linear model — that mimics the black box in the *neighbourhood* of the instance. It generates perturbed samples around the instance (for tabular data by sampling feature values; for text by removing words; for images by switching off super-pixels), obtains the black-box predictions for them, weights them by proximity to the original instance and fits the surrogate, as summarised in Algorithm 1.23 and formalised in Equation (1.78). LIME is intuitive and works for any data type, but its explanations can be unstable: different random perturbations or kernel widths can yield different explanations for the same instance.

$$\xi(\mathbf{x}) = \arg\min_{g \in G} \; \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g) \tag{1.78}$$

where f is the black-box model, g a surrogate from an interpretable class G, π_x a proximity kernel around x, $\mathcal{L}$ the weighted fidelity loss and Ω(g) a complexity penalty (e.g. number of non-zero weights).

Algorithm 1.23 LIME for Tabular Data

Input: Black-box model f; instance x; number of samples N; kernel width σ; number of features K

Output: Weights of a sparse linear explanation for f(x)

for i = 1 **to** N **do**
  $z_i \leftarrow$ perturbation of x (sample features from training distribution)
  $y_i \leftarrow f(z_i)$ ▷ *query black box*
  $w_i \leftarrow \exp(-d(x, z_i)^2 \,/\, \sigma^2)$ ▷ *proximity weight*
select K features (e.g. by forward selection **or** lasso path)
fit weighted linear model g on $\{(z_i, y_i, w_i)\}$ using the K features
return coefficients of g as the explanation

Listing 1.25 Permutation importance, partial dependence and SHAP for a random forest.

```python
import shap
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance, PartialDependenceDisplay

X, y = load_diabetes(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=0)
model = RandomForestRegressor(n_estimators=300, random_state=0).fit(X_tr, y_tr)

# Global: permutation importance on held-out data
pi = permutation_importance(model, X_te, y_te, n_repeats=20, random_state=0)
for name, imp in sorted(zip(X.columns, pi.importances_mean), key=lambda t: -t[1])[:5]:
    print(f"{name:5s} {imp:.3f}")
```

```
# Global: partial dependence of the two most important features
PartialDependenceDisplay.from_estimator(model, X_te, ["bmi", "s5"], kind="both")

# Local + global: SHAP values with the fast TreeExplainer
explainer = shap.TreeExplainer(model)
sv = explainer(X_te)
shap.plots.waterfall(sv[0])        # explanation of one prediction
shap.plots.beeswarm(sv)            # global summary of all predictions
```

> **Note | Explanations Are Not Causal**
>
> Feature importance, PDPs and SHAP values describe how the *model* uses its inputs, not how the world works. A feature can receive high importance because it is a proxy for a true cause or because of data leakage. Strongly correlated features make all these methods harder to interpret, and PDPs can extrapolate into unrealistic feature combinations. Use explanations to debug and communicate models, and use causal inference methods when the question is about interventions.

1.43 Machine Learning Pipelines and Automation

A trained model is only one component of a working machine learning system. Real systems ingest raw data, validate it, clean and transform it, engineer features, train and evaluate models, and then serve predictions and monitor them in production. Sculley et al. (2015) famously observed that in mature systems the model code is a small fraction of the whole, surrounded by data collection, configuration, feature extraction, serving and monitoring infrastructure, and that neglecting this surrounding "glue" creates *hidden technical debt*. **Pipelines** organise these steps into reproducible, automated workflows.

1.43.1 Pipelines in Code

At the modelling level, a pipeline chains preprocessing steps and an estimator into a single object that is fitted and applied as a unit. This has three benefits. It **prevents data leakage**, because statistics such as means, scaling factors and imputation values are learned only from the training portion of each cross-validation fold. It **simplifies deployment**, because the exact same transformations are applied to new data. And it **enables joint tuning** of preprocessing and model hyperparameters. Listing 1.26 builds a scikit-learn pipeline for mixed numerical and categorical data with a `ColumnTransformer`, and Figure 1.53 shows its structure.

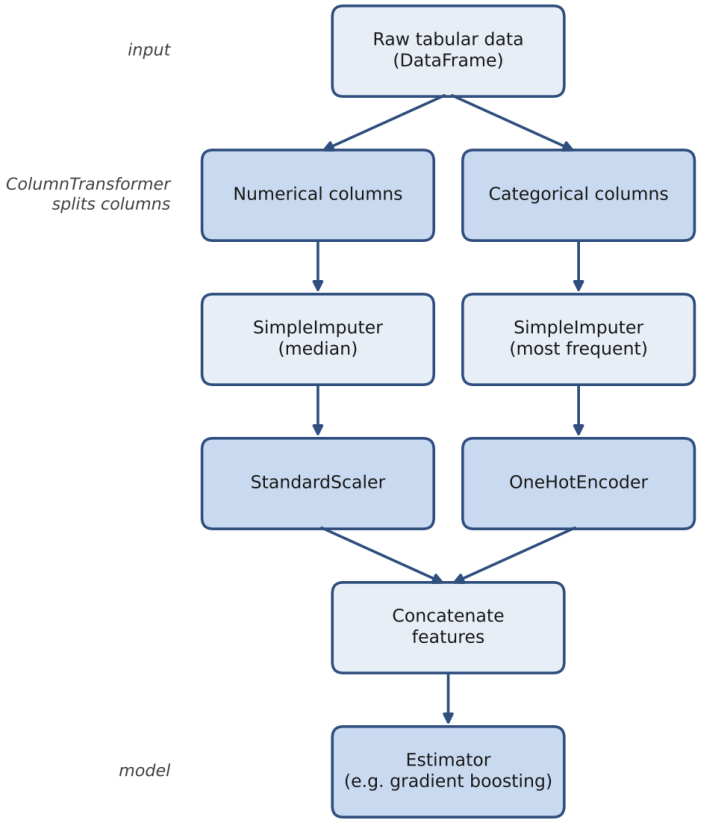


Figure 1.53 Structure of a scikit-learn pipeline for tabular data: numerical and categorical columns are processed in parallel branches of a ColumnTransformer, concatenated and passed to the estimator; the whole object is fitted and tuned as one model.

Listing 1.26 A leakage-safe preprocessing and modelling pipeline for mixed-type data.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.model_selection import cross_val_score
import joblib

df = pd.read_csv("customers.csv")                       # hypothetical churn dataset
X, y = df.drop(columns="churned"), df["churned"]
num_cols = X.select_dtypes("number").columns
cat_cols = X.select_dtypes(exclude="number").columns
```

```

preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), num_cols),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), cat_cols)])

model = Pipeline([("prep", preprocess),
                  ("clf", HistGradientBoostingClassifier(random_state=0))])

print(cross_val_score(model, X, y, cv=5, scoring="roc_auc").mean())  # no leakage
model.fit(X, y)
joblib.dump(model, "churn_model.joblib")                # one artefact for deployment
```

1.43.2 From Pipelines to MLOps

At the system level, **MLOps** applies DevOps principles — automation, version control, continuous integration and delivery, and monitoring — to machine learning. A typical end-to-end workflow, sketched in Figure 1.54, includes data ingestion and validation (checking schemas, ranges and distributions), feature engineering (often via a *feature store* that serves the same features for training and inference), training and evaluation, registration of approved models in a *model registry*, deployment as a batch job or online service, and continuous monitoring of input data and predictions. Monitoring closes the loop: when **data drift** (a change in the input distribution) or **concept drift** (a change in the relationship between inputs and target) degrades performance, retraining is triggered automatically or by an engineer. Google's maturity model distinguishes level 0 (manual, script-driven), level 1 (automated training pipeline with continuous training) and level 2 (automated CI/CD of the pipeline itself).

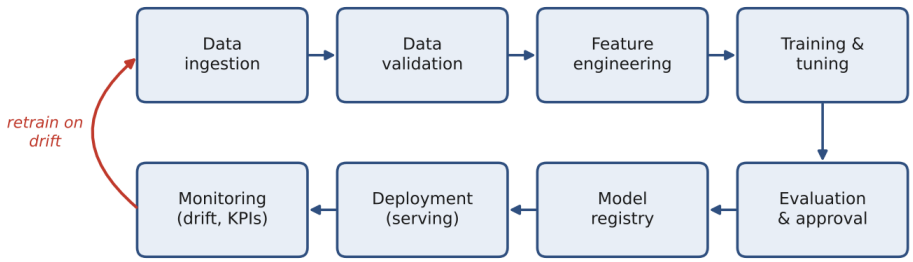


Figure 1.54 An automated machine learning lifecycle (MLOps): production monitoring feeds back into data collection and triggers retraining, forming a continuous loop.

Table 1.44 Tools supporting ML pipelines and MLOps.

Stage	Purpose	Example tools
Pipeline definition (in-process)	Chain preprocessing and model	scikit-learn Pipeline, Spark ML Pipelines
Workflow orchestration	Schedule and run multi-step jobs	Apache Airflow, Kubeflow Pipelines, Prefect
Experiment tracking	Log parameters, metrics, artefacts	MLflow, Weights & Biases
Data and model versioning	Reproducibility of datasets and models	DVC, MLflow Model Registry
Data validation	Detect schema and distribution problems	Great Expectations, TensorFlow Data Validation
Feature store	Consistent features for training and serving	Feast, Tecton
Serving	Expose models as APIs or batch jobs	FastAPI, BentoML, TensorFlow Serving, TorchServe
Monitoring	Track drift and performance	Evidently, Prometheus/Grafana, cloud monitoring services

T Practice Tip | Reproducibility Checklist

- Version code, data and configuration together; record random seeds.
- Wrap all preprocessing inside the pipeline, never in ad hoc notebook cells.
- Log every experiment's parameters, metrics and environment (library versions).
- Validate input data at training *and* serving time with the same checks.
- Store the fitted pipeline as a single artefact and test it on held-out data before deployment.

1.44 AutoML and Automated Feature Engineering

Building a good model involves many decisions: how to preprocess each feature, which features to construct, which algorithm to use, and how to set its hyperparameters. **Automated machine learning** (AutoML) aims to automate these decisions, making machine learning accessible to non-experts and freeing experts from tedious trial and error. The field grew out of hyperparameter optimisation and was popularised by systems such as Auto-WEKA (Thornton et al., 2013) and Auto-sklearn (Feurer et al., 2015).

1.44.1 The CASH Problem

AutoML is often formalised as the **combined algorithm selection and hyperparameter optimisation** (CASH) problem of Equation (1.79): given a set of algorithms, each with its own hyperparameter space, find the algorithm and configuration that minimise the cross-validated loss. The search space is large, conditional (an SVM's kernel width matters only if the SVM is chosen) and expensive to evaluate, so AutoML systems combine the search techniques of the cross-validation section — Bayesian optimisation, successive halving, evolutionary search — with additional tricks. Auto-sklearn uses **meta-learning**, warm-starting the search with configurations that worked well on similar datasets, and builds a final **ensemble** of the best models found. AutoGluon (Erickson et al., 2020) instead relies mainly on multi-layer stacking of many strong default models, which often outperforms extensive hyperparameter search.

$$A^{*}_{\lambda^{*}} \in \arg\min_{A^{(j)} \in \mathcal{A},\ \lambda \in \Lambda^{(j)}} \frac{1}{k}\sum_{i=1}^{k} \mathcal{L}\left(A^{(j)}_{\lambda},\ D^{(i)}_{\text{train}},\ D^{(i)}_{\text{valid}}\right) \tag{1.79}$$

where $\mathcal{A}$ is the set of candidate algorithms, $\Lambda^{(j)}$ the hyperparameter space of algorithm $A^{(j)}$ and $\mathcal{L}$ the validation loss on the i-th of k cross-validation folds.

A generic AutoML workflow is shown in Figure 1.55. Beyond classical tabular learning, **neural architecture search** (NAS) applies the same idea to the design of deep networks, searching over layer types, connections and widths; early reinforcement-learning-based NAS (Zoph and Le, 2017) required thousands of GPU-days, while later weight-sharing and differentiable methods such as DARTS reduced this to a few GPU-days.

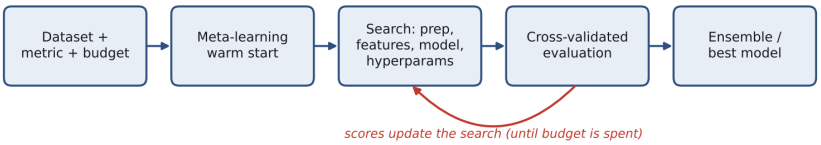


Figure 1.55 A generic AutoML workflow: the system searches jointly over preprocessing, feature engineering, algorithms and hyperparameters, evaluates candidates by cross-validation and returns the best model or an ensemble.

1.44.2 Automated Feature Engineering

Feature engineering, discussed earlier in this chapter, is often where domain expertise adds the most value — and where most time is spent. **Automated feature engineering** systematically generates candidate features and selects the useful ones. For a

single table, generators apply mathematical transformations (logarithms, powers, ratios and products of pairs of features), binning and date-part extraction. For relational data, **Deep Feature Synthesis** (Kanter and Veeramachaneni, 2015), implemented in the Featuretools library, follows relationships between tables and stacks aggregation primitives (count, mean, max, trend) and transform primitives — for example creating "mean amount of a customer's transactions in the last 30 days" from a customers table and a transactions table. For time series, libraries such as tsfresh extract hundreds of statistical descriptors and filter them with hypothesis tests. Because generation produces many redundant or irrelevant features, it must be followed by rigorous selection inside cross-validation to avoid overfitting.

Listing 1.27 AutoML on tabular data with AutoGluon and FLAML.

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("customers.csv")                       # hypothetical churn dataset
train, test = train_test_split(df, test_size=0.2, stratify=df["churned"],
    random_state=0)

# Option 1: AutoGluon - multi-layer stacked ensembles with strong defaults
from autogluon.tabular import TabularPredictor
predictor = TabularPredictor(label="churned", eval_metric="roc_auc").fit(
    train, time_limit=600, presets="best_quality")
print(predictor.leaderboard(test).head())

# Option 2: FLAML - fast, cost-aware search over learners and hyperparameters
from flaml import AutoML
automl = AutoML()
automl.fit(X_train=train.drop(columns="churned"), y_train=train["churned"],
           task="classification", metric="roc_auc", time_budget=300)
print(automl.best_estimator, automl.best_config)
```

> **Note | Limits of AutoML**
>
> AutoML automates model *search*, not problem formulation. It cannot decide what to predict, whether the labels are trustworthy, whether features will be available at prediction time, or whether the model is fair. Its results can also overfit the validation data when the search is very large. Treat AutoML output as a strong baseline to be scrutinised, not as a finished solution.

1.45 Machine Learning Tools and Frameworks

The machine learning ecosystem offers a rich set of open-source libraries, each with a particular focus. Choosing well depends on the data type, model family, scale and de-

Table 1.45 Selected AutoML systems.

System	Main technique	Notes
Auto-WEKA	Bayesian optimisation (SMAC) over WEKA	Early CASH system (2013)
Auto-sklearn	Bayesian optimisation + meta-learning + ensembling	Built on scikit-learn
TPOT	Genetic programming over pipelines	Exports Python pipeline code
H2O AutoML	Random search + stacked ensembles	Scales on clusters; Java core
AutoGluon	Multi-layer stacking of default models	Strong tabular, text and image performance
FLAML	Cost-frugal hyperparameter search	Lightweight, fast
Cloud AutoML services	Proprietary search and NAS	Google Vertex AI, Azure AutoML, Amazon SageMaker Autopilot

ployment target. Python dominates the field thanks to its readability and the maturity of its scientific stack, although R remains strong in statistics, and Julia, Scala (Spark) and Java (WEKA, H2O) have their niches.

1.45.1 The Scientific Python Stack

Nearly every project rests on a small set of foundation libraries. **NumPy** provides the n-dimensional array and vectorised numerical operations; **pandas** provides labelled data frames for loading, cleaning, joining and aggregating tabular data (newer alternatives such as Polars offer faster, multi-threaded execution); **SciPy** adds optimisation, linear algebra, statistics and sparse matrices; and **Matplotlib** and **seaborn** provide visualisation, with Plotly for interactive charts. **Jupyter** notebooks combine code, output and narrative and are the standard environment for exploration and teaching.

1.45.2 Classical Machine Learning Libraries

scikit-learn (Pedregosa et al., 2011) is the standard library for classical machine learning. Its consistent estimator API — `fit`, `predict`, `transform`, `score` — covers preprocessing, supervised and unsupervised algorithms, model selection and pipelines, and most methods in this chapter are available in it. For gradient-boosted trees, the specialised libraries **XGBoost** (Chen and Guestrin, 2016), **LightGBM** (Ke et al., 2017) and **CatBoost** (Prokhorenkova et al., 2018) offer state-of-the-art accuracy and speed on tabular data and dominate many data-science competitions; CatBoost is notable for its native handling of categorical features. **statsmodels** complements these with classi-

cal statistical models and inference (p-values, confidence intervals), and **imbalanced-learn** adds resampling methods such as SMOTE.

1.45.3 Deep Learning Frameworks and Platforms

For neural networks, **PyTorch** (released by Facebook AI Research in 2016) and **TensorFlow** (released by Google in 2015) are the two dominant frameworks. PyTorch's define-by-run (eager) style made it the favourite in research, while TensorFlow, with its high-level **Keras** API, has strong production and mobile deployment tooling (TensorFlow Serving, TensorFlow Lite). Keras 3 now runs on TensorFlow, PyTorch or **JAX**, Google's library for composable automatic differentiation and just-in-time compilation. **Hugging Face Transformers** provides thousands of pre-trained models for language, vision and audio. These are covered in Chapter 2. For big data, **Apache Spark MLlib** distributes training across clusters, and **ONNX** provides a common format for exchanging trained models between frameworks and runtimes. Table 1.46 summarises the landscape, and Figure 1.56 shows how the layers fit together.

Table 1.46 Major machine learning libraries and frameworks.

Library	Primary purpose	Strengths	Typical users
NumPy / pandas	Arrays and data frames	Ubiquitous, fast vectorised operations	Everyone
scikit-learn	Classical ML	Consistent API, excellent documentation	Data scientists, students
XGBoost / LightGBM / CatBoost	Gradient-boosted trees	Top accuracy on tabular data, fast	Practitioners, competitions
statsmodels	Statistical modelling	Inference, diagnostics, time series	Statisticians, economists
PyTorch	Deep learning	Flexible, Pythonic, research-friendly	Researchers, industry
TensorFlow / Keras	Deep learning	Production and mobile deployment, high-level API	Industry, education
JAX	Differentiable numerical computing	JIT compilation, functional transforms, TPUs	Research labs
Hugging Face Transformers	Pre-trained models	Thousands of ready-to-use models	NLP and vision practitioners
Spark MLlib	Distributed ML	Scales to cluster-size data	Data engineers
WEKA / KNIME / Orange	GUI-based ML	No-code workflows, teaching	Analysts, beginners

> **T Practice Tip | Choosing a Tool**
>
> For tabular data, start with scikit-learn for baselines and a gradient-boosting library for the strongest model. For images, text and audio, use PyTorch or TensorFlow/Keras with pre-trained models. Use a GUI tool such as KNIME or Orange when collaborators do not program, and Spark when data no longer fit on one machine.

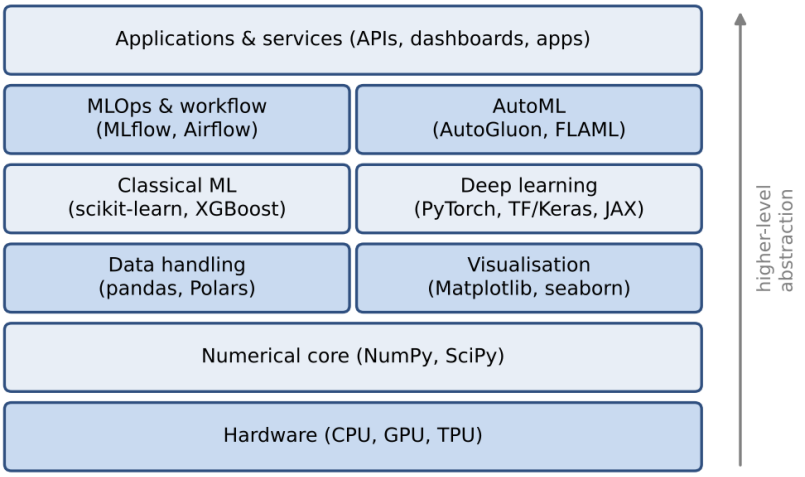


Figure 1.56 Layers of a typical Python machine learning stack, from hardware and numerical foundations up to modelling libraries, workflow tools and applications.

1.46 Practical Machine Learning Using Python, Scikit-learn, Google Colab, and KNIME

This section brings the chapter together in practice. It shows how a complete supervised-learning project is carried out in two complementary environments: **Python with scikit-learn**, run in the cloud-hosted notebook service **Google Colab**, and the visual, no-code analytics platform **KNIME**. The workflow is the same in both — load, explore, preprocess, split, train, tune, evaluate, interpret and save — but the means of expressing it differ.

1.46.1 Working in Google Colab

Google Colaboratory (Colab) is a free, browser-based Jupyter notebook environment hosted by Google. It requires no installation, comes with NumPy, pandas, scikit-learn, TensorFlow and PyTorch pre-installed, and offers free (time-limited) access to GPUs and TPUs, which makes it popular for teaching and prototyping. Notebooks are saved to Google Drive and can be shared like documents. Practical points to remember are that the virtual machine is *ephemeral* — files written to the local disk disappear when the session ends, so data and models should be saved to a mounted Google Drive or downloaded — and that additional packages are installed per session with `!pip install`. GPUs are enabled via *Runtime* → *Change runtime type*.

1.46.2 An End-to-End scikit-learn Project

Listing 1.28 implements a complete project on the Wisconsin breast-cancer dataset (569 tumours, 30 numerical features, malignant or benign). It follows the workflow developed throughout this chapter: a stratified hold-out test set, a pipeline that combines scaling with a classifier, cross-validated comparison of several algorithms, randomised hyperparameter search, a single final evaluation on the test set, permutation-importance interpretation and persistence of the fitted pipeline. Figure 1.57 summarises the steps.

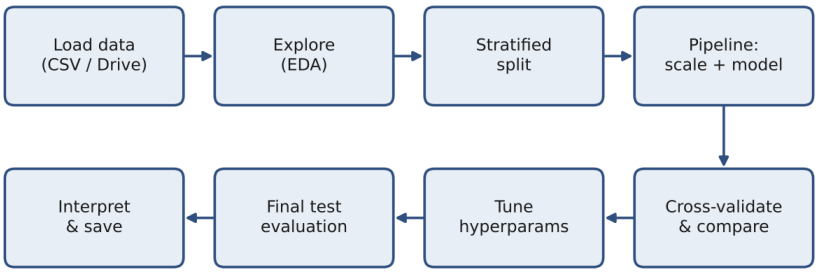


Figure 1.57 Steps of the end-to-end project implemented in Python and, equivalently, as KNIME nodes. The test set is used only once, in the final evaluation step.

Listing 1.28 An end-to-end classification project with scikit-learn (runs unchanged in Google Colab).

```
import numpy as np, joblib
from scipy.stats import loguniform
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score,
↪  RandomizedSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.inspection import permutation_importance

data = load_breast_cancer(as_frame=True)
X, y = data.data, 1 - data.target                    # 1 = malignant (positive class)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y,
↪  random_state=42)
cv = StratifiedKFold(5, shuffle=True, random_state=0)

candidates = {"logreg": LogisticRegression(max_iter=2000),
```

```
              "rf": RandomForestClassifier(n_estimators=300, random_state=0),
              "svm": SVC(probability=True)}
for name, clf in candidates.items():
    pipe = Pipeline([("scale", StandardScaler()), ("clf", clf)])
    print(name, cross_val_score(pipe, X_tr, y_tr, cv=cv,
    ↪  scoring="roc_auc").mean().round(4))

pipe = Pipeline([("scale", StandardScaler()), ("clf", SVC(probability=True))])
search = RandomizedSearchCV(pipe, {"clf__C": loguniform(1e-2, 1e3), "clf__gamma":
↪  loguniform(1e-4, 1)},
                            n_iter=40, cv=cv, scoring="roc_auc",
                            ↪  random_state=0).fit(X_tr, y_tr)
best = search.best_estimator_

proba = best.predict_proba(X_te)[:, 1]              # touch the test set once
print(classification_report(y_te, (proba >= 0.5).astype(int), target_names=["benign",
↪  "malignant"]))
print("test ROC-AUC:", roc_auc_score(y_te, proba).round(4))

pi = permutation_importance(best, X_te, y_te, scoring="roc_auc", n_repeats=10,
↪  random_state=0)
print(X.columns[np.argsort(pi.importances_mean)[::-1][:5]].tolist())
joblib.dump(best, "breast_cancer_svm.joblib")       # in Colab, save to a mounted Drive
↪  folder
```

1.46.3 Visual Workflows with KNIME

KNIME Analytics Platform (the Konstanz Information Miner, first released by the University of Konstanz in 2006) is a free, open-source desktop application in which analyses are built by connecting **nodes** on a canvas into a **workflow**. Each node performs one task — reading a file, filtering rows, normalising columns, training a model — and passes a data table to the next. Workflows are self-documenting, easy to explain to non-programmers, and reproducible; they can also embed Python or R scripts and call Weka, H2O, XGBoost and Keras integrations. Table 1.47 maps each step of the Python project to the corresponding KNIME nodes. Model selection with cross-validation is built with the *X-Partitioner/X-Aggregator* loop, and hyperparameter tuning with *Parameter Optimization Loop* nodes.

> **T Practice Tip | Python or KNIME?**
>
> Use Python notebooks when you need flexibility, custom models, deep learning, version control with Git or deployment as code. Use KNIME when the analysis must be understood and maintained by analysts who do not program, when building repeatable data-preparation and reporting workflows, or for teaching the ML workflow conceptually. The two combine well: KNIME's Python Script node runs scikit-learn code inside a workflow.

Table 1.47 Equivalent steps in scikit-learn and KNIME.

Workflow step	Python / scikit-learn	KNIME node(s)
Read data	pandas.read_csv	CSV Reader, Excel Reader
Explore	df.describe(), plots	Statistics, Data Explorer, Scatter Plot
Handle missing values	SimpleImputer	Missing Value
Encode categories	OneHotEncoder	One to Many, Category to Number
Scale features	StandardScaler	Normalizer (+ Normalizer (Apply))
Train/test split	train_test_split(stratify=y)	Table Partitioner (stratified sampling)
Train model	RandomForestClassifier().fit	Random Forest Learner
Predict	model.predict / predict_proba	Random Forest Predictor
Cross-validation	cross_val_score	X-Partitioner ... X-Aggregator loop
Tune hyperparameters	GridSearchCV / RandomizedSearchCV	Parameter Optimization Loop Start/End
Evaluate	classification_report, roc_auc_score	Scorer, ROC Curve
Save model	joblib.dump	Model Writer, PMML Writer

1.47 Machine Learning Applications Across Different Industries

Machine learning is now embedded in almost every sector of the economy. The same small set of problem types — classification, regression, clustering, anomaly detection, recommendation, forecasting and sequential decision-making — recurs across very different domains. Table 1.48 maps representative applications to the techniques of this chapter.

1.47.1 Healthcare and Life Sciences

Healthcare is one of the most active and consequential application areas. Supervised models estimate the risk of diabetes, heart disease or kidney disease from routine clinical measurements; deep convolutional networks classify skin lesions, retinal scans and histopathology slides at specialist-level accuracy in research settings; unsupervised clustering identifies patient sub-types with different prognoses; and survival models predict time to events. These applications highlight the themes of this chapter: imbalanced classes call for recall, PR-AUC and calibrated probabilities rather than accuracy; clinicians require explanations (SHAP, PDPs); and models must be validated externally on data from other hospitals because of dataset shift.

Table 1.48 Representative machine learning applications by industry.

Industry	Application	Problem type	Typical techniques
Healthcare	Disease risk prediction, medical image diagnosis	Classification	Gradient boosting, CNNs, SHAP for explanation
Healthcare	Patient phenotyping, readmission risk	Clustering, classification	k-means, hierarchical clustering, logistic regression
Finance	Credit scoring	Classification	Logistic regression, gradient boosting with monotonic constraints
Finance	Fraud and anti-money-laundering detection	Anomaly detection	Isolation Forest, autoencoders, graph methods
Retail and e-commerce	Product recommendation, market-basket analysis	Recommendation, association rules	Matrix factorisation, Apriori, FP-Growth
Retail and e-commerce	Customer segmentation, demand forecasting	Clustering, regression	k-means, gradient boosting, time-series models
Manufacturing	Predictive maintenance, visual quality inspection	Anomaly detection, classification	One-Class SVM, random forests, CNNs
Transport and logistics	ETA prediction, route optimisation, ride pricing	Regression, RL	Gradient boosting, reinforcement learning
Agriculture	Crop-yield prediction, plant-disease detection	Regression, classification	Random forests, CNNs on drone images
Energy	Load forecasting, grid and cooling control	Regression, RL	Gradient boosting, LSTMs, deep RL
Telecommunicatio	Churn prediction, network intrusion detection	Classification, anomaly detection	Gradient boosting, LOF
Education	Student dropout prediction, adaptive learning	Classification, RL	Logistic regression, bandits
Cybersecurity	Malware and phishing detection	Classification	Random forests, gradient boosting, explainable ML

1.47.2 Finance, Commerce and Industry

Banks have used statistical credit scoring for decades and now combine it with gradient-boosted models under regulatory requirements for explanation and fairness. Payment networks score every transaction for fraud in milliseconds. Online retailers attribute a substantial share of their sales to recommender systems, and use association rules for product placement and bundling. In manufacturing, vibration and temperature sensors feed anomaly detectors that predict machine failures before they occur, reducing unplanned downtime. Figure 1.58 summarises which problem types dominate across the industries in Table 1.48.

C Case Study 1.7 | Machine Learning in Agriculture

Smallholder farmers lose a large share of their crops to pests and diseases. Mobile applications now let a farmer photograph a leaf and receive an instant diagnosis from a convolutional network trained on hundreds of thousands of labelled images, such as those in the public PlantVillage dataset. Combined with yield models that use satellite imagery, weather and soil data, such tools support earlier interventions and better planning — though per-

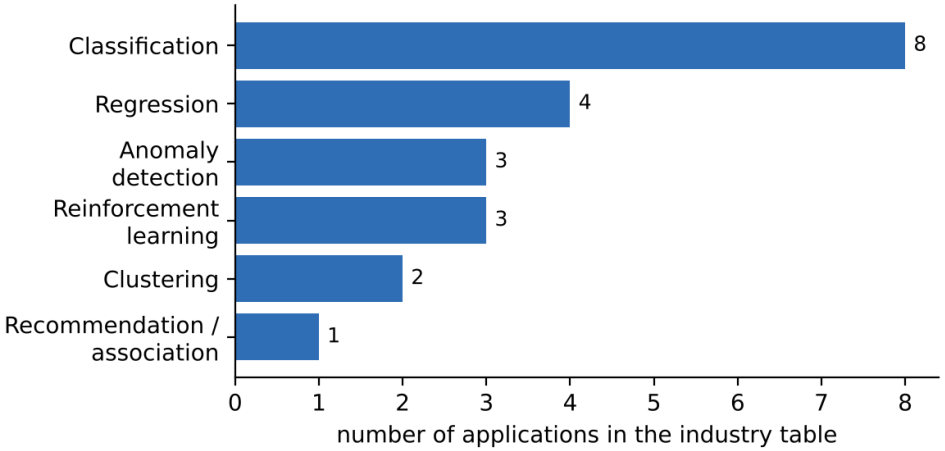


Figure 1.58 How often each machine learning problem type appears among the thirteen representative applications in the industry table; classification dominates, but unsupervised and sequential-decision methods are widespread.

formance often drops on field photographs that differ from the curated training images, a reminder of the importance of representative data.

1.48 Challenges, Limitations, and Future Trends in Machine Learning

Despite its successes, machine learning faces significant technical, ethical and practical challenges. Understanding them is as important as mastering the algorithms, because they determine whether a model that performs well in a notebook will deliver value — and avoid harm — in the real world.

1.48.1 Technical Challenges

- **Data quality and quantity.** Models inherit the errors, gaps and biases of their training data. Labels are often noisy or expensive, and many problems suffer from severe class imbalance.
- **Distribution shift.** Production data drift away from training data — new customer behaviour, new sensors, a pandemic — degrading performance silently unless monitored.
- **Generalisation and robustness.** Models can exploit spurious correlations (shortcut learning) and can be fooled by small, deliberately crafted *adversarial* perturba-

tions.

- **Interpretability.** The most accurate models are often the least transparent, complicating debugging, trust and regulatory compliance.
- **Reproducibility.** Results depend on random seeds, data splits, software versions and undocumented preprocessing; many published results are hard to reproduce.
- **Computational and environmental cost.** Training large models requires substantial energy and specialised hardware, concentrating capability in a few organisations (Strubell, Ganesh and McCallum, 2019).

1.48.2 Ethical, Social and Legal Challenges

Machine learning systems can encode and amplify social bias. The 2016 ProPublica analysis of the COMPAS recidivism tool reported that black defendants who did not reoffend were far more likely than white defendants to be labelled high risk, and subsequent research proved that several intuitive **fairness** criteria — calibration within groups and equal false positive and false negative rates — cannot in general be satisfied simultaneously when base rates differ (Kleinberg, Mullainathan and Raghavan, 2017; Chouldechova, 2017). **Privacy** is at risk when models memorise training data; techniques such as **differential privacy** and **federated learning** (training across devices without centralising raw data; McMahan et al., 2017) mitigate this. **Accountability** requires documentation such as datasheets for datasets and model cards, and regulation is growing, notably the EU AI Act. Table 1.49 summarises the main challenges and current mitigation strategies.

1.48.3 Future Trends

Several trends are shaping the field. **Foundation models** — very large models pretrained by self-supervision on broad data and adapted to many tasks — have shifted practice from training models from scratch towards fine-tuning, prompting and retrieval augmentation; large language models and agentic systems that plan and use tools are discussed in Chapter 3. **Data-centric AI** emphasises systematic improvement of data quality over model tinkering. **Efficient and edge ML** (TinyML, quantisation, distillation) brings inference to phones and microcontrollers. **Causal machine learning** aims to move from correlation to intervention-level reasoning. **Trustworthy AI** integrates fairness, robustness, privacy and explainability into the development lifecycle. **AutoML and MLOps** continue to industrialise the workflow, while **quantum machine learning** and **neuromorphic computing** remain active research frontiers. Figure 1.59 sketches how the dominant paradigm has shifted over time.

Table 1.49 Major challenges in machine learning and common mitigations.

Challenge	Example	Mitigation strategies
Data scarcity	Rare diseases, new products	Transfer learning, semi-/self-supervised learning, data augmentation
Class imbalance	Fraud is < 1% of transactions	Resampling, class weights, PR-AUC, threshold tuning
Distribution shift	Model trained before a market change	Drift monitoring, retraining, domain adaptation
Bias and fairness	Unequal error rates across groups	Fairness audits, constraints, representative data
Lack of transparency	Unexplained credit refusal	Interpretable models, SHAP/LIME, counterfactuals
Privacy	Memorisation of personal data	Differential privacy, federated learning, anonymisation
Adversarial vulnerability	Perturbed images misclassified	Adversarial training, input validation
Reproducibility	Results differ across runs	Seeds, versioning, pipelines, experiment tracking
Compute and energy cost	Large-model training	Efficient architectures, distillation, pruning, quantisation

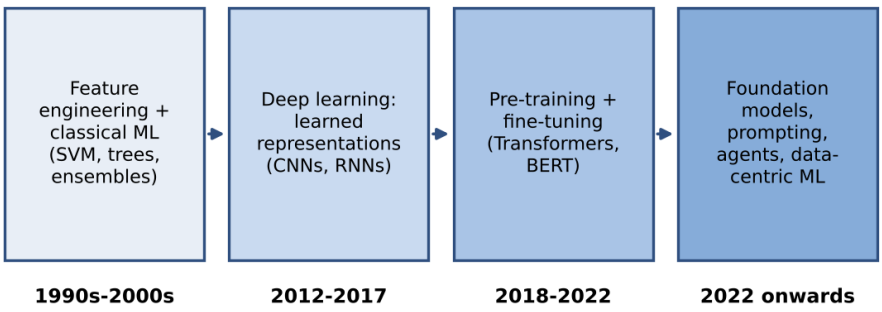


Figure 1.59 Shifts in the dominant machine learning paradigm, from hand-engineered features and classical models to representation learning and pre-trained foundation models adapted to many tasks (approximate periods).

Key Point | The Enduring Fundamentals

Tools and architectures change quickly, but the fundamentals of this chapter do not: define the problem and the metric carefully, understand and prepare the data, establish baselines, guard against leakage and overfitting with proper validation, interpret and question the model, and monitor it after deployment.

Chapter Summary

This chapter introduced machine learning as the study of algorithms that improve their performance at a task through experience. We traced its history from early perceptrons and statistical pattern recognition to today's data-driven systems, defined its key concepts — features, labels, models, parameters, hyperparameters, loss functions and generalisation — and distinguished its main paradigms: supervised, unsupervised, semi-supervised, self-supervised and reinforcement learning. We followed the machine learning workflow from problem definition through data collection, cleaning, exploratory data analysis, feature engineering and feature scaling to the division of data into training, validation and test sets.

The chapter then examined the central tension of the field, the bias–variance trade-off, and its practical manifestations as under- and over-fitting, together with regularisation (ridge, lasso, elastic net) as a remedy. We studied the core supervised algorithms — linear and logistic regression, decision trees, random forests, support vector machines, k-nearest neighbours and naive Bayes — and the ensemble strategies of bagging, boosting and stacking that combine them into stronger predictors.

Turning to unlabelled data, we covered clustering with k-means (Lloyd's algorithm, k-means++ seeding, the elbow method and silhouette analysis), hierarchical clustering with its linkage criteria and dendrograms, and density-based clustering with DBSCAN. We explained the curse of dimensionality, derived principal component analysis from the eigen-decomposition of the covariance matrix, and surveyed non-linear embeddings such as t-SNE and UMAP. Association rule learning with support, confidence and lift, the Apriori and FP-Growth algorithms, and anomaly detection with statistical rules, Isolation Forest, LOF and One-Class SVM completed the unsupervised toolkit. Semi-supervised methods (self-training, pseudo-labelling, co-training, label propagation) and self-supervised pretext and contrastive tasks showed how unlabelled data can strengthen learning from few labels.

Reinforcement learning was introduced through Markov decision processes, policies, value functions, the Bellman equations and the exploration–exploitation dilemma, leading to Q-learning and Deep Q-Networks with experience replay and target

networks. We then addressed how to evaluate and improve models rigorously: classification metrics derived from the confusion matrix, ROC and precision–recall curves, regression metrics (MAE, MSE, RMSE, R^2), cross-validation schemes, and grid, random and Bayesian hyperparameter search, as well as the gradient-based and second-order optimisation algorithms that train most models.

Finally, we considered machine learning as an engineering and social practice: explaining models with feature importance, partial dependence, SHAP and LIME; building leakage-safe pipelines and MLOps workflows; automating model and feature search with AutoML; choosing among tools and frameworks; carrying out end-to-end projects in Python, Google Colab and KNIME; applying machine learning across industries; and confronting its challenges of data quality, distribution shift, fairness, privacy and cost. Chapter 2 builds on these foundations to develop deep learning, where representations themselves are learned by multi-layer neural networks.

Key Terms

Table 1.50 Key terms introduced in Chapter 1.

Term	Definition
Machine learning	The study of algorithms that improve at a task through experience (data) without being explicitly programmed for it.
Supervised learning	Learning a mapping from inputs to known target labels or values.
Unsupervised learning	Discovering structure, such as groups or low-dimensional representations, in unlabelled data.
Reinforcement learning	Learning a behaviour policy that maximises cumulative reward through interaction with an environment.
Feature engineering	Creating, transforming and selecting input variables to improve model performance.
Feature scaling	Transforming features to comparable ranges, e.g. standardisation or min–max normalisation.
Data leakage	Use of information during training that would not be available at prediction time, inflating performance estimates.
Bias–variance trade-off	The tension between error from overly simple assumptions (bias) and from sensitivity to training data (variance).
Overfitting	Fitting noise in the training data so that performance on new data deteriorates.
Regularisation	Adding a penalty on model complexity to the objective to reduce overfitting (e.g. L1, L2).
Ensemble learning	Combining several models (bagging, boosting, stacking) to obtain better predictions than any single model.
Clustering	Partitioning data into groups of similar observations without labels.

Term	Definition
k-means	Partitional clustering that minimises the within-cluster sum of squared distances to k centroids.
Silhouette coefficient	Internal clustering measure comparing a point's cohesion with its own cluster to its separation from the nearest other cluster.
Dendrogram	Tree diagram recording the sequence and heights of merges in hierarchical clustering.
Linkage criterion	Rule defining the distance between clusters in hierarchical clustering (single, complete, average, Ward).
DBSCAN	Density-based clustering that grows clusters from core points and labels sparse points as noise.
Curse of dimensionality	Phenomena such as data sparsity and distance concentration that arise as the number of features grows.
Principal component analysis	Linear projection onto orthogonal directions of maximum variance, found by eigen-decomposition of the covariance matrix.
Explained variance ratio	The fraction of total variance captured by a principal component.
Association rule	An implication $X \Rightarrow Y$ between itemsets, evaluated by support, confidence and lift.
Lift	Ratio of a rule's confidence to the consequent's support; values above 1 indicate positive association.
Isolation Forest	Anomaly detector that scores points by how few random splits are needed to isolate them.
Pseudo-labelling	Using a model's confident predictions on unlabelled data as training targets.
Pretext task	A self-supervised task with automatically generated targets used to learn transferable representations.
Markov decision process	Formal model of sequential decision-making with states, actions, transitions, rewards and a discount factor.
Bellman equation	Recursive relation expressing a state's value as immediate reward plus discounted value of successor states.
Q-learning	Off-policy temporal-difference algorithm that learns optimal action values from sampled transitions.
Experience replay	Storing past transitions and training on random mini-batches of them to decorrelate updates.
Confusion matrix	Table cross-tabulating actual and predicted classes (TP, FP, FN, TN).
Precision and recall	The fraction of predicted positives that are correct, and the fraction of actual positives that are found.
ROC-AUC	Area under the ROC curve; the probability that a random positive is ranked above a random negative.
Cross-validation	Repeated partitioning of data into training and validation folds to estimate generalisation performance.
Hyperparameter	A setting chosen before training that controls model capacity or the learning process.

Term	Definition
Gradient descent	Iterative optimisation that updates parameters in the direction of the negative gradient of the loss.
SHAP	Additive feature attributions based on Shapley values from cooperative game theory.
AutoML	Automation of algorithm selection, hyperparameter optimisation and pipeline construction.
MLOps	Practices for automating and monitoring the deployment and maintenance of machine learning systems.

Review Questions

1. Explain how machine learning differs from traditional rule-based programming, and give an example of a problem for which machine learning is clearly preferable.
2. Outline the key milestones in the history of machine learning, from the perceptron to modern deep learning, and explain what enabled the resurgence of neural networks after 2010.
3. Distinguish supervised, unsupervised, semi-supervised, self-supervised and reinforcement learning, giving one realistic application of each.
4. Describe the stages of the machine learning workflow. Why is problem definition, including the choice of evaluation metric, placed at the very beginning?
5. Compare three strategies for handling missing values. Under what circumstances can imputation itself introduce bias?
6. What is the purpose of exploratory data analysis? List four specific checks you would perform on a new tabular dataset before modelling.
7. Why must scaling and imputation parameters be learned on the training set only? Describe how a pipeline prevents this form of data leakage.
8. Explain the bias–variance trade-off and relate it to underfitting and overfitting. How do learning curves help diagnose which problem a model has?
9. Compare L1 (lasso) and L2 (ridge) regularisation in terms of their effect on coefficients and their usefulness for feature selection.
10. Why is logistic regression a classification method despite its name? Explain the role of the sigmoid function and the cross-entropy loss.
11. How does a random forest reduce the variance of individual decision trees? Contrast bagging with boosting.

12. Explain the kernel trick in support vector machines and why it allows non-linear decision boundaries to be learned efficiently.
13. Why does k-means fail on non-convex clusters, and how do DBSCAN and single-linkage hierarchical clustering overcome this limitation?
14. Explain why the eigenvectors of the covariance matrix give the principal components, and how you would choose the number of components to retain.
15. Define support, confidence and lift. Why can a rule with high confidence still be uninteresting?
16. Compare Isolation Forest and Local Outlier Factor. For what kind of data would you prefer LOF?
17. What assumptions must hold for unlabelled data to improve a semi-supervised classifier? Describe the risk of confirmation bias in self-training.
18. Write the Bellman optimality equation for q* and explain how Q-learning turns it into a sample-based update. Why is Q-learning called off-policy?
19. When is accuracy a misleading metric? Explain why the precision–recall curve is often preferred to the ROC curve for highly imbalanced data.
20. Explain nested cross-validation and why it is needed when hyperparameters are tuned. Why is random search often more efficient than grid search?
21. Discuss the difference between global and local explanations, and compare SHAP and LIME as local explanation methods.
22. Identify three ethical challenges in deploying machine learning systems and propose a technical or procedural mitigation for each.

Exercises

1. **k-means by hand.** Cluster the points $(1, 1)$, $(1.5, 2)$, $(3, 4)$, $(5, 7)$, $(3.5, 5)$, $(4.5, 5)$, $(3.5, 4.5)$ into $k = 2$ clusters starting from centroids $(1, 1)$ and $(5, 7)$. Show each assignment and update step until convergence and compute the final inertia.
2. **Hierarchical clustering.** For the one-dimensional data $\{2, 4, 5, 10, 12, 20\}$, construct the single-linkage and complete-linkage dendrograms, giving the height of every merge. Where would you cut each dendrogram to obtain three clusters?
3. **PCA.** For the data $(1, 2)$, $(2, 3)$, $(3, 5)$, $(4, 4)$, $(5, 6)$, compute the covariance matrix, its eigenvalues and eigenvectors, and the proportion of variance explained by the first principal component. Project the data onto PC1.

4. **Association rules.** Using the transactions in the market-basket table of this chapter, run Apriori with a minimum support of 0.25 and list all frequent itemsets. Generate all rules with confidence ≥ 0.6 and rank them by lift.

5. **Outliers.** For the values 10, 12, 12, 13, 12, 11, 14, 13, 15, 102, 12, 14, compute the z-score, robust z-score and IQR fences, and state which values each method flags.

6. **Classification metrics.** A classifier's confusion matrix on 1,000 cases is TP = 45, FN = 15, FP = 90, TN = 850. Compute accuracy, precision, recall, specificity, F1 and F_2. Would you deploy this model for (a) cancer screening and (b) automatic spam deletion? Justify your answer.

7. **ROC by hand.** Six test cases have scores $(0.9, 0.8, 0.7, 0.6, 0.4, 0.2)$ and true labels $(1, 1, 0, 1, 0, 0)$. Plot the ROC curve by sweeping the threshold and compute the AUC both geometrically and as the proportion of correctly ordered positive–negative pairs.

8. **Regression metrics.** For $y = (10, 12, 15, 20, 25)$ and $\hat{y} = (11, 11, 16, 18, 30)$, compute MAE, MSE, RMSE and R^2. Recompute after replacing the last prediction with 25 and comment on which metrics change most.

9. **Q-learning.** In a 1×4 corridor with states s_1–s_4 (s_4 terminal, reward $+10$ on entry, -1 for every other move), actions left and right, $\alpha = 0.5$ and $\gamma = 0.9$, perform Q-learning updates for the episode $s_1 \to s_2 \to s_3 \to s_2 \to s_3 \to s_4$ starting from $Q = 0$. Report the final Q-table.

10. **Gradient descent.** Implement batch, mini-batch and stochastic gradient descent in NumPy for linear regression on synthetic data $y = 4 + 3x + \varepsilon$. Plot the training loss against the number of gradient evaluations for three learning rates and discuss convergence.

11. **Programming: clustering study.** Using scikit-learn, apply k-means, agglomerative clustering (Ward) and DBSCAN to the `make_moons` and `make_blobs` datasets and to the standardised Iris data. Choose hyperparameters using the silhouette score and k-distance plot, and report the adjusted Rand index against the true labels.

12. **Programming: end-to-end project.** For a public tabular dataset of your choice (e.g. a UCI heart-disease or diabetes dataset), build a scikit-learn pipeline with imputation, encoding and scaling; compare three classifiers with stratified 5-fold cross-validation; tune the best with randomised search inside nested cross-validation; report test-set metrics with bootstrap confidence intervals; and explain the final model with permutation importance and SHAP. Re-implement the same workflow in KNIME and compare the two experiences.

Further Reading

- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press.
- Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- Han, J., Kamber, M. and Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2nd ed.). Self-published.
- Hutter, F., Kotthoff, L. and Vanschoren, J. (eds.) (2019). *Automated Machine Learning: Methods, Systems, Challenges*. Springer.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30*.
- Ester, M., Kriegel, H.-P., Sander, J. and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD-96*, 226–231.
- Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 487–499.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13, 281–305.

CHAPTER 1

Machine Learning

Machine learning is the study of computer programs that improve their performance on a task by learning from data rather than by following rules written explicitly by a programmer. Over the last three decades it has moved from a specialised corner of artificial intelligence research to a core engineering discipline: it ranks web search results, filters spam, detects fraudulent card transactions, recommends films and products, reads medical images, forecasts demand and translates between languages. Behind each of these applications sit a small number of recurring ideas — representing examples as feature vectors, choosing a family of candidate models, measuring error with a loss function, fitting parameters by optimisation and, above all, checking that what has been learned generalises to data the model has never seen.

AT A GLANCE

48 Sections	**59** Figures	**50** Tables
23 Algorithms	**79** Equations	**28** Code Listings

IN THIS CHAPTER YOU WILL LEARN TO

- ✓ define machine learning formally and distinguish it from traditional rule-based programming, statistics and the broader field of artificial intelligence.
- ✓ outline the historical development of machine learning and identify the milestones that shaped today's methods.
- ✓ classify a problem as supervised, unsupervised, semi-supervised, self-supervised or reinforcement learning, and choose an appropriate formulation.
- ✓ describe each stage of the machine learning lifecycle, from problem framing and data collection to deployment and monitoring.
- ✓ clean, preprocess, explore, encode, scale and split a raw dataset without introducing data leakage.

ABOUT THE AUTHOR

Luxshi Karunakaran
B.Sc. (Hons) in Computer Science and Technology
M.Sc. in Data Science and Artificial Intelligence (Reading)
Department of Social Statistics, University of Sri Jayewardenepura, Sri Lanka